



Build a Roguelike in Godot

A PRACTICAL GUIDE - 1ST EDITION

Daniel Buckley
Pablo Farias

academy.zenva.com
youtube.com/c/zenva
linkedin.com/company/zenva

©2026 Zenva Pty Ltd, all rights reserved

Table of Contents

Introduction.....	8
How to use this Book.....	8
Project Setup.....	10
Prerequisites.....	10
Project Files.....	11
Godot Installation and Version.....	13
How to Install Version 4.6.....	13
Game Design.....	14
Game Overview.....	14
Core Gameplay Mechanics.....	15
Player Controls.....	16
Enemy Behavior.....	16
User Interface.....	17
Project Setup.....	20
Setting Up the File System Structure.....	20
Creating the First Scene.....	22
Changing the Background Color.....	24
Setting Up Inputs.....	24
Creating the Player Character.....	27
Setting Up the Player Node.....	27
Adding Visuals.....	28
Adjusting Texture Settings.....	29
Selecting a Sprite Region.....	31
Adding Collision.....	33
Setting Up the Weapon.....	34
Adding a Muzzle Point.....	38
Setting Up the Camera.....	40
Scripting the Player.....	42
Defining Variables.....	43
Implementing Movement.....	44
Implementing Aiming.....	45
Flipping the Sprite.....	45
Saving the Player Scene.....	48

Designing Room Templates.....	49
Understanding the Room Structure.....	49
Setting Up the Room Node.....	50
Adding the TileMapLayer.....	51
Drawing the Room Corners.....	54
Adding Wall Colliders.....	56
Creating Entrances.....	59
Adding Doors and Tunnels.....	60
Adding Door Sprites and Collisions.....	63
Adding Exit Triggers and Player Spawn.....	64
Creating the Barrier.....	66
Adding Scripts to Manage Entrances.....	69
Duplicating Entrances for Other Directions.....	70
Saving the Room Template.....	72
Procedural Room Generation.....	75
The Generation Process.....	75
Blocking Out Room Positions.....	75
Setting Up the Room Generation Node.....	77
Defining Variables.....	79
Preloading the Room Scene.....	80
Creating Functions.....	80
Implementing the Generation Logic.....	81
The Generate Function.....	81
Helper Functions for the Map.....	82
The Recursive Check Room Function.....	82
Deciding Expansion Directions.....	84
Verifying the Generation.....	85
Instantiating Room Scenes.....	86
Adjusting the Camera.....	87
Room Logic and Navigation.....	89
The Room Entrance Script.....	89
Defining Directions.....	89
Setting Up Variables.....	90
Entrance Functions.....	90
Initialization.....	91
Controlling Doors.....	92

Toggling the Barrier.....	93
Setting the Neighbor.....	93
Determining the Neighbor's Entry Direction.....	93
The Room Script.....	94
Configuring Entrances in the Editor.....	94
Room Variables.....	95
Room Functions.....	96
Opening and Closing Doors.....	96
Handling Player Entry.....	97
Setting Neighbors.....	98
Connecting Room Neighbors.....	98
Updating Room Instantiation.....	98
Helper Functions for Map Position.....	99
Linking Neighbors.....	100
Spawning the Player.....	100
Player Room Traversal.....	102
Identifying the Player.....	102
Triggering the Transition.....	103
Conclusion.....	104
Combat Mechanics and Signals.....	105
Player Shooting.....	105
Creating the Projectile Scene.....	105
Scripting the Projectile.....	109
Updating the Player.....	112
Projectile Lifecycle.....	112
Implementing Shooting in Player.....	115
Global Signals.....	118
What are Global Signals?.....	118
Setting Up the Autoload.....	118
Defining the Signal.....	119
Emitting the Signal.....	119
Connecting the Camera.....	120
Benefits of Global Signals.....	123
Creating Enemies.....	124
Creating the Enemy Character.....	124
Adding Components.....	125

Scripting the Enemy.....	127
Defining Variables.....	127
Defining Functions.....	128
Movement Logic.....	129
Connecting the Player.....	129
Setting Up Collision Layers and Masks.....	131
Configuring the Player.....	132
Configuring the Enemy.....	133
Configuring the Projectile.....	134
Combat Logic.....	137
Stopping Within Range.....	137
Attacking the Player.....	138
Taking Damage and Dying.....	139
Room Integration.....	140
Updating the Enemy Script.....	140
Locking Doors.....	141
Updating the Room Script.....	142
Items and Dungeon Expansion.....	145
Different Room Types.....	145
Clearing the Room Template.....	145
Creating Inherited Room Scenes.....	146
Updating the Room Generation Script.....	151
Testing the Room Generation.....	153
Challenge: Four Enemy Room.....	155
Items.....	157
Creating the Base Item Scene.....	158
Programming the Item Script.....	161
Extending the Player Script.....	164
Creating Potion Scenes.....	165
Item Room.....	168
Setting Up the Item Room.....	168
Spawning Random Items.....	171
Testing the Item Room.....	174
User Interface.....	176
Creating the Mini-Map.....	176
The CanvasLayer Node.....	176

Adding UI Borders.....	177
Setting Up the Mini-Map Grid.....	182
Scripting the Mini-Map Logic.....	187
Player Health UI.....	191
Creating the Health Container.....	191
Scripting the Health UI.....	194
Connecting the Health Signal.....	195
Audio and Visual Feedback.....	198
Visual Feedback: Damage Flash.....	198
Implementing Flash on Enemies.....	198
Implementing Flash on the Player.....	199
Implementing Sound Effects.....	200
Player Shoot Sound.....	200
Player Damage Sound.....	202
Enemy Damage Sound.....	203
Item Collection Sound.....	205
Adding Background Music.....	206
Creating the Boss Battle.....	209
Creating the Boss Enemy.....	209
Creating the Boss Room.....	211
Guaranteeing the Boss Room Spawn.....	213
Creating the Next Level Portal.....	216
Scripting the Portal.....	220
Main Menu and Game Loop.....	226
Creating the Menu Scene.....	226
Setting Up the Title.....	226
Adding Buttons.....	229
Creating the Menu Script.....	232
Testing the Menu.....	235
Connecting the Menu to the Game.....	236
Handling Player Death.....	236
Handling Level Completion.....	237
Conclusion.....	237
Closing Words.....	238
What You Accomplished.....	238
Where to Go From Here.....	239

Final Thoughts.....	240
Appendix: Full Source Code.....	241
camera_controller.gd.....	241
enemy.gd.....	241
global_signals.gd.....	243
item.gd.....	244
menu.gd.....	245
minimap.gd.....	245
next_level_portal.gd.....	246
player.gd.....	247
player_health_ui.gd.....	250
projectile.gd.....	251
room.gd.....	251
room_entrance.gd.....	254
room_generation.gd.....	255
room_item_spawner.gd.....	260

Introduction

Welcome to the world of Godot. If you have ever wanted to bring your own game ideas to life, you are in the right place. This book is not just a technical manual; it is a guided tour through the creative process of building games.

Godot has grown into one of the most popular game engines available today. It is lightweight, powerful, and surprisingly intuitive once you understand its core concepts. We wrote this book to help you bridge the gap between “I want to make a game” and “I just made a game.” Whether you are just taking your first steps out of the beginner level or an experienced developer switching from another engine, we are glad you chose to pursue your journey here.

Over the coming chapters, you will build a complete roguelike dungeon crawler from the ground up. Think along the lines of *The Binding of Isaac* or *Enter the Gungeon*: procedurally generated rooms, enemies that swarm you on sight, collectible items that power up your character, a boss fight, and a full game loop tying it all together. By the end, you will have a playable game and the confidence to expand it with your own ideas.

How to use this Book

We have structured this journey to be practical and hands-on. Across its many chapters, we will build our project and introduce new concepts layer by layer. Instead of memorizing abstract definitions, you will learn by doing.

To get the most out of this experience, keep these tips in mind:

- **Code along with us.** It is easy to read through code snippets and nod your head, but typing them out yourself builds muscle memory.
- **Experiment freely.** If you wonder what happens if you change a number or move a node, try it. Breaking the game is often the fastest way to understand how it works.
- **Use the resources.** If you get stuck or want to dive deeper into a specific node, the official Godot documentation is your best friend. We will also provide a full appendix of the code at the end of this book.

We are excited to be part of your development journey. When you are ready, turn the page, set up your environment, and let's build something great together.

Project Setup

Let's start by looking at the big picture: the game we are building, the tools we need, and the project structure that will carry us through every chapter. By the end of this chapter, you will have Godot installed, a new project created, and your folder structure and input mappings ready to go.

Prerequisites

This book assumes you are super comfortable with the basics of Godot and GDScript, and are ready for something more intermediate. Specifically, you should be familiar with:

- Godot's editor interface: the Scene dock, Inspector, FileSystem dock, and Node tab.
- Nodes, scenes, and the scene tree.
- GDScript fundamentals: variables, types, operators, conditionals, and functions.
- 2D coordinate systems and basic vector math.
- Exporting variables with `@export`.
- Working with Godot's TileMap system.

If any of these topics are new to you, the following resources are a great place to start:

- [Learn Godot From Scratch](https://academy.zenva.com/product/intro-to-godot-ebook/)
(<https://academy.zenva.com/product/intro-to-godot-ebook/>)
- [Mini-Projects with Godot](https://academy.zenva.com/product/godot-mini-projects-ebook/)
(<https://academy.zenva.com/product/godot-mini-projects-ebook/>)

Alternatively, you can follow the video course versions of these topics:

- [Intro to Godot 4 Game Development](https://academy.zenva.com/product/intro-to-godot-4-game-development/)
(<https://academy.zenva.com/product/intro-to-godot-4-game-development/>)
- [Godot 4 Mini-Projects](https://academy.zenva.com/product/godot-4-mini-projects/)
(<https://academy.zenva.com/product/godot-4-mini-projects/>)

Having built a couple of small projects in Godot (even simple demos or experiments) will give you a solid head start. That said, we walk through every step in detail, so if you aren't an expert just yet, you can absolutely follow along.

Project Files

To follow along with this tutorial, you will need the project assets. You can also download the completed project for reference.

- Assets:
<https://academy.zenva.com/wp-content/uploads/2026/03/Assets-Godot-Roguelike.zip>
- Completed Project:
<https://academy.zenva.com/wp-content/uploads/2026/03/Complete-Project-Godot-Roguelike-v4.6.zip>

Third Party Asset Licenses

Some of the assets provided in this course are subject to third-party licenses:

Assets by VEXED

Tiles, people, potions, weapons, creatures, and UI sprites

URL: <https://v3x3d.itch.io/>

License: Creative Commons CC0
(<https://creativecommons.org/publicdomain/zero/1.0/>)

Assets by Alexander Ehlers (tricksntraps)

Background music

URL: <https://opengameart.org/users/tricksntraps>

License: Creative Commons CC0
(<https://creativecommons.org/publicdomain/zero/1.0/>)

Assets by artisticdude

Collect item SFX

URL: <https://opengameart.org/users/artisticdude>

License: Creative Commons CC0
(<https://creativecommons.org/publicdomain/zero/1.0/>)

Assets by Rick Hoppmann (TinyWorlds)

Player Damaged SFX

URL: <https://opengameart.org/users/tinyworlds>

License: Creative Commons CC0
(<https://creativecommons.org/publicdomain/zero/1.0/>)

Assets by Kenney

Room Enter SFX

URL: <https://www.kenney.nl/>

License: Creative Commons CC0
(<https://creativecommons.org/publicdomain/zero/1.0/>)

Zenva-Created Assets

All other artwork and media assets included in the project files that are not listed above were created by **Zenva** and are released under:

Creative Commons CC0 (Public Domain)
<https://creativecommons.org/publicdomain/zero/1.0/>

Zenva-created source code included in the project files for this course is released under the MIT License (<https://opensource.org/license/MIT>).

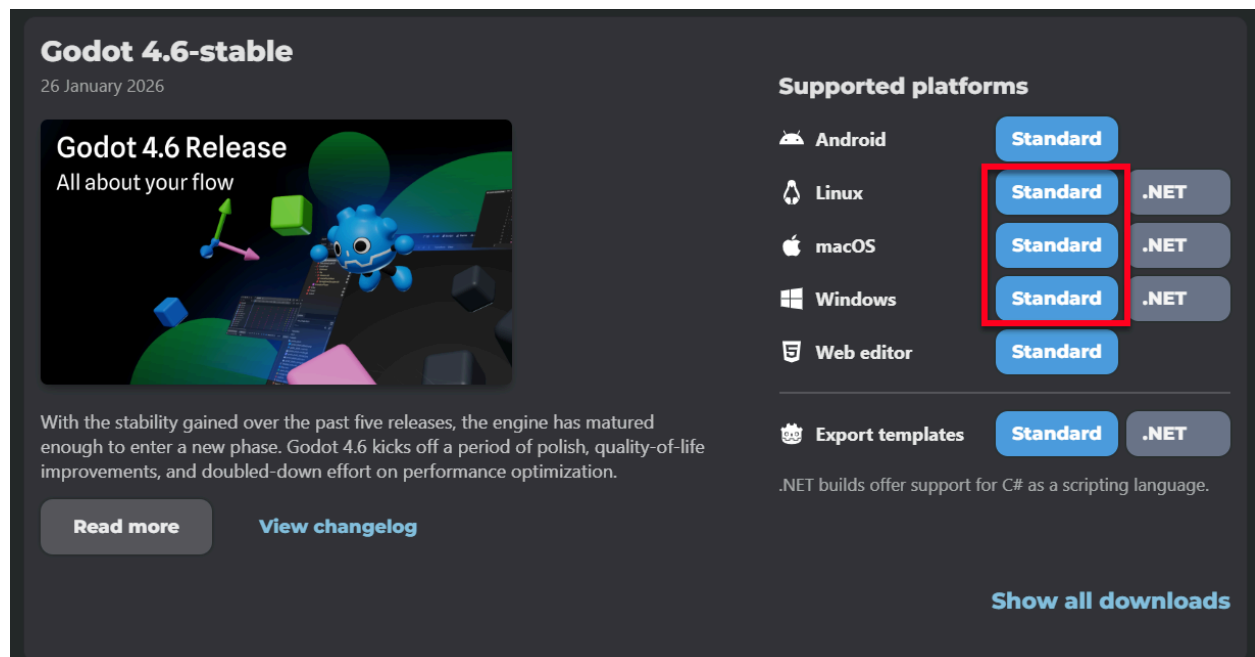
Third-party libraries and dependencies remain under their respective licenses.

Godot Installation and Version

For this project, we are using **Godot 4.6**. It is important to use this specific version (or a compatible stable release) to ensure the course material works as expected.

How to Install Version 4.6

You can download the 4.6 version of the engine by visiting the official Godot download archive (<https://godotengine.org/download/archive/4.6-stable/>). On this page, select the Standard option that matches your operating system. Note that instructions provided in this ebook are intended for desktops only.



Once the file is downloaded, unzip it to a location of your choice. Godot does not require a traditional installation process; simply double-click the application file to launch the engine.

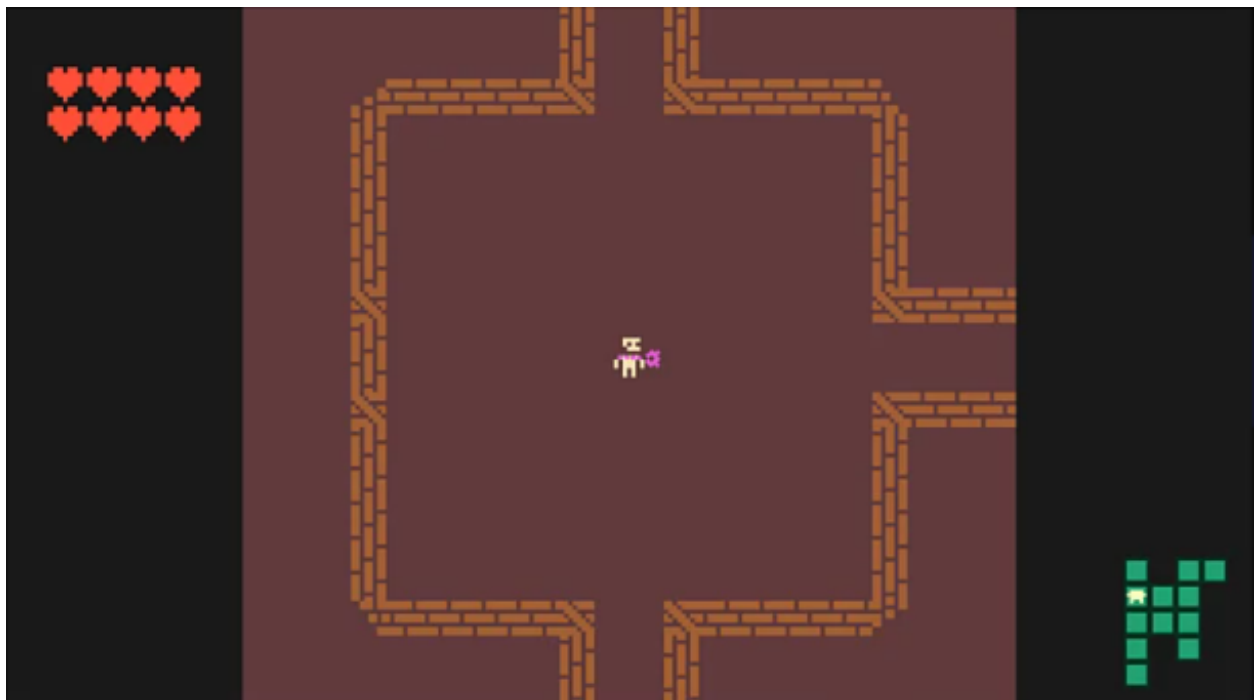
Game Design

Before we dive into the development process, let's take a moment to understand what we will be building. Our goal is to create a game similar to popular titles like *The Binding of Isaac* or *Enter the Gungeon*. These are run-and-gun shooters where the player enters a room, defeats all enemies to proceed to the next room, collects items, and faces a boss to advance to the next level.

Game Overview

The game will feature several key systems:

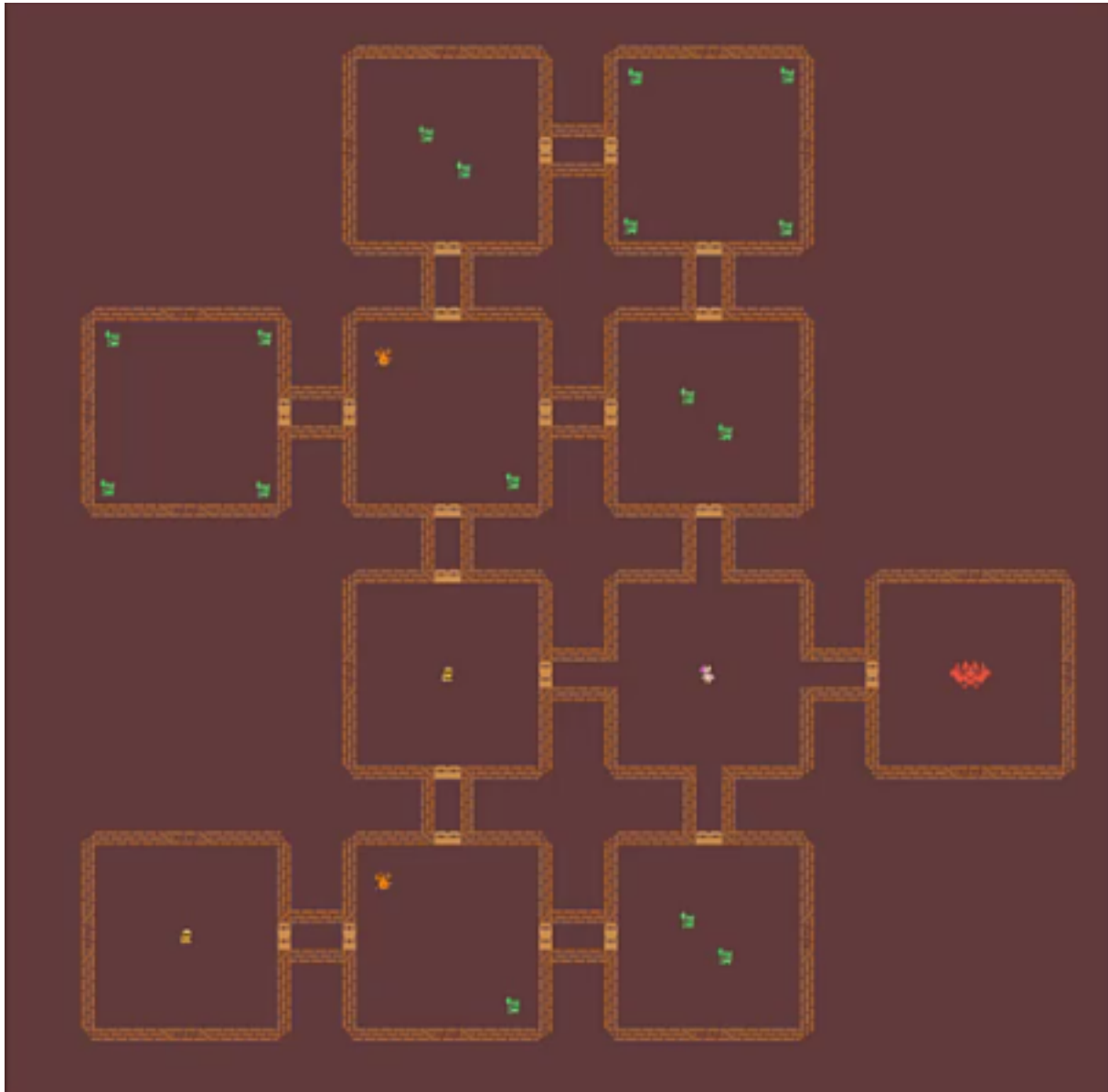
- Procedurally generated rooms with different types and connecting corridors.
- Items to collect and enhance the player's abilities.
- Enemies that activate when the player enters their room.
- A mini-map to display the room layout and player's position.
- A health system displayed as hearts.
- A menu system to allow players to start or quit the game.



Core Gameplay Mechanics

The main focus of this book will be on room generation. Here is how the loop will work:

1. At the start of the game, a room layout will be procedurally generated.
2. Each room will have doors that open or close based on the presence of enemies.
3. Doors will close when the player enters a room with enemies and will only open once all enemies are defeated.

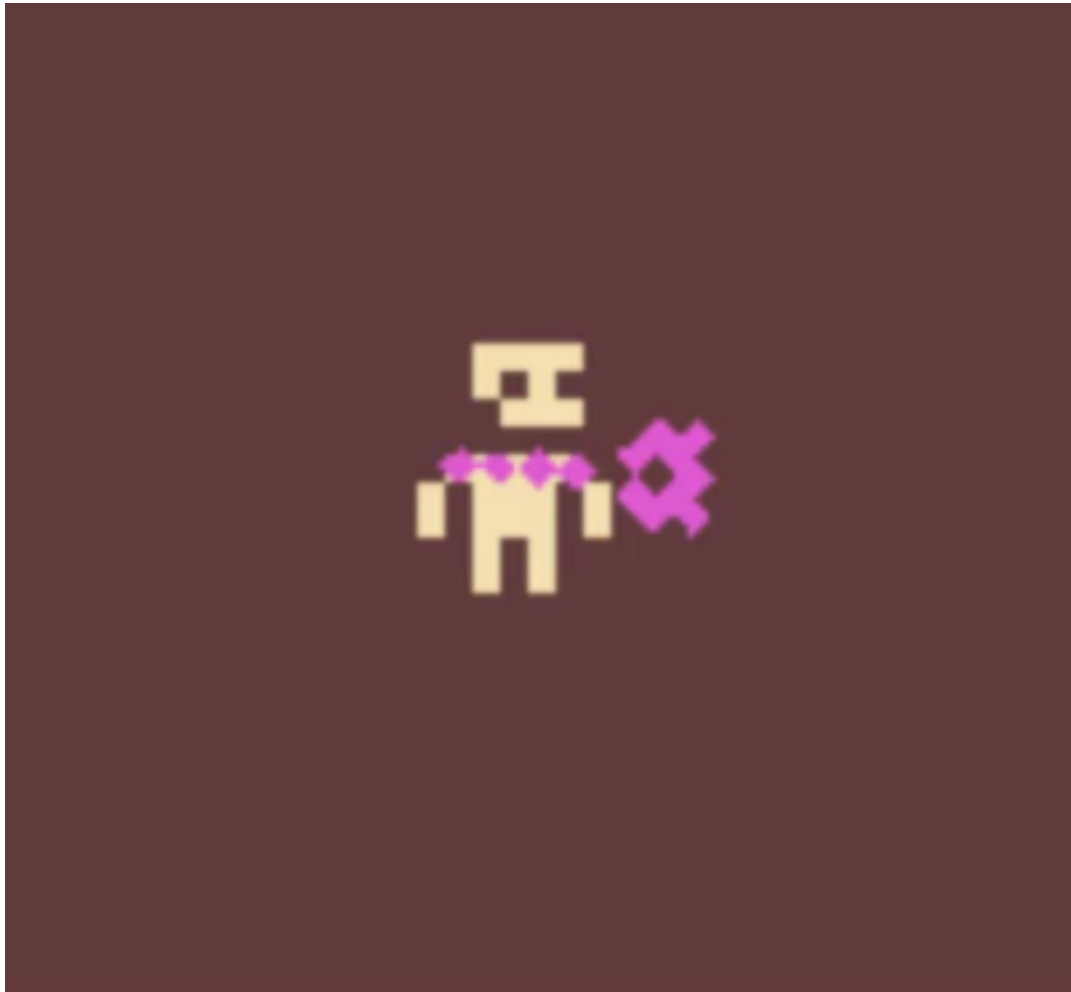


Player Controls

The player will control their character using standard keyboard and mouse inputs:

- **Movement:** W, A, S, and D keys.
- **Aiming:** Mouse position.
- **Shooting:** Left mouse button.

The player will shoot projectiles that deal damage to enemies upon impact.

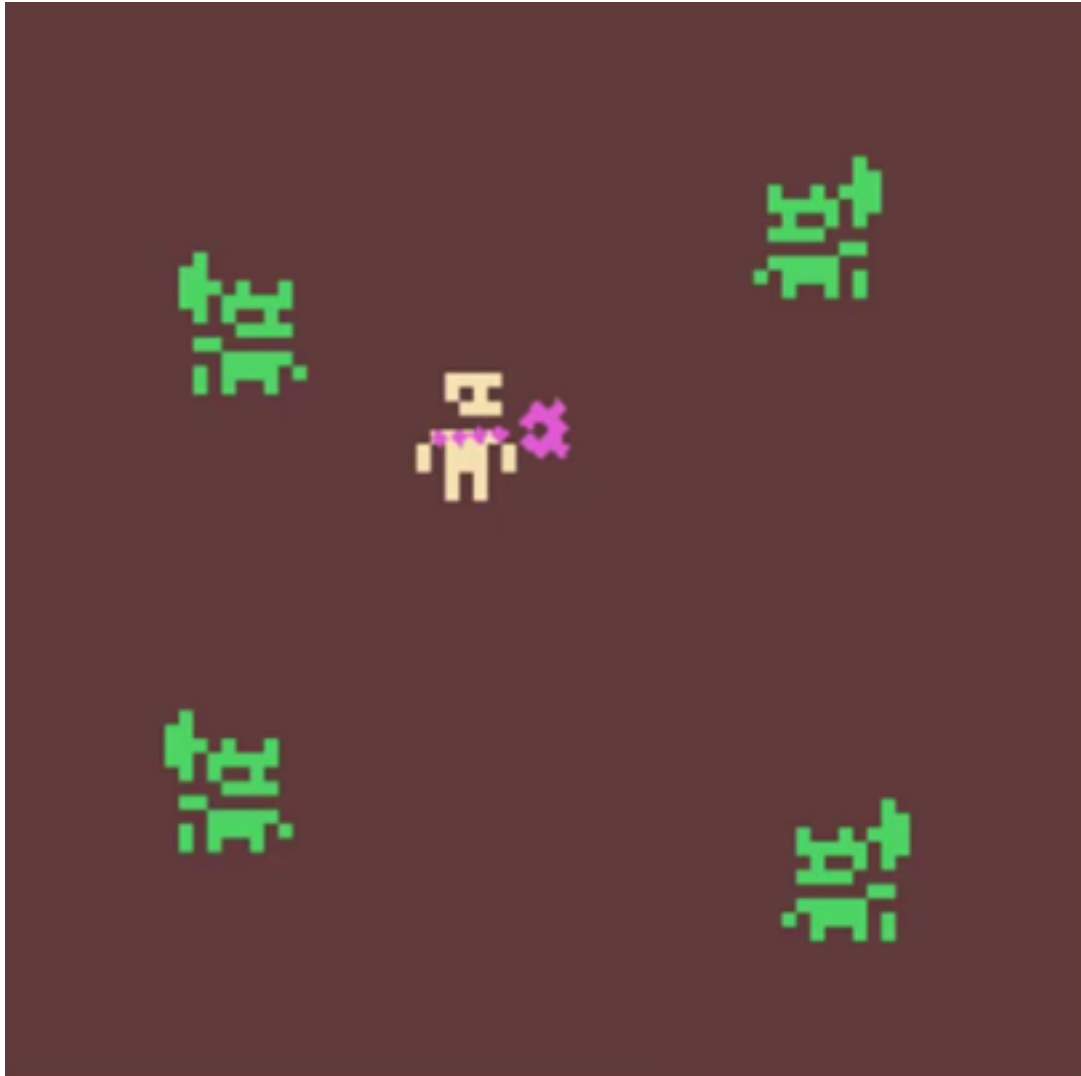


Enemy Behavior

Enemies will be designed with specific behaviors to challenge the player:

- Enemies activate only when the player enters their room.
- Enemies chase the player and deal damage when in range.

- Enemies can be defeated by the player's projectiles.
- When all enemies in a room are defeated, the doors will open.



User Interface

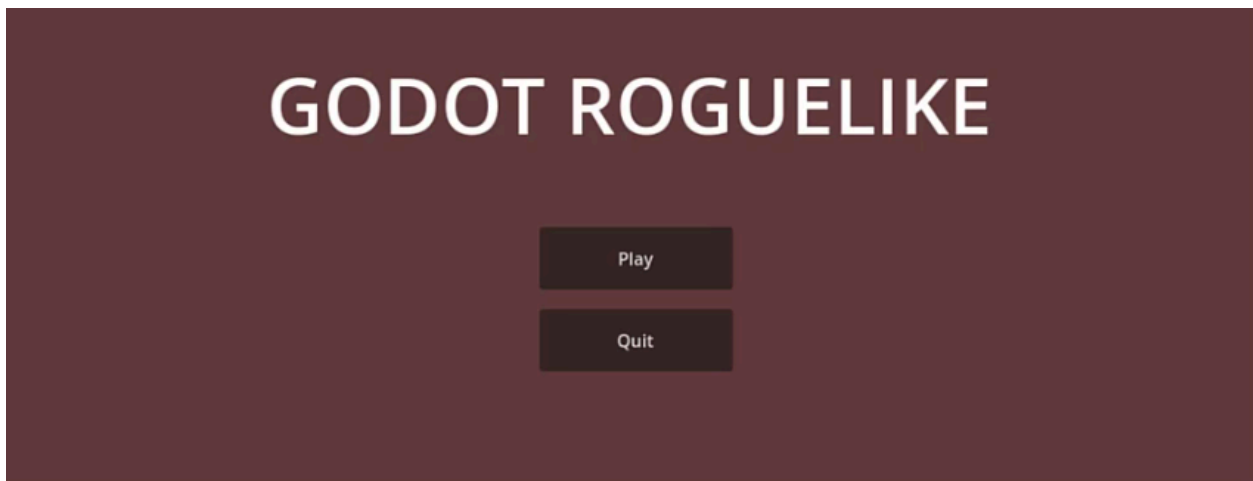
The game will include three primary UI elements. First, the **Mini-map** displays the room layout and player's position.



Second, the **Health System** displays the player's health as full or empty hearts.



Finally, the **Menu System** allows players to start or quit the game.



Now that we have a clear understanding of what we will be creating, let's get started with the project setup.

Project Setup

A clean, well-organized project saves time as the codebase grows. In this section, we will create a new project, set up a folder structure for scenes, scripts, and assets, and configure the basic settings our roguelike needs.

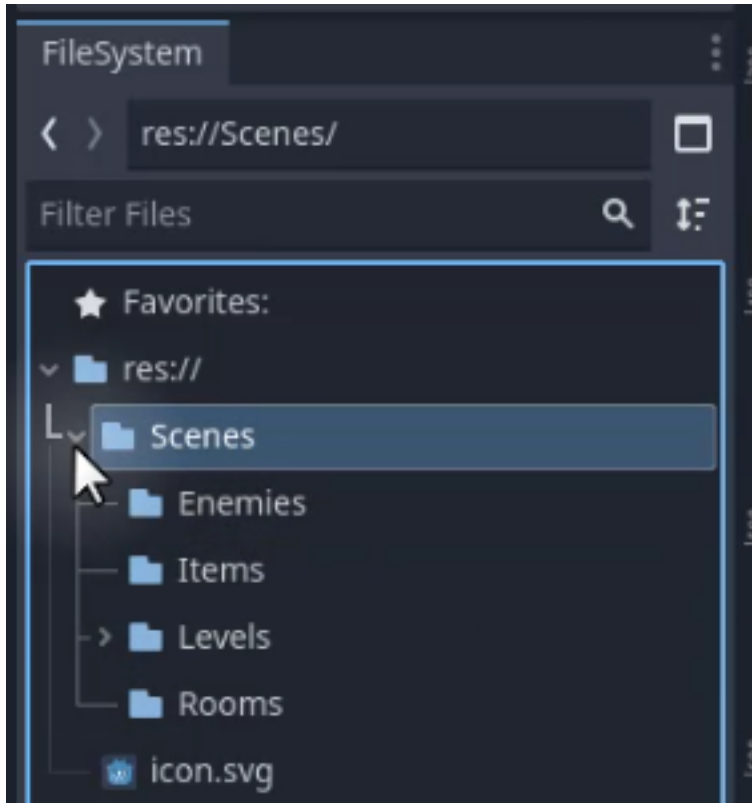
To begin, let's open Godot and create a brand new project. The project will start empty, providing a clean slate for our development.

Setting Up the File System Structure

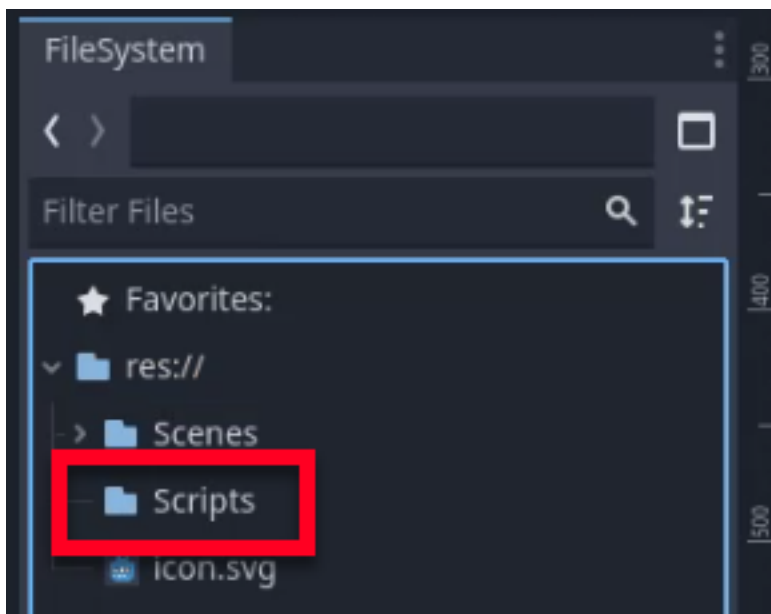
Organizing your project's file system structure is crucial for maintaining clarity and efficiency. We will create a specific folder hierarchy to keep our assets and scenes sorted.

Follow these steps to set up your folders:

1. Create a new folder named `Scenes`. This folder will store all the scenes you create.
2. Within the `Scenes` folder, create a subfolder named `Levels`. This will hold the different levels of your game.
3. Create additional folders within the `Scenes` folder:
 - `Items`: For storing item scenes.
 - `Rooms`: For storing different room types.
 - `Enemies`: For storing enemy scenes.

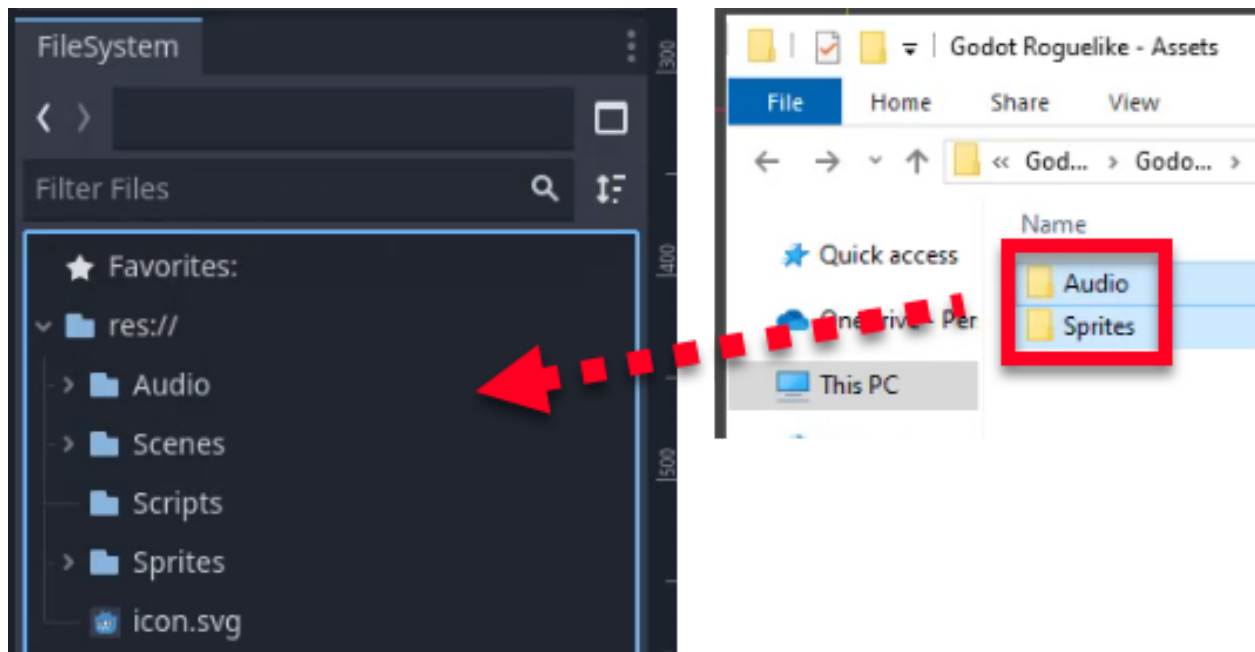


Next, create a Scripts folder in the root directory to store all your GDScript files.



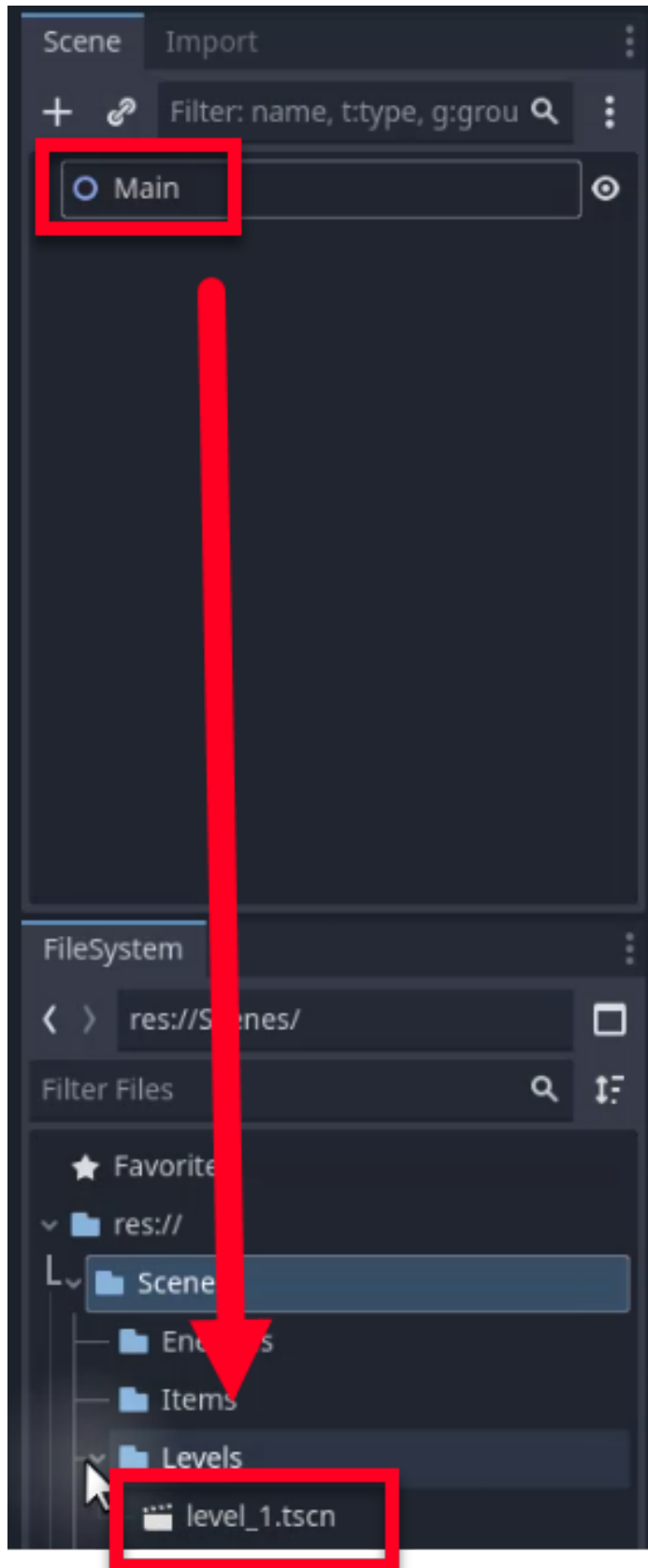
Finally, we need to import the course assets. Download and extract the course files (linked in the **Project Files** section above), which include Audio

and Sprites folders. Once extracted, drag them directly into your project's FileSystem dock.



Creating the First Scene

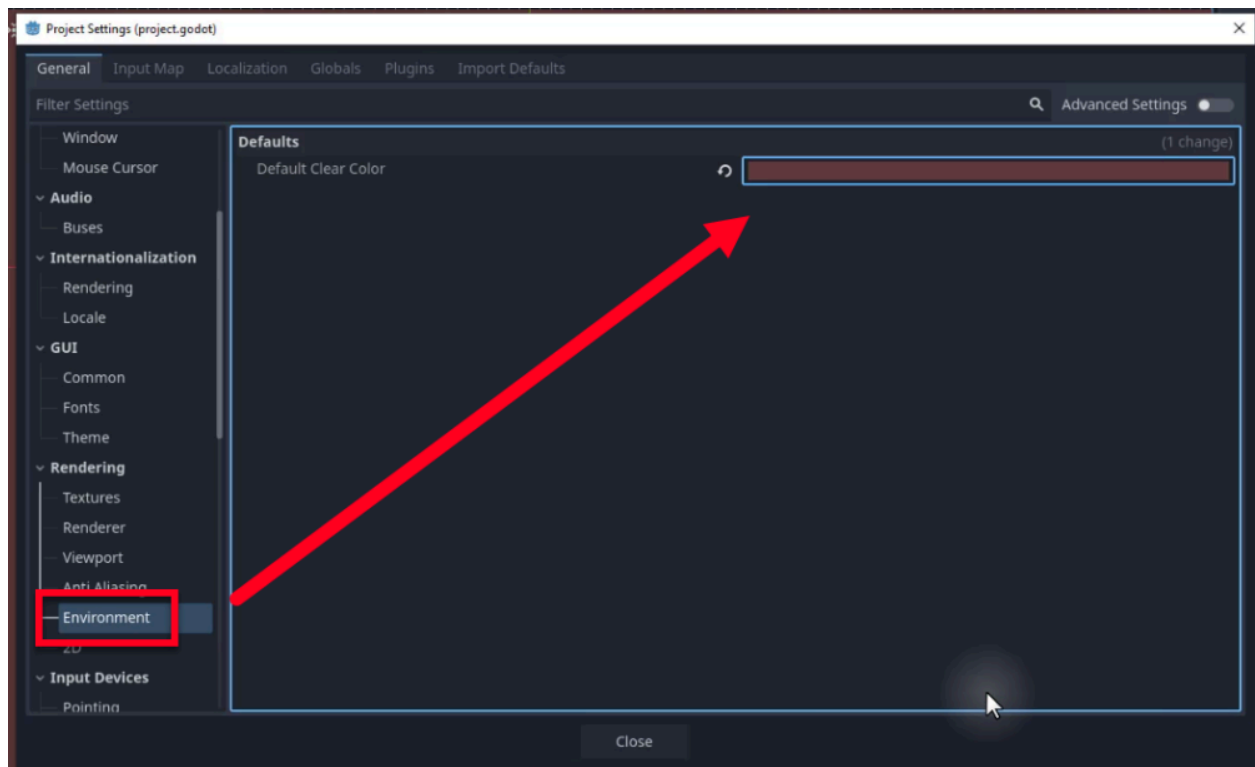
Let's create our first scene, which will serve as the main level where the player can walk around. In the Scene dock, select **2D Scene** and rename the root node to **Main**. Then, save this scene in the Scenes/Levels folder as `level_1.tscn`.



Changing the Background Color

For our game, we want a specific background color that complements our tileset. To set this, go to **Project -> Project Settings**, navigate to the **Rendering** section, and find the **Environment** option.

Then, open the **Default Clear Color** setting and modify it to your desired color. For this guide, we recommend a brownish-red color that matches the provided tileset (#613c3c).



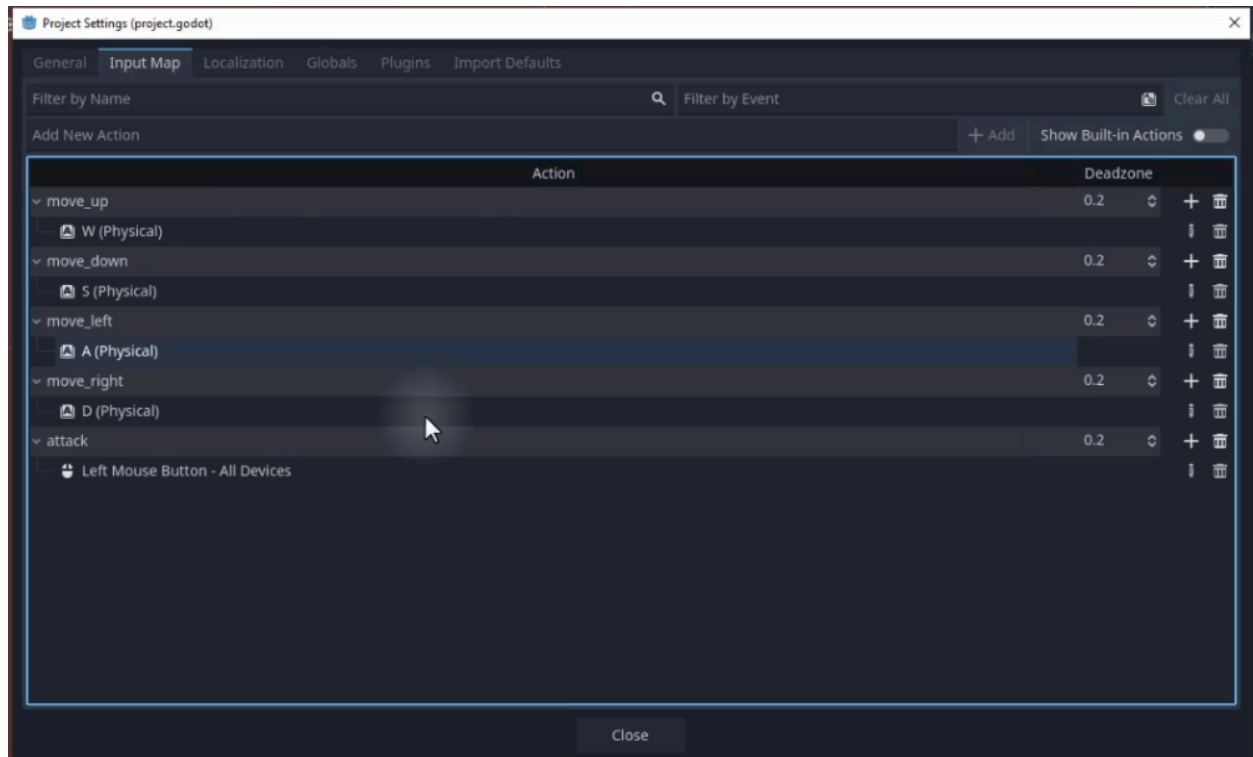
Setting Up Inputs

In order for our player to work, we need to define the inputs for movement and attacking. This allows us to map physical keys (like W, A, S, D) to abstract actions (like `move_up`) that our code will listen for.

From the **Project -> Project Settings** window, navigate to the **Input Map** tab. Then, add the following new actions and assign the corresponding keys or buttons:

- `move_up`: Assign the **W** key.
- `move_down`: Assign the **S** key.

- move_left: Assign the A key.
- move_right: Assign the D key.
- attack: Assign the Left Mouse Button.



Godot Insight: The Input Map system

Godot's Input Map creates a layer of abstraction between physical inputs and game actions. Instead of checking for a specific key like "W" directly in your code, you check for a named action like `move_up`. This means you can reassign keys, add gamepad support, or even let players remap controls without changing a single line of gameplay code.

The actions we defined here will be used throughout the project with functions like `Input.get_vector()` and `Input.is_action_pressed()`, which accept action names rather than key codes.

Learn more: [Input](https://docs.godotengine.org/en/4.6/classes/class_input.html)

(https://docs.godotengine.org/en/4.6/classes/class_input.html)

With these steps completed, our project is correctly configured and ready for development. In the next chapter, we will begin setting up our player character.

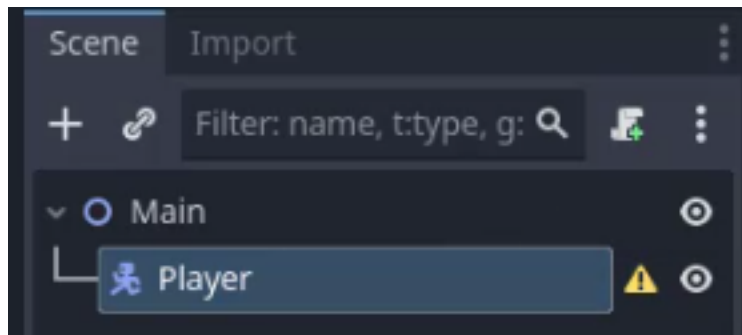
Creating the Player Character

In this chapter, we will establish the foundation of our game by creating the player character. We will set up the necessary node structure, configure the visuals and collision physics, and implement a script to handle movement and weapon aiming. By the end of this chapter, you will have a controllable character capable of navigating the game world and aiming a weapon.

Setting Up the Player Node

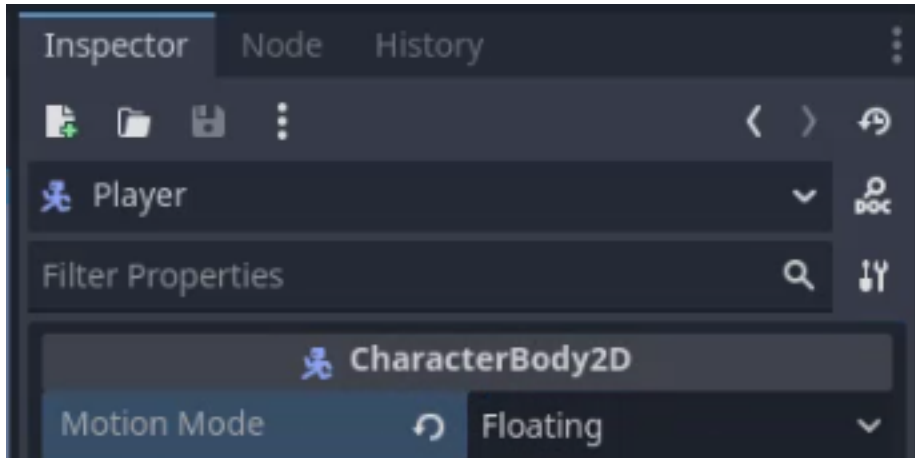
To begin, we need to create the root node for our player. In Godot, a `CharacterBody2D` is the ideal node type for characters that require physics-based movement and collision handling.

As such, create a new node of type `CharacterBody2D` in your main scene. Let's rename this node to `Player`.



Since our game is a top-down shooter, we need to adjust how the physics engine handles the character. By default, Godot might apply gravity or other platformer-specific behaviors.

In the Inspector, change the `Motion Mode` to `Floating`. This setting ensures that movement calculations are appropriate for a top-down perspective where gravity is not a factor.



Godot Insight: CharacterBody2D and motion modes

Godot provides three main physics body types for 2D, each suited to a different role.

CharacterBody2D is moved entirely by your code. It is the right choice for player characters and NPCs where you want full control over movement, including custom gravity, acceleration, and collision response.

StaticBody2D never moves on its own. It is used for walls, floors, and other immovable geometry.

RigidBody2D is driven by the physics engine. Use it for objects that should respond to gravity, collisions, and forces automatically, such as crates or rolling boulders.

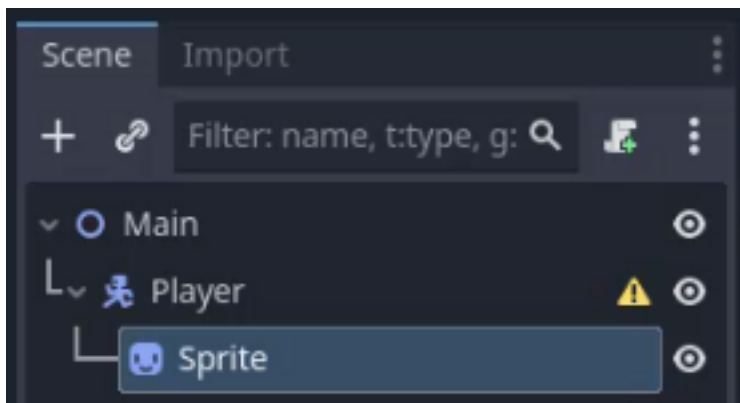
The **Floating** motion mode we selected tells the engine to skip gravity calculations entirely, which is exactly what a top-down game needs. In a platformer, you would use the default **Grounded** mode instead.

Learn more: [CharacterBody2D](https://docs.godotengine.org/en/4.6/classes/class_characterbody2d.html)
(https://docs.godotengine.org/en/4.6/classes/class_characterbody2d.html)

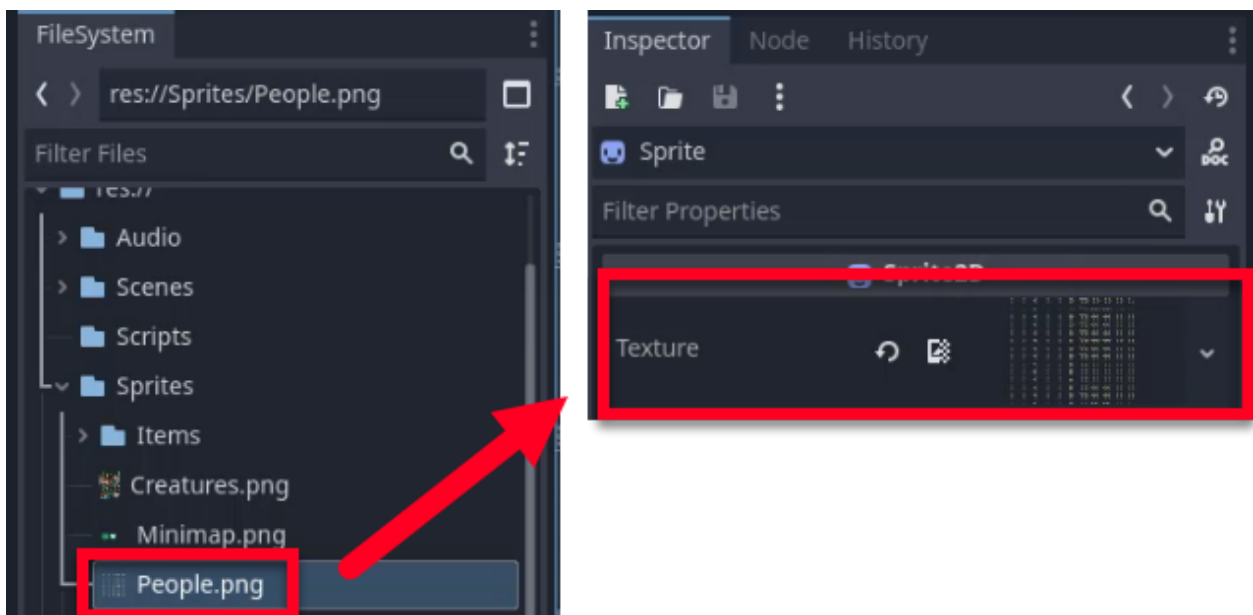
Adding Visuals

Next, the player needs a visible sprite so the user can see and control their character on screen. We will configure the sprite first, as the size of the collision shape often depends on the visual dimensions of the character.

Create a child node of type Sprite2D under the Player node and rename it to Sprite.

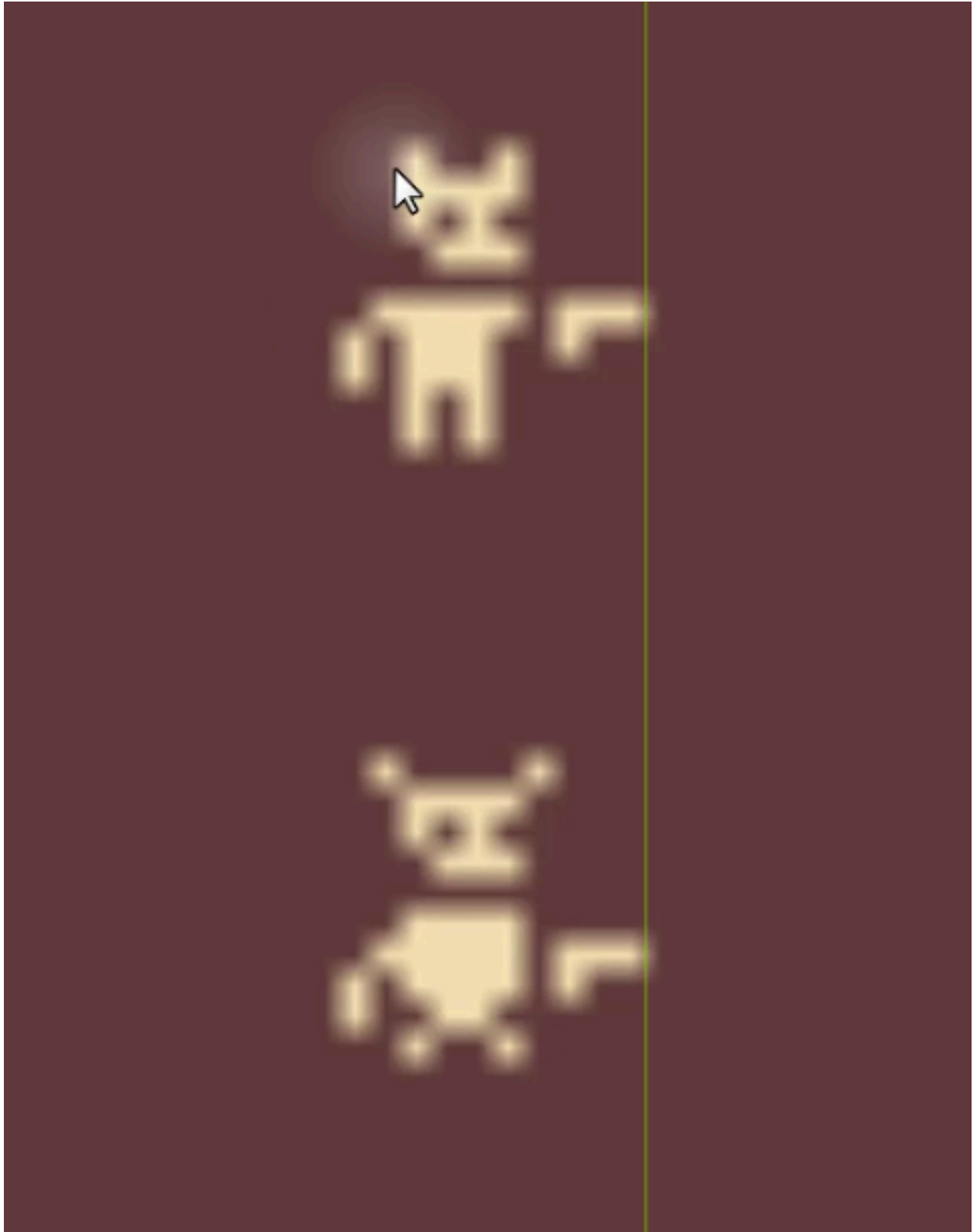


Now we need to assign a texture to this sprite. Navigate to your Sprites folder in the FileSystem dock, find the appropriate sprite sheet (e.g., People.png), and drag it into the **Texture** field of the Sprite node in the Inspector.



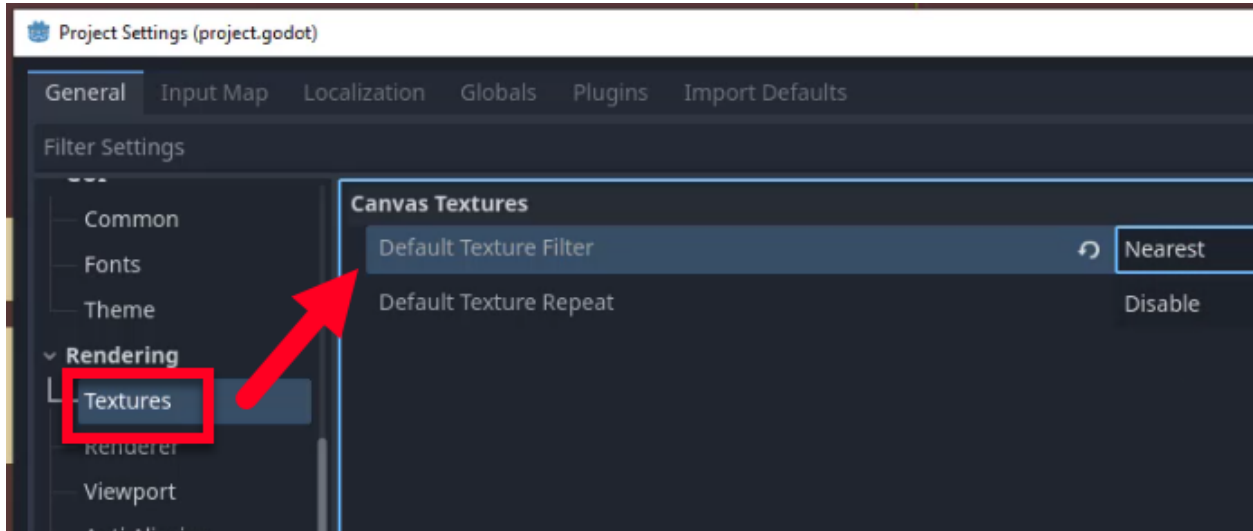
Adjusting Texture Settings

The first thing you might notice is that the sprite appears blurry. This occurs because Godot defaults to linear interpolation, which smooths out pixels. For pixel art games, we want crisp, sharp edges.



To fix this globally for the project:

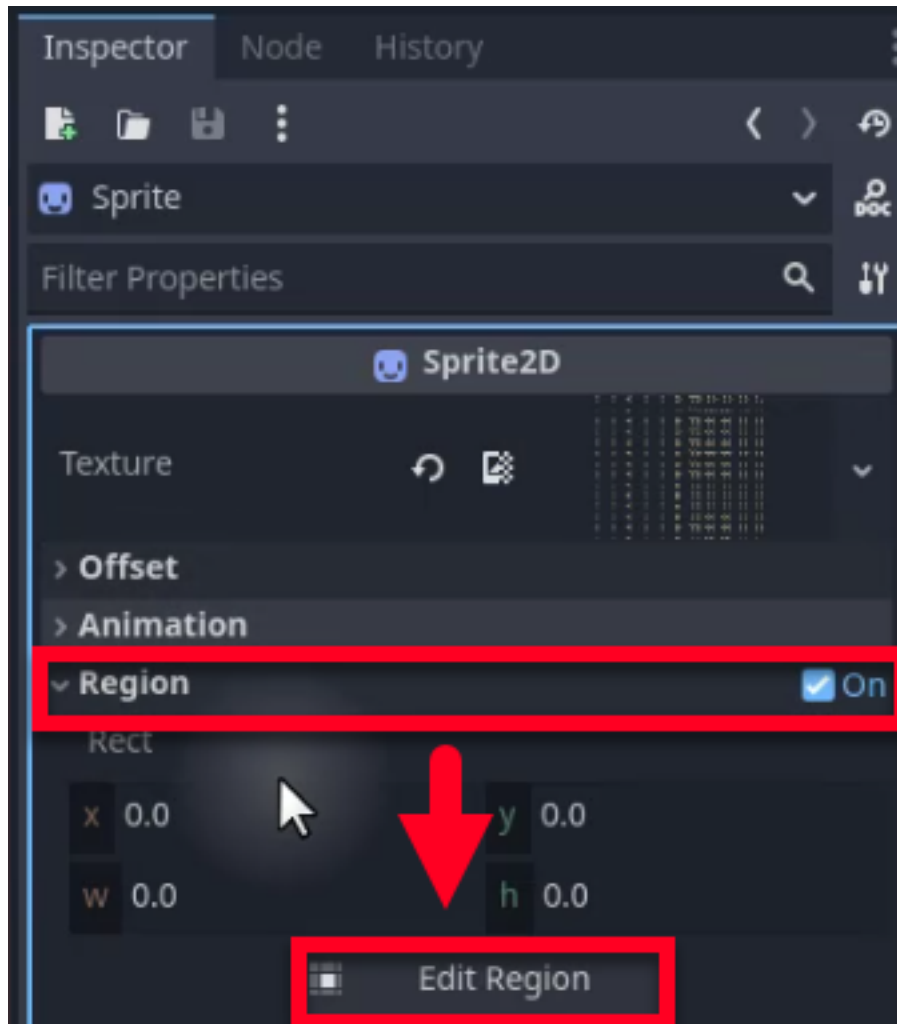
1. Go to **Project** -> **Project Settings**.
2. Navigate to **General** -> **Rendering** -> **Textures**.
3. Change the Default Texture Filter from Linear to Nearest.



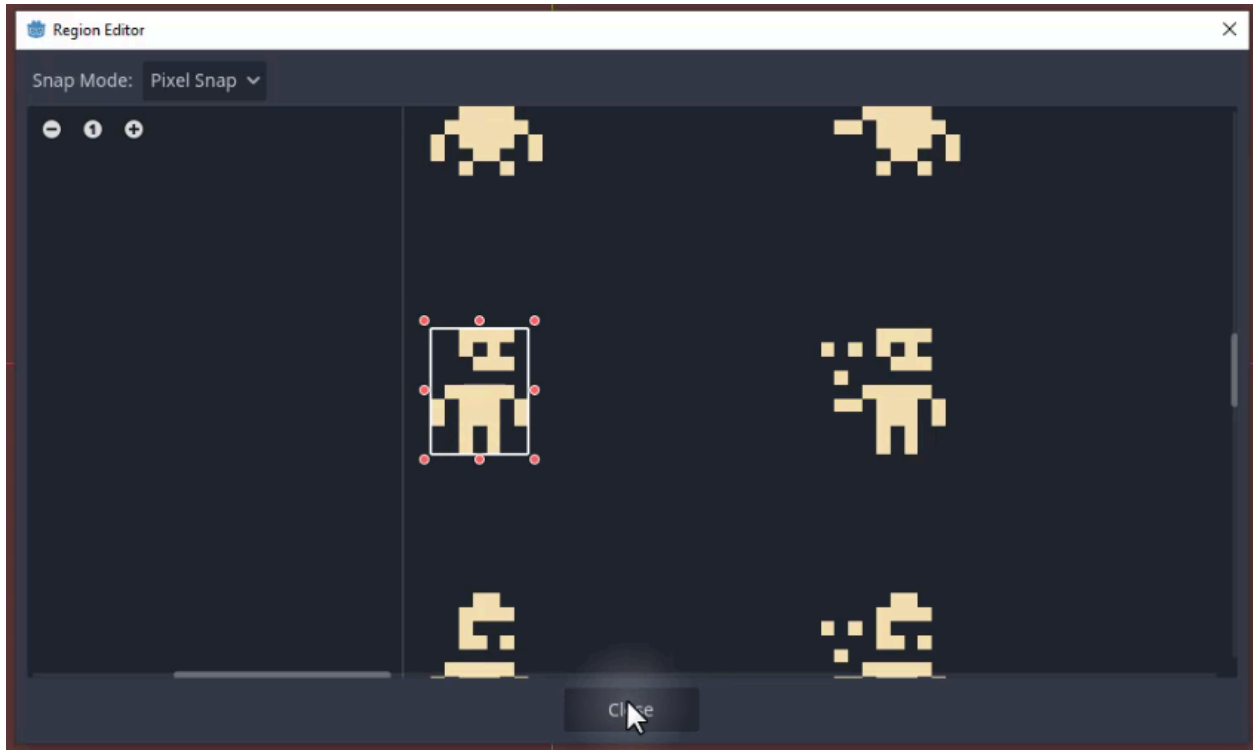
Now the pixel art should be crisp with hard edges as we'd expect for the art style.

Selecting a Sprite Region

Our texture is a sprite sheet containing multiple characters, so at the moment we will see all of them at once. We need to isolate just the player character. To do so, first select the Sprite node. In the Inspector, expand the **Region** section and enable the Enabled property. Then, click the Edit Region button that appears.



This opens the Region Editor at the bottom of the screen. Here, you can select the specific part of the sprite sheet you want to use. To make this easier, change the Snap Mode to Pixel Snap, then drag a selection box around the character you wish to use.

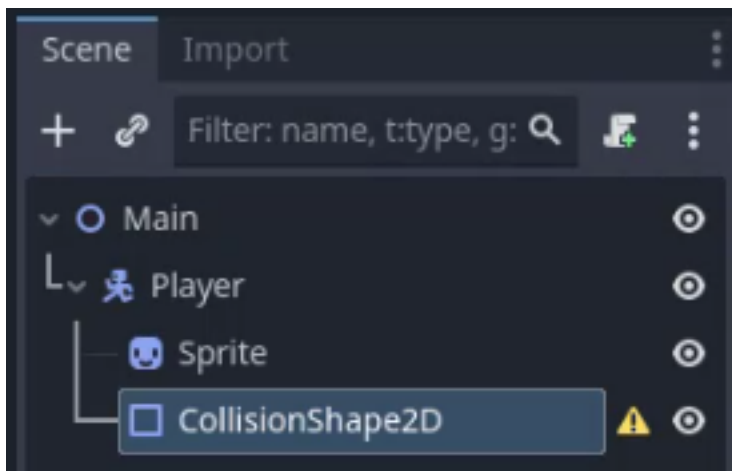


After you make your selection, completely close the window. You'll see we now only have a single sprite visible for our character.

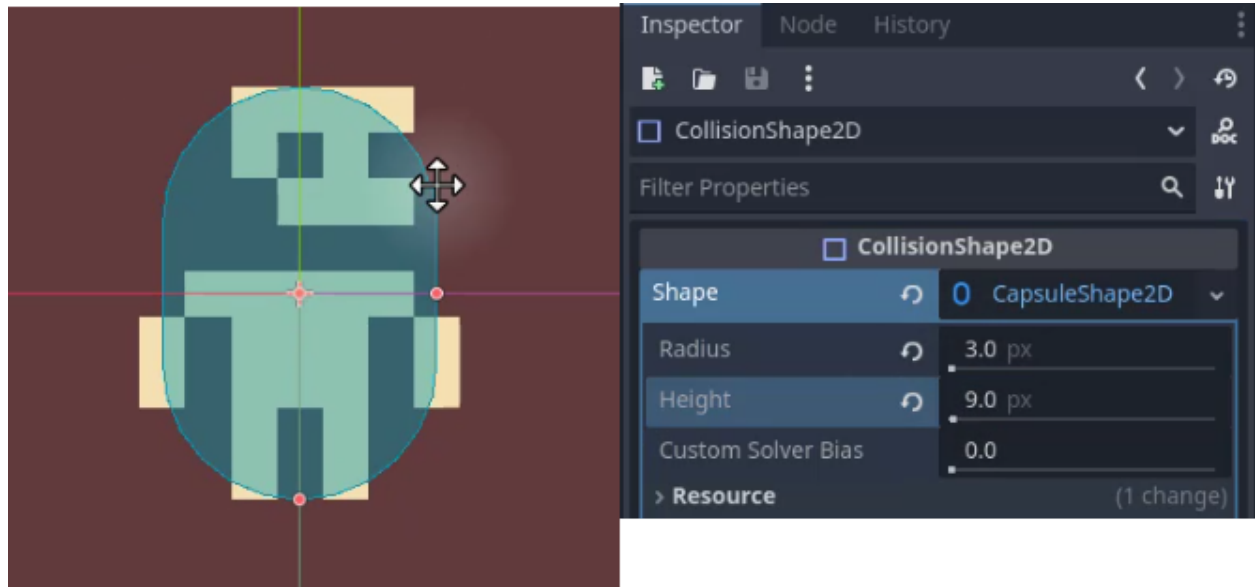
Adding Collision

With our visual representation in place, we can now define the physical shape of the player. This shape determines how the player interacts with walls, enemies, and other objects.

Add a child node of type `CollisionShape2D` under the `Player` node.



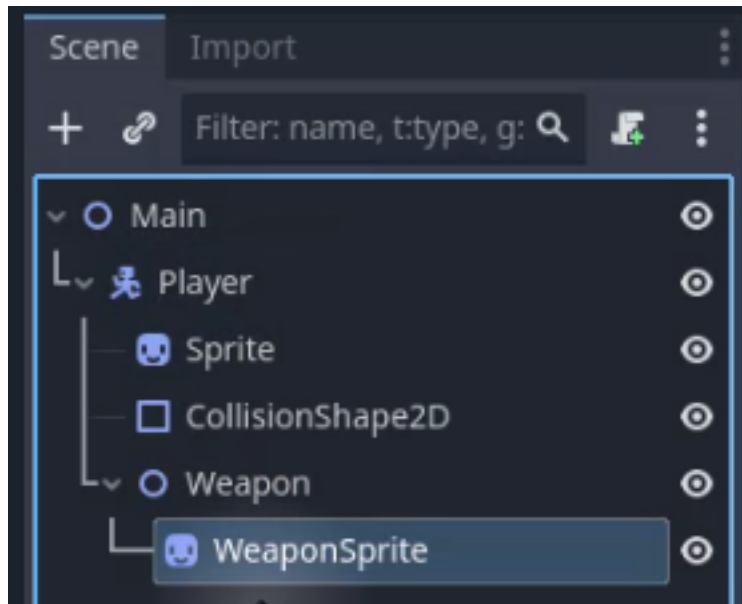
In the Inspector, set the **Shape** property to `CapsuleShape2D`, and then adjust the **Radius** and **Height** to fit your sprite. For the sprite used in this example, a radius of 3px and a height of 9px works well.



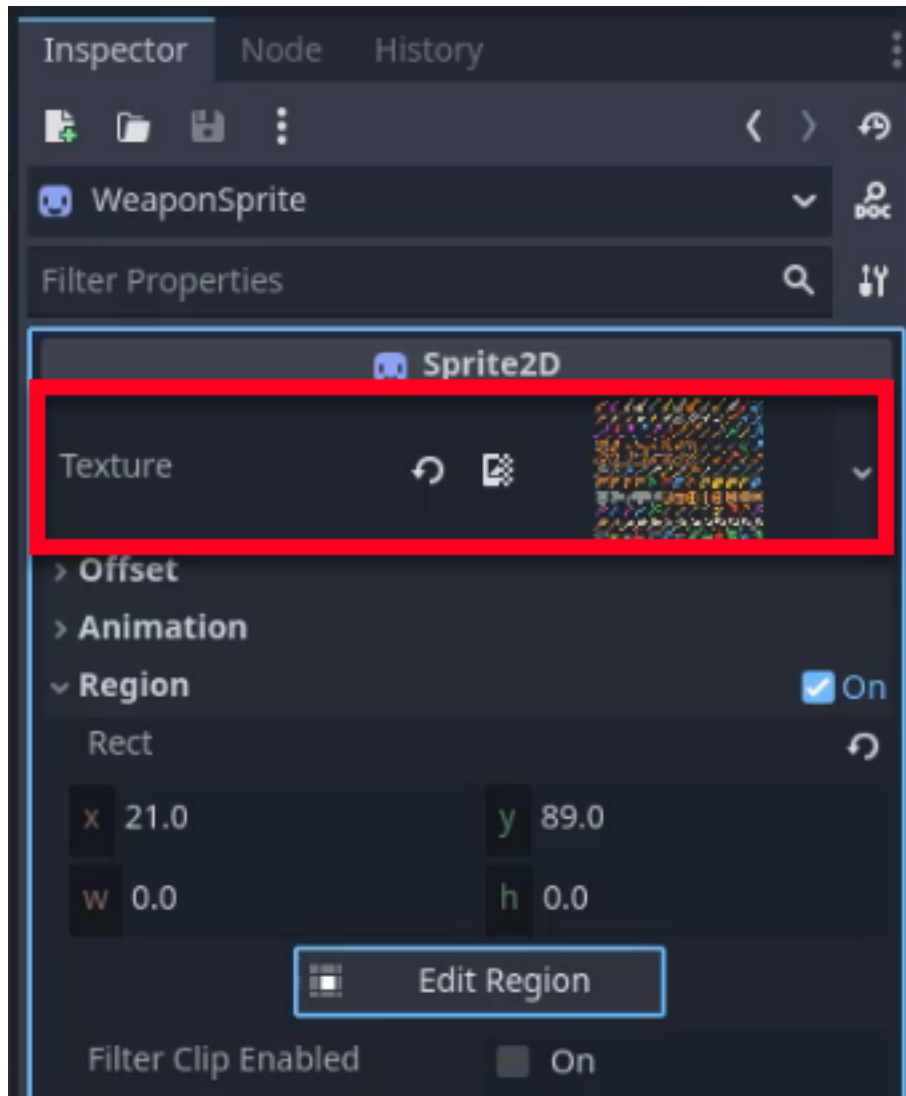
Setting Up the Weapon

Beyond its base body, our player needs a weapon that can rotate independently to aim at the mouse cursor. This is what will indicate to the player where they are shooting.

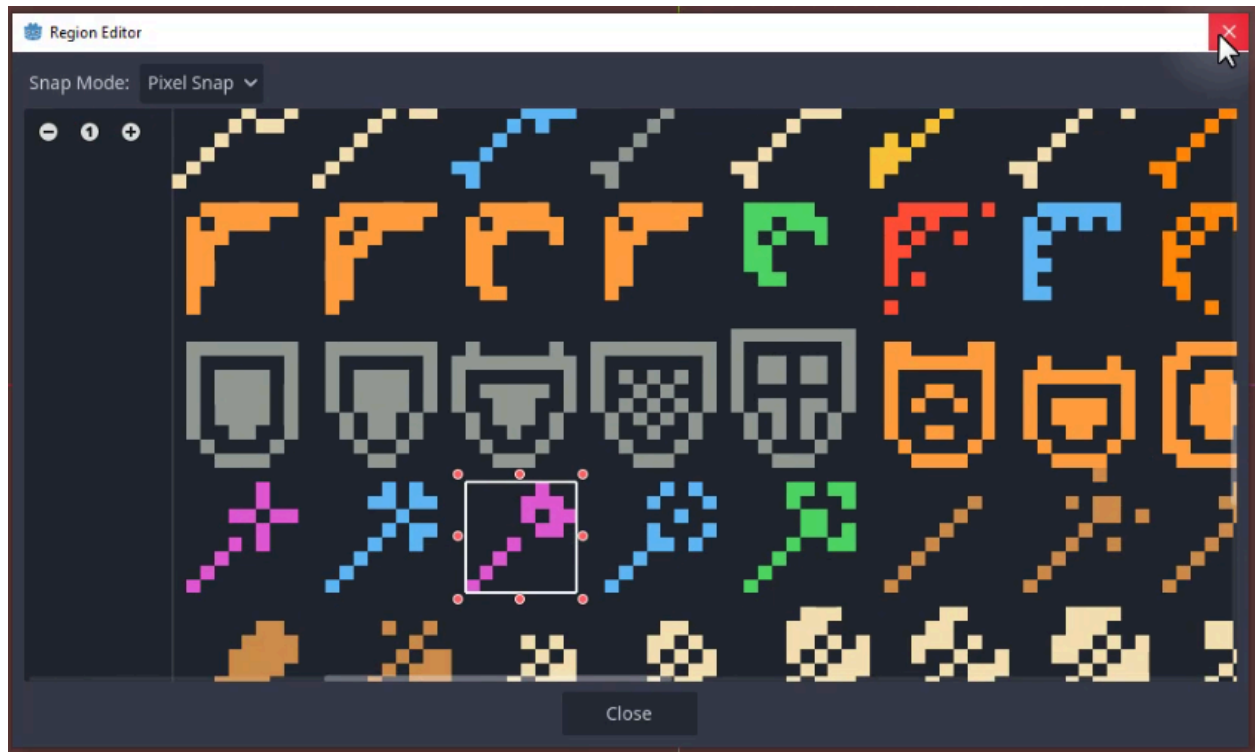
We will create a dedicated container for the weapon to handle this rotation. First, create a new node of type `Node2D` under the `Player` node and rename it to `Weapon`. Then, add a child node of type `Sprite2D` under the `Weapon` node and rename it to `WeaponSprite`.



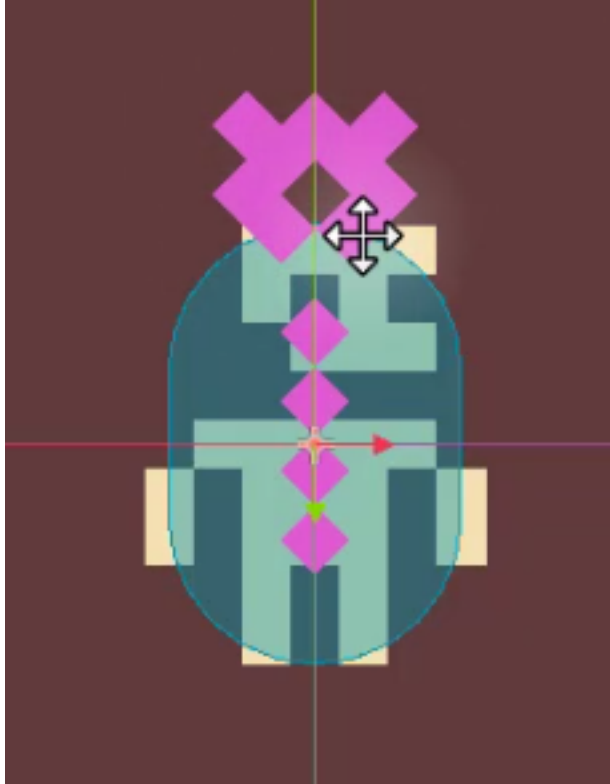
Assign the weapon sprite sheet (e.g., Weapons.png) to the **Texture** field of the WeaponSprite.



Just like with the player sprite, enable the **Region** property and use the Region Editor to select a specific weapon, such as a wand.



Once the sprite is selected, rotate the `WeaponSprite` so that it points upwards. You can use the rotate tool in the viewport, holding `Control` (or `Command` on macOS) to snap to precise angles (e.g., 0, 90, 180 degrees). You may also need to adjust the position so it aligns with the player's hand.

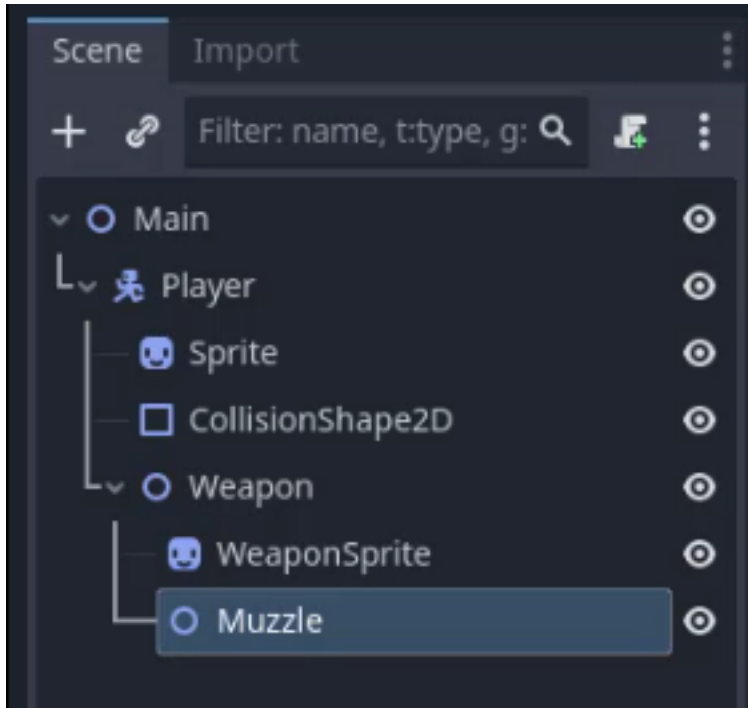


This is partly up to what you think looks good aesthetically, so feel free to adjust as we develop our game.

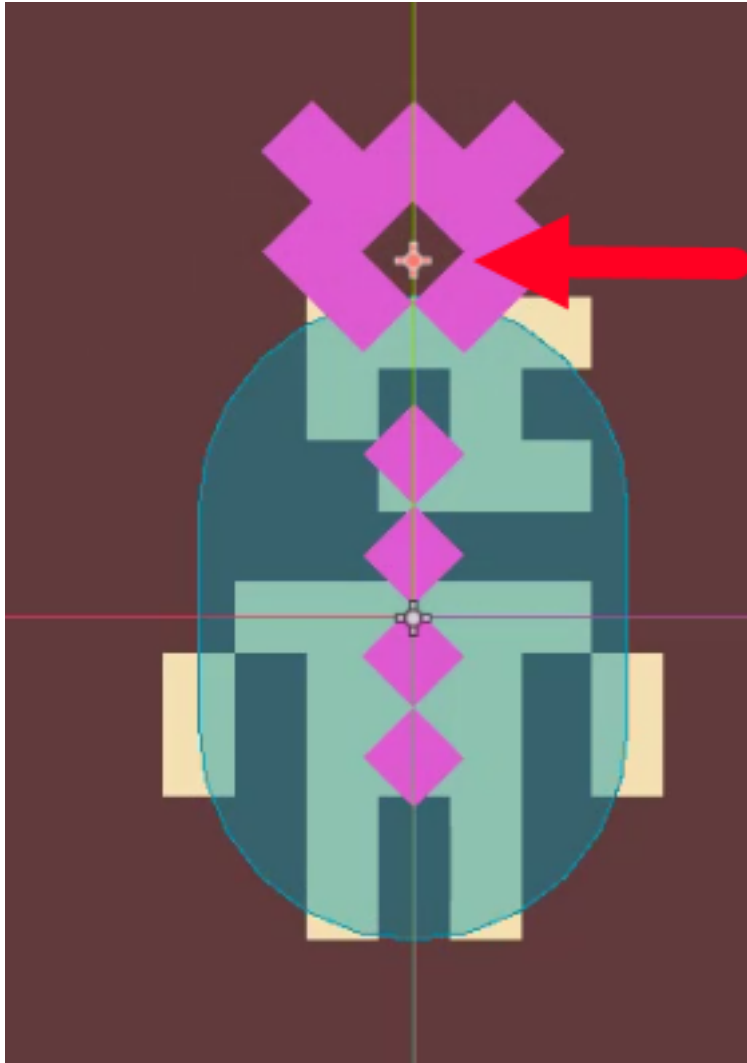
Adding a Muzzle Point

Though we can see our weapon, it isn't quite done yet. We need a reference point to tell the game where projectiles should spawn when the player shoots.

Let's add a child node of type `Node2D` under the `Weapon` node and rename it to `Muzzle`.



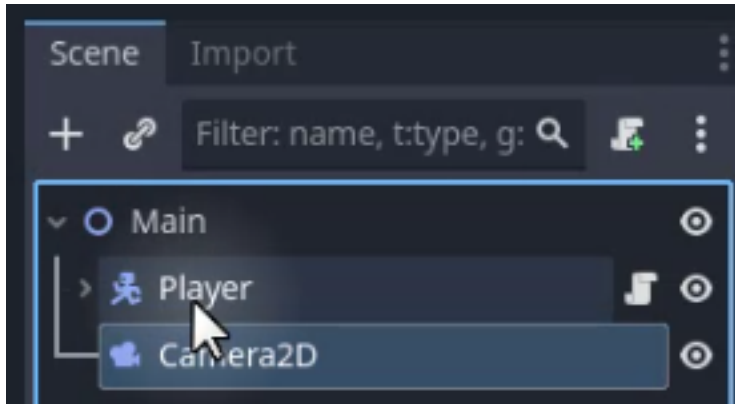
Then, position the `Muzzle` node at the tip of the weapon sprite. This ensures projectiles appear to come out of the weapon itself.



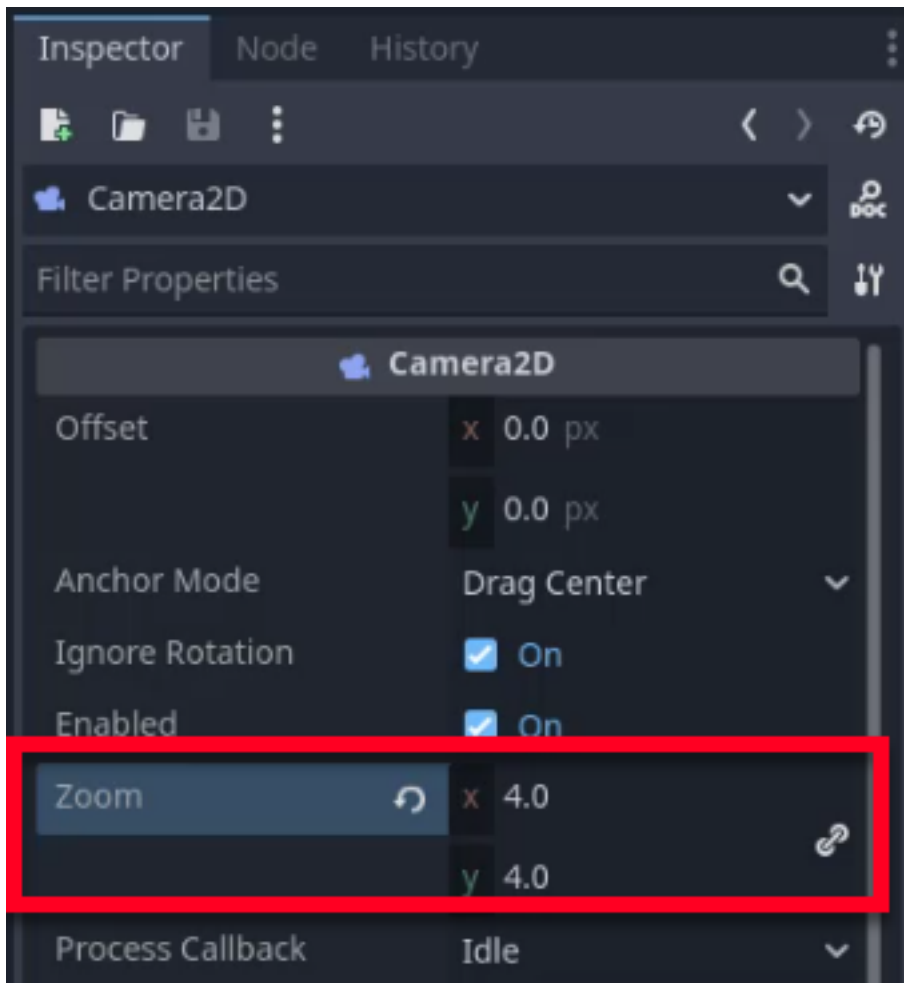
Setting Up the Camera

To properly view our player in the game window, we need a camera. For now, we will add a camera to the main scene to verify our setup.

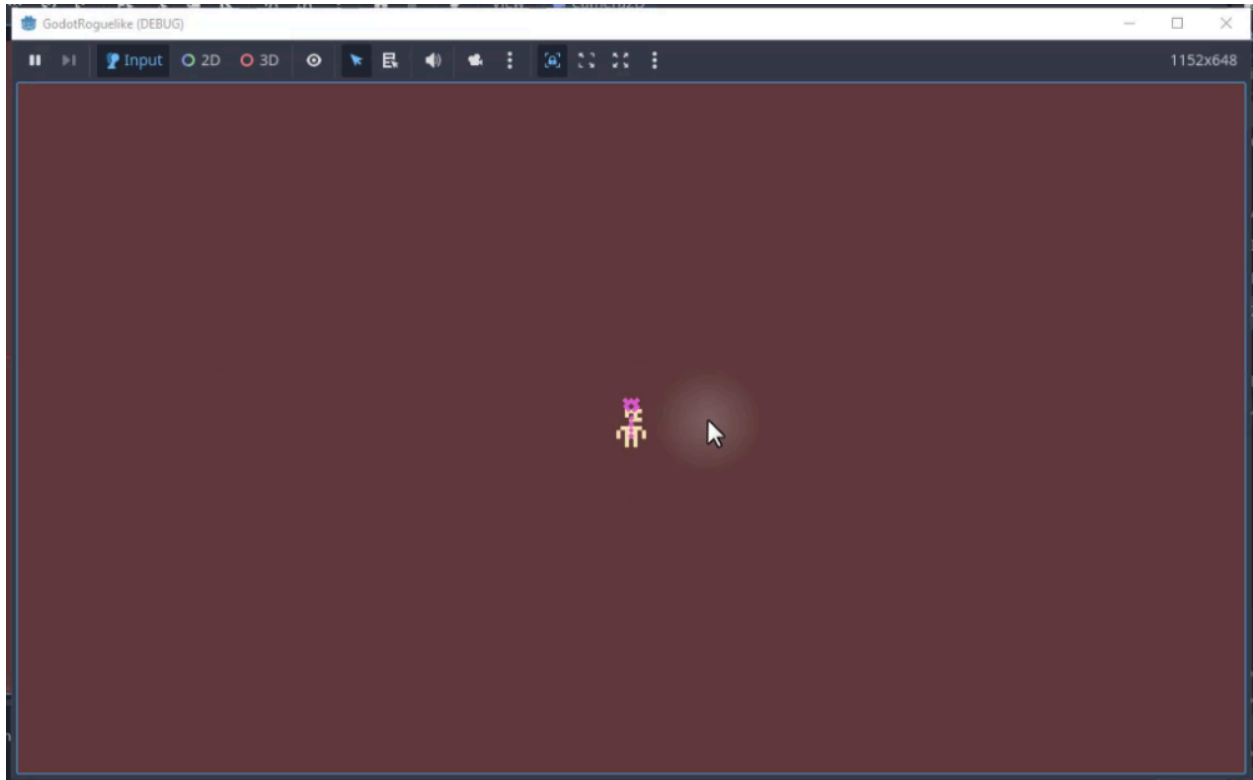
Select the `Main` node (the root of your scene) and add a child node of type `Camera2D`.



Since we are working with small pixel art, we need to zoom in. In the Inspector for the Camera2D, set the **Zoom** level to 4 for both X and Y. This means every pixel in our art will take up 4 pixels on the screen.



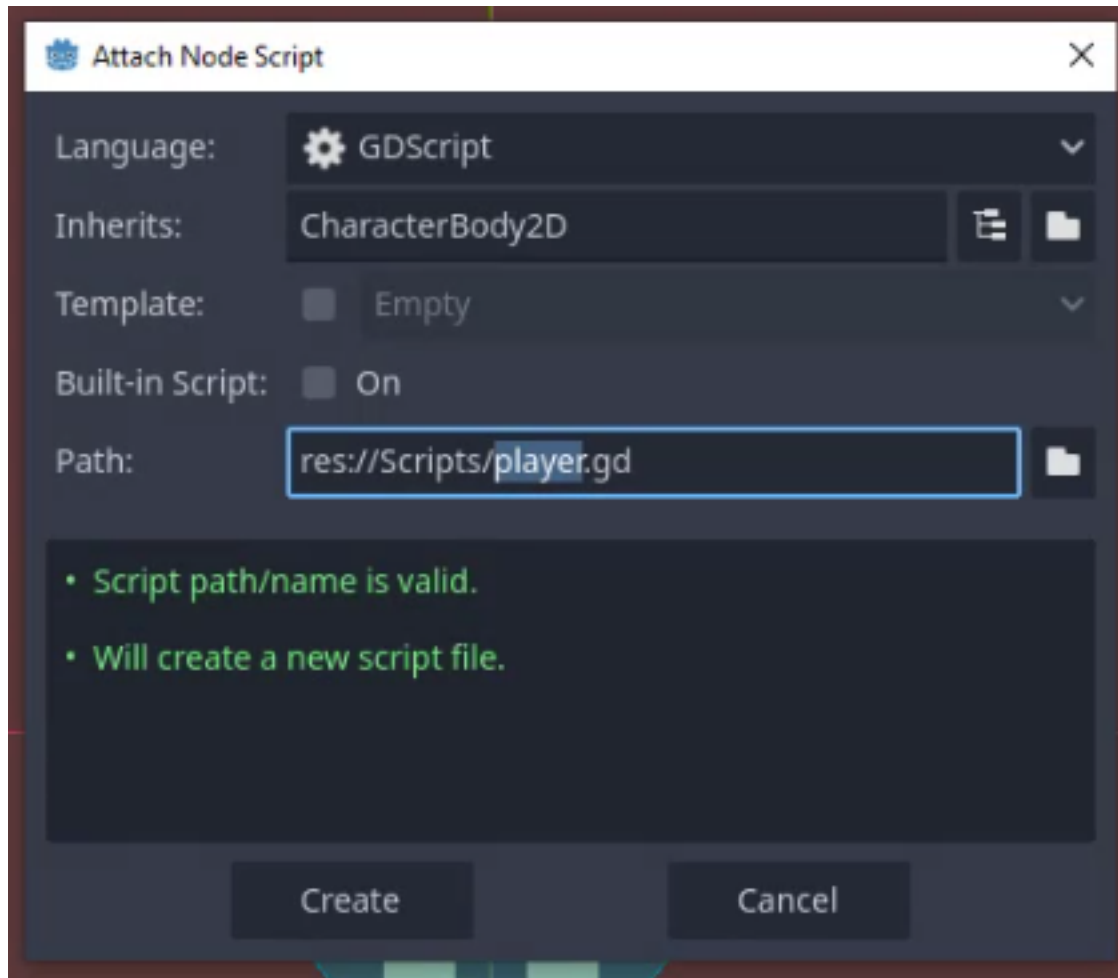
If you run the scene now, you should see your player character rendered crisply in the center of the screen.



Scripting the Player

Now that our nodes are set up, we need to give the player the ability to move and aim. We will create a script to handle input and update the player's position and rotation.

Select the Player node and attach a new script named `player.gd`, saving it in the Scripts folder.



We can then open the script to start working on it.

Defining Variables

First, we will define the variables needed to control the player and reference our nodes.

- `move_speed`: Controls how fast the player moves. We export this so it can be tuned in the Inspector.
- `sprite`: A reference to the player's sprite, used for flipping the character.
- `weapon_origin`: A reference to the weapon container, used for rotation.
- `muzzle`: A reference to the projectile spawn point.

Add the following code to your script:

```
extends CharacterBody2D
```

```
@export var move_speed : float = 50
```

```
@onready var sprite : Sprite2D = $Sprite
```

```
@onready var weapon_origin : Node2D = $Weapon
```

```
@onready var muzzle : Node2D = $Weapon/Muzzle
```

Implementing Movement

We will handle movement in the `_physics_process` function. This built-in function runs at a fixed time step, making it ideal for physics calculations.

1. We use `Input.get_vector` to capture the WASD keys we defined in the Project Settings.
2. We calculate the velocity by multiplying the input vector by our speed.
3. We call `move_and_slide()` to apply the movement and handle collisions automatically.

Add this function to your script:

```
# Handle WASD movement each physics frame  
func _physics_process(_delta):  
    var move_input : Vector2 = Input.get_vector("move_left",  
"move_right", "move_up", "move_down")  
    velocity = move_input * move_speed  
    move_and_slide()
```

Godot Insight: `_physics_process` vs `_process`

Godot offers two main update loops. `_physics_process(delta)` runs at a fixed rate (60 times per second by default), which makes it ideal for movement, collision detection, and anything that interacts with the physics engine. `_process(delta)` runs once per rendered frame, so its rate varies with the player's hardware.

As a rule of thumb, use `_physics_process` for movement and physics, and `_process` for visual updates, input polling, and anything that does not need to be perfectly synchronized with the physics engine. In this project, we use

`_physics_process` for the player's movement and `_process` for aiming, since aiming is purely visual.

Learn more: [Node](https://docs.godotengine.org/en/4.6/classes/class_node.html)
(https://docs.godotengine.org/en/4.6/classes/class_node.html)

Implementing Aiming

For aiming, we want the weapon to rotate toward the mouse cursor. We will handle this in the `_process` function, which runs every frame. To implement this, we:

1. Get the global mouse position.
2. Calculate the direction vector from the player to the mouse.
3. Calculate the angle of that direction and apply it to the `weapon_origin`.
4. We add 90 degrees to the rotation because our weapon sprite points upwards (which is -90 degrees in Godot's coordinate system relative to the standard right-facing 0 degrees).

Add this function to your script:

```
# Aim weapon toward the mouse and handle shooting input
func _process(delta):
    var mouse_pos : Vector2 = get_global_mouse_position()
    var mouse_dir : Vector2 = (mouse_pos -
global_position).normalized()
    # +90 because the sprite points up, but Godot's angle() 0 points right
    weapon_origin.rotation_degrees = rad_to_deg(mouse_dir.angle()) +
90
```

Flipping the Sprite

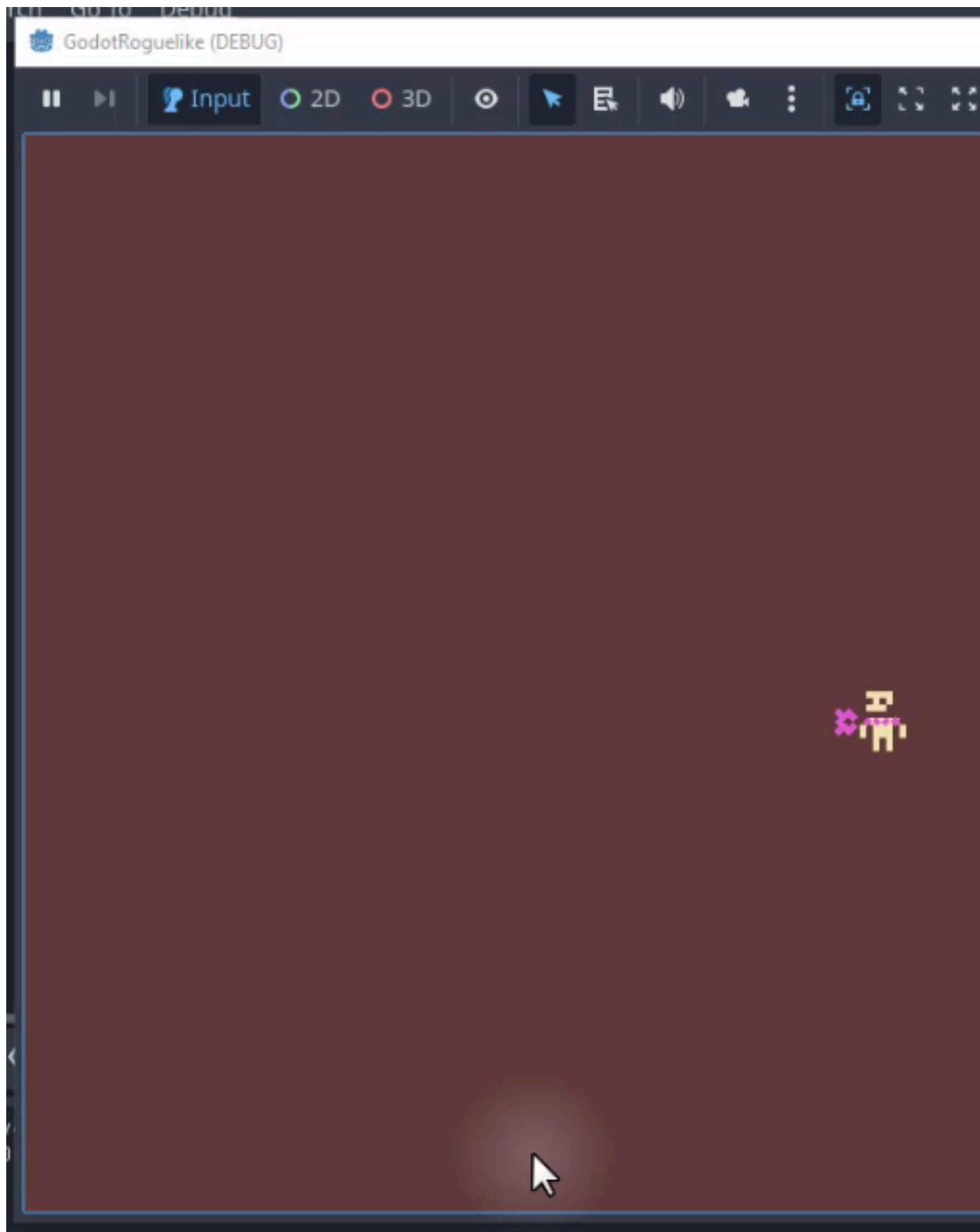
To make the character look more dynamic, we can flip the sprite horizontally so the player faces the direction they are aiming.

Add this line to the end of your `_process` function:

```
sprite.flip_h = mouse_dir.x < 0
```

If the mouse is to the left of the player (`mouse_dir.x < 0`), the sprite flips. Otherwise, it faces right.

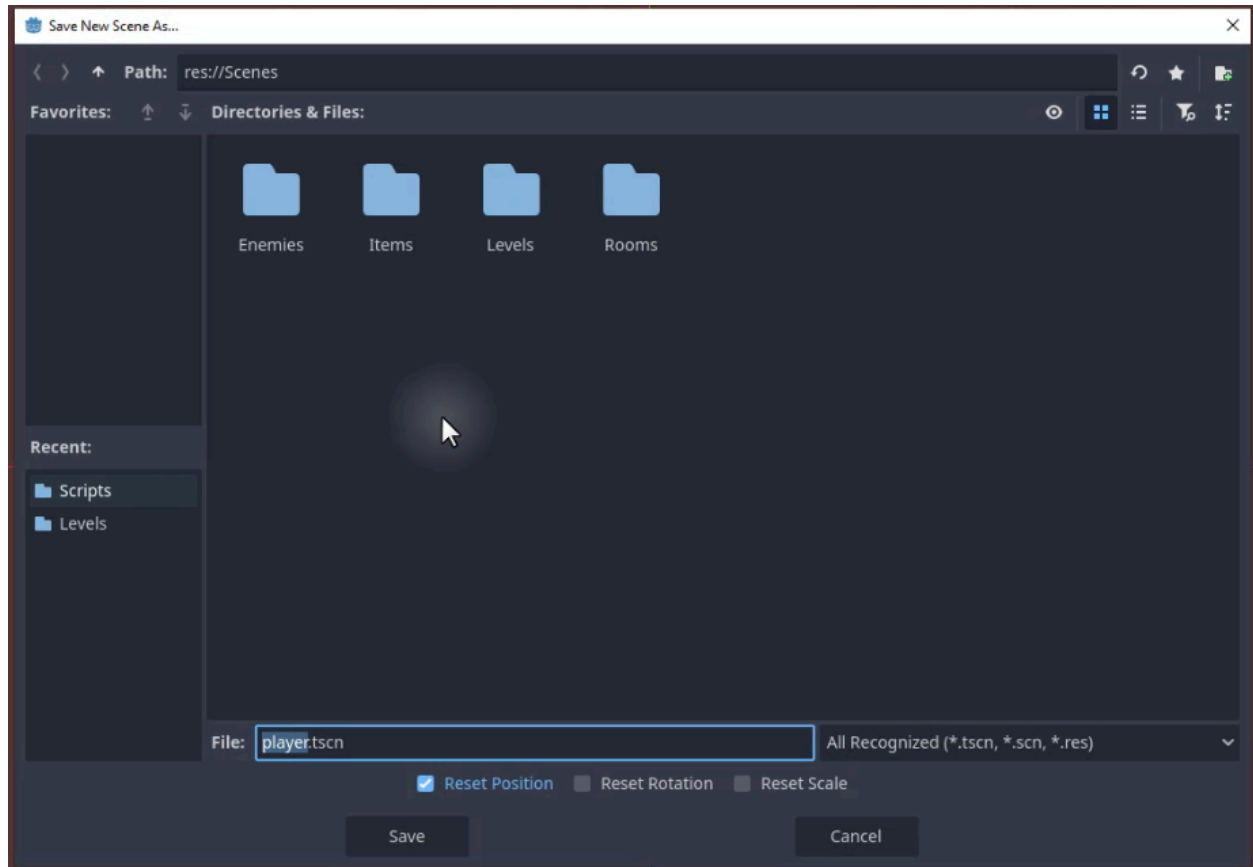
With these three implementations, your player should now be able to move, aim, and face the cursor.



Saving the Player Scene

Before we move on, we should save the player as its own scene. This allows us to instantiate the player in different levels or easily edit the character without affecting the main level file.

In the Scene panel, right-click the Player node and select **Save Branch as Scene**. Then, save the file as `player.tscn` in the Scenes folder.



You have now successfully created a fully functional player character that can move around an environment. In the next chapter, we will begin working on the procedural generation systems to create our dungeon rooms.

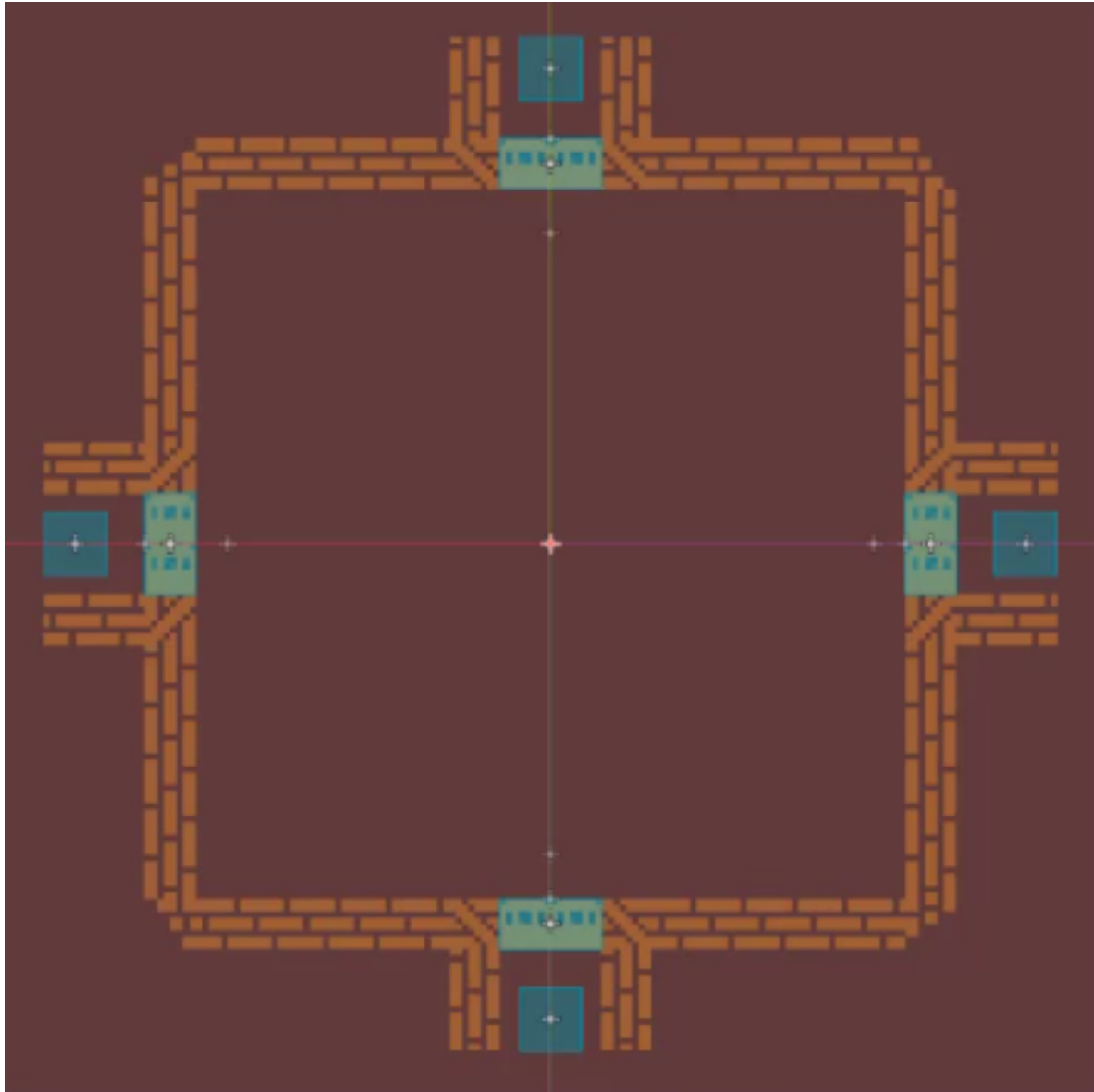
Designing Room Templates

In this chapter, we will create our base room template. This template will serve as the foundation for all the rooms in our game. Each room will be a square, connected to others via tunnels and doors. The entrances will cover all four cardinal directions, and will be set up to be open and closed dynamically. By the end, we'll have everything we need to implement the procedural generation aspects of our map generation.

Understanding the Room Structure

Before we dive into the creation process, let's understand the basic structure of our room. The room is designed as a square with four sides. Each side can have either a door or a barrier:

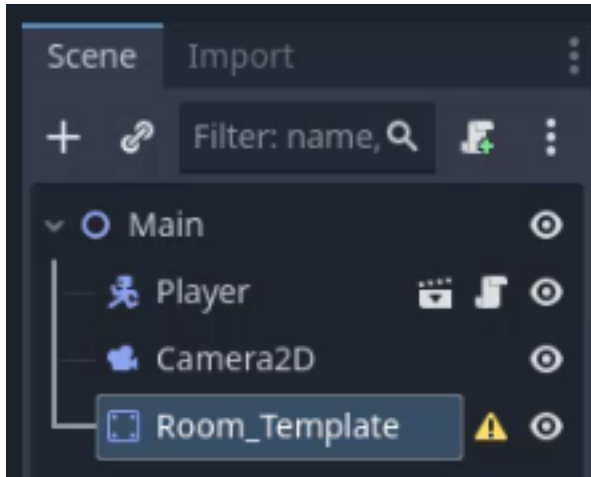
- A **barrier** is a flat wall used when there is no adjacent room in that direction.
- A **door** is an opening that the player can pass through to enter a connected room. Doors can be opened or closed, depending on the presence of enemies in the room.



Setting Up the Room Node

We will start by creating the root node for our room. Since the room consists primarily of walls and static obstacles, a `StaticBody2D` is the appropriate node type.

As such, create a new `StaticBody2D` node in your scene and rename it to `Room_Template`. This node will serve as the container for all room components.



Godot Insight: Why StaticBody2D for rooms

We chose StaticBody2D as the root of our room template because the room itself never moves. A StaticBody2D participates in the physics system (so characters collide with its walls) but costs almost nothing in terms of performance because the engine knows it will never change position. If we needed the room to move (for example, a sliding platform), we would use AnimatableBody2D instead.

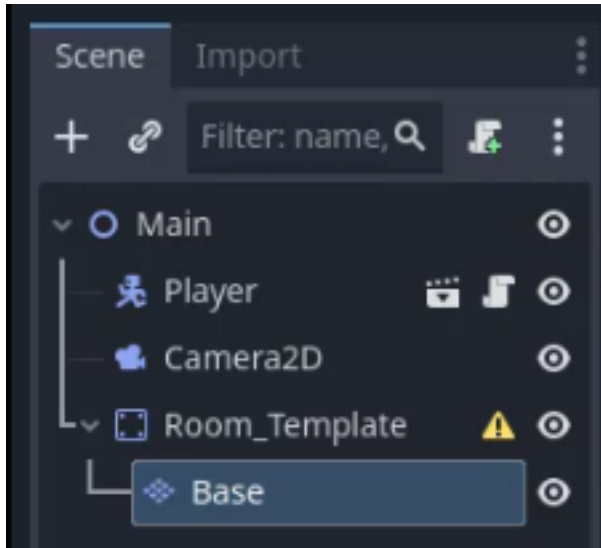
Learn more: [StaticBody2D](https://docs.godotengine.org/en/4.6/classes/class_staticbody2d.html)

(https://docs.godotengine.org/en/4.6/classes/class_staticbody2d.html)

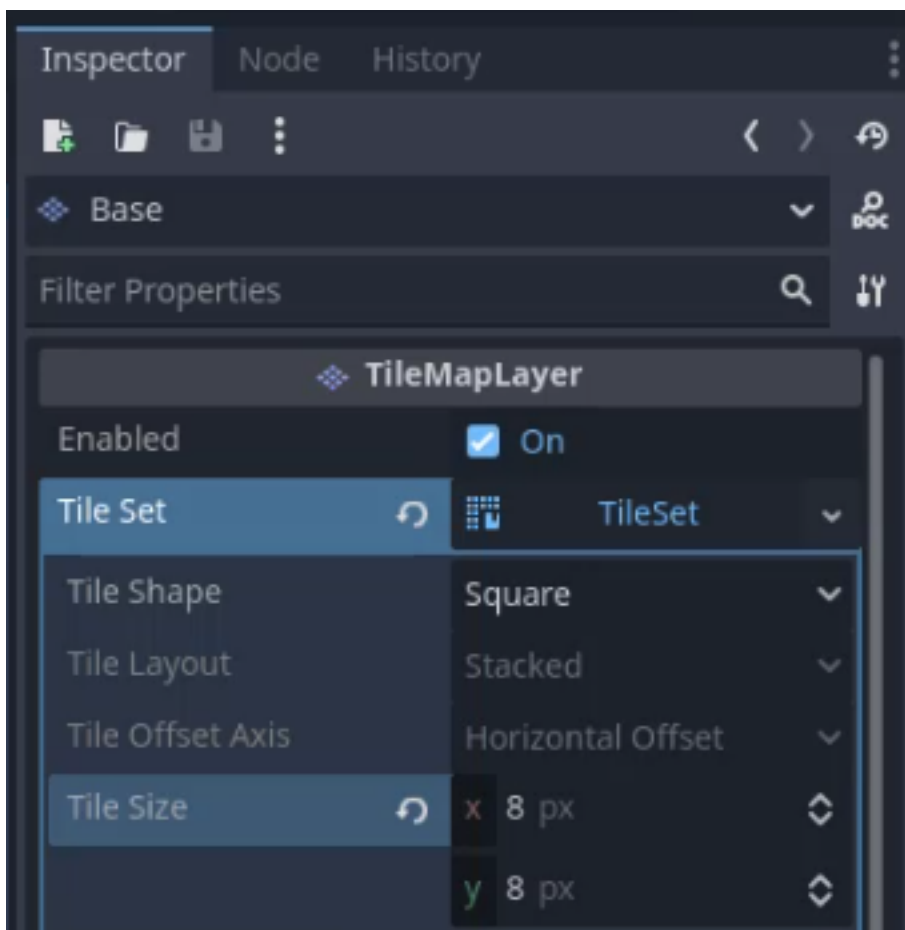
Adding the TileMapLayer

Next, we need to set up the visual layer for our room using Godot's TileMap system.

Create a new TileMapLayer node as a child of Room_Template and name it Base.

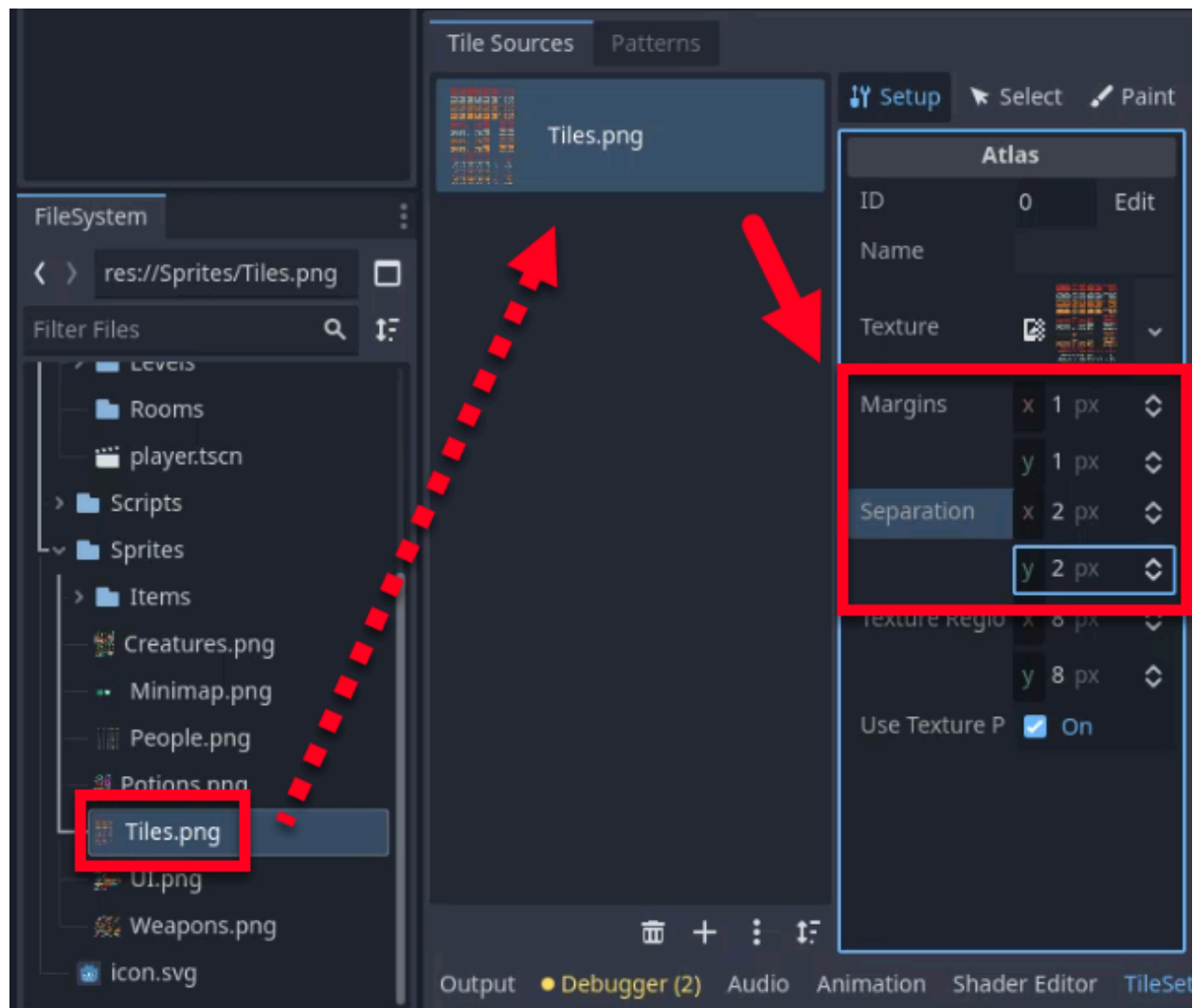


With the Base node selected, go to the Inspector to configure a new **TileSet** resource, which dictates what tiles are available to use. For this resource, set the **Tile Size** to 8x8 pixels to match the pixel art assets we'll use.



Now we need to import our tileset image itself. Drag your tileset image (e.g., Tiles.png) from the FileSystem into the **Tile Set** panel at the bottom of the editor.

To ensure the tiles are properly spaced, adjust the **Margins** and **Separation** settings. For the provided assets, set the **Margins** to 1px (X and Y) and the **Separation** to 2px (X and Y).



This will ensure the tiles are sliced correctly in their grid so that we can paint them onto our level.

Godot Insight: TileMapLayer and tile-based design

`TileMapLayer` is Godot's node for painting grid-based levels. Each layer holds a reference to a `TileSet` resource, which defines the available tiles, their sizes, and any physics or navigation data they carry. By separating tile data (the `TileSet`) from tile placement (the `TileMapLayer`), Godot lets you reuse the same tileset across many different scenes and layers.

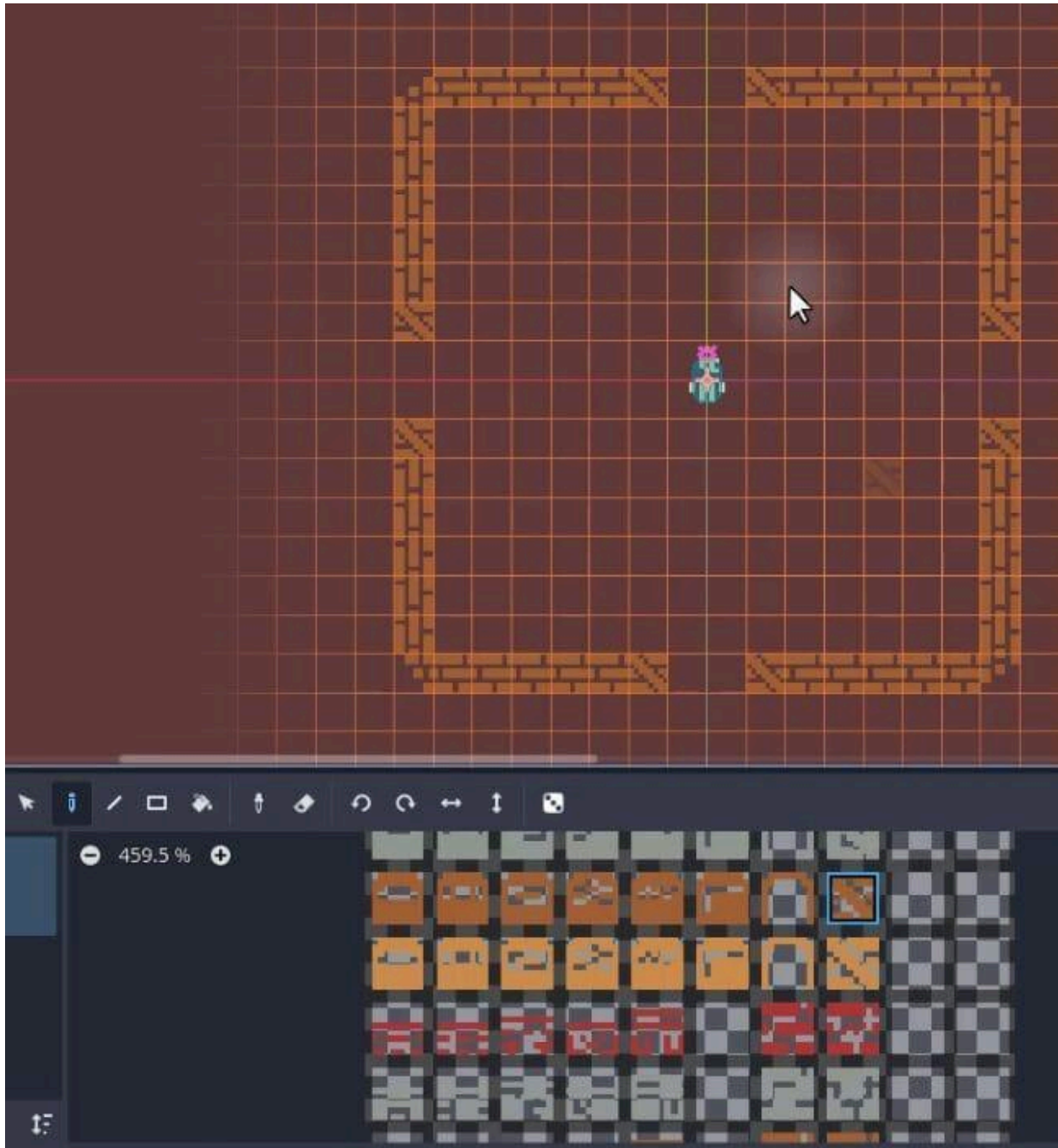
In our room template, we will use multiple `TileMapLayer` nodes: one for the base room geometry and separate ones inside each entrance for the tunnel visuals. This keeps each piece independently editable.

Learn more: [TileMapLayer](https://docs.godotengine.org/en/4.6/classes/class_tilemaplayer.html)

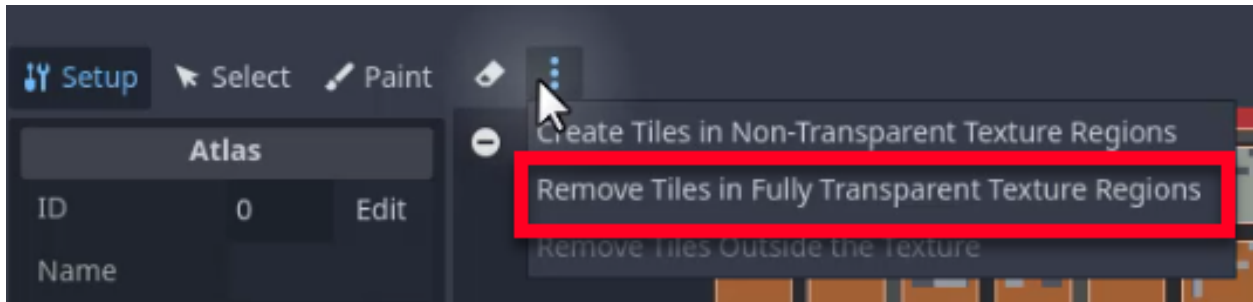
(https://docs.godotengine.org/en/4.6/classes/class_tilemaplayer.html)

Drawing the Room Corners

With everything configured, we can draw the basic shape of the room. Using the Base layer, draw the four corners of the room. We will add the doors and barriers as separate nodes later, so leave the middle sections of the walls empty for now.



If you encounter debugger errors related to the tile set at any point, (specifically regarding transparent regions), you can fix them in the TileSet editor. Open the TileSet panel, click the three vertical dots (options menu), and select **Remove Tiles in Fully Transparent Texture Regions**.

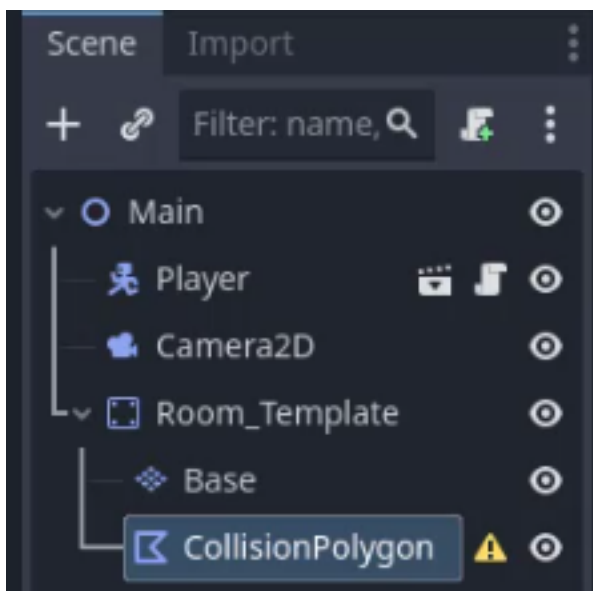


This should resolve the errors and allow the game to run without errors.

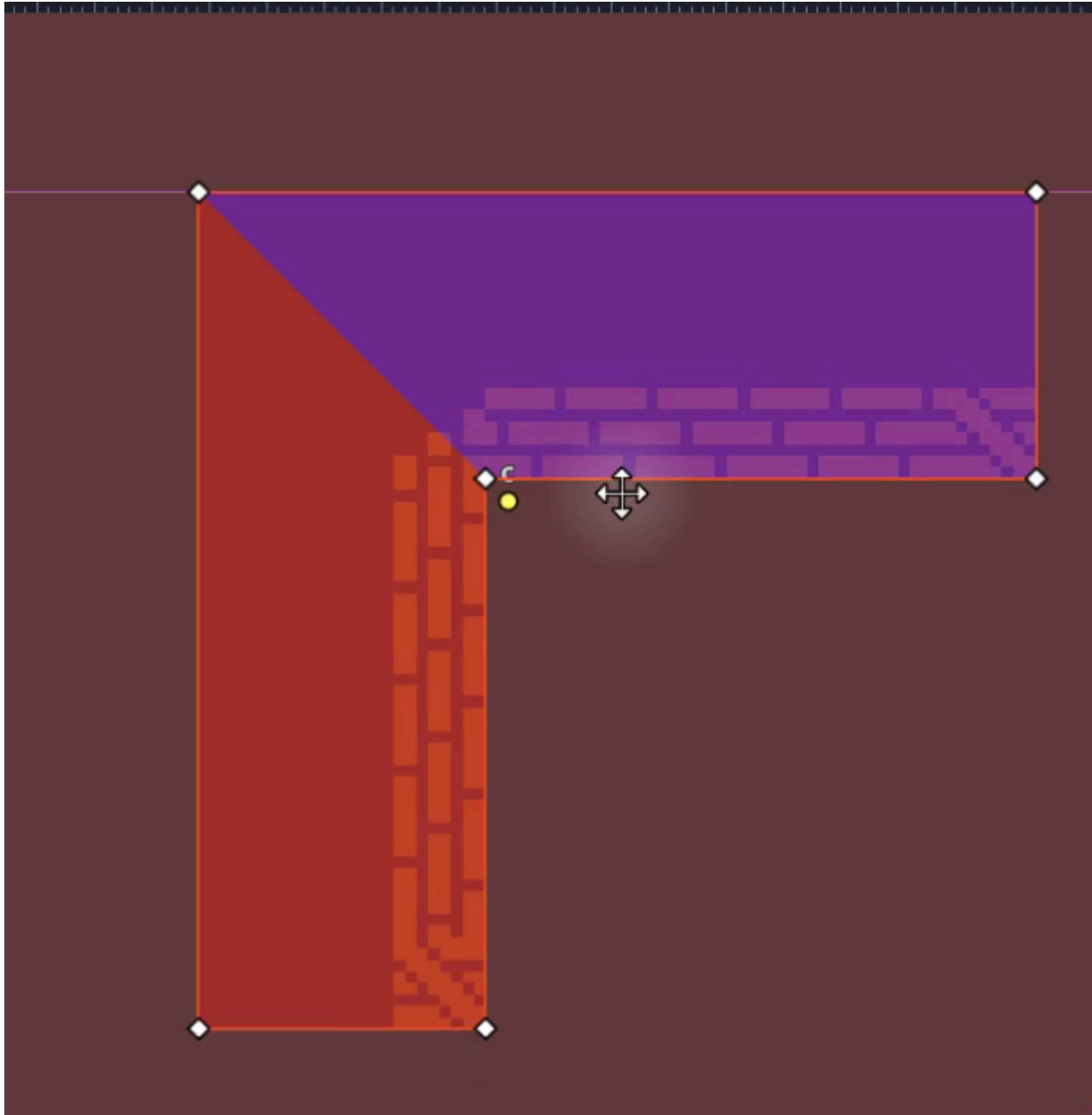
Adding Wall Colliders

To prevent the player from walking through walls, we need to define collision boundaries.

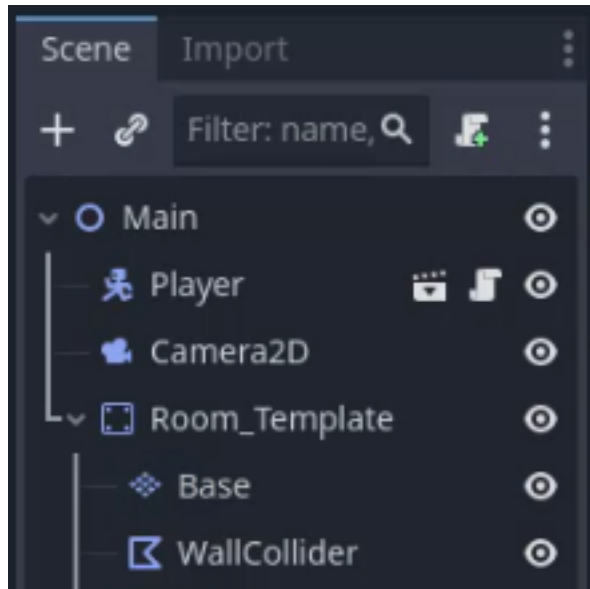
To do this, add a `CollisionPolygon2D` node as a child of `Room_Template`.



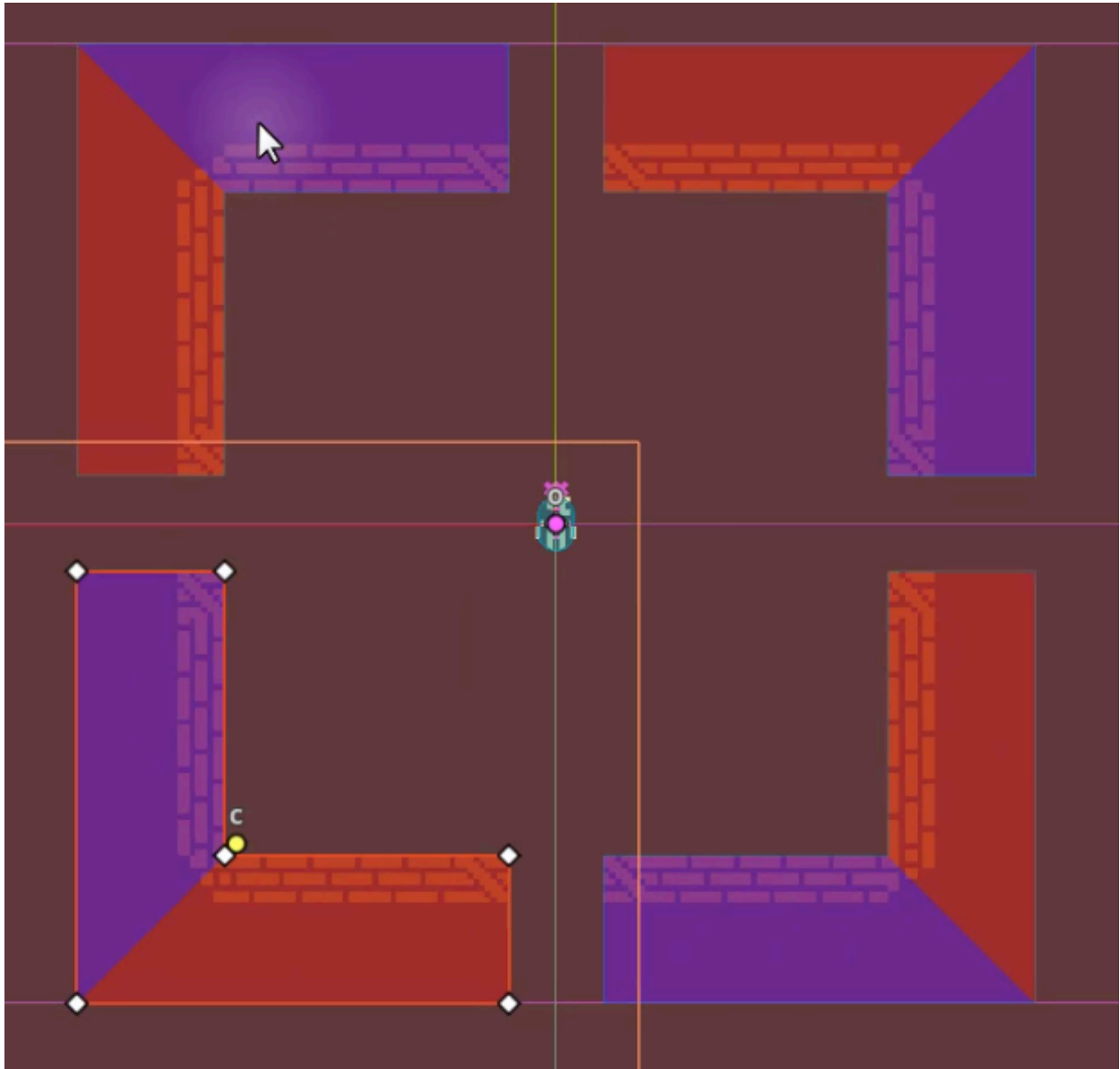
First, rename this node to `WallCollider` so we know what it's for later. We can then use the viewport to click and draw points, defining a custom shape that covers one of the corner walls.



Once the first collider is defined, you can duplicate the `WallCollider` node and rotate or adjust it to cover the remaining three corners.



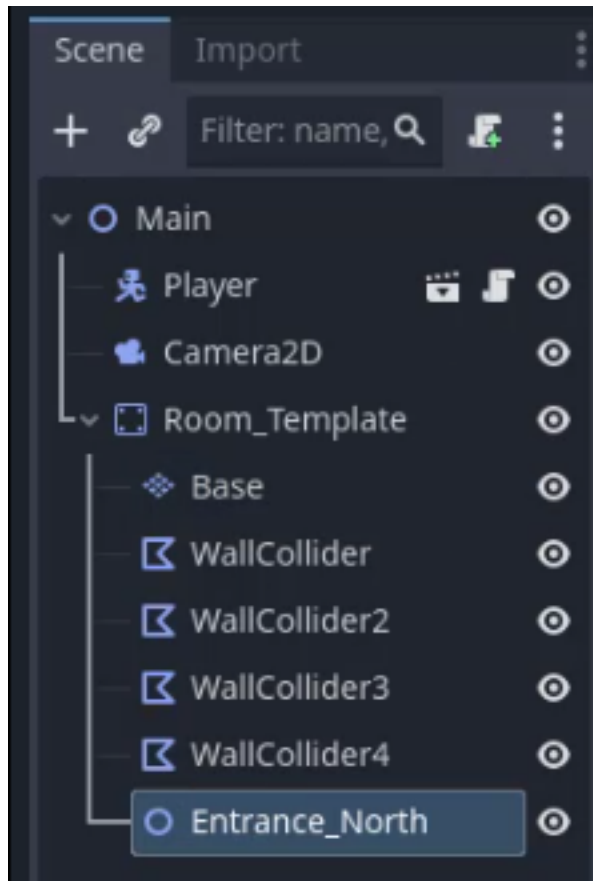
Ensure all solid walls have corresponding collision shapes.



Creating Entrances

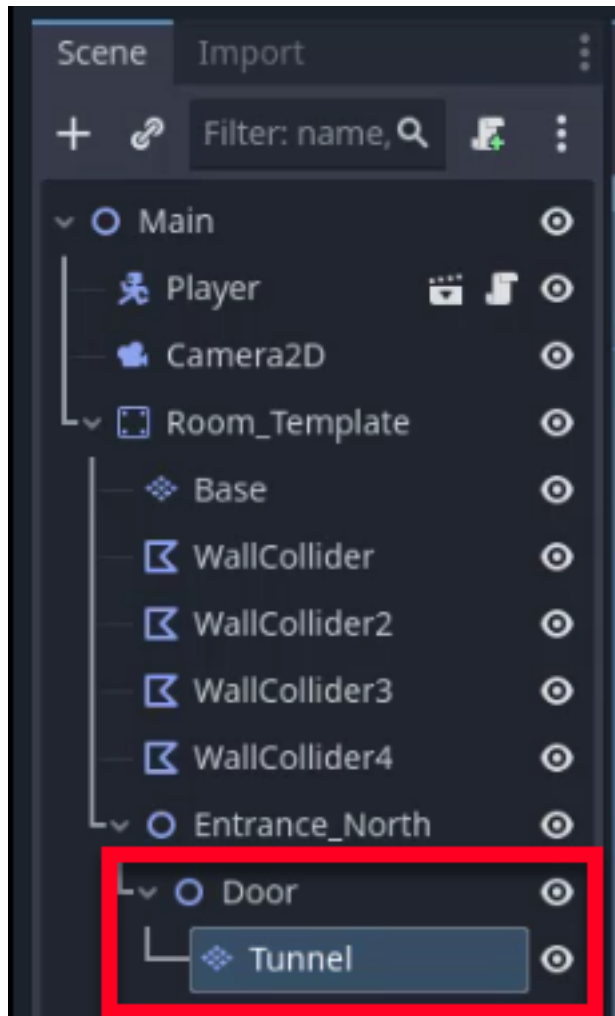
We will now create the entrances, starting with the North entrance. Each entrance will be a self-contained structure that includes the door visuals, the tunnel, and the logic for entering or exiting.

Create a new Node2D as a child of Room_Template and name it Entrance_North.

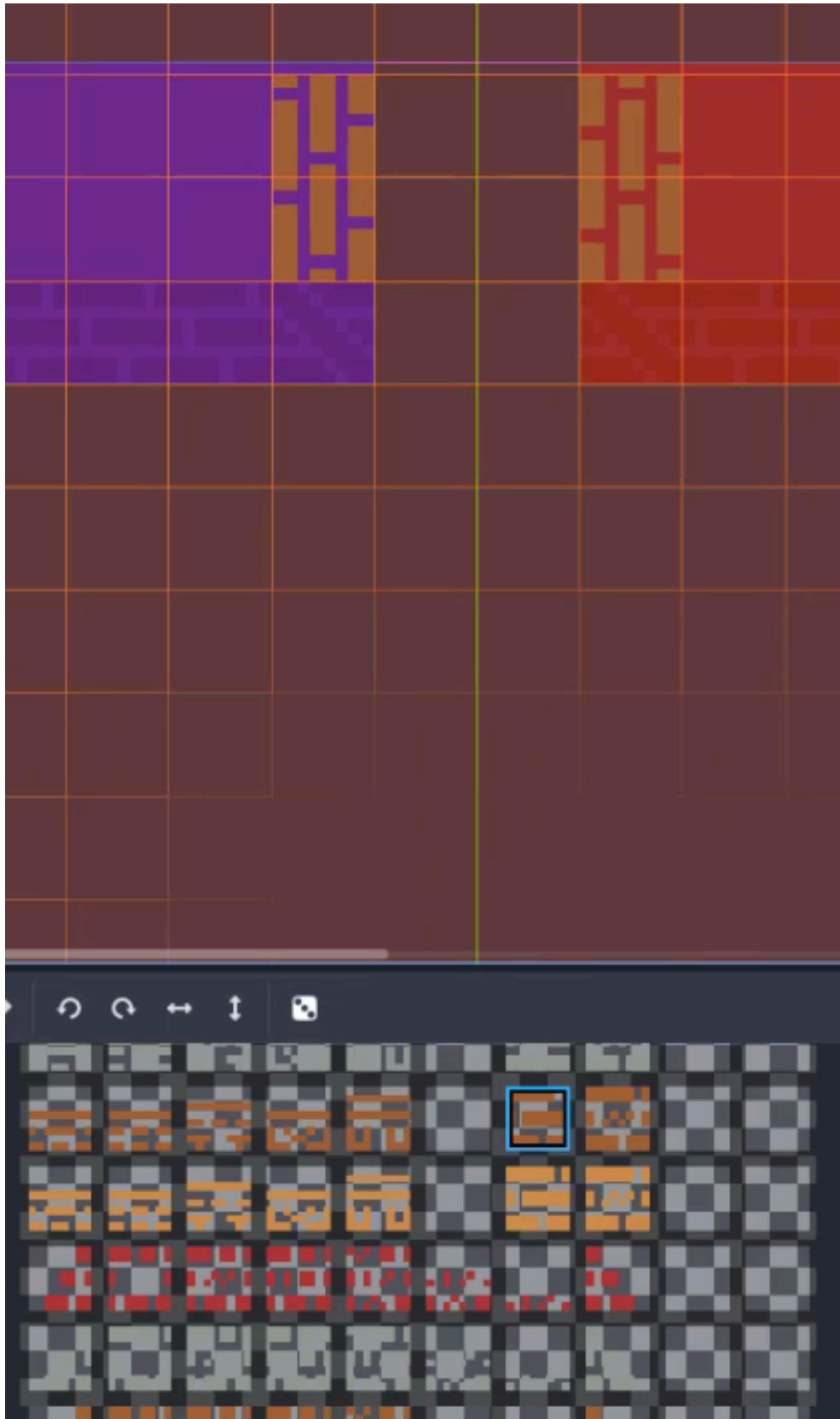


Adding Doors and Tunnels

Inside `Entrance_North`, we need to visualize the tunnel leading out of the room. First, add a `Node2D` child to `Entrance_North` and name it `Door`. Then, add a `TileMapLayer` child to the `Door` node and name it `Tunnel1`.



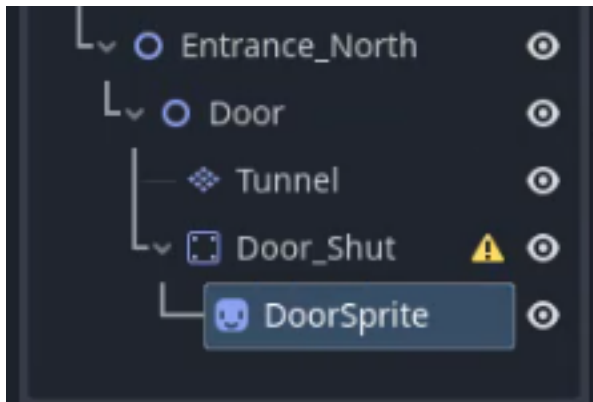
Use the Tunnel TileMap to draw the floor tiles that create the appearance of a passage leading out of the room. Remember that we're working on the north entrance when painting.



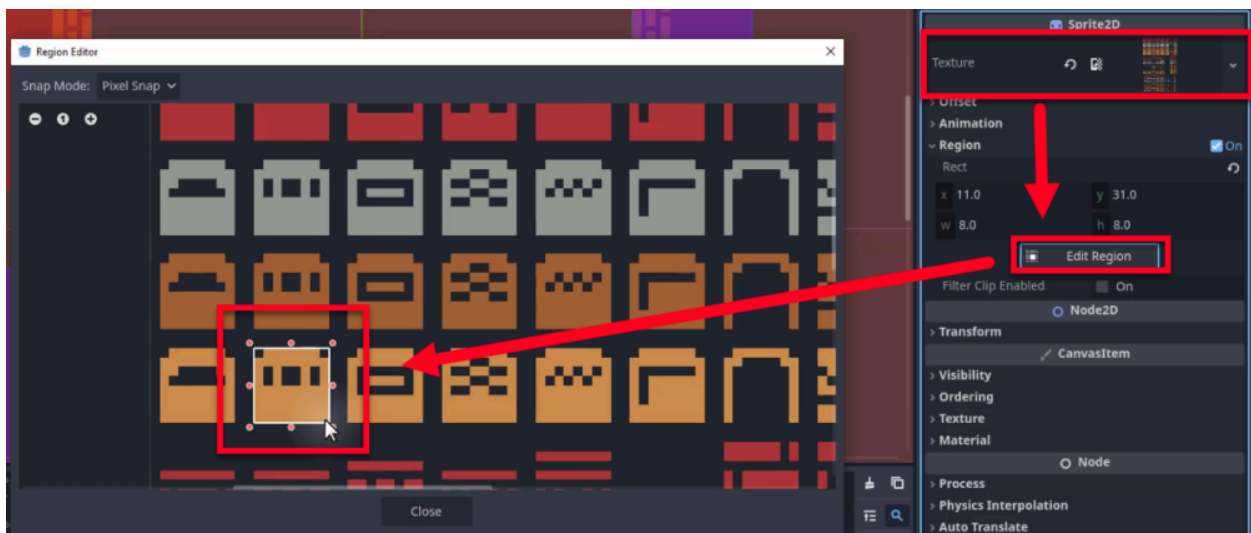
Adding Door Sprites and Collisions

Our next step is to set up doors that block the player from leaving the room when they're shut.

The door itself needs to be a physical object that can block the player when closed. As such, let's first add a `StaticBody2D` as a child of the `Door` node and name it `Door_Shut`. We'll then add a `Sprite2D` as a child of `Door_Shut` and rename it to `DoorSprite`.



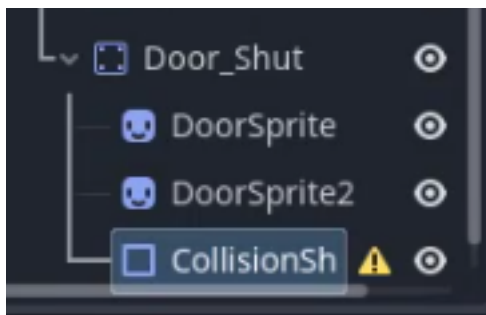
Next, assign `Tiles.png` to the `DoorSprite`'s `Texture` property. Use the **Region** settings (Edit Region) to select the specific door tile from the sprite sheet.



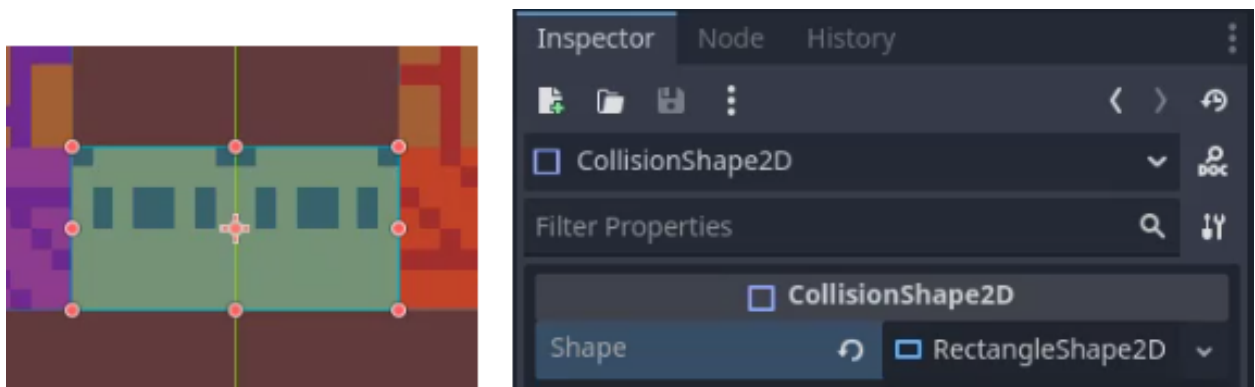
Since our tunnel is two tiles wide, we need to duplicate the `DoorSprite` to create a double door, positioning them side-by-side to cover the entrance.



To make the door solid, we must add a `CollisionShape2D` as a child of `Door_Shut`.



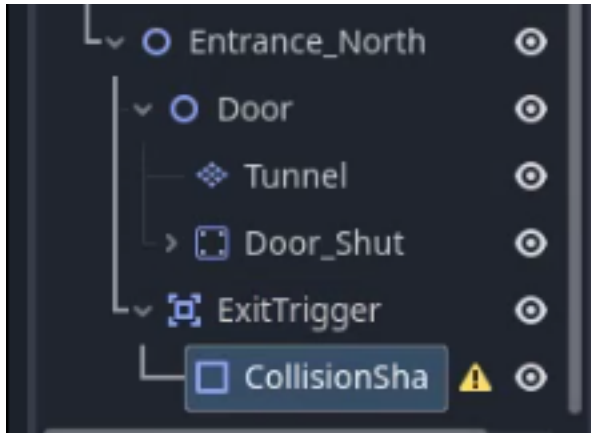
Set the shape to `RectangleShape2D` and resize it to cover both door sprites. This ensures the player cannot pass through when the door is closed.



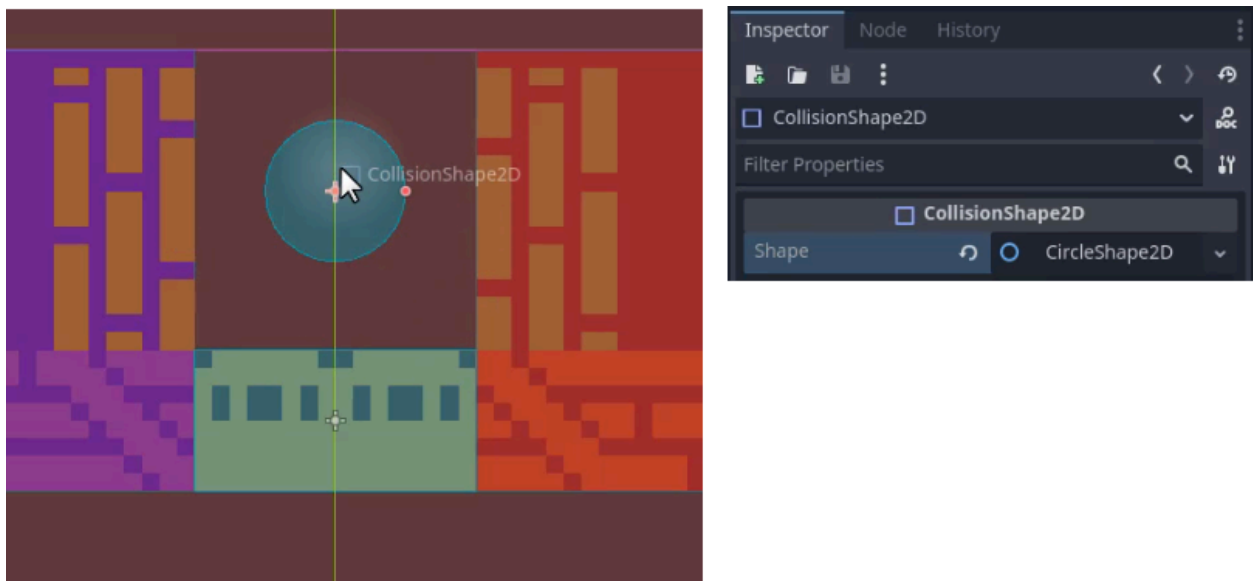
Adding Exit Triggers and Player Spawn

Beyond the tunnels and doors, we need to detect when the player walks through the open door to transition them to the next room.

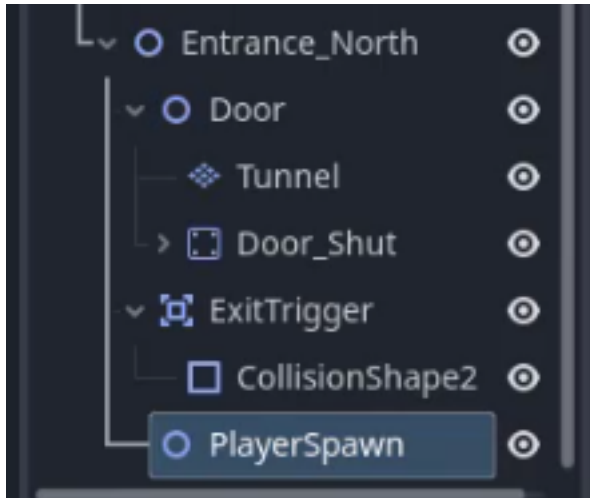
To set this up, let's add an `Area2D` as a child of `Entrance_North` and name it `ExitTrigger`. We'll then add a `CollisionShape2D` to the `ExitTrigger`.



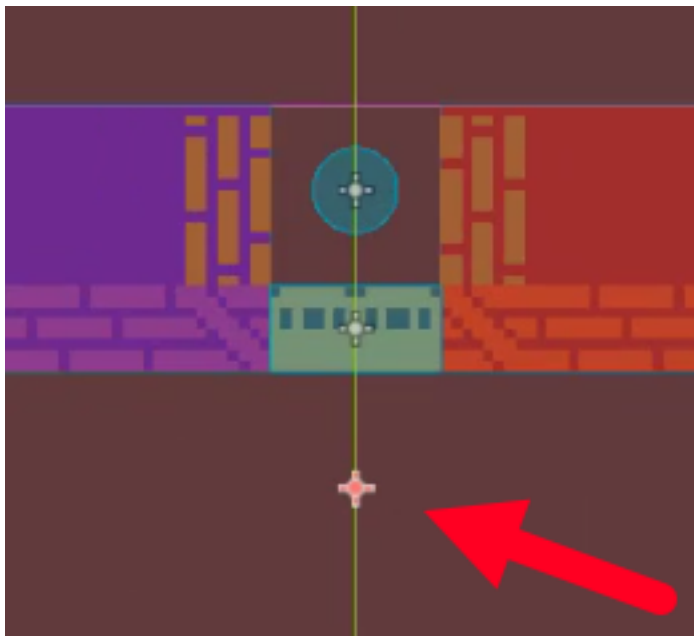
In this case, set the collision shape to a `CircleShape2D`. Place it inside the tunnel area so the player triggers it when walking out of the room.



Finally, we need to define where the player should appear when entering the room from this direction. Add another `Node2D` as a child of `Entrance_North` and name it `PlayerSpawn`.



We want to position this node just inside the room, near the door. This coordinate will be used to place the player character when they enter through the North gate.

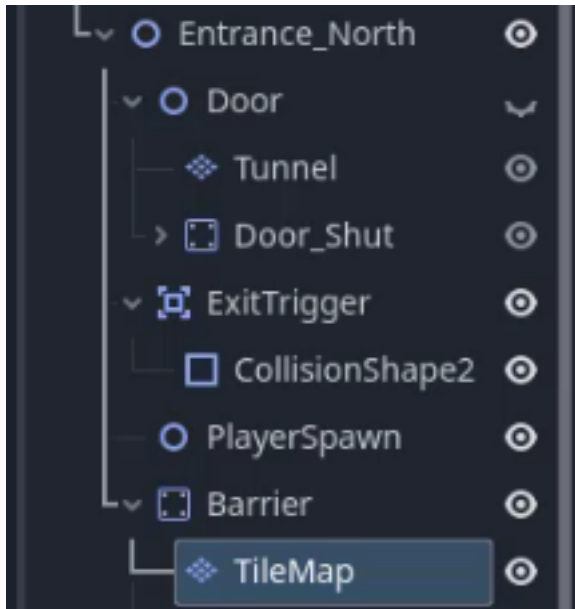


Creating the Barrier

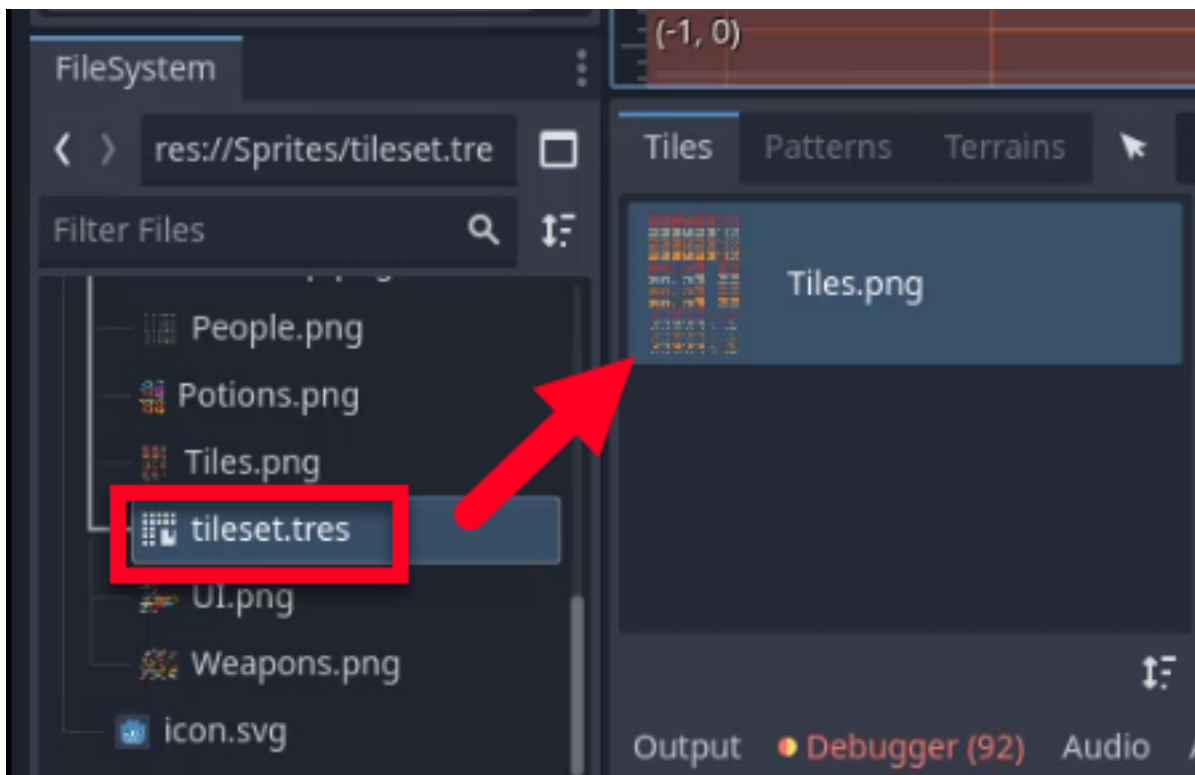
So far, we've set up our northern entrance to be able to handle any sort of room connection.

However, a complete entrance system needs to handle cases where there is no connected room. In such cases, the door should be replaced by a solid wall or "barrier."

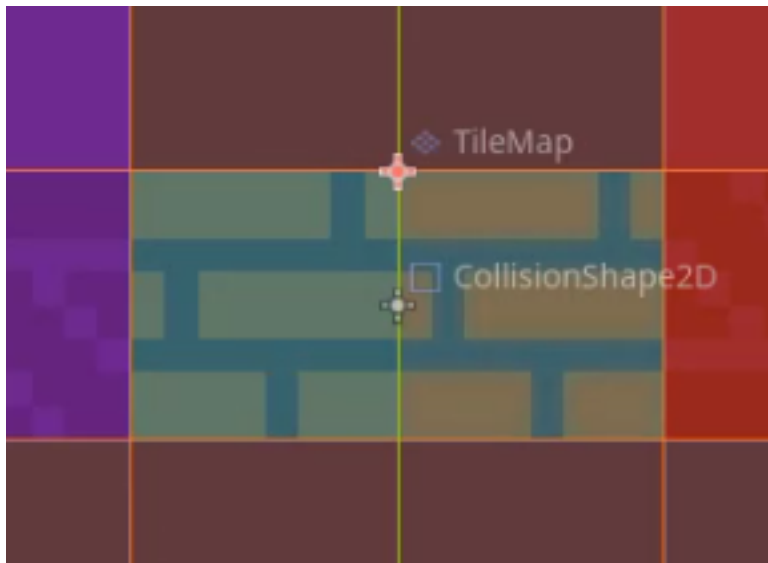
The last thing we'll add is a barrier component to our Entrance_North node. First, create a new StaticBody2D node as a child of Entrance_North and rename it to Barrier. After, add another TileMapLayer as a child of the Barrier node.



For this barrier TileMapLayer, assign the same tileset used for our base.



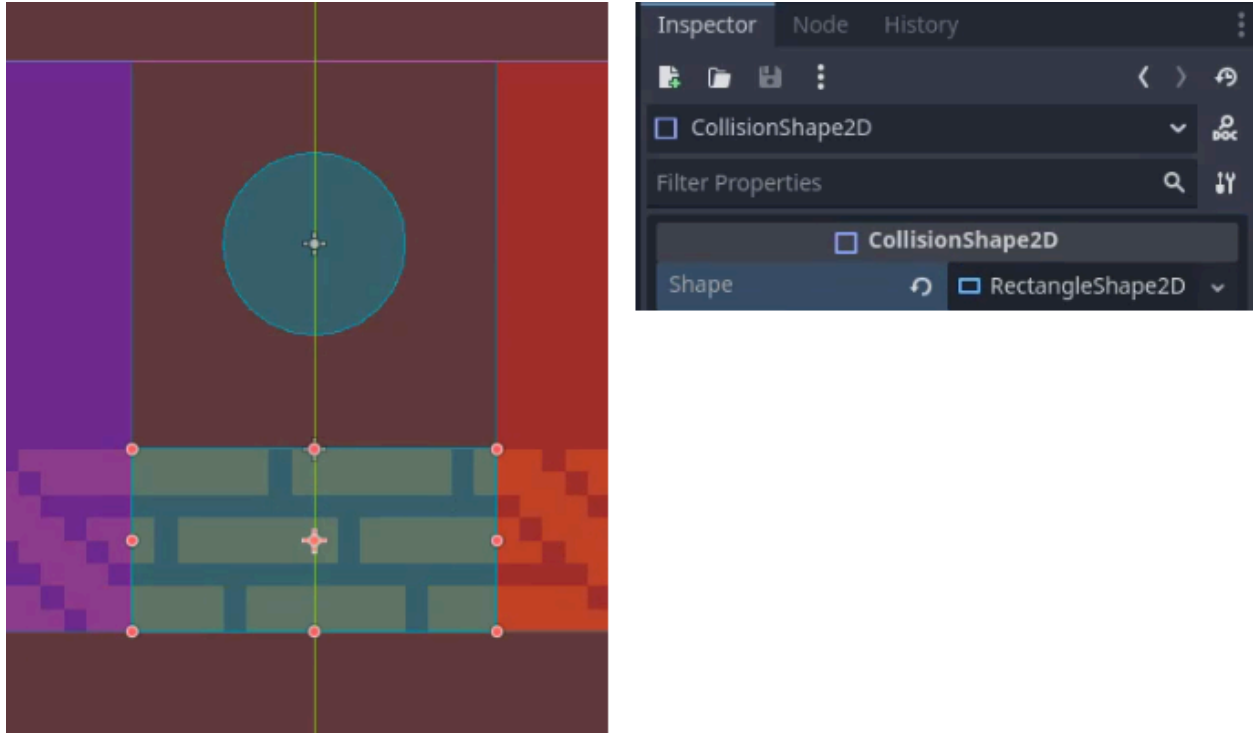
We can then draw a wall section to fill the gap where the door would be. This visual will be shown when the entrance is disabled.



To make the barrier solid, add a CollisionShape2D to the Barrier node.



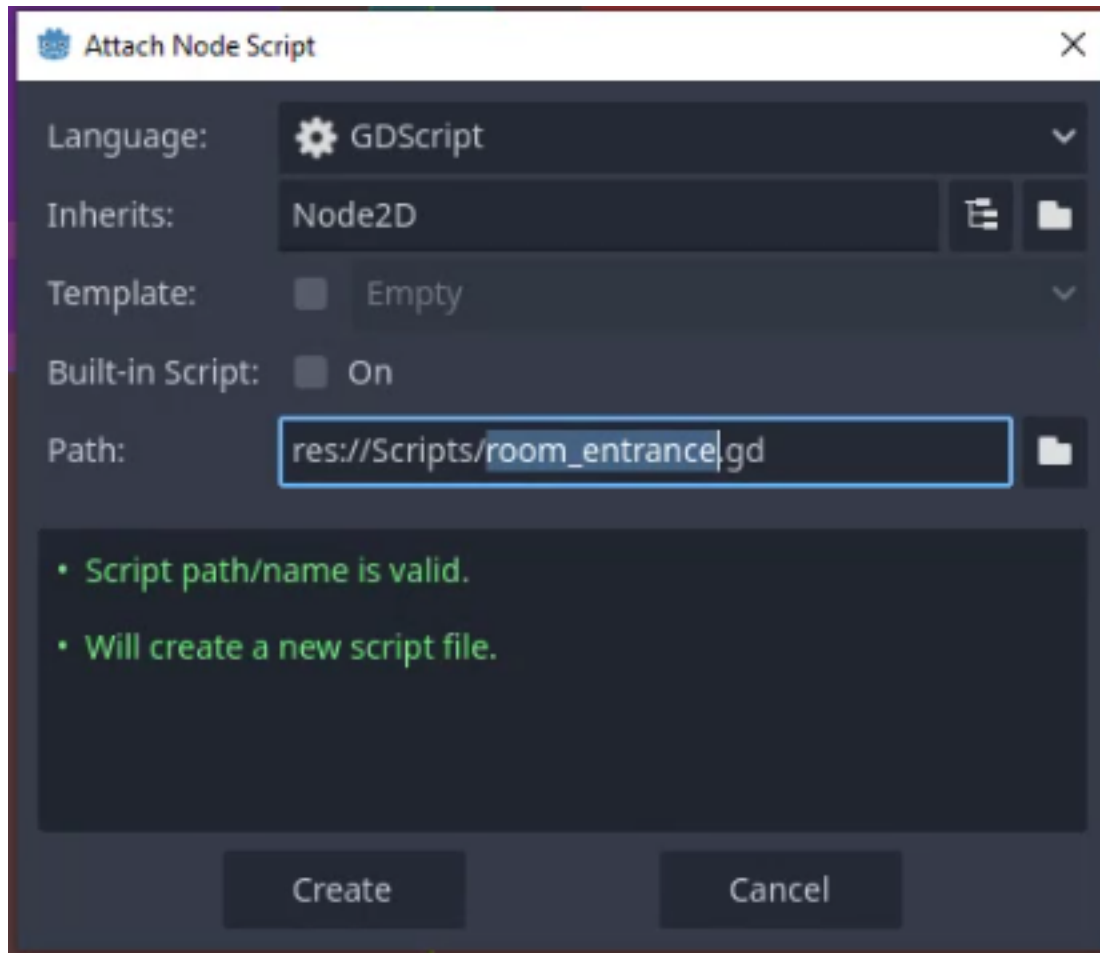
Set the shape to a rectangle in the Inspector and fit it to the bounds of the barrier tiles you just drew.



Adding Scripts to Manage Entrances

Each entrance will eventually need logic to switch between the “Door” and “Barrier” states. We will create a script for this purpose now, though we will implement the full logic later. As we’ll be duplicating this entrance to create the others, attaching the script at this point ensures every copy of the entrance inherits it automatically.

Select the Entrance_North node. Then, attach a new script named `room_entrance.gd` and save it in the Scripts folder.

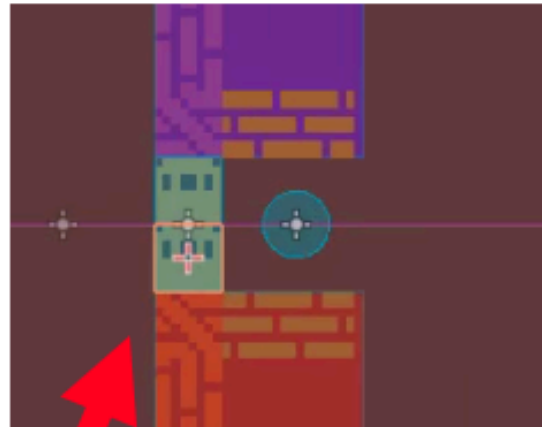
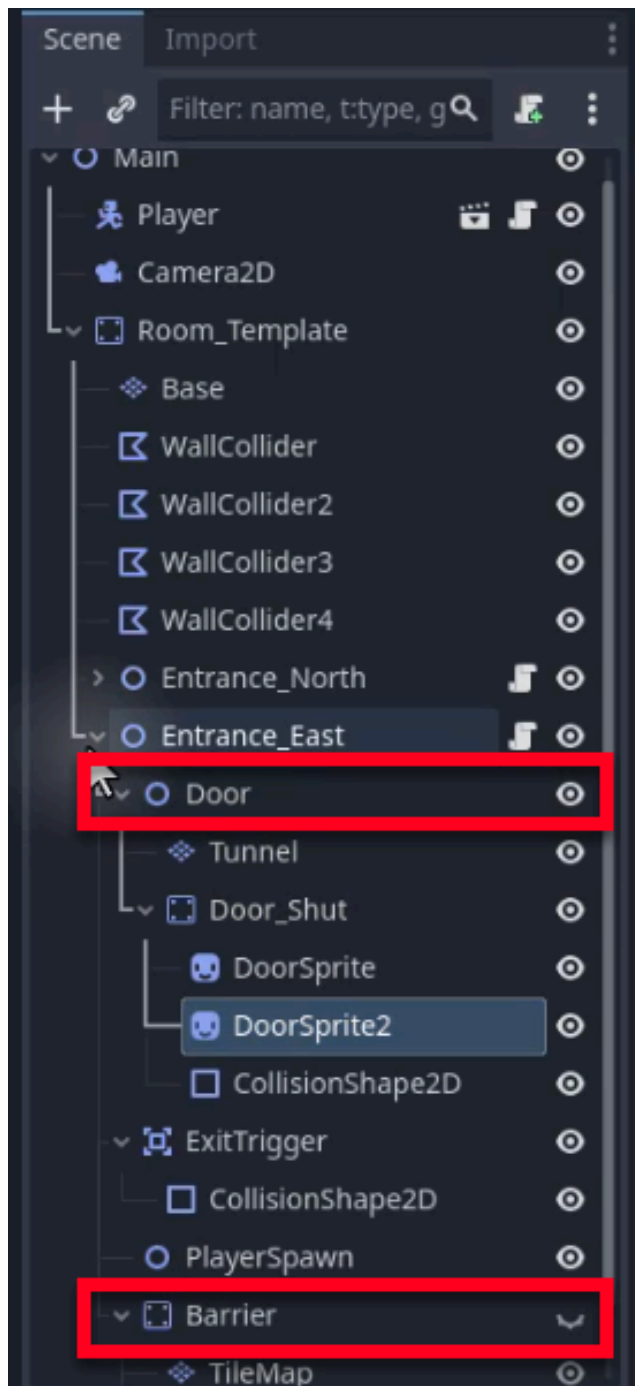


Duplicating Entrances for Other Directions

Now that we have a fully configured North entrance, we can create the East, West, and South entrances by duplicating and rotating it.

1. Select Entrance_North and duplicate it (Ctrl + D or Cmd + D).
2. Rename the duplicate to Entrance_East.
3. Rotate the node **90 degrees clockwise**.
4. Adjust the position to align with the east wall.

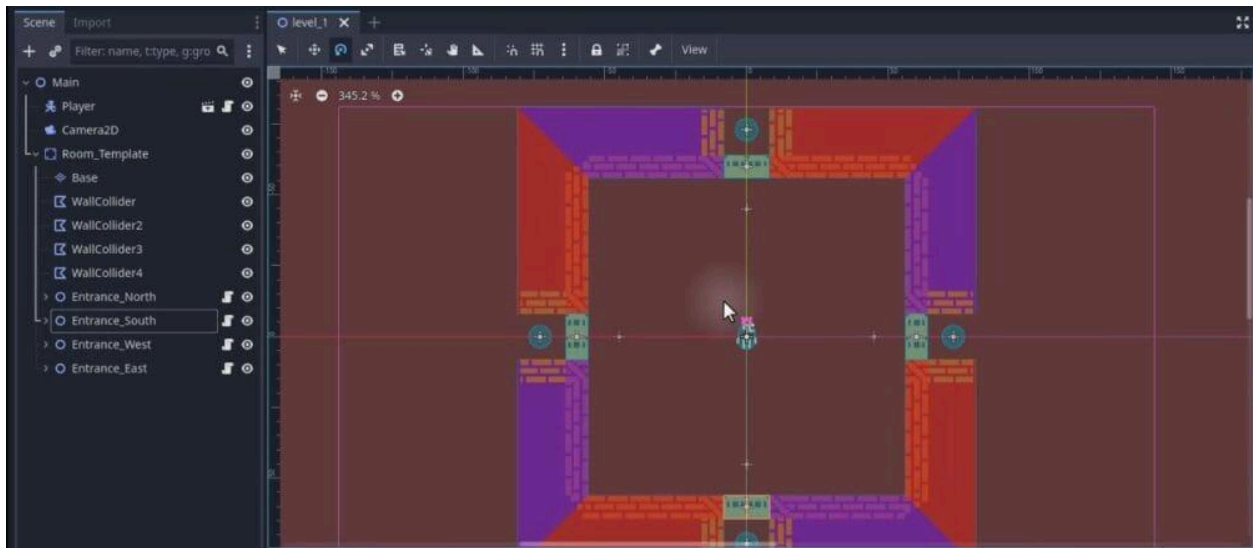
You may need to rotate the door sprites or adjust the barrier tiles to ensure they face the correct direction visually.



Repeat this process for the remaining directions:

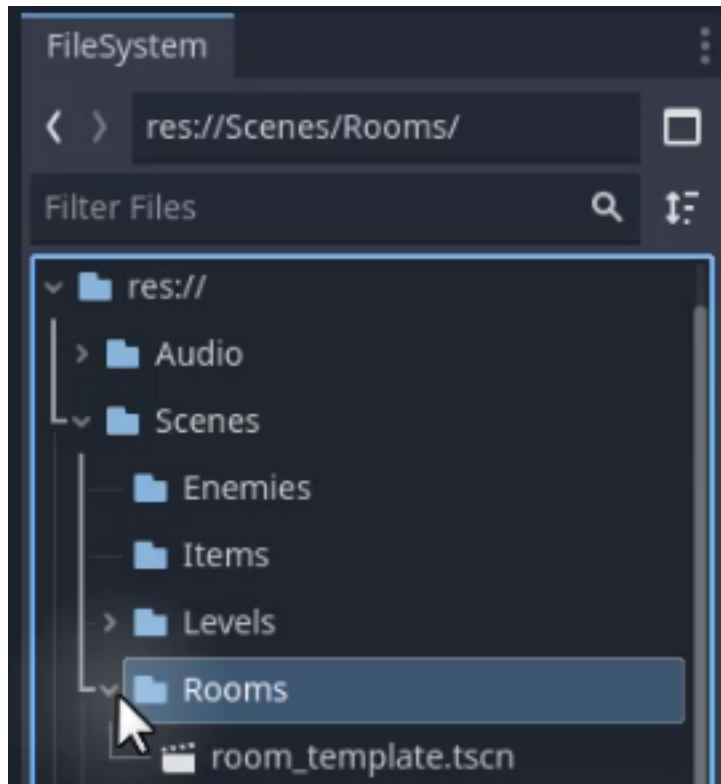
- **West Entrance:** Duplicate and rotate **90 degrees counter-clockwise** (or 270 degrees clockwise).
- **South Entrance:** Duplicate and rotate **180 degrees**.

Ensure that for each entrance, the door sprites and barrier tiles are correctly oriented.

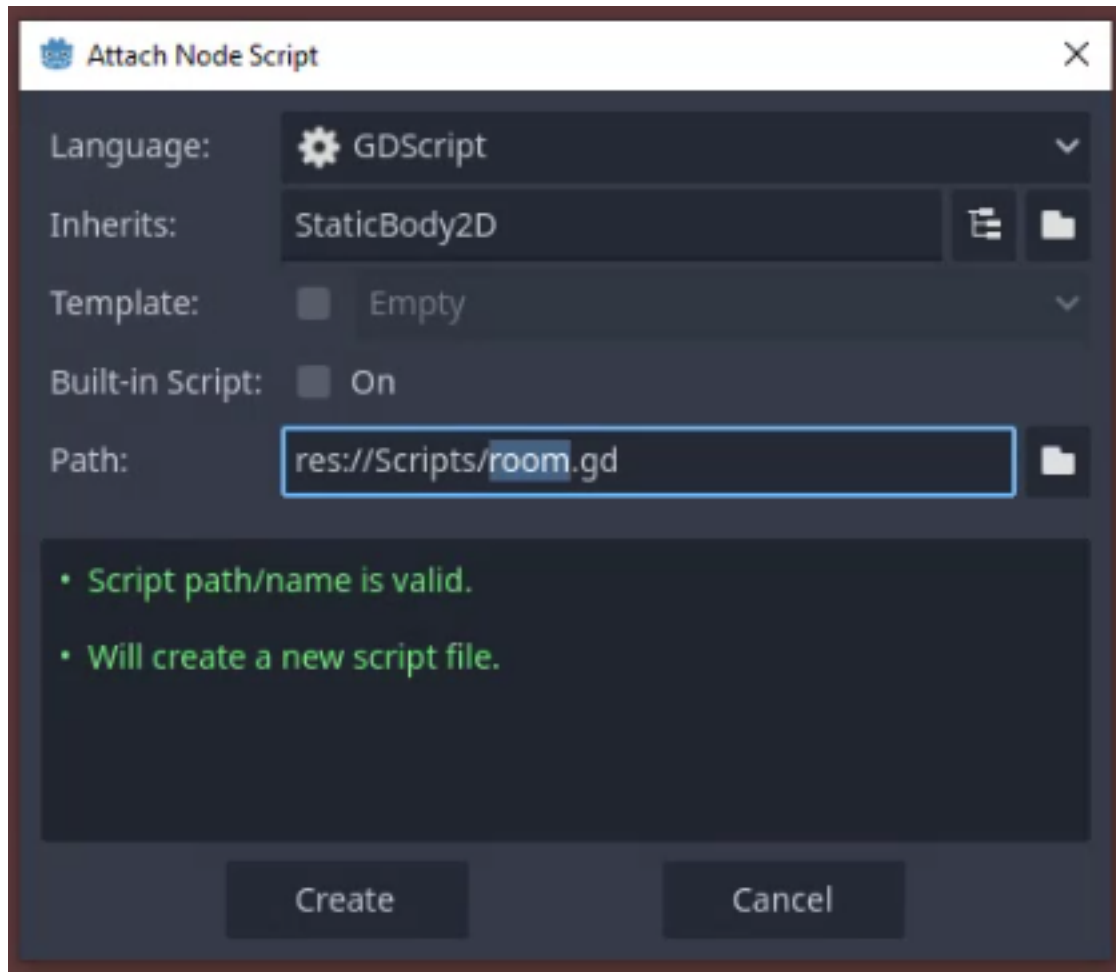


Saving the Room Template

With all four entrances set up, we should save this room as a reusable scene. In the FileSystem, navigate to Scenes/Rooms. Then, right-click the Room_Template node in the Scene tree and select **Save Branch as Scene**. Name the file `room_template.tscn`.



Before we consider our template done, we also need a script for the root node to handle room-specific logic. So, attach a new script to the Room_Template node and name it `room.gd`.



For now, add the following code to `room.gd` to define the class name. This will allow us to easily reference the Room class in other scripts as we start setting up our logic.

```
class_name Room
extends StaticBody2D
```

We now have a complete room template with functional entrances and barriers ready for scripting.

In the next chapter, we will use this template to begin implementing the procedural generation algorithms that will build our dungeon.

Procedural Room Generation

In this chapter, we will begin setting up the room generation system for our game. This system is one of the most significant components of the project, as it allows us to procedurally generate unique levels for every run. This involves placing rooms, connecting them, and ensuring that the game remains replayable with different layouts.

Our objective is to generate a random layout of rooms, ensure they are all connected, and place the starting room in the center of the map.



The Generation Process

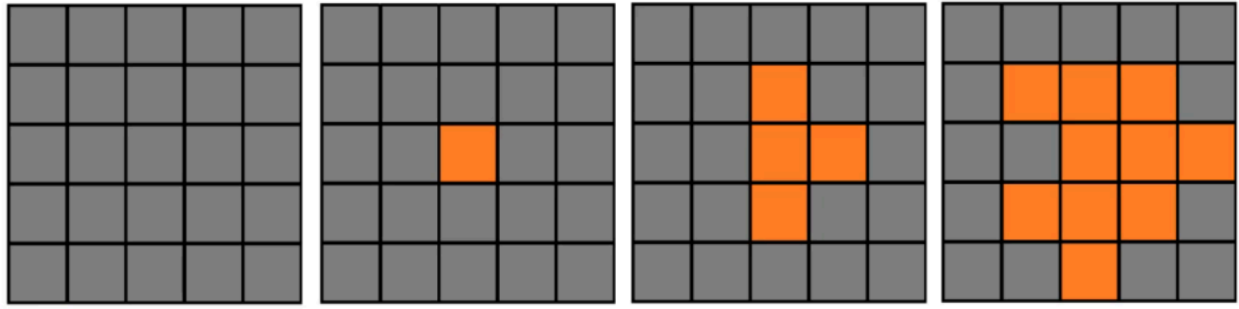
We will break this process down into three distinct steps:

1. **Block out room positions:** We first determine where rooms will be placed on a logical grid.
2. **Instantiate room scenes:** Once positions are decided, we spawn the actual room scenes.
3. **Initialize rooms:** Finally, we inform each room about its neighbors and configure the doors accordingly.

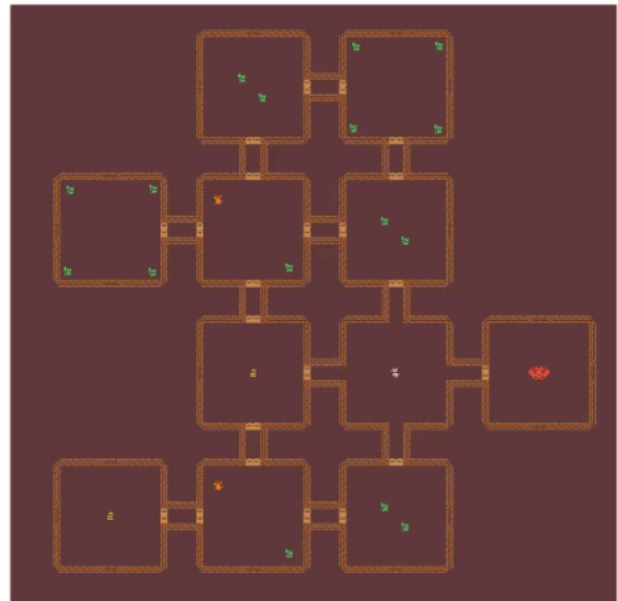
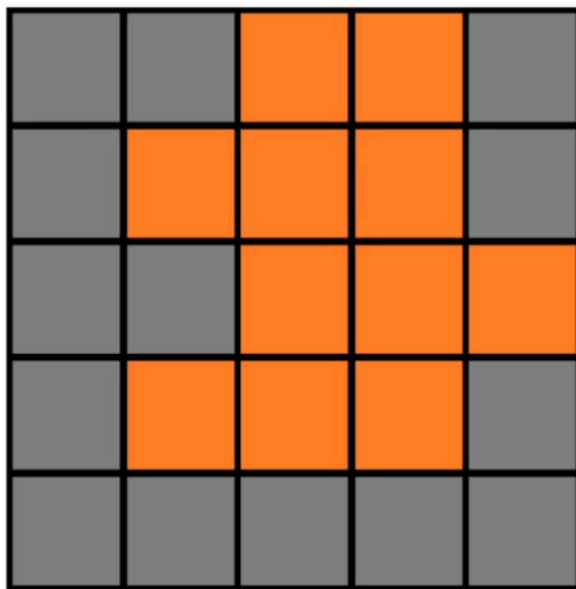
Blocking Out Room Positions

To block out room positions, we will use a 2D array. A 2D array is a data structure that holds multiple values, accessed by two indices (x and y), similar to a grid.

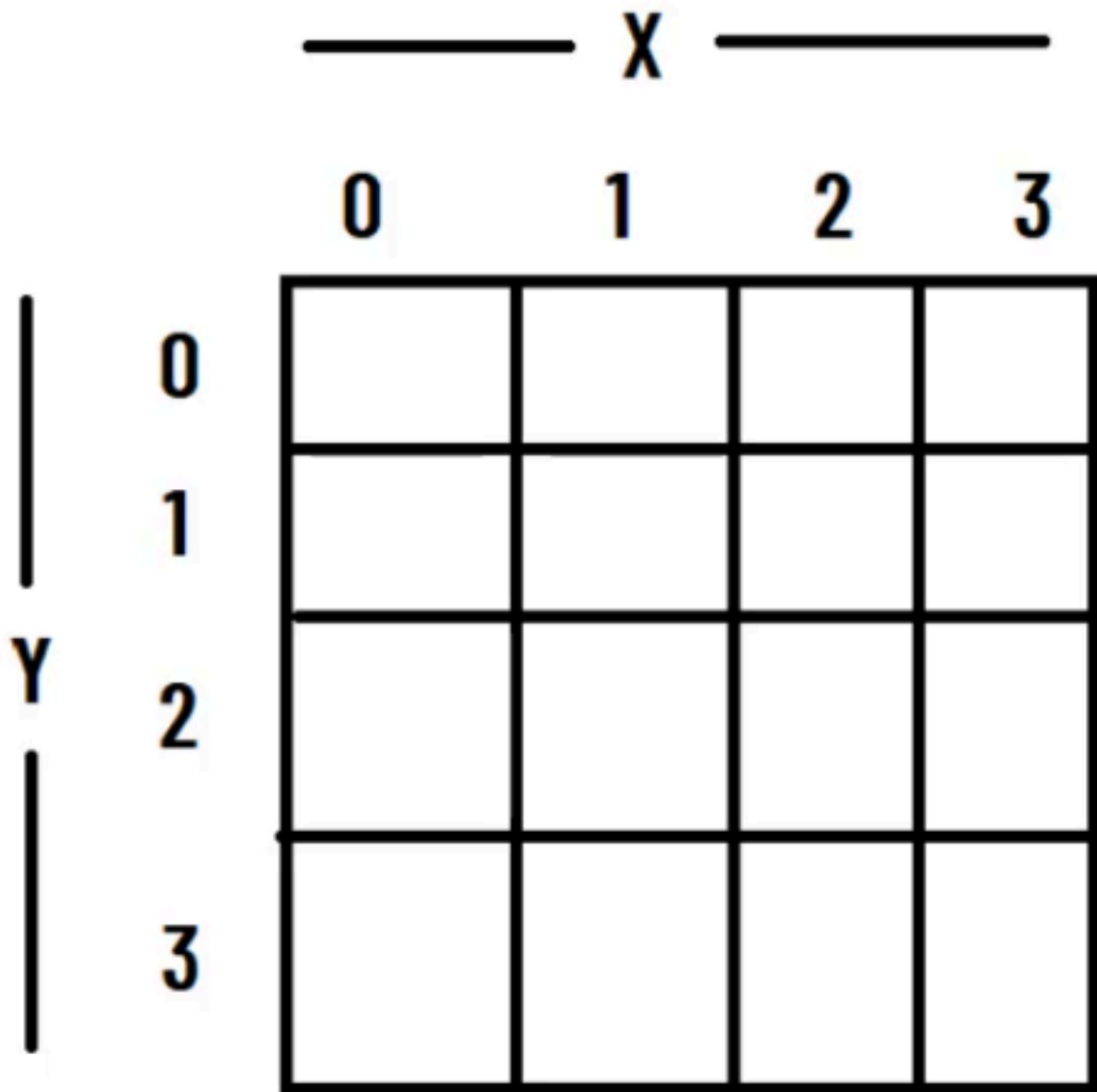
The diagram below shows how the algorithm starts at the center and branches out to create a connected structure.



Once the logical grid is populated, we translate that data into actual game objects.



In Godot, we do not have a built-in 2D array type, but we can simulate one using a standard 1D array and some math. Our array will use boolean values to indicate the presence (true) or absence (false) of a room at specific coordinates.



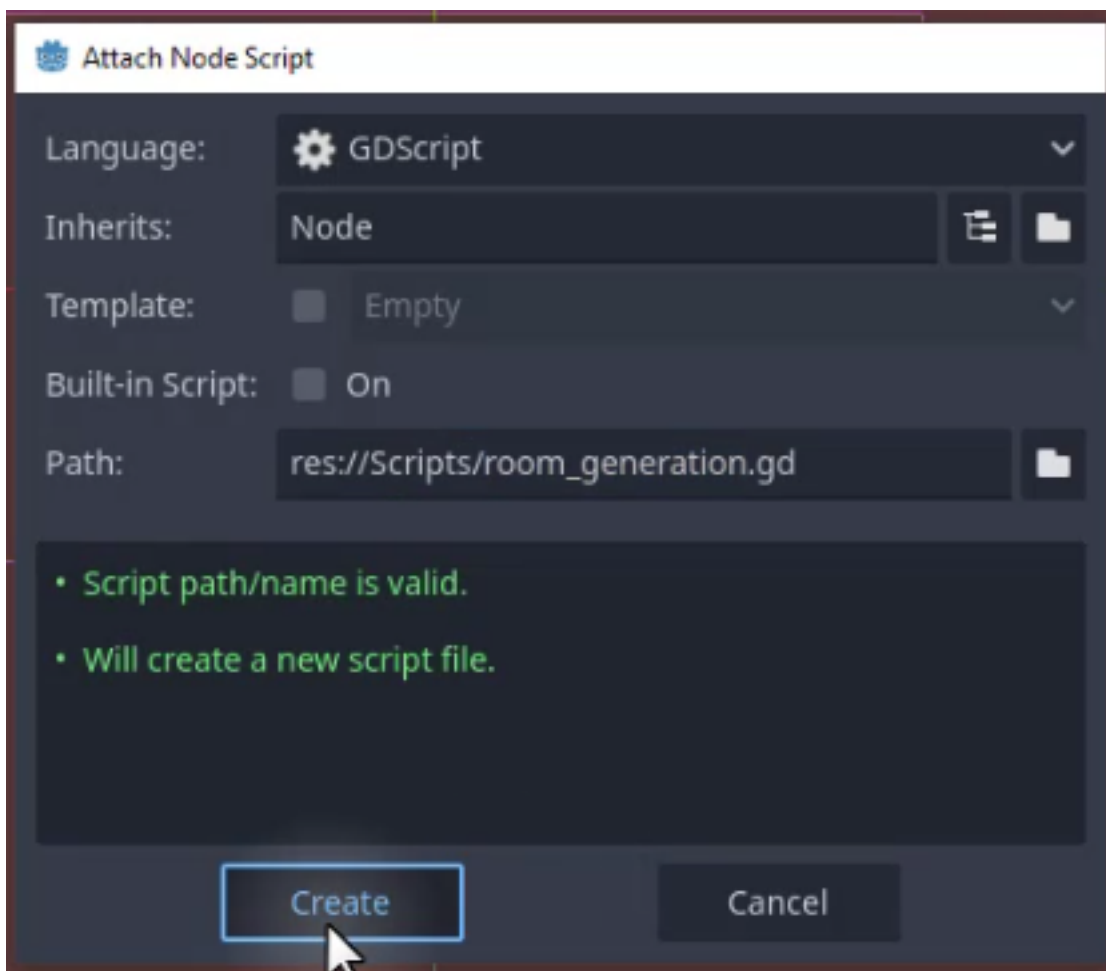
Setting Up the Room Generation Node

With the generation process planned out, we need a dedicated node to drive it at runtime. Let's create the node that will manage this process.

In the **Main** scene, create a new generic Node named `RoomGeneration`. If you still have the `Room_Template` scene in the tree from the previous chapter, go ahead and remove it, as we will be spawning rooms dynamically from this point.



Attach a new script to this node named `room_generation.gd` and save it inside the Scripts folder.



Defining Variables

Our algorithm needs to track several things: the grid size, the target room count, which cells contain rooms, and where the starting room is. In the `room_generation.gd` script, we define these as variables along with the arrays that will hold our map data.

Note that we will also add a class name to this script so we can easily reference it when we need to.

Add the following code to your script:

```
class_name RoomGeneration
extends Node

@export var map_size : int = 7
@export var rooms_to_generate : int = 12
var room_count : int = 0
var map : Array[bool]
var rooms : Array[Room]
var room_pos_offset : float = 160

var first_room_x : int = 3
var first_room_y : int = 3
var first_room : Room

@export var player : CharacterBody2D
```

Here is what each variable handles:

- `map_size`: Defines the width and height of our logical grid (7x7).
- `rooms_to_generate`: The target number of rooms we want to create.
- `room_count`: Tracks how many rooms have been generated so far.
- `map`: The array acting as our 2D grid to define room presence.
- `rooms`: An array to keep track of the actual room nodes we instantiate.
- `room_pos_offset`: The distance (in pixels) between the centers of adjacent rooms. This is based on our room templates size.
- `first_room_x` and `first_room_y`: The coordinates for the starting room (center of the 7x7 grid).
- `first_room`: Stores a reference to the starting room node.

- `player`: A reference to the player character, which we will need later to position them.

Preloading the Room Scene

To spawn rooms later, we need a reference to the room template we created in the previous chapter. To set this up, add the following variable at the top of your script with the rest we just added:

```
var room_scene : PackedScene =  
preload("res://Scenes/Rooms/room_template.tscn")
```

Godot Insight: PackedScene and preload

A `PackedScene` is Godot's way of storing an entire scene (nodes, properties, connections) as a single resource that can be created on demand. Calling `instantiate()` on a `PackedScene` produces a fresh copy of that scene's node tree, ready to be added to the game world with `add_child()`.

We use `preload()` here rather than `load()` because `preload` reads the file at compile time, guaranteeing it is available instantly when needed. Use `load()` instead when the path is not known until runtime.

Learn more: [PackedScene](https://docs.godotengine.org/en/4.6/classes/class_packedscene.html)
(https://docs.godotengine.org/en/4.6/classes/class_packedscene.html)

Creating Functions

Next, we will outline the functions required for the generation logic. We will define them as empty stubs for now and fill them in step-by-step in the proceeding sections.

```
func _ready():  
    pass
```

```
func _generate():  
    pass
```

```
func _check_room(x : int, y : int, desired_direction : Vector2,  
is_first_room : bool = false):
```

```

    pass

func _instantiate_rooms():
    pass

```

- `_ready`: Initializes the generation process when the game starts.
- `_generate`: Sets up the map array and kicks off the recursive generation.
- `_check_room`: The core recursive function that determines where rooms are placed.
- `_instantiate_rooms`: Converts the logical map into actual room scenes in the game world.

Implementing the Generation Logic

Now we can begin filling in our functions. We start with `_ready`, which, as just mentioned, simply calls our custom `_generate` function. This ensures our map is generated right when the scene opens up.

```

func _ready ():
    _generate()

```

The Generate Function

The `_generate` function will be what prepares the system for a new run. It resets the room count, resizes the map array to fit our grid (7x7 = 49 slots), and starts the generation process at the center of the map.

```

func _generate ():
    room_count = 0
    map.resize(map_size * map_size)

    # start the check room chain at the center of the grid
    _check_room(first_room_x, first_room_y, Vector2.ZERO, true)

    # once we've defined our map, spawn in the room scenes
    _instantiate_rooms()

```

We call `_check_room` with the starting coordinates (3, 3). The `Vector2.ZERO` indicates no specific direction preference for the first room, and the `true` flag marks it as the starting room. This is essential information we'll need to ensure our map generates correctly. However, we'll explore this aspect further on as we start tackling the `_check_room` function itself.

Helper Functions for the Map

Before we dive into our `_check_room` function, we need to consider our current setup for our map. Since we are using a 1D array to represent a 2D grid, we need helper functions to convert x and y coordinates into a single index.

Add these two functions to your script:

```
# 2D-to-1D index: row-major layout in a flat array
func _get_map (x : int, y : int) -> bool:
    return map[x + y * map_size]

func _set_map (x : int, y : int, value : bool):
    map[x + y * map_size] = value
```

- `_get_map`: Returns true if a room exists at the given (x, y) coordinates. This allows us to see if a room is in a specific position or not already.
- `_set_map`: Sets the value at the given (x, y) coordinates. In other words, we tell our array “yes” or “no” as to whether a room exists at a specific coordinate.

You'll see how these functions are applied later and help simplify several checks we must do in order to generate our map.

The Recursive Check Room Function

The `_check_room` function is the heart of our procedural generation. It uses recursion (a technique where a function calls itself) to “walk” across the grid and place rooms.

Here is the logic we need to implement:

1. Check if we have already generated enough rooms (based on our `rooms_to_generate` variable)
2. Check if the current coordinates are out of bounds.
3. Check if a room already exists at this position. This is where our `_get_map` helper function will help us.
4. If all checks pass, place a room at the given coordinates.
5. Randomly decide which neighboring directions (North, South, East, West) to expand into.
6. Recursively call `_check_room` for those directions.

Let's implement the early return conditions first, as we don't want to generate rooms if we simply can't.

```
func _check_room(x: int, y: int, desired_direction: Vector2,
is_first_room: bool = false):
    # return if we've created enough rooms
    if room_count >= rooms_to_generate:
        return

    # return if out of bounds
    if x < 0 or x > map_size - 1 or y < 0 or y > map_size - 1:
        return

    # return if room already exists
    if _get_map(x, y):
        return
```

If the function survives these checks, it means we can validly place a room here. We increment the count and update the map with our `_set_map` helper function.

```
# Place the room
room_count += 1
_set_map(x, y, true)
```

Deciding Expansion Directions

Now we need to decide where to go next. For our project, we want the generation to favor the `desired_direction` (the direction we just moved in) to create corridors, but still have a chance to branch out. This is why `desired_direction` is one of the parameters in our `_check_room` function, as it allows us to notify our algorithm of this direction.

We use `randf()` to generate a random number between 0 and 1. If the number is greater than a certain threshold, we expand in that direction. We selectively lower the threshold (making it more likely) if it matches our desired direction.

Here is the logic for all four directions:

```
# define the directions we want to expand in
var go_north: bool = randf() > (0.2 if desired_direction ==
Vector2.UP else 0.8)
var go_south: bool = randf() > (0.2 if desired_direction ==
Vector2.DOWN else 0.8)
var go_east: bool = randf() > (0.2 if desired_direction ==
Vector2.RIGHT else 0.8)
var go_west: bool = randf() > (0.2 if desired_direction ==
Vector2.LEFT else 0.8)
```

Finally, we recursively call `_check_room` for any direction that was selected. Note that for the first room, we force expansion in all selected directions. Otherwise, we set the new `desired_direction` appropriately.

Note that this is why we pass `true` for the `is_first_room` parameter of the `_check_room` function when it first runs; it allows us to toggle the one case where this is relevant to ensure proper expansion.

```
# continue the check room chain in those directions
if go_north or is_first_room:
    _check_room(x, y - 1, Vector2.UP if is_first_room else
desired_direction)
if go_south or is_first_room:
    _check_room(x, y + 1, Vector2.DOWN if is_first_room else
desired_direction)
```



```

    if go_east or is_first_room:
        _check_room(x + 1, y, Vector2.RIGHT if is_first_room else
desired_direction)
    if go_west or is_first_room:
        _check_room(x - 1, y, Vector2.LEFT if is_first_room else
desired_direction)

```

Verifying the Generation

Before we instantiate the actual scenes, it is helpful to verify that our logic works. We can print a simple ASCII representation of the map to the console.

Add this temporary loop to the end of your `_ready` function (after calling `_generate`):

```

# room layout preview
for x in range(map_size):
    var line : String
    for y in range(map_size):
        line += "# " if _get_map(x, y) else " "
    print(line)

```

If you run the game now and check the Output console, you should see a grid of # symbols representing your generated rooms. Feel free to run this several times to see how the map is randomized with each separate run.



Once you are satisfied that the logic is producing connected shapes, you can remove or comment out this printing loop.

Instantiating Room Scenes

Now that we have our logical map and know its working, we need to spawn the actual room nodes into the game world. We will use the `_instantiate_rooms` function to loop through our map array and do this.

For every coordinate where `_get_map` returns true, we will:

1. Instantiate a new room scene.
2. Add it to the scene tree.
3. Position it based on its grid coordinates multiplied by the `room_pos_offset`.
4. Store it in our rooms array for future reference.

Add the following code to `_instantiate_rooms`:

```
# instantiate each room scene at their designated position
func _instantiate_rooms ():
    for x in range(map_size):
        for y in range(map_size):
            if _get_map(x, y) == false:
                continue

            var room : Room = room_scene.instantiate()
            # get the proper root node of the scene
            get_tree().root.get_node("/root/Main")
                .add_child.call_deferred(room)
            rooms.append(room)

            room.global_position = Vector2(x, y) * room_pos_offset
```

Godot Insight: Why call_deferred matters

`call_deferred()` tells Godot to postpone a function call until the current frame's processing is complete. This is essential when modifying the scene tree (adding or removing nodes) during callbacks like `_ready()` or signal handlers, because the engine may still be iterating over the tree at that point. Modifying it mid-iteration can cause crashes or unpredictable behavior.

Whenever you see `add_child.call_deferred(node)` in this project, it means we are safely queuing the addition for the next idle frame rather than forcing it immediately.

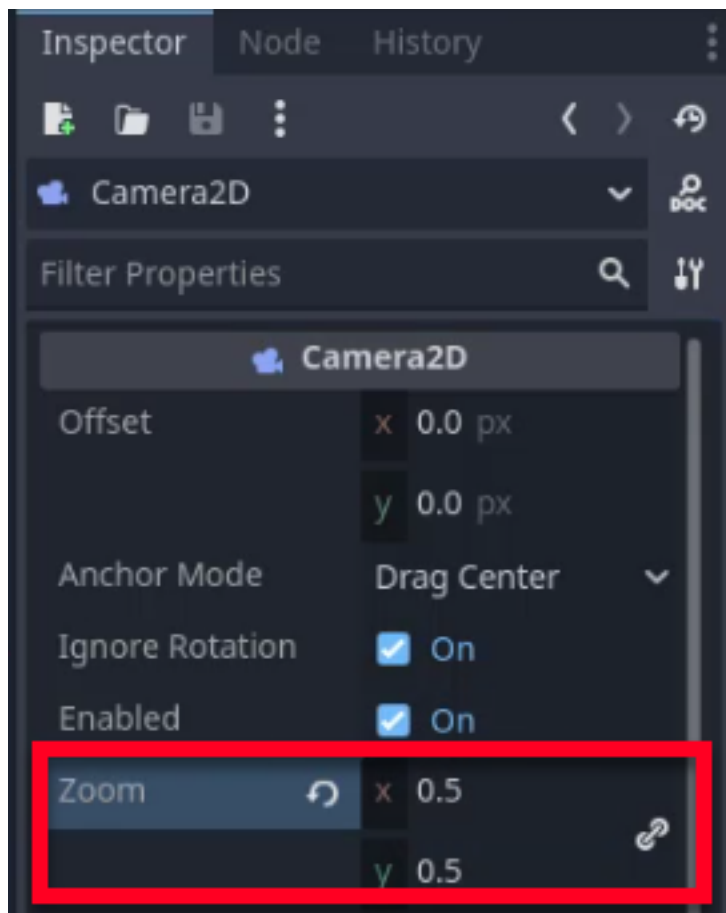
Learn more: [Object](#)

(https://docs.godotengine.org/en/4.6/classes/class_object.html)

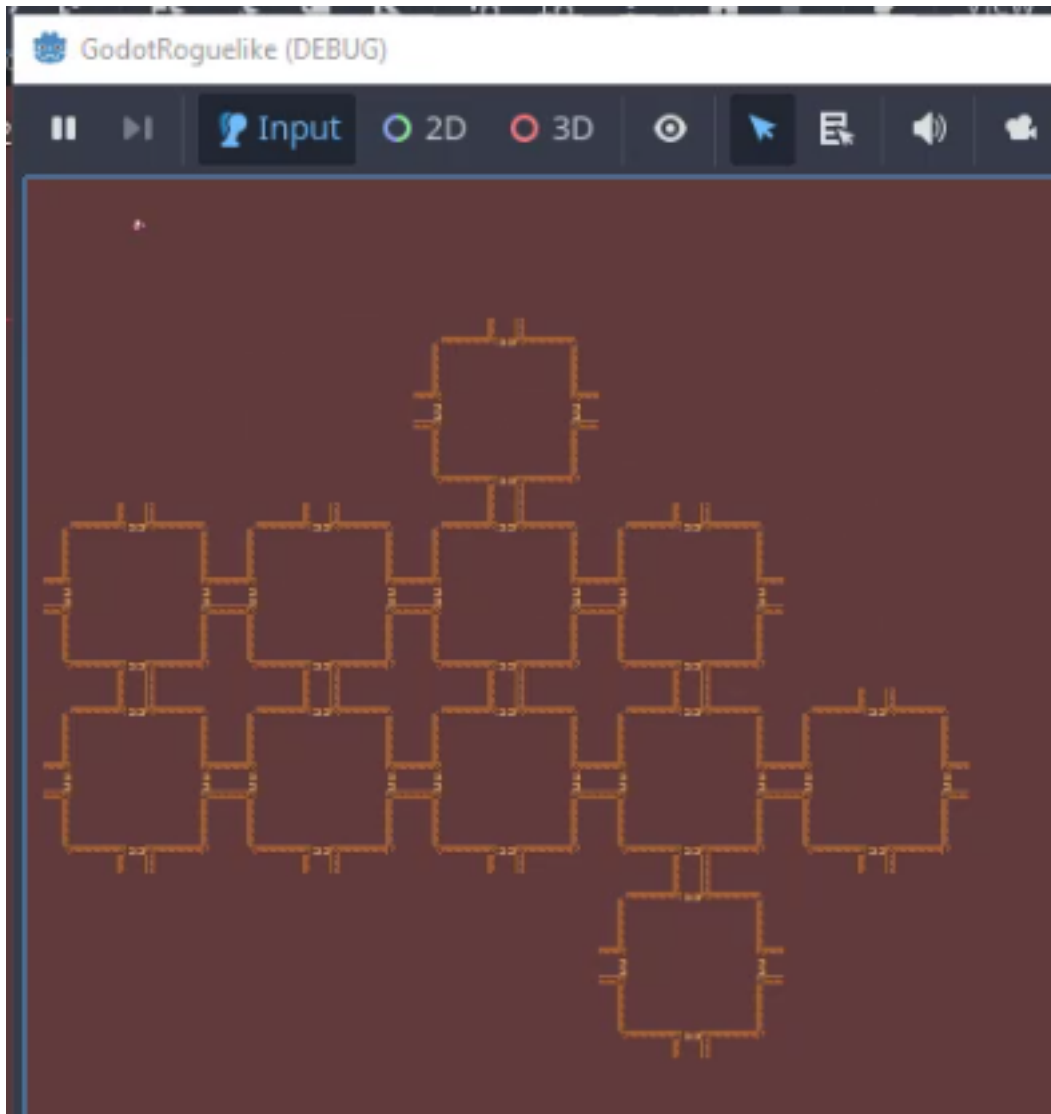
Adjusting the Camera

If you run the game now, the rooms will be generated, but they might be spread out beyond the current camera view. To visualize the entire layout, let's temporarily zoom out the camera.

Select the Camera2D node in your Main scene and change the **Zoom** property to 0.5 for both X and Y.



Now, when you run the game, you should see a branching structure of rooms. The entrances won't be connected yet (we will handle that in the next chapter), but the layout itself is complete.



We have successfully implemented the core of our procedural generation system. In the next chapter, we will focus on the logic to connect these rooms, opening doors where neighbors exist and keeping them closed where they do not.

Room Logic and Navigation

In the previous chapter, we successfully generated a procedural layout of rooms. However, these rooms are currently isolated islands; the player cannot move between them, and the doors do not function.

In this chapter, we will implement the logic required to connect these rooms, allowing the player to traverse the dungeon.

We will focus on three main scripts:

1. **RoomEntrance**: Manages individual doorways, barriers, and transition triggers.
2. **Room**: Manages the state of a specific room, including its four entrances and enemy tracking.
3. **RoomGeneration**: Updates the generation process to link neighbors and spawn the player.

The Room Entrance Script

We will start with the `RoomEntrance` script. This script is responsible for the behavior of a single entrance, such as opening or closing the door, toggling the barrier (wall) if no room exists in that direction, and detecting when the player leaves the room.

Defining Directions

Before we write the entrance logic, we need a centralized way to define directions (North, South, East, West). We will define this enumeration in the `Room` script so it can be accessed globally by other scripts.

Open `room.gd` (which is attached to your `Room_Template` node) and define the `Direction` enum as follows:

```
class_name Room
extends StaticBody2D

enum Direction
{
    NORTH,
```

```

        SOUTH,
        EAST,
        WEST
    }
}

```

Setting Up Variables

Let's start working on the entrances now that we have our directions. Open the `room_entrance.gd` script (which is attached to your entrance nodes). We need to define variables to store references to the door visuals, colliders, and the specific direction the entrance represents.

Add the following variables to `room_entrance.gd`:

```

@export var direction : Room.Direction = Room.Direction.NORTH

@onready var barrier : StaticBody2D = $Barrier
@onready var barrier_collider : CollisionShape2D =
$Barrier/CollisionShape2D
@onready var door : Node2D = $Door
@onready var door_shut : StaticBody2D = $Door/Door_Shut
@onready var door_shut_collider : CollisionShape2D =
$Door/Door_Shut/CollisionShape2D
@onready var player_spawn : Node2D = $PlayerSpawn
@onready var exit_trigger : Area2D = $ExitTrigger
var neighbor : Room

```

As a quick breakdown:

- `direction`: Allows us to specify which way this entrance points in the Inspector.
- `barrier` and `door`: References to the visual and physical components we built in the room template.
- `neighbor`: Will store a reference to the Room node that this entrance leads to.

Entrance Functions

Next, we're going to need several functions to handle the state of the entrance. Let's outline them first as below. We'll cover each one in-depth

further along, but this skeleton gives us a good, pseudo “to-do” list of what we need to set up.

```
func _ready():  
    pass  
  
func set_neighbor (neighbor_room : Room):  
    pass  
  
func toggle_barrier (toggle : bool):  
    pass  
  
func open_door ():  
    pass  
  
func close_door ():  
    pass  
  
func _on_body_entered_exit_trigger (body):  
    pass
```

Initialization

Let’s start filling in our functions.

First, in the `_ready` function, we connect the exit trigger’s signal. This will allow us to detect when the player walks into the tunnel to leave the room. Note that we will implement the `_on_body_entered_exit_trigger` logic later in this chapter; for now, the connection makes sure the function is ready.

```
func _ready():  
    exit_trigger.body_entered.connect(_on_body_entered_exit_trigger)
```

Godot Insight: Signals and the observer pattern

Signals are Godot’s implementation of the observer pattern. Instead of one node constantly checking another node’s state, the observed node emits a signal when something interesting happens, and any number of listeners respond automatically. This keeps nodes independent: the emitter does not need to know who is listening.

In GDScript, you connect a signal to a callback with `signal.connect(callback)`. The `body_entered` signal on `Area2D` fires whenever a physics body overlaps the area, passing that body as an argument to the connected function. This is how our exit triggers detect the player without polling every frame.

Learn more: [Signal](https://docs.godotengine.org/en/4.6/classes/class_signal.html)
(https://docs.godotengine.org/en/4.6/classes/class_signal.html)

By default, all entrances should be barriers (walls). They should only become doors if `set_neighbor` is called (meaning the room has a neighbor). To enforce this, we will update the `_ready` function so the barrier is active from the start.

```
func _ready():
    exit_trigger.body_entered.connect(_on_body_entered_exit_trigger)
    # Start as a solid wall; set_neighbor will swap to a door if linked
    toggle_barrier(true)
```

Controlling Doors

Let's work on toggling our doors next. The `open_door` and `close_door` functions manage the visual and physical state of the door. Importantly, if the entrance is currently a solid wall (barrier), we should ignore door commands.

```
func open_door():
    # Do nothing if this entrance is a barrier (no neighbor)
    if barrier.visible:
        return
    door_shut.visible = false
    door_shut.collider.disabled = true

func close_door():
    # Do nothing if this entrance is a barrier (no neighbor)
    if barrier.visible:
        return
    door_shut.visible = true
    door_shut.collider.disabled = false
```


Toggling the Barrier

Next, let's set up the function that sets up our barriers. Recall that our barriers are meant to show up when the room has no neighbor.

The `toggle_barrier` function switches the entrance between being a door (leading to another room) and a barrier (a solid wall).

```
# Swap between impassable wall (barrier) and openable door
func toggle_barrier(toggle : bool):
    barrier.visible = toggle
    barrier.collider.disabled = !toggle
    door.visible = !toggle
```

Setting the Neighbor

Finally let's work on aspects related to our neighbors. The `set_neighbor` function links the entrance to another room. When a neighbor is assigned, we disable the barrier to reveal the door.

```
# Reveal a passable door when this entrance connects to another room
func set_neighbor(neighbor_room : Room):
    neighbor = neighbor_room
    toggle_barrier(false)
```

Determining the Neighbor's Entry Direction

When the player exits a room to the North, they enter the next room from the South. We need a helper function to calculate this opposite direction.

Add the `_get_neighbor_entry_direction` function:

```
# Map to the opposite direction so the player enters from the correct side
func _get_neighbor_entry_direction() -> Room.Direction:
    if direction == Room.Direction.NORTH:
        return Room.Direction.SOUTH
    elif direction == Room.Direction.SOUTH:
        return Room.Direction.NORTH
    elif direction == Room.Direction.EAST:
```

```
        return Room.Direction.WEST
    else:
        return Room.Direction.EAST
```

In the proceeding sections, we'll go over in more detail how our “neighbor” system works.

For now, though, this script provides all the necessary tools to manage the entrance state. We'll jump into the next step which involves actually calling these functions we just created.

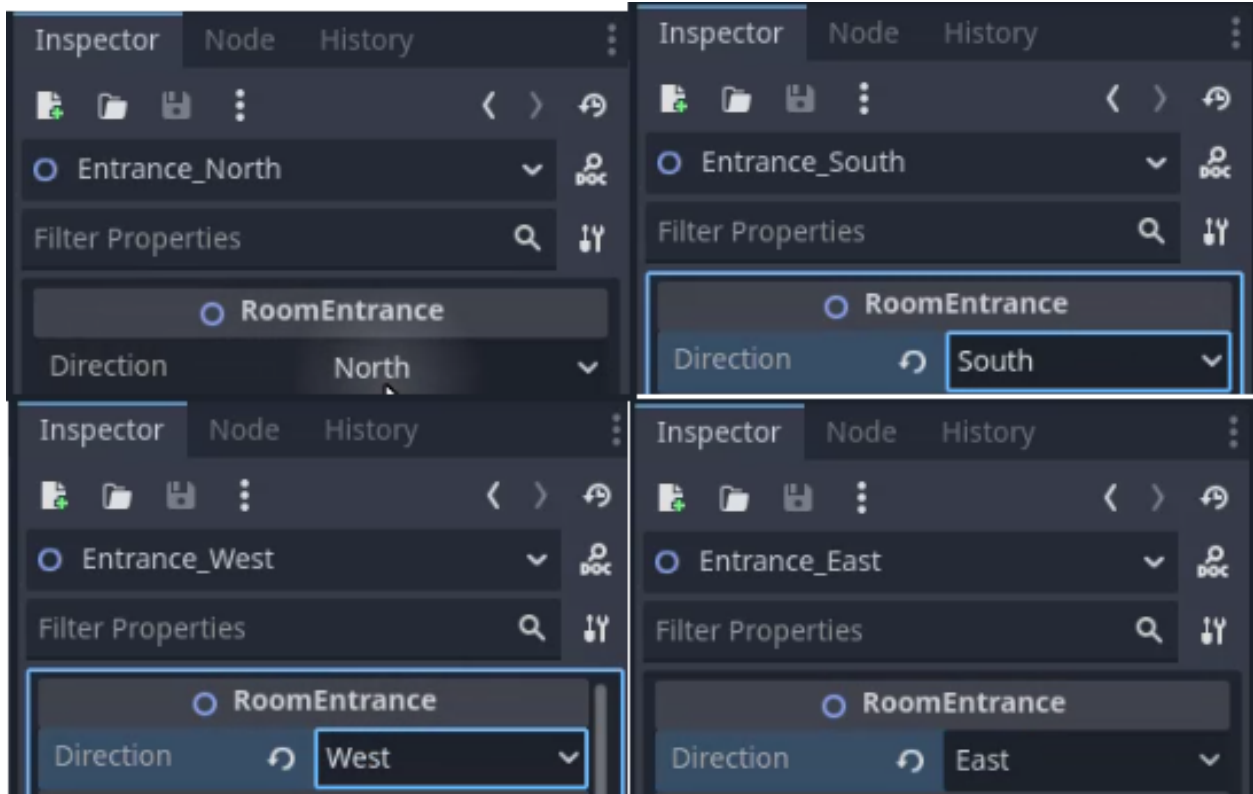
The Room Script

We will now move to the Room script. This script acts as the manager for the entire room, coordinating the four entrances and handling player entry.

Configuring Entrances in the Editor

Before coding, let's make sure each entrance in the room template points in the correct direction. Select each RoomEntrance node (Entrance_North, Entrance_South, etc.) and set its **Direction** property in the Inspector to match.

- Set Entrance_South to South.
- Set Entrance_West to West.
- Set Entrance_East to East.
- Entrance_North should already be North by default.



Room Variables

Now let's open `room.gd` and add the following variables. We need references to all four entrances to control them simultaneously. We also want variables to know if the doors should always be open, and how many enemies are in the room. We don't have any enemies yet, but you'll see how this value comes into play when setting up our room.

```
@export var doors_always_open : bool = false

@onready var entrance_north : RoomEntrance = $Entrance_North
@onready var entrance_south : RoomEntrance = $Entrance_South
@onready var entrance_east : RoomEntrance = $Entrance_East
@onready var entrance_west : RoomEntrance = $Entrance_West

var enemies_in_room : int
```

Room Functions

Next, we will define functions to initialize the room, handle player entry, and manage doors. Like with the room entrances, we won't fill these out yet, but use the skeleton to guide our next steps.

```
func _ready ():  
    pass  
  
func set_neighbor (neighbor_direction : Direction, neighbor_room :  
Room):  
    pass  
  
# called when the player enters the room  
func player_enter (entry_direction : Direction, player :  
CharacterBody2D, first_room : bool = false):  
    pass  
  
func _on_defeat_enemy (enemy):  
    pass  
  
func open_doors ():  
    pass  
  
func close_doors ():  
    pass
```

Opening and Closing Doors

Let's work on the open_doors and close_doors functions first. These functions simply delegate the command to all four entrances. This is what we can use to toggle the state of the doors based on whether enemies are in the room or not.

Note that to avoid any errors with Godot's physics system, we use call_deferred so that this action occurs at the end of the frame.

```
# call_deferred avoids toggling physics bodies mid-callback  
func open_doors():  
    entrance_north.open_door.call_deferred()  
    entrance_south.open_door.call_deferred()  
    entrance_east.open_door.call_deferred()
```

```
entrance_west.open_door.call_deferred()
```

```
func close_doors():  
    entrance_north.close_door.call_deferred()  
    entrance_south.close_door.call_deferred()  
    entrance_east.close_door.call_deferred()  
    entrance_west.close_door.call_deferred()
```

Handling Player Entry

Next, let's work with the `player_enter` function. The `player_enter` function is critical as it is called whenever the player moves into this room. Its responsibilities are:

1. Positioning the player at the correct spawn point (based on where they came from). This is where the entry points we added in the room template come into play. Note that if the room is the first room, we simply spawn the player in the middle.
2. Checking if enemies are present.
3. Closing doors if enemies are present (to lock the player in), or opening them if the room is clear.

```
func player_enter(entry_direction : Direction, player :  
CharacterBody2D, first_room : bool = false):  
    # set their position based on where they entered from  
    if entry_direction == Direction.NORTH:  
        player.global_position =  
entrance_north.player_spawn.global_position  
    elif entry_direction == Direction.SOUTH:  
        player.global_position =  
entrance_south.player_spawn.global_position  
    elif entry_direction == Direction.EAST:  
        player.global_position =  
entrance_east.player_spawn.global_position  
    elif entry_direction == Direction.WEST:  
        player.global_position =  
entrance_west.player_spawn.global_position  
  
    # if this is the first room, spawn the player in the middle  
    if first_room:  
        player.global_position = global_position
```

```

# close the doors if there are enemies to defeat
if enemies_in_room > 0 and not doors_always_open:
    close_doors()
else:
    open_doors()

```

Setting Neighbors

Next, let's dive into the concept of neighbors again. The `set_neighbor` function in the `Room` script acts as a router. It takes a direction and a room, and passes that room to the correct `RoomEntrance`.

```

func set_neighbor(neighbor_direction : Direction, neighbor_room :
Room):
    if neighbor_direction == Direction.NORTH:
        entrance_north.set_neighbor(neighbor_room)
    elif neighbor_direction == Direction.SOUTH:
        entrance_south.set_neighbor(neighbor_room)
    elif neighbor_direction == Direction.EAST:
        entrance_east.set_neighbor(neighbor_room)
    elif neighbor_direction == Direction.WEST:
        entrance_west.set_neighbor(neighbor_room)

```

While we haven't implemented everything, we have enough to proceed to the next major step.

Connecting Room Neighbors

With our room base logic in place, we need to update the `RoomGeneration` script to actually connect these rooms. Currently, the generator places rooms but doesn't tell them who their neighbors are.

Updating Room Instantiation

In `room_generation.gd`, update the `_instantiate_rooms` function. We need to identify the first room (the starting room) so we can set our variable for it appropriately.

```

# instantiate each room scene at their designated position
func _instantiate_rooms():
    for x in range(map_size):
        for y in range(map_size):
            if _get_map(x, y) == false:
                continue

            var room : Room = room_scene.instantiate()
            var is_first_room : bool = first_room_x == x and
first_room_y == y

            get_tree().root.get_node("/root/Main")
                .add_child.call_deferred(room)
            rooms.append(room)

            room.global_position = Vector2(x, y) * room_pos_offset

            if is_first_room:
                first_room = room

```

Helper Functions for Map Position

To connect neighbors, we need to know where each room is on the grid. We'll add two helper functions.

`_get_map_index` converts a room's world position back into grid coordinates (`Vector2i`).

```

func _get_map_index(room: Room) -> Vector2i:
    return Vector2i(room.global_position / room_pos_offset)

```

`get_room_from_map`, meanwhile, finds a specific room object based on grid coordinates.

```

func get_room_from_map(x: int, y: int) -> Room:
    for room in rooms:
        var pos = _get_map_index(room)
        if pos.x != x or pos.y != y:
            continue
        return room
    return null

```

Linking Neighbors

With these helper functions, we can now loop through all generated rooms and check their surroundings. If a room exists to the North, South, East, or West, we connect them.

Add this logic to the end of `_instantiate_rooms`:

```
# set room neighbors
for room in rooms:
    var map_pos : Vector2i = _get_map_index(room)
    var x = map_pos.x
    var y = map_pos.y

    if y > 0 and _get_map(x, y - 1):
        room.set_neighbor.call_deferred(Room.Direction.NORTH,
        get_room_from_map(x, y - 1))

    if y < map_size - 1 and _get_map(x, y + 1):
        room.set_neighbor.call_deferred(Room.Direction.SOUTH,
        get_room_from_map(x, y + 1))

    if x < map_size - 1 and _get_map(x + 1, y):
        room.set_neighbor.call_deferred(Room.Direction.EAST,
        get_room_from_map(x + 1, y))

    if x > 0 and _get_map(x - 1, y):
        room.set_neighbor.call_deferred(Room.Direction.WEST,
        get_room_from_map(x - 1, y))
```

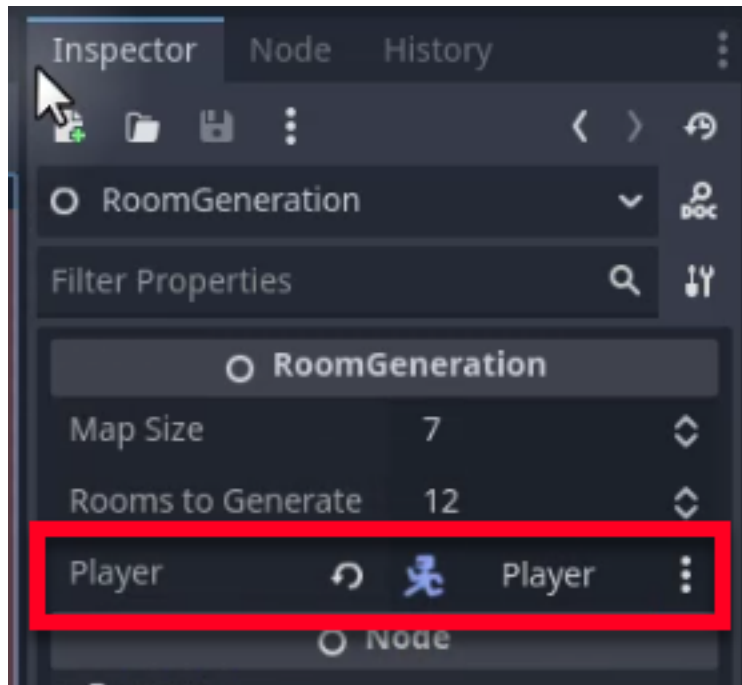
We use `call_deferred` here to ensure that all rooms are fully initialized before we start calling functions on them.

Spawning the Player

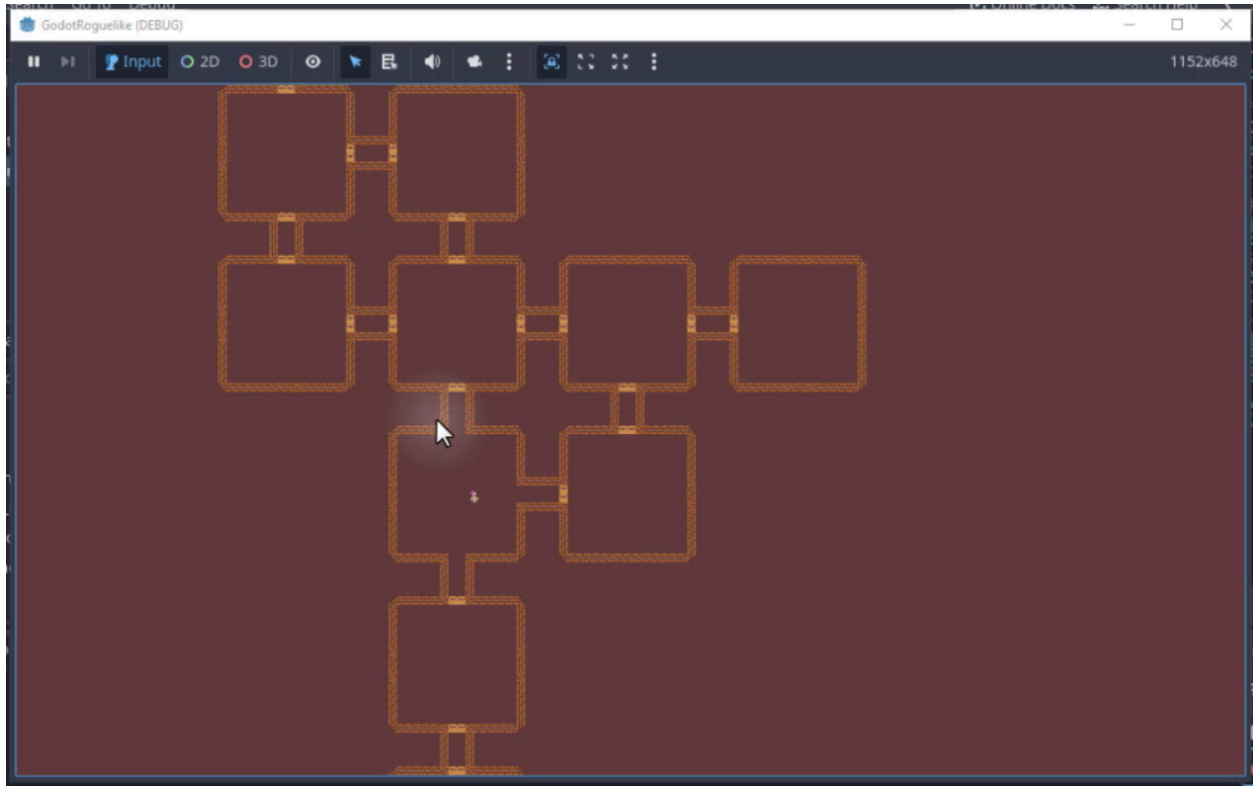
Finally, we need to place the player in the starting room. To do so, add this line to the very end of `_instantiate_rooms` in `room_generation.gd`:

```
# put the player in the first room
first_room.player_enter.call_deferred(Room.Direction.NORTH,
player, true)
```


If you haven't done so on your own already, make sure you have assigned your Player node to the Player export variable in the RoomGeneration node in the Inspector.



If you run the game now, you should see the map generated with correct walls and doors. The player will spawn in the center room.



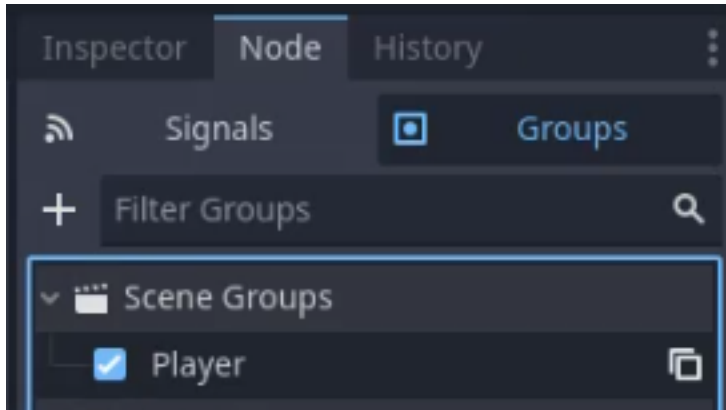
Player Room Traversal

The final piece of the puzzle is allowing the player to walk through the tunnels to enter adjacent rooms.

Identifying the Player

Before we set up our logic, we need to ensure that only the player triggers the room transition. To do this cleanly, we will use Godot's Group system.

1. Open your Player scene.
2. Select the root node (Player).
3. Go to the **Node** tab next to the Inspector.
4. Select **Groups**.
5. Add a new group named Player.



Godot Insight: Groups for node identification

Groups are a lightweight way to tag nodes. Any node can belong to any number of groups, and you can query group membership at runtime with `is_in_group("name")` or retrieve every node in a group with `get_tree().get_nodes_in_group("name")`. Unlike collision layers, groups are not limited to physics; they work on any node type.

We use the “Player” group here so that the exit trigger can confirm it was the player who entered, not a projectile or enemy. This is simpler and more robust than comparing node names or types.

Learn more: [Node](#)

(https://docs.godotengine.org/en/4.6/classes/class_node.html)

Triggering the Transition

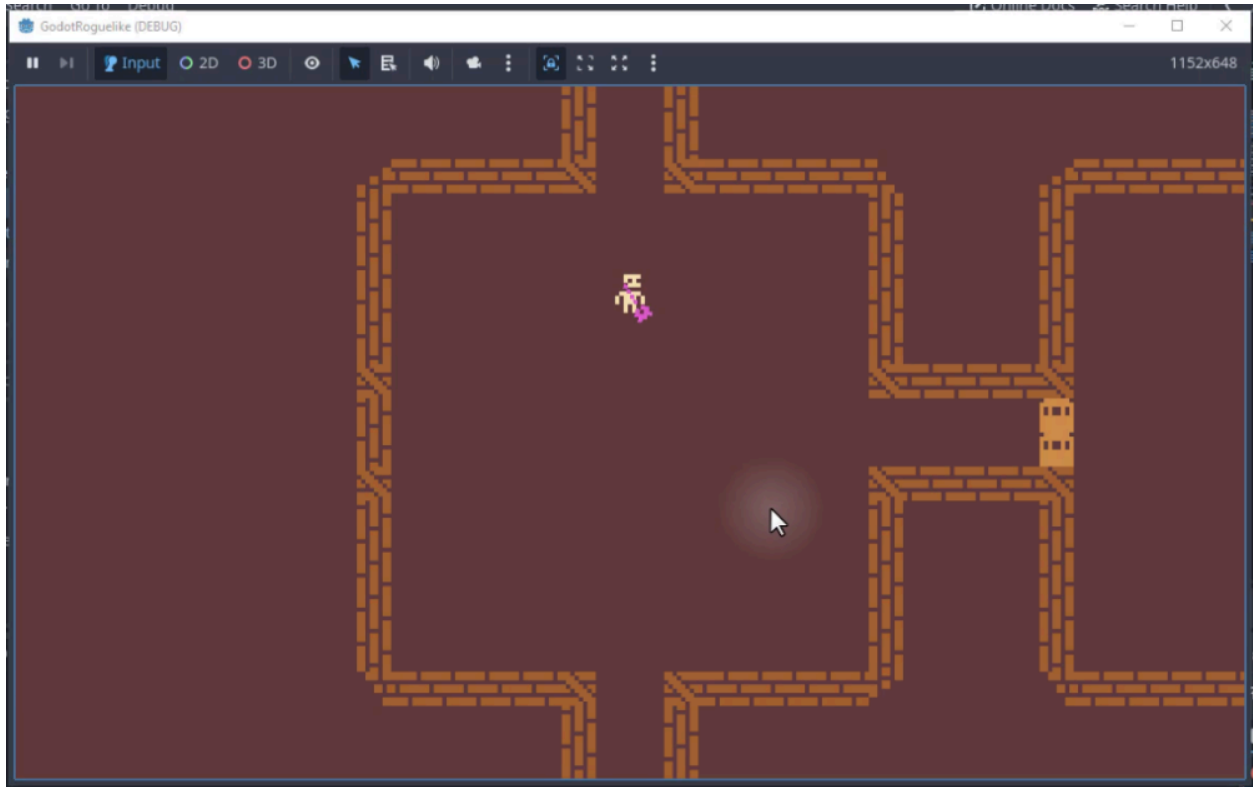
Now, let's return to `room_entrance.gd` and implement the `_on_body_entered_exit_trigger` function finally.

```
# Transition the player into the neighboring room
func _on_body_entered_exit_trigger(body):
    if body.is_in_group("Player"):
        neighbor.player_enter(_get_neighbor_entry_direction(), body)
```

When the player enters the trigger, we call `player_enter` on the neighbor room, passing the correct entry direction.

Conclusion

With these changes, your room navigation system is complete. The player can walk through tunnels, triggering a transition to the next room. Doors will open and close correctly, and barriers will block paths where no rooms exist.



In the next chapter, we will set up the player's combat abilities in preparation for adding actual enemies to our roguelike.

Combat Mechanics and Signals

In this chapter, we will implement the player's ability to defend themselves by shooting projectiles. This involves setting up a projectile scene, handling aiming with the mouse, and spawning projectiles that travel towards the cursor.

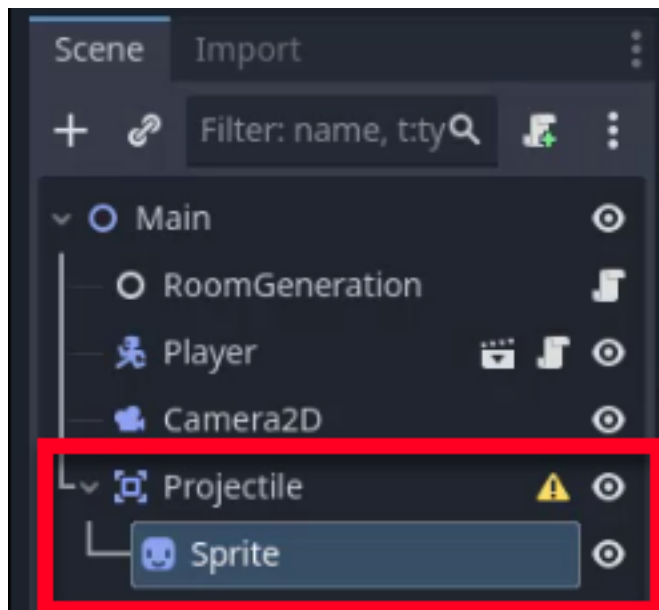
Player Shooting

To enable combat, we first need to create the object that the player will fire. We will create a projectile scene that handles its own movement and collision detection.

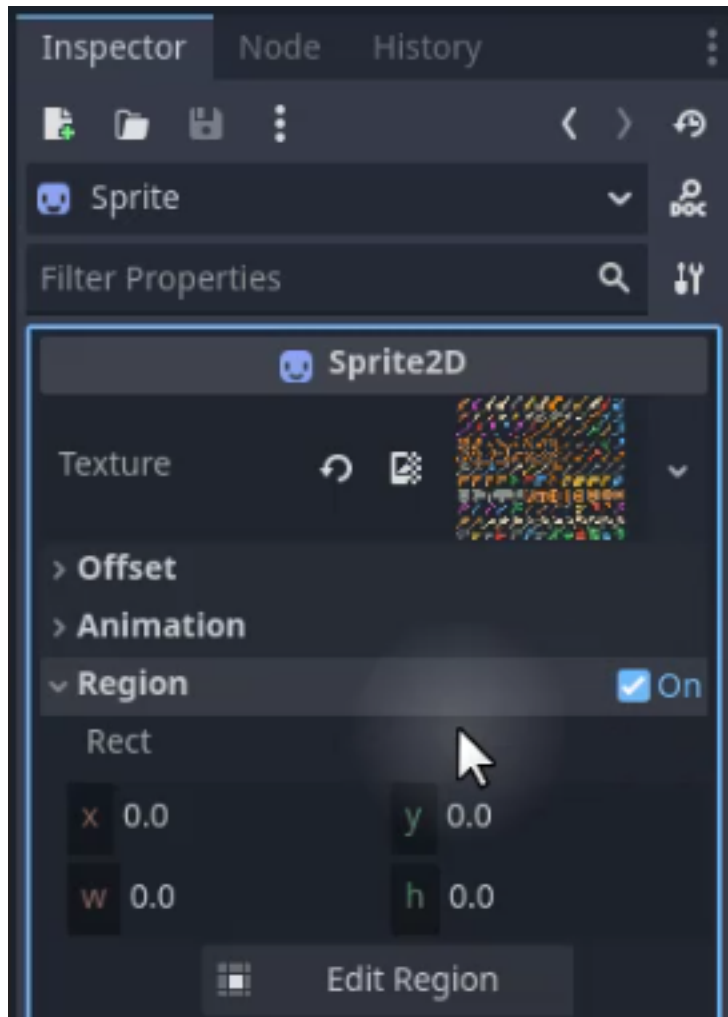
Creating the Projectile Scene

The projectile will be a physics object that detects when it hits something. An Area2D node is perfect for this, as it detects overlaps without handling complex physics collisions like bouncing.

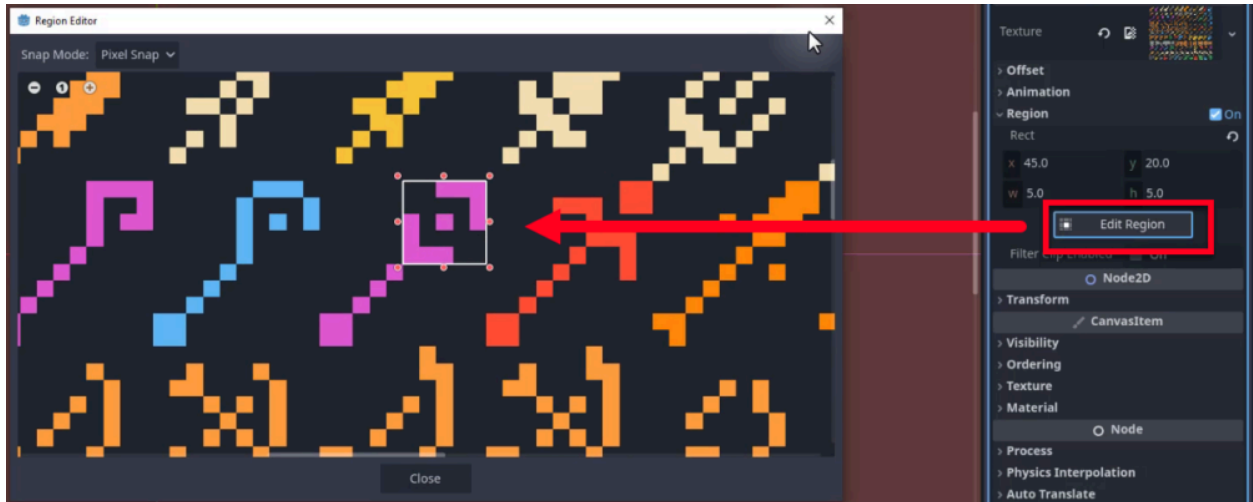
Start by creating a new scene with an Area2D as the root node and renaming it to Projectile. Then, add a Sprite2D node as a child and rename it to Sprite.



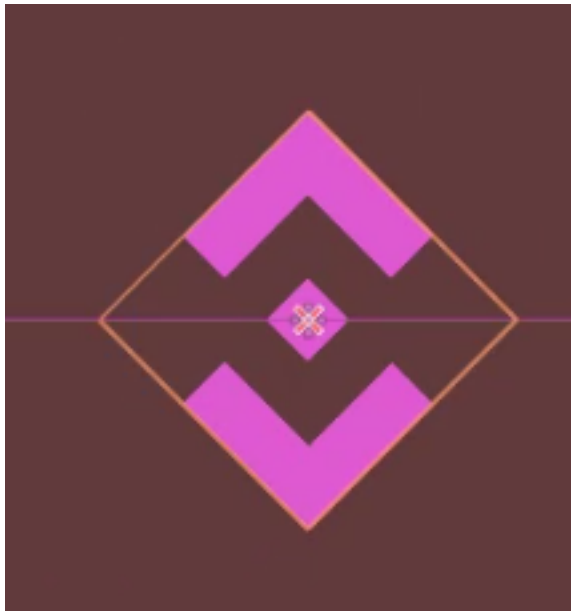
Next, we need to assign a visual to our projectile. Select the Sprite node and, in the Inspector, drag the `Weapons.png` file from your sprites folder into the **Texture** property.



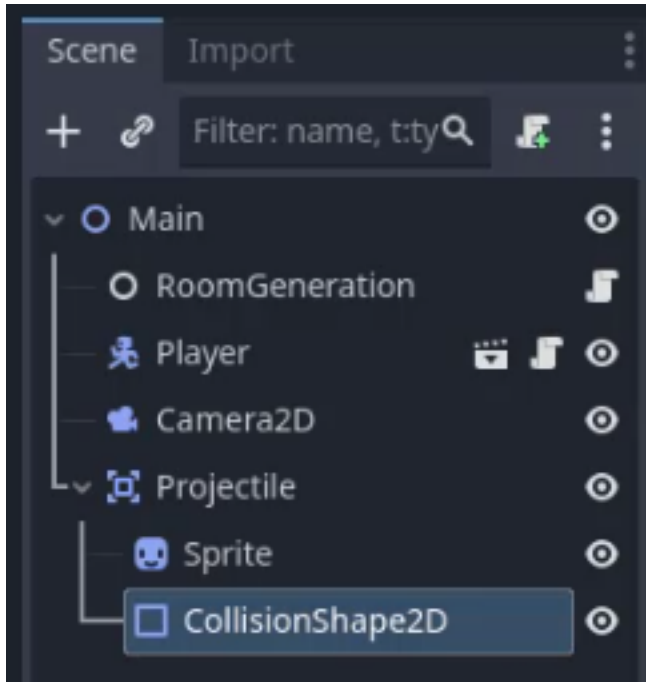
Since we are using a sprite sheet, we need to isolate the specific projectile image (like we did with the player). To do so, enable the **Region** property in the Inspector, and then click **Edit Region** to open the Region Editor. From here, simply select a part of the texture that looks like a projectile (e.g., a small energy ball or bullet).



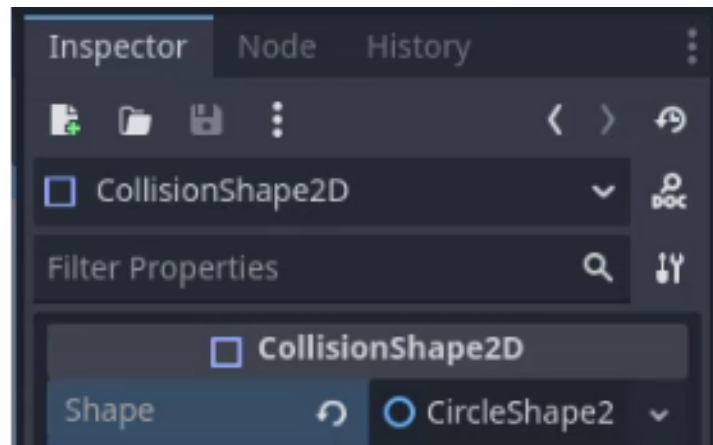
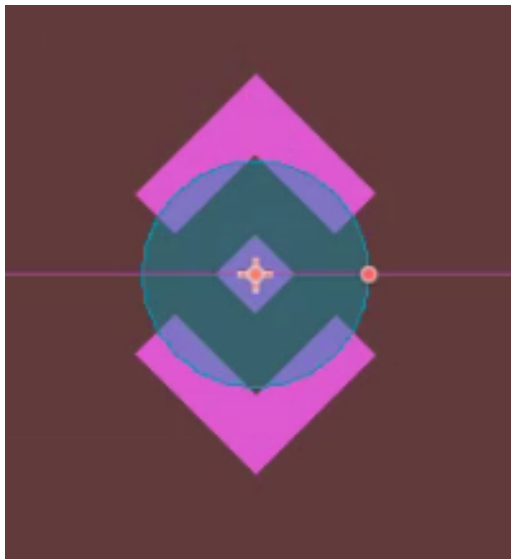
Once the region is selected, rotate the sprite so that it points upwards. This standardizes the orientation, making it easier to calculate rotation in our scripts.



Finally, we need to define the collision boundary. Add a `CollisionShape2D` node as a child of `Projectile`.



For this one, we'll set the **Shape** property to `CircleShape2D`. We can then adjust the radius to match the size of our projectile sprite.



Your scene hierarchy should now look like this:

```

Projectile (Area2D)
├── Sprite (Sprite2D)
└── CollisionShape2D
  
```


Godot Insight: Area2D for detection zones

Area2D detects overlaps but does not physically block or push other objects. This makes it ideal for projectiles, item pickups, trigger zones, and any situation where you want to know “did something enter this space?” without affecting its movement. In contrast, StaticBody2D and CharacterBody2D participate in solid-body collisions that stop or redirect movement.

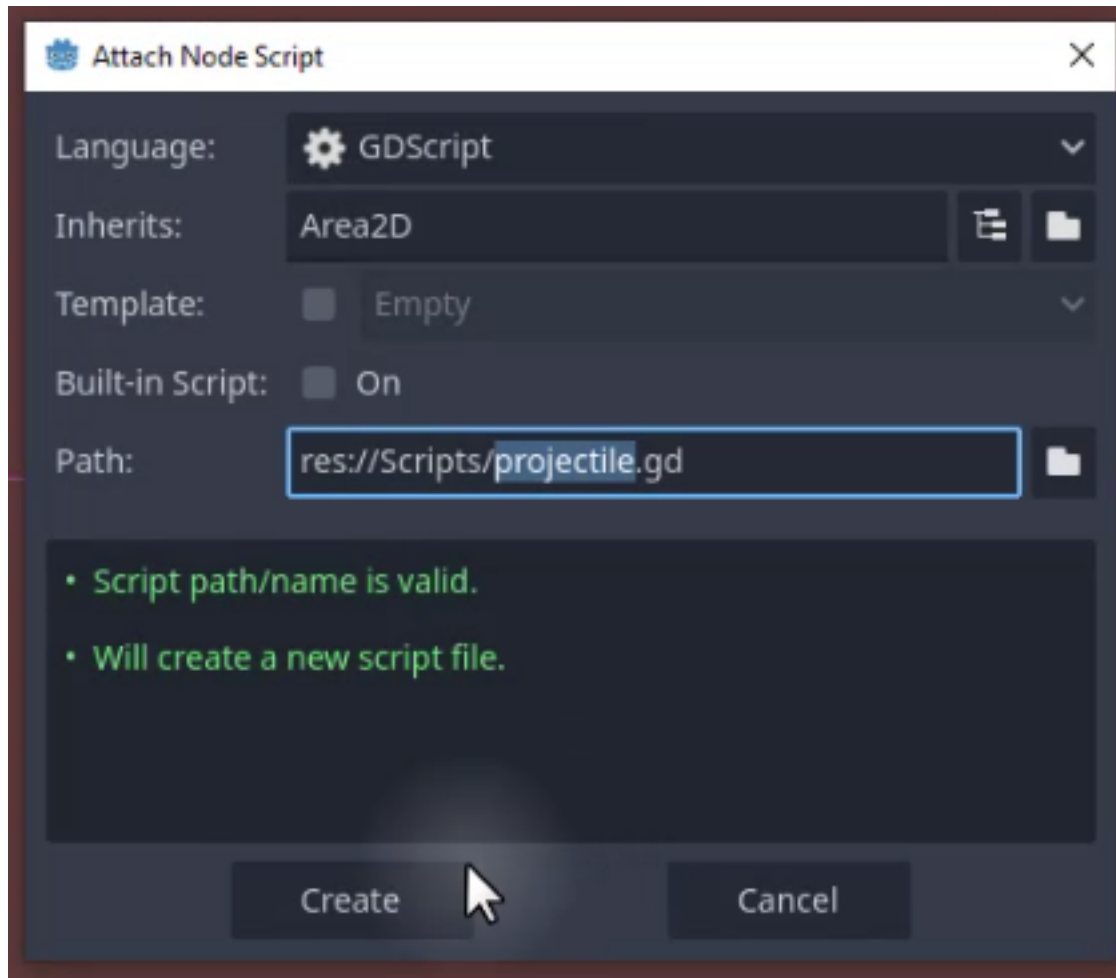
Our projectile uses Area2D because it should pass through space and react to contact (deal damage, then disappear), not bounce off targets like a physics object would.

Learn more: [Area2D](#)

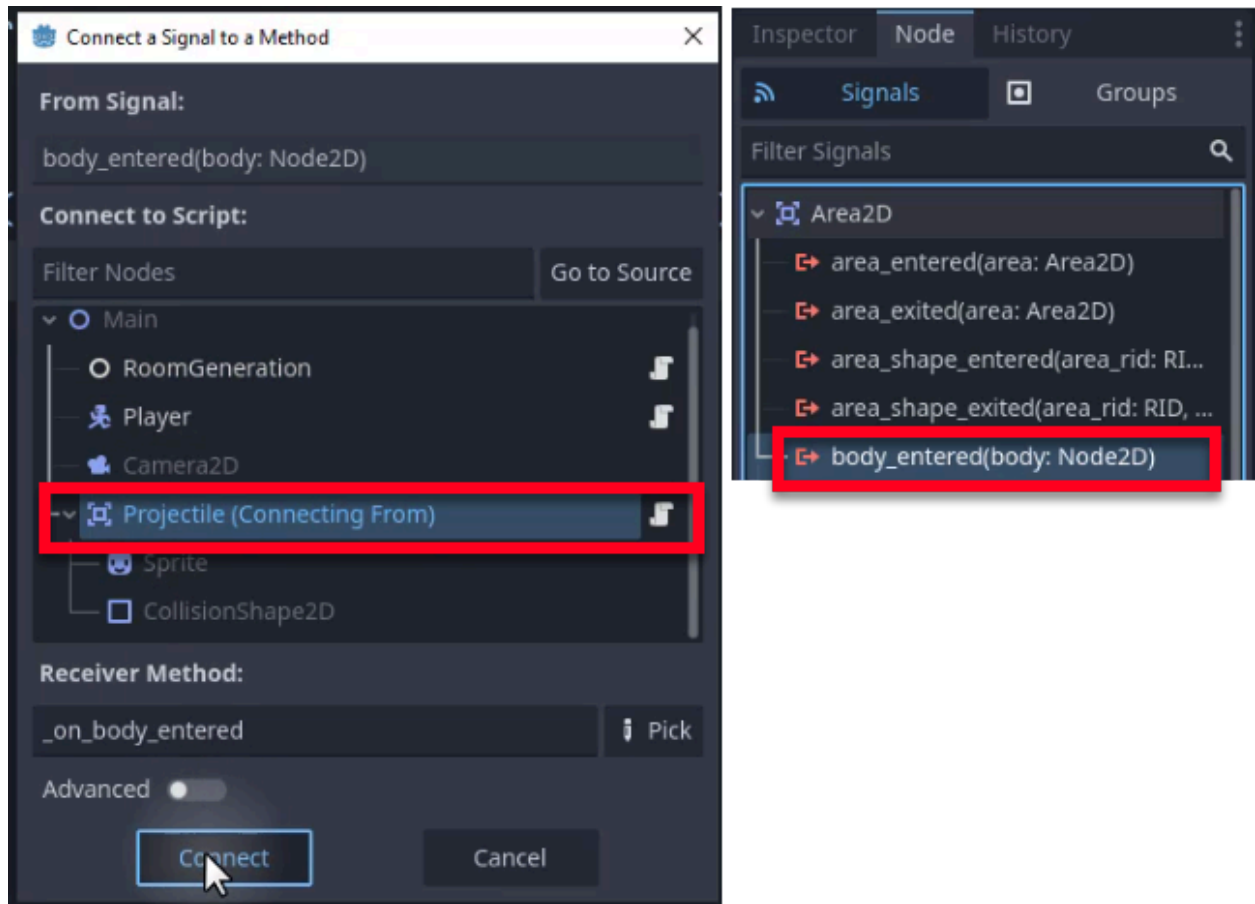
(https://docs.godotengine.org/en/4.6/classes/class_area2d.html)

Scripting the Projectile

Now we need to give the projectile logic to move and inflict damage. Attach a new script to the Projectile node and name it `projectile.gd`.



As mentioned, we need to detect when the projectile hits a body (like an enemy or a wall). To do this, we will connect the Area2D's built-in signal. Select the Projectile node and open the **Node** tab (Signals). Then, double-click the `body_entered` signal and connect it to the `projectile.gd` script.



Add the following code to the script, whose logic we'll explain below:

```
extends Area2D
```

```
@export var base_speed : float = 100
var owner_character : CharacterBody2D
```

```
func _process(delta):
    # Move along local up (-Y); rotation determines travel direction
    translate(-transform.y * base_speed * delta)
```

```
func _on_body_entered(body):
    # Ignore the character who fired this projectile
    if body == owner_character:
        return

    # Duck-typed damage: any body with take_damage() can be hurt
    if body.has_method("take_damage"):
        body.take_damage(1)
```

```
queue_free()
```

Here is how the script works:

- `base_speed`: Controls how fast the projectile travels.
- `owner_character`: Stores a reference to who fired the projectile (the player). This will help us prevent the projectile from hitting the player immediately upon spawning.
- `_process`: Moves the projectile forward along its local Y-axis (upwards relative to the sprite).
- `_on_body_entered`: When a collision occurs, it checks if the body is the owner. If not, it attempts to deal damage and then destroys itself (`queue_free`).

Updating the Player

As seen, the projectile script calls a `take_damage` function on the body it hits. Currently, our player script does not have this function, which means the player wouldn't take any damage.

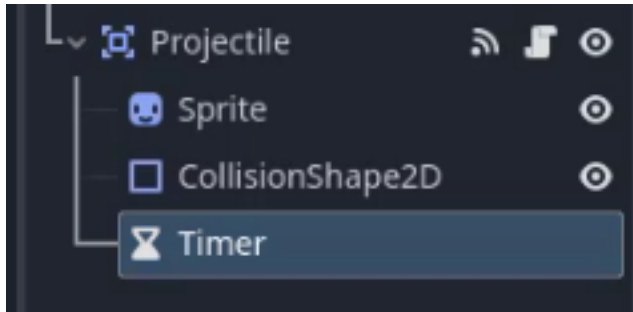
To fix this, let's open `player.gd` and add this placeholder function. We will implement the full health logic in a later chapter.

```
func take_damage (amount : int):  
    pass
```

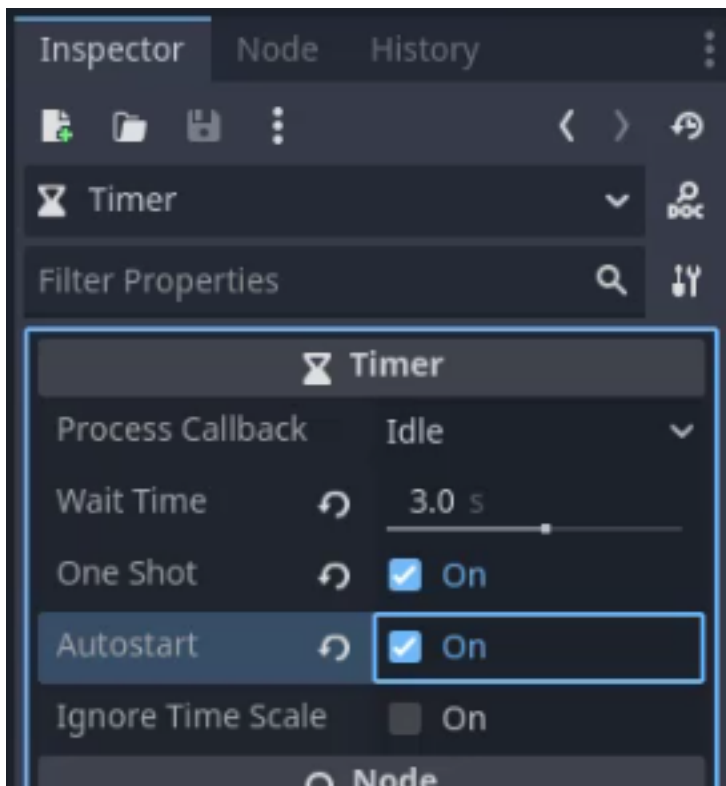
Projectile Lifecycle

Without cleanup, each missed projectile stays in the scene tree as a live node, processing every frame even after flying off-screen. To prevent this buildup, we should destroy projectiles after a certain amount of time.

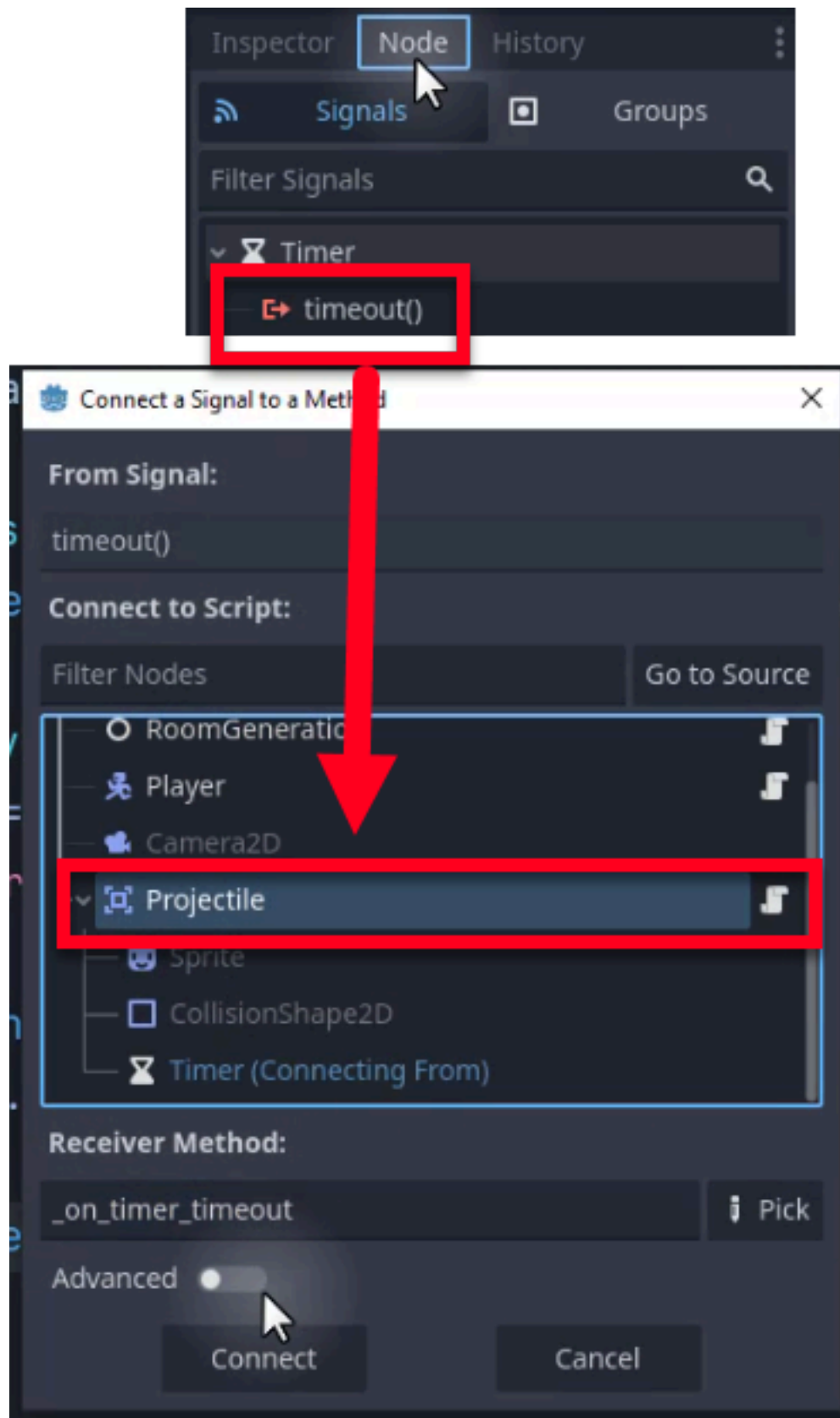
To do this, first add a `Timer` node as a child of `Projectile`.



We'll set the **Wait Time** to 3 seconds and enable **Autostart** and **One Shot**.



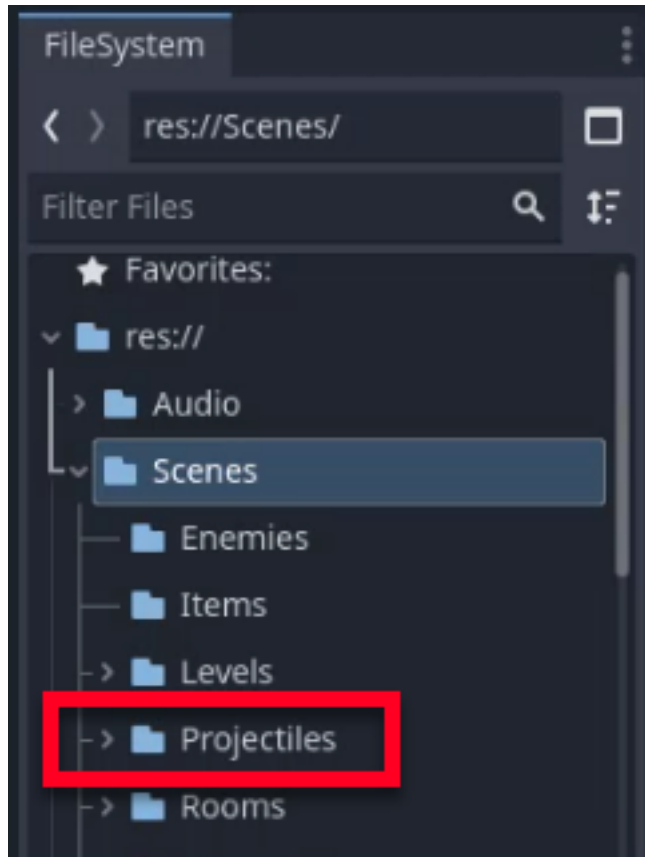
Now, we can connect the timeout signal of the Timer to the projectile script.



In the generated function, simply delete the projectile:

```
func _on_timer_timeout():  
    queue_free()
```

Our projectile itself is set up, so save the scene as `projectile.tscn` in a new folder `Scenes/Projectiles`.



Implementing Shooting in Player

Now that we have a projectile, we need to let the player fire it. As such, let's open the `player.gd` script.

First, we need to add the necessary variables at the top of the script:

```
@export var shoot_rate : float = 0.4  
var last_shoot_time : float
```

```
var projectile_scene : PackedScene =
preload("res://Scenes/Projectiles/projectile.tscn")
```

These variables work as follows:

- shoot_rate: The minimum time (in seconds) between shots.
- last_shoot_time: Stores the timestamp of the last shot.
- projectile_scene: Preloads the projectile resource so we can instantiate it.

Next, update the `_process` function to handle shooting input. We already calculate the direction to the mouse and rotate the weapon itself. Thus, now we just need to check for the attack button and make sure we can shoot based on the time and `shoot_rate`.

```
func _process(delta):
    var mouse_pos : Vector2 = get_global_mouse_position()
    var mouse_dir : Vector2 = (mouse_pos -
global_position).normalized()
    weapon_origin.rotation_degrees = rad_to_deg(mouse_dir.angle()) +
90

    sprite.flip_h = mouse_dir.x < 0

    if Input.is_action_pressed("attack"):
        # Enforce cooldown between shots
        if Time.get_unix_time_from_system() - last_shoot_time >
shoot_rate:
            _shoot()
```

Finally, we need to implement the `_shoot` function to spawn the projectile.

```
func _shoot():
    last_shoot_time = Time.get_unix_time_from_system()

    # Spawn a projectile at the muzzle and orient it to match the weapon
    var proj = projectile_scene.instantiate()
    get_tree().root.add_child(proj)
    proj.global_position = muzzle.global_position
    proj.rotation = weapon_origin.rotation
```



```
# Tag projectile so it won't damage the player who fired it  
proj.owner_character = self
```

This function:

1. Updates the shoot timer.
2. Creates a new instance of the projectile.
3. Adds it to the scene root (so it moves independently of the player).
4. Positions it at the Muzzle node we created earlier.
5. Matches the projectile's rotation to the weapon.
6. Sets the player as the owner to prevent self-damage.

Run the game, and you should be able to aim with the mouse and fire projectiles by left-clicking.



Once you're done with any testing, make sure no projectiles are left on the Player scene itself; otherwise, the projectile will damage the player when they load in. The player should only spawn projectiles via code, not have one attached by default.

Our player is now ready for combat with our enemies once we add them. In the next chapter, we're going to set up some essential architecture for the rest of our project to ensure adding our enemies goes smoothly.

Global Signals

In this chapter, we will introduce a powerful architectural pattern in Godot called Global Signals. This system will allow different parts of our game (such as the rooms and the camera) to communicate efficiently without creating tight dependencies. As we build our game, you'll see why this pattern makes adding features like enemies much easier to manage.

What are Global Signals?

As our game grows, different systems need to communicate. For example, when the player enters a room, the camera needs to move, enemies need to activate, and the UI needs to update. Connecting all these nodes directly can lead to “spaghetti code,” where everything depends on everything else. This sort of chained dependency makes our game very prone to errors.

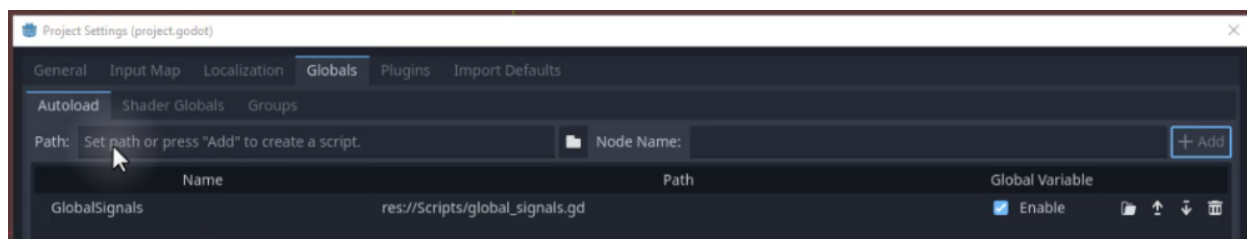
To solve this, we will use **Global Signals**. This involves creating a central “hub” script that defines signals. Any node can emit a signal through this hub, and any other node can listen for it.

Setting Up the Autoload

We will use Godot's **AutoLoad** feature to create a globally accessible singleton. To set this up:

1. Go to **Project -> Project Settings** and navigate to the **Globals** tab, and then the **AutoLoad** tab.
2. Enter `GlobalSignals` as the **Node Name**.
3. Create a new script named `global_signals.gd` in your **Scripts** folder, assign it to the **Path**, and click **Add**.

Once created, ensure the **Global Variable** checkbox is checked. This allows us to access `GlobalSignals` from anywhere in our code.



Godot Insight: AutoLoad singletons

An AutoLoad is a scene or script that Godot loads automatically before any other scene and keeps alive for the entire lifetime of the application. Because it is always present, any script can access it by name (for example, `GlobalSignals`) without needing a node reference. This makes AutoLoads perfect for global state, event buses, and services that multiple systems depend on.

The “Global Variable” checkbox we enabled is what lets us write `GlobalSignals.OnPlayerEnterRoom` anywhere in the codebase. Without it, the script would still load automatically, but we would have to look it up manually in the scene tree.

Learn more: [Node](https://docs.godotengine.org/en/4.6/classes/class_node.html)
(https://docs.godotengine.org/en/4.6/classes/class_node.html)

Defining the Signal

With our AutoLoad created, we can now define our first signal. Open the `global_signals.gd` script and define a signal that carries information about the room the player just entered. You’ll see below that the script here is quite simple.

```
# Global event bus (AutoLoad): lets any node broadcast or listen for  
game events  
# without needing direct references to other nodes  
extends Node  
  
signal OnPlayerEnterRoom (room : Room)
```

Emitting the Signal

Now we need to trigger this signal. The best place to do this is in the `Room` script, specifically when the player enters.

Open `room.gd` and find the `player_enter` function. Add a line to emit the signal before we close or open the doors:

```

func player_enter(entry_direction : Direction, player :
CharacterBody2D, first_room : bool = false):
    # ... Existing code ...

    # If this is the first room, spawn the player in the middle
    if first_room:
        player.global_position = global_position

    # Emit the global signal
    GlobalSignals.OnPlayerEnterRoom.emit(self)

    # Close the doors if there are enemies to defeat
    if enemies_in_room > 0 and not doors_always_open:
        close_doors()
    else:
        open_doors()

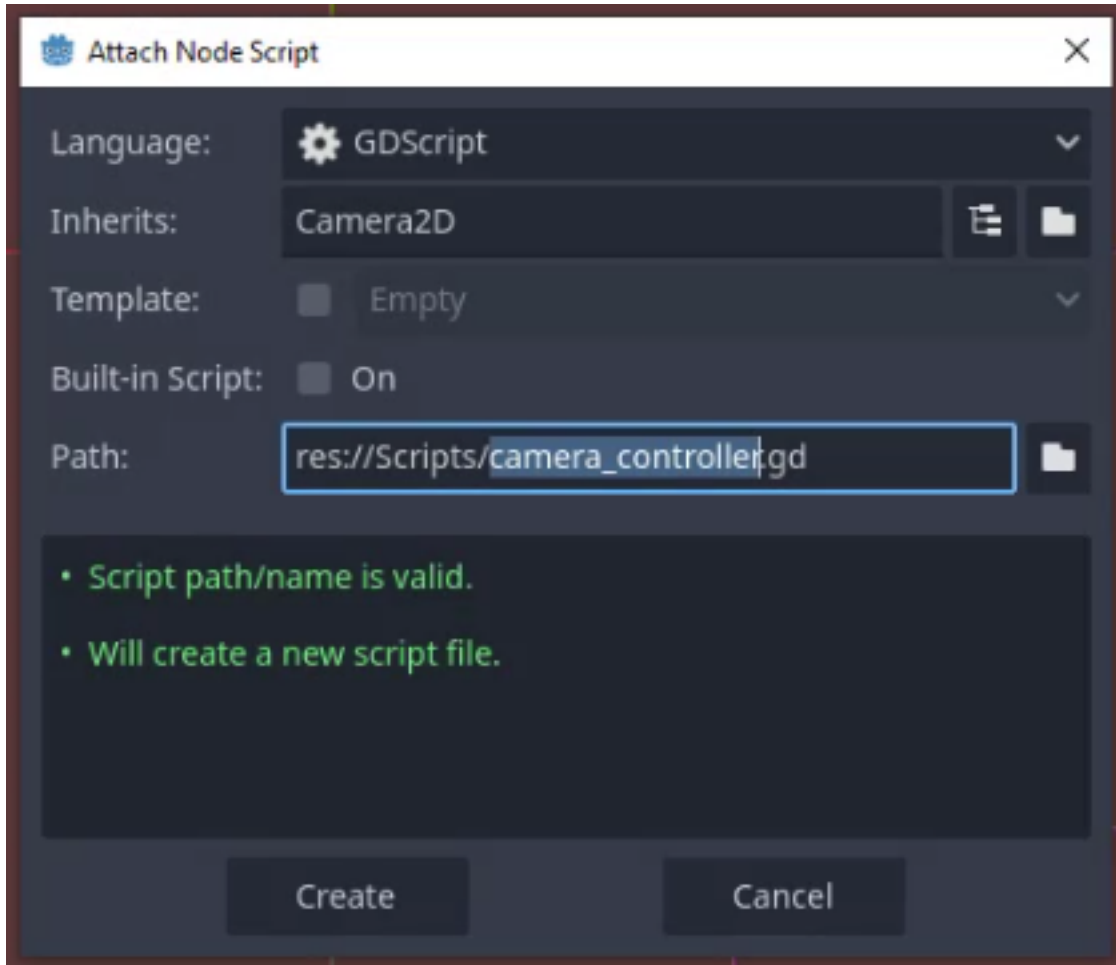
```

Now, whenever `player_enter` runs, the entire game is notified via `GlobalSignals`.

Connecting the Camera

Let's use this signal to control the camera as a practical example. We want the camera to snap to the center of the new room whenever the player enters it.

First, select the `Camera2D` node in your Main scene and attach a new script named `camera_controller.gd`.



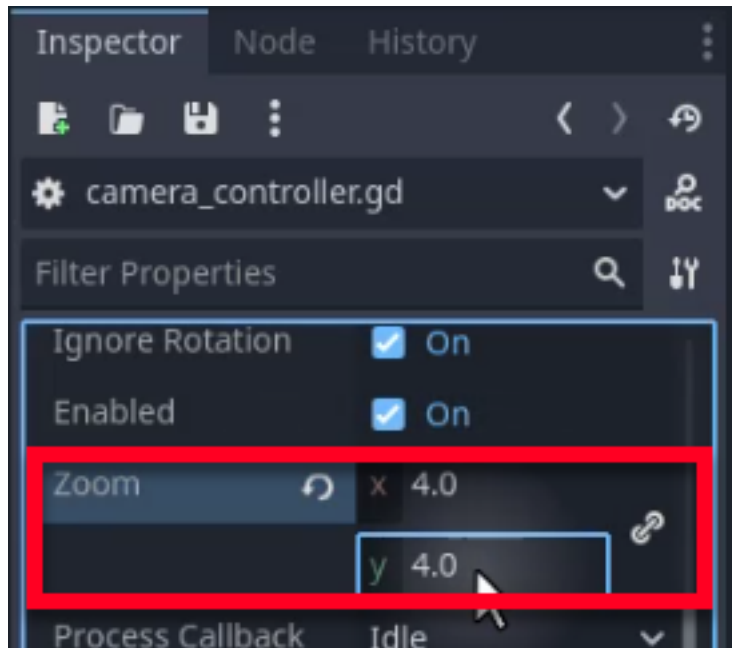
In this script, we connect to the global signal in `_ready`. When the signal is received, the `_on_player_enter_room` function is called, updating the camera's position to match the room's position.

```
extends Camera2D
```

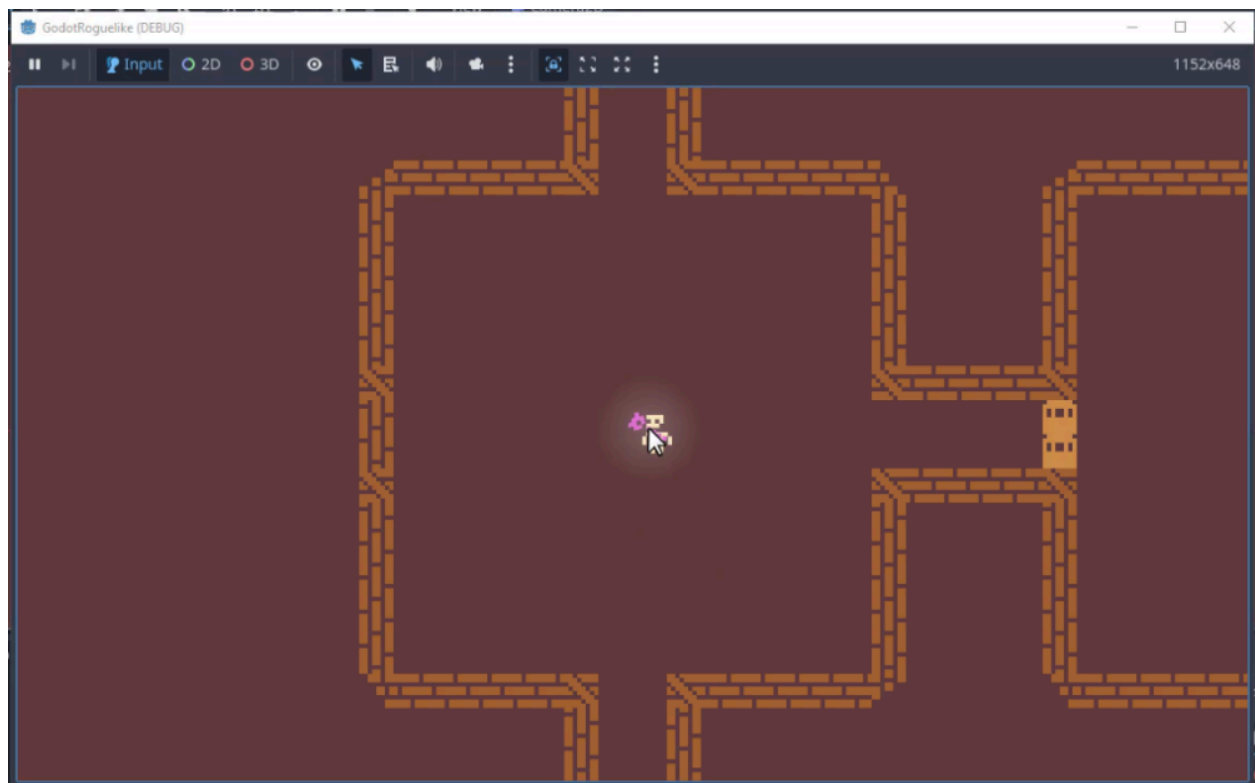
```
func _ready():  
    # Listen for room changes via the global event bus  
    GlobalSignals.OnPlayerEnterRoom.connect(_on_player_enter_room)
```

```
func _on_player_enter_room(room : Room):  
    # Snap the camera to the center of the newly entered room  
    global_position = room.global_position
```

To verify this works, reset the camera zoom in the Inspector to 4 (since we zoomed it out in the previous chapter for debugging).



Run the game. As you walk between rooms, the camera should instantly snap to center on the new room.



Benefits of Global Signals

Using this pattern offers several advantages:

- **Decoupling:** The room script doesn't need to know the camera exists. It just announces "The player entered me!" and anyone interested can respond.
- **Scalability:** We can easily add more listeners later. For example, a UI script could listen to the same signal to update the mini-map name.
- **Simplicity:** We avoid passing references (like the camera node) down through multiple layers of the scene tree.

In the next chapter, we will expand on our combat system by creating enemies for the player to fight.

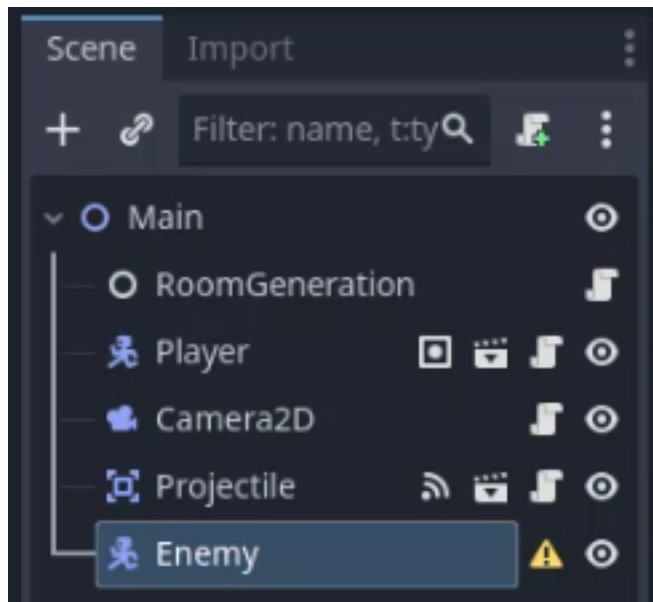
Creating Enemies

In this chapter, we will bring our dungeon to life by adding enemies. Enemies are essential for adding challenge and excitement to the game. We will create an enemy character, add the necessary components for visuals and physics, and write a script to control its behavior. We will also implement a system where enemies only activate when the player enters their room, and doors remain locked until all enemies are defeated.

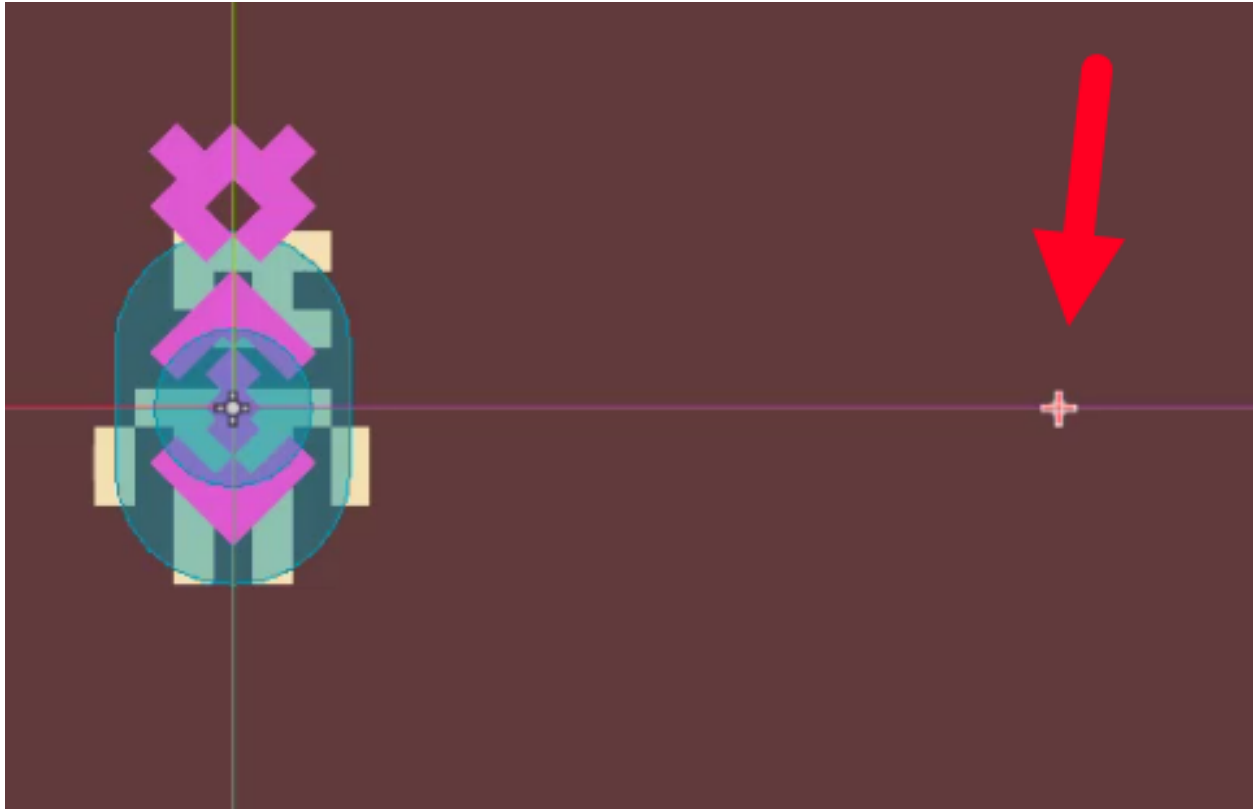
Creating the Enemy Character

To begin, we need to create a new scene for our enemy. This process is similar to how we set up the player.

First, create a new `CharacterBody2D` node and name it `Enemy`. Make sure to save the scene as `enemy.tscn` in the `Scenes/Enemies` folder.



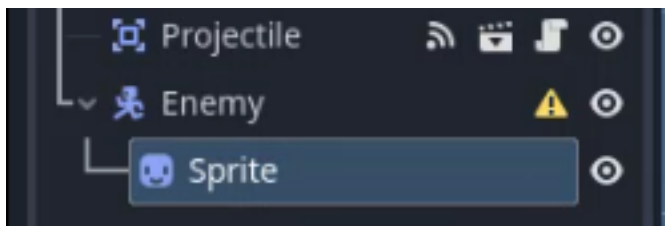
For now, it is helpful to position the enemy character to the right side of the screen initially so it does not overlap with other objects while we configure it.



Adding Components

Next, we will add the visual and physical components to our enemy character.

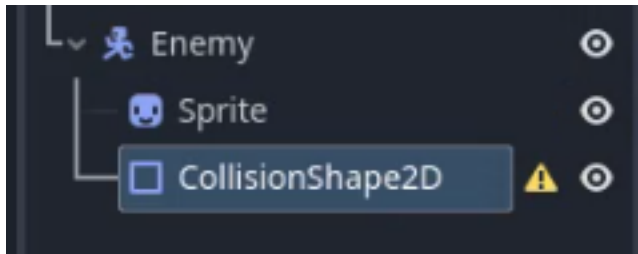
To begin, add a `Sprite2D` child node to the enemy and name it `Sprite`.



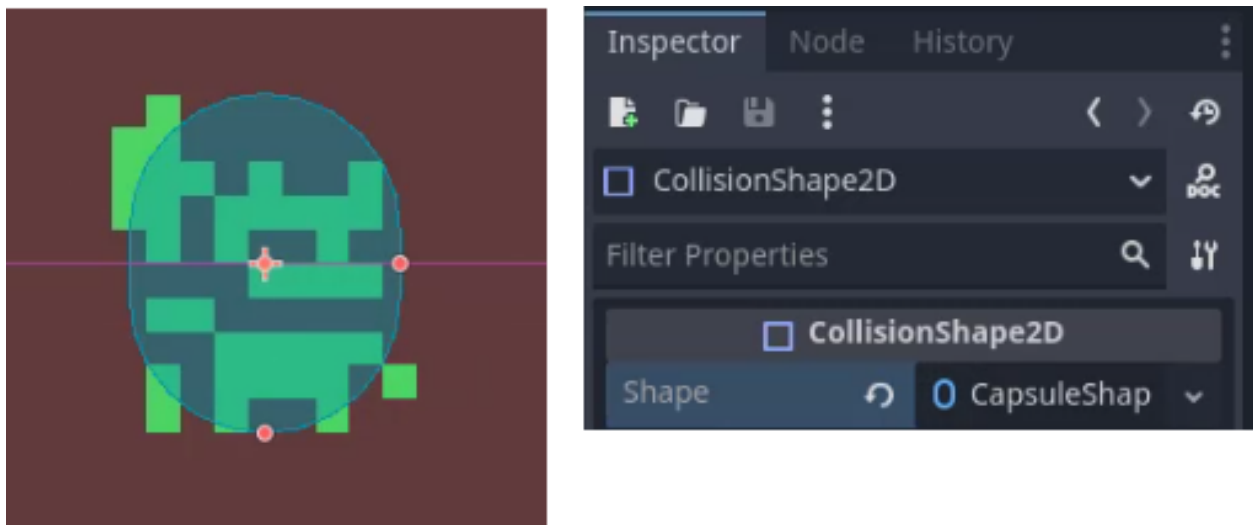
For the sprite's texture, we will use the `Creatures.png` file from the assets folder. Since this is a sprite sheet, enable **Region** editing in the Inspector and select a specific enemy sprite, such as the green slime or goblin.



Now we need to configure the collision shape. Add a `CollisionShape2D` child node to the enemy and set its **Shape** to `CapsuleShape2D`.

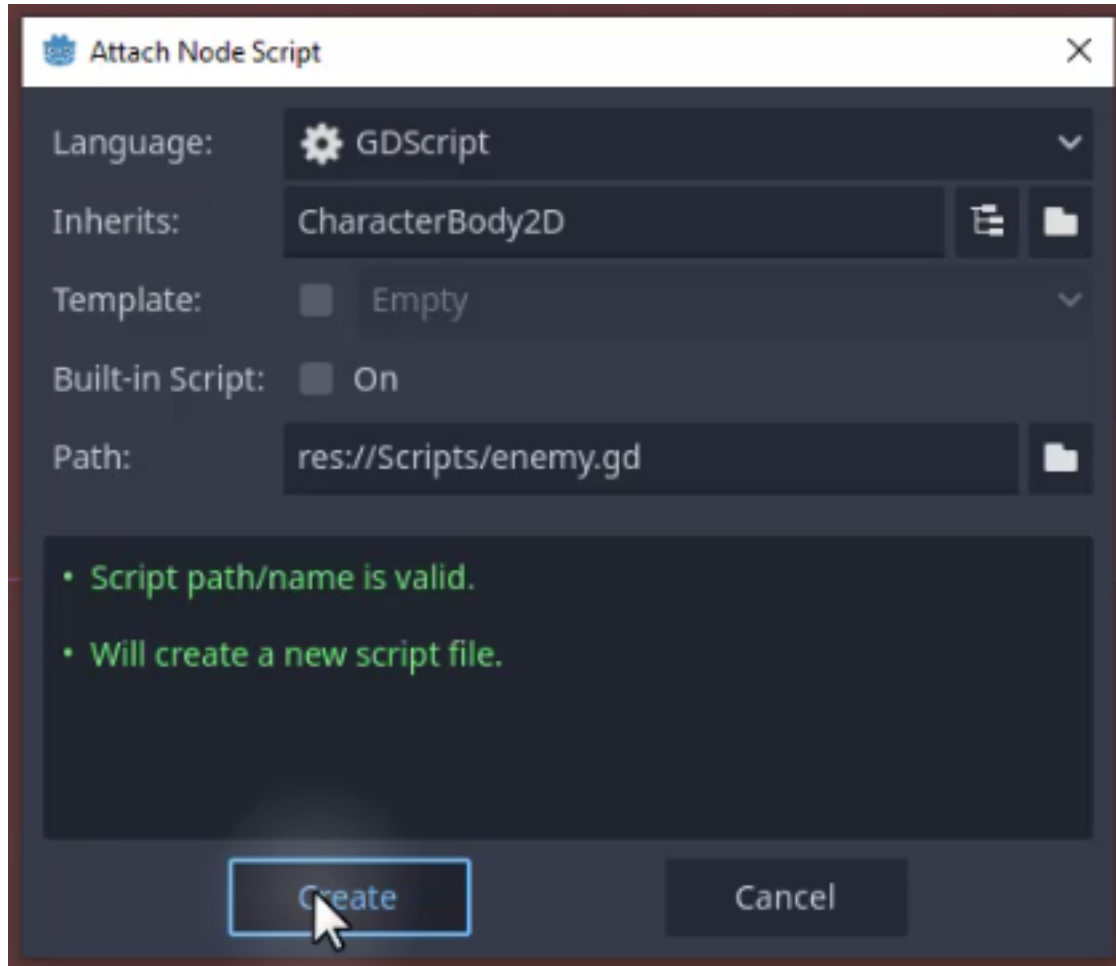


In the viewport, adjust the size of the capsule to fit the enemy sprite you selected.



Scripting the Enemy

With the scene set up, we need to add a script to control the enemy's behavior. Select the Enemy node and attach a new script named `enemy.gd`.



Defining Variables

Inside the script, we will define the properties that control the enemy's stats and state. Add the following code to `enemy.gd`:

```
extends CharacterBody2D

@export var cur_hp : int = 4
@export var max_hp : int = 4
@export var move_speed : float = 20

@export var attack_damage : int = 1
```

```

@export var attack_range : float = 10
@export var attack_rate : float = 0.5
var last_attack_time : float

@onready var sprite : Sprite2D = $Sprite

var room : Room
var is_active : bool = false
var player : CharacterBody2D

var player_direction : Vector2
var player_distance : float

```

These variables track the enemy's health, movement speed, and combat stats. We also keep references to the player, the room the enemy belongs to, and whether the enemy is currently active.

Defining Functions

We will implement several functions to handle the enemy's lifecycle and behavior. Let's start by defining the structure, and we'll implement each function in the subsequent sections:

```

func _ready ():
    pass

func initialize (in_room : Room):
    pass

func _on_player_enter_room (player_room : Room):
    pass

func _physics_process(delta):
    pass

func _try_attack ():
    pass

func take_damage (amount : int):
    pass

func die ():
    pass

```

Movement Logic

The most basic behavior for our enemy is to chase the player. We will implement this in the `_physics_process` function. The logic for this is as follows:

1. Calculate the direction and distance to the player.
2. Flip the sprite to face the player.
3. Move the enemy towards the player.

Update `_physics_process` with the following code:

```
func _physics_process(delta):
    player_direction =
global_position.direction_to(player.global_position)
    player_distance =
global_position.distance_to(player.global_position)

    sprite.flip_h = player_direction.x < 0

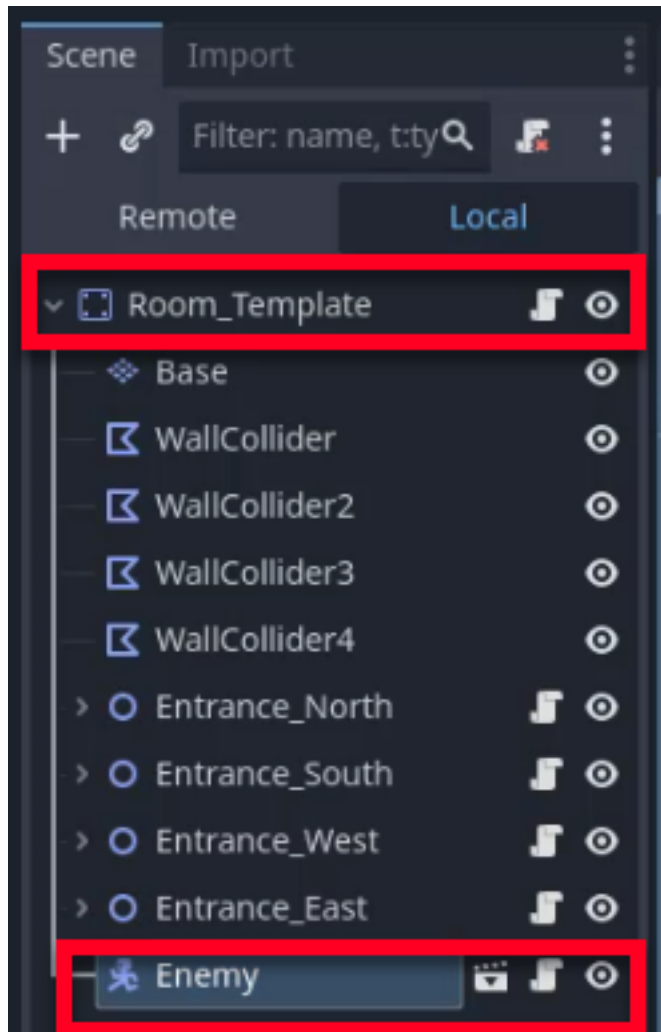
    velocity = player_direction * move_speed
    move_and_slide()
```

Connecting the Player

For the movement logic to work, the enemy needs to know who the player is. We can find the player node in the `_ready` function using Godot's group system (assuming you added the player to the "Player" group in the previous chapter).

```
func _ready():
    # Find the player node via the "Player" group instead of a hardcoded path
    player = get_tree().get_first_node_in_group("Player")
```

To test this, you can add an instance of the Enemy scene to your `Room_Template`.



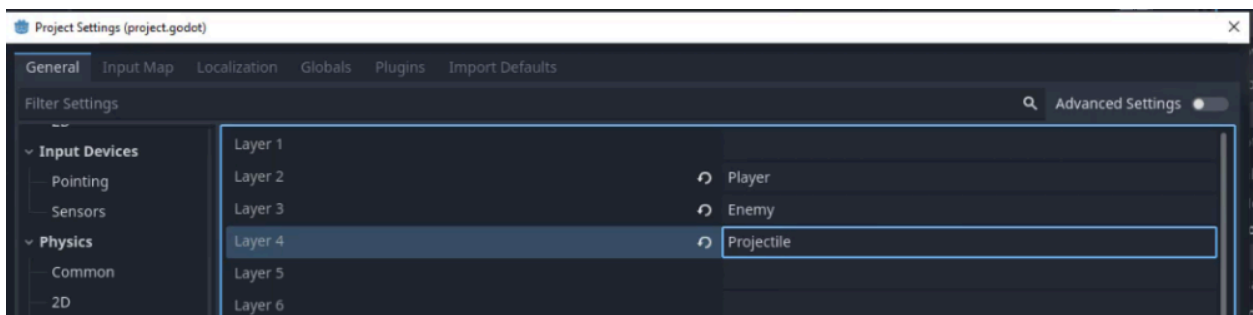
When you run the game, the enemy should move towards the player, even if you move the player around.



Setting Up Collision Layers and Masks

Currently, the enemy and player might collide and push each other in undesirable ways. To manage this, we will use Godot's collision layers. Before we configure our objects, it'd be helpful for us to first name our layers for easy reference. To do this:

1. Go to **Project -> Project Settings -> Layer Names -> 2D Physics**.
2. Name Layers 2, 3, and 4 as Player, Enemy, and Projectile, respectively.
3. Leave Layer 1 as unnamed, as we will use this layer for the walls.

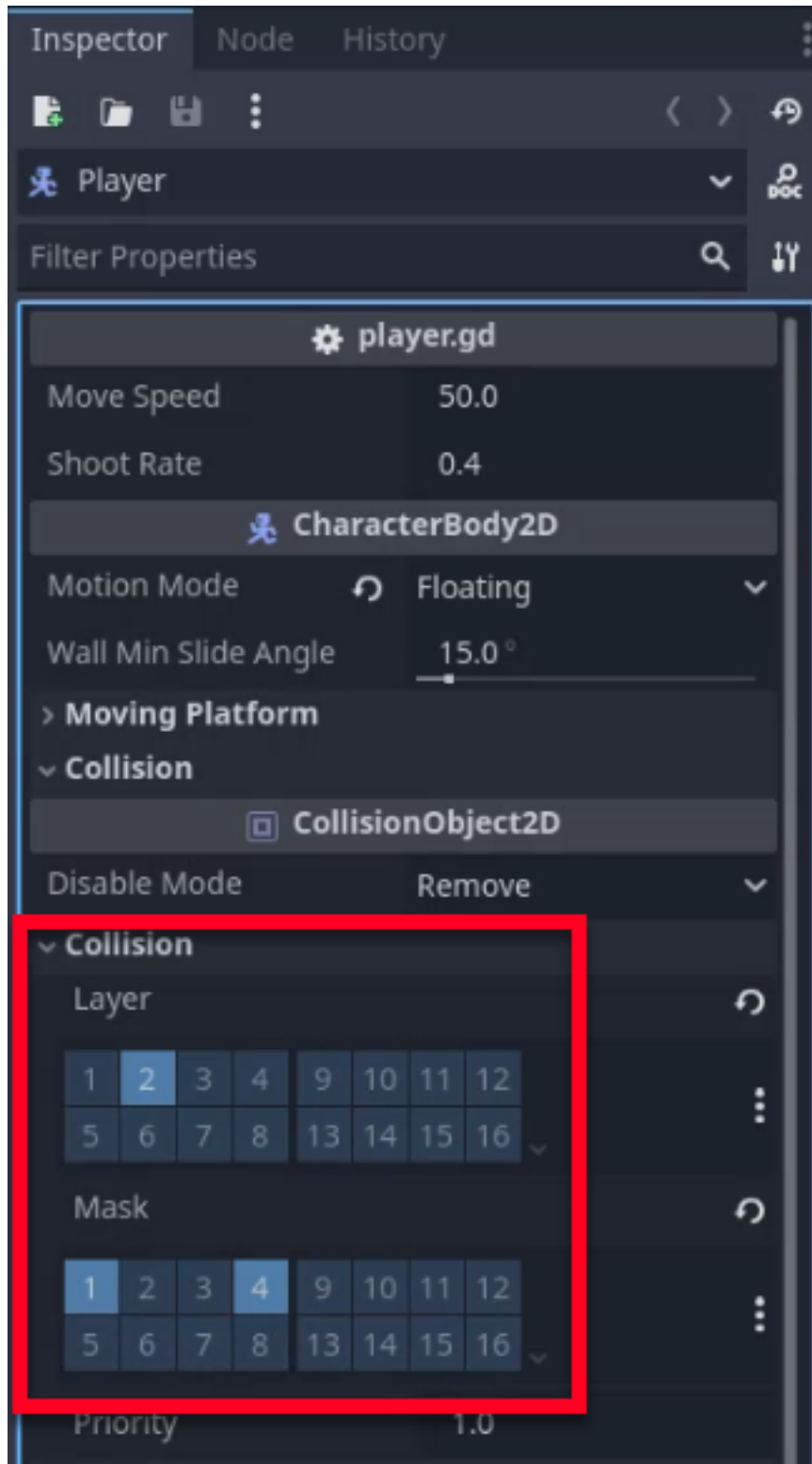


Configuring the Player

Now that our layers are set up, let's configure everything. First, select the Player node and update its **Collision** settings in the Inspector:

- **Layer:** Enable Layer 2 (Player).
- **Mask:** Enable Layer 1 (Barriers) and Layer 4 (Projectiles). Disable Layers 2 and 3.

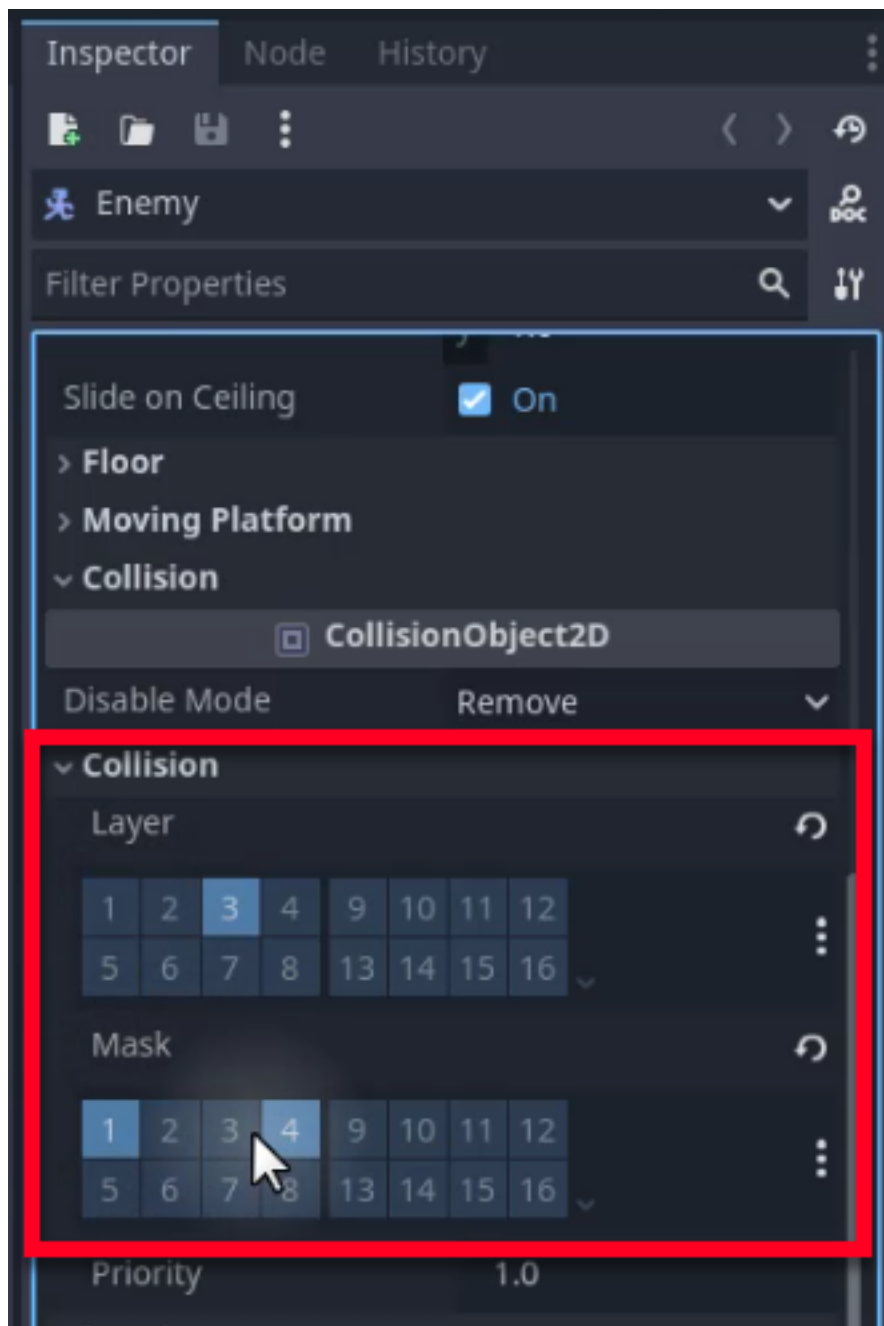
This ensures the player collides with walls and projectiles but does not physically push enemies.



Configuring the Enemy

Next, select the Enemy node and update its **Collision** settings:

- **Layer:** Enable Layer 3 (Enemy).
- **Mask:** Enable Layer 1 (Barriers) and Layer 4 (Projectiles).

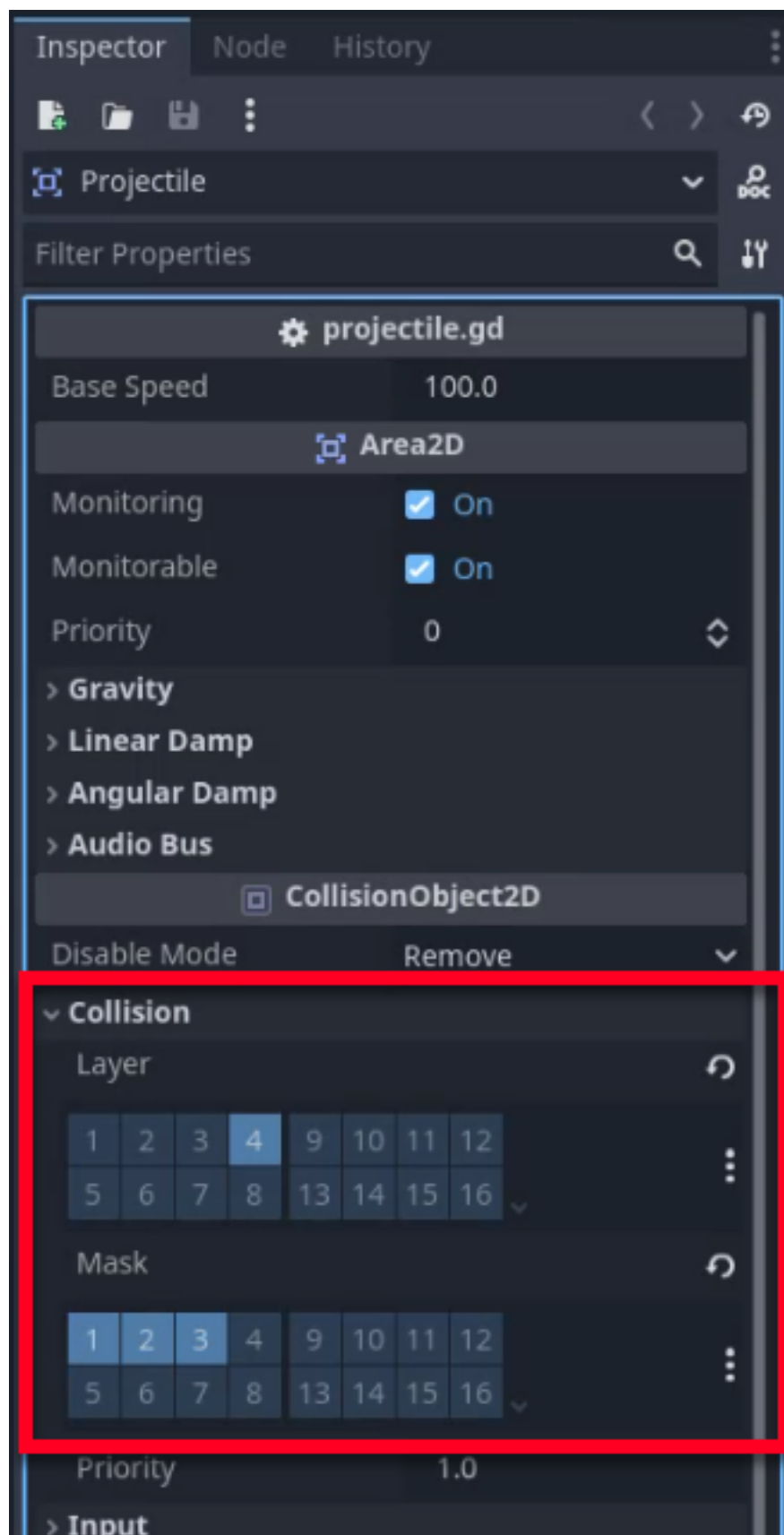


Like the player, this ensures the enemies collide with walls and projectiles but does not physically push the player.

Configuring the Projectile

Finally, let's update the Projectile scene to ensure it hits enemies.

- **Layer:** Enable Layer 4 (Projectile).
- **Mask:** Enable Layer 1 (Barriers), Layer 2 (Player), and Layer 3 (Enemy).



Godot Insight: Collision layers vs masks

Godot separates collision configuration into two concepts. A node's **layer** answers the question "what am I?" while its **mask** answers "what do I want to detect?" A collision only happens when node A's mask includes node B's layer, or vice versa.

For example, our player sits on Layer 2 (Player) and has Mask bits for Layer 1 (Barriers) and Layer 4 (Projectile). This means the player collides with walls and can be hit by projectiles, but does not physically push enemies. The enemy, meanwhile, has its own layer (3) and masks for barriers and projectiles. By designing these relationships carefully, we avoid unwanted interactions like enemies shoving the player across the room.

Learn more: [CollisionObject2D](https://docs.godotengine.org/en/4.6/classes/class_collisionobject2d.html)

(https://docs.godotengine.org/en/4.6/classes/class_collisionobject2d.html)

Combat Logic

Now that we've resolved our collisions, we will refine the enemy's behavior to stop moving when close to the player and attack.

Stopping Within Range

We'll first update the `_physics_process` function in `enemy.gd`. We will add a check to see if the enemy is within `attack_range`. If so, it should stop moving and try to attack.

```
func _physics_process(delta):
    player_direction =
global_position.direction_to(player.global_position)
    player_distance =
global_position.distance_to(player.global_position)

    sprite.flip_h = player_direction.x < 0

    # Stop moving and attack when close enough to the player
    if player_distance < attack_range:
        _try_attack()
    return
```

```
velocity = player_direction * move_speed  
move_and_slide()
```

Attacking the Player

Next, let's implement the `_try_attack` function. This function checks if enough time has passed since the last attack (based on `attack_rate`). If the cooldown is over, it resets the cooldown and deals damage to the player.

```
# Attempts an attack if the cooldown period has elapsed  
func _try_attack():  
    # Enforce cooldown using real time to prevent attack spamming  
    if Time.get_unix_time_from_system() - last_attack_time <  
attack_rate:  
        return  
  
    last_attack_time = Time.get_unix_time_from_system()  
  
    player.take_damage(attack_damage)
```

To make this work, update the `take_damage` function in `player.gd` to print a message, allowing us to verify the attack works.

```
func take_damage(amount):  
    print("Take Damage")
```

If you test the game now, you should see “Take Damage” printed in the console when the enemy gets close.



Taking Damage and Dying

Besides the player, we also need the enemy to take damage from the player's projectiles. Let's implement the `take_damage` and `die` functions in `enemy.gd`.

```
func take_damage (amount : int):  
    cur_hp -= amount  
  
    if cur_hp <= 0:  
        die()  
  
func die ():  
    queue_free()
```

Now, when you shoot the enemy, it should eventually disappear (die) after taking enough hits.



Room Integration

Activating all enemies at once would waste processing power on off-screen rooms and create unfair situations where the player takes damage from enemies they cannot see. Enemies should only be active when the player is in the same room, and we will use the `is_active` variable to control this.

Updating the Enemy Script

First, update `_physics_process` to do nothing if the enemy is inactive or the player is missing. This prevents anything from processing, wasting precious computing power.

```
func _physics_process(delta):  
    # Skip processing when inactive or if the player reference is lost  
    if not is_active or not player:  
        return
```



```
# ... rest of movement logic ...
```

Next, we'll implement the room connection logic. When the enemy is initialized, we assign it to a room. We then listen for the global `OnPlayerEnterRoom` signal we set up earlier to check if the player has entered *this* enemy's room.

```
func initialize(in_room):  
    is_active = false  
    room = in_room  
  
func _on_player_enter_room(player_room):  
    # Activate only when the player is in this enemy's room  
    is_active = player_room == room  
  
func _ready():  
    # Find the player node via the "Player" group instead of a hardcoded path  
    player = get_tree().get_first_node_in_group("Player")  
    GlobalSignals.OnPlayerEnterRoom.connect(_on_player_enter_room)
```

Locking Doors

Locking the player in a room until all enemies are defeated is a classic roguelike mechanic that creates tension during encounters and prevents players from bypassing combat by simply running to the next room. We can implement this by counting enemies in the room script.

First, give the enemy a class name so we can easily identify it. Add this to the top of `enemy.gd`:

```
class_name Enemy  
extends CharacterBody2D
```

Next, we need a way to notify the room when an enemy dies. Open `global_signals.gd` and add a new signal:

```
signal OnDefeatEnemy(enemy: Enemy)
```

We then update the die function in enemy.gd to emit this signal before removing the enemy:

```
func die():  
    # Emit signal before queue_free so listeners can react while the  
    node still exists  
    GlobalSignals.OnDefeatEnemy.emit(self)  
    queue_free()
```

With these architectural changes, we can now update the room logic itself.

Updating the Room Script

Finally, let's open room.gd. We need to count the enemies when the room starts and listen for their defeat.

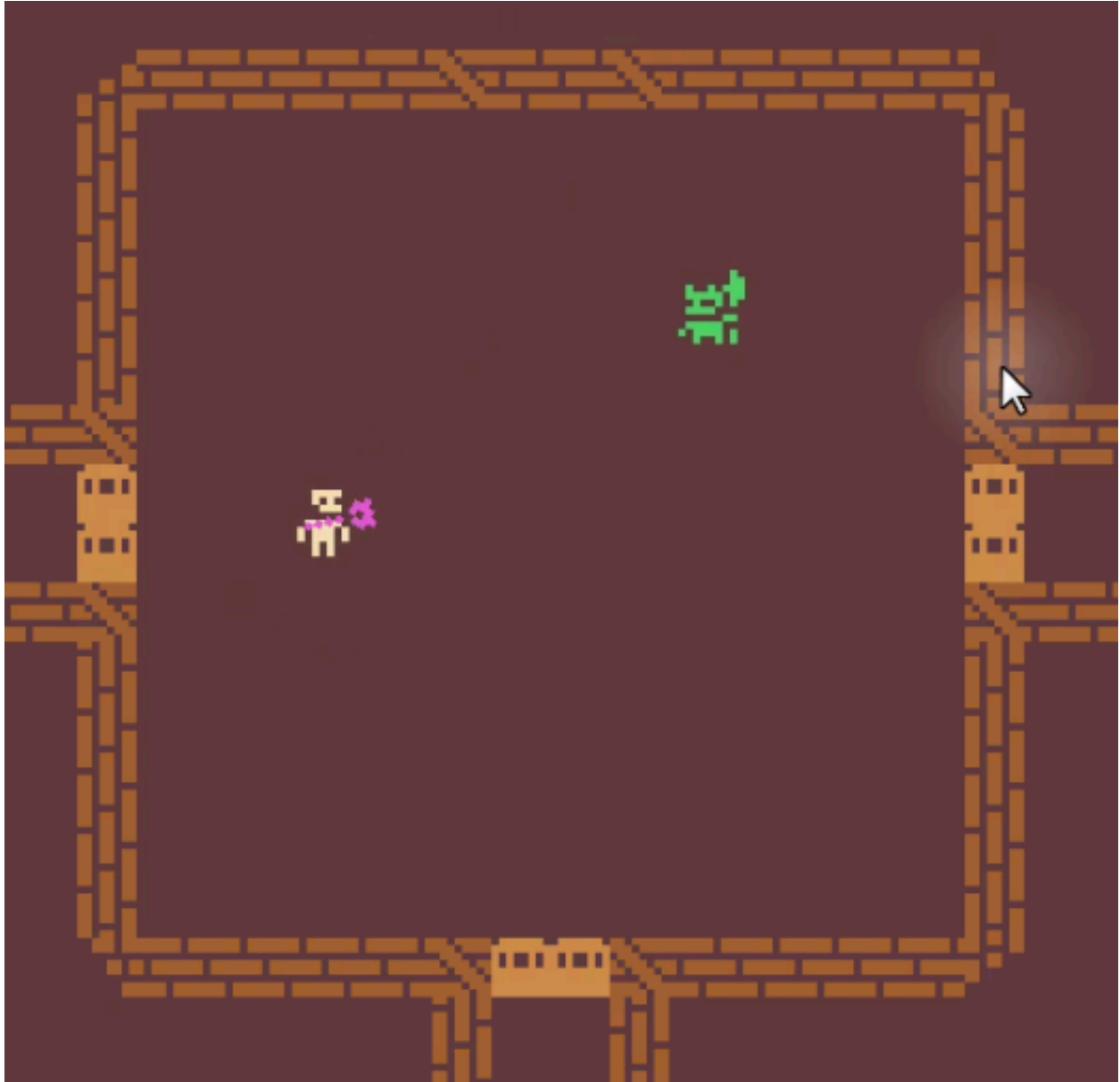
Update the _ready function to count children that are of type Enemy. We also initialize them here so they can probably toggle their active state.

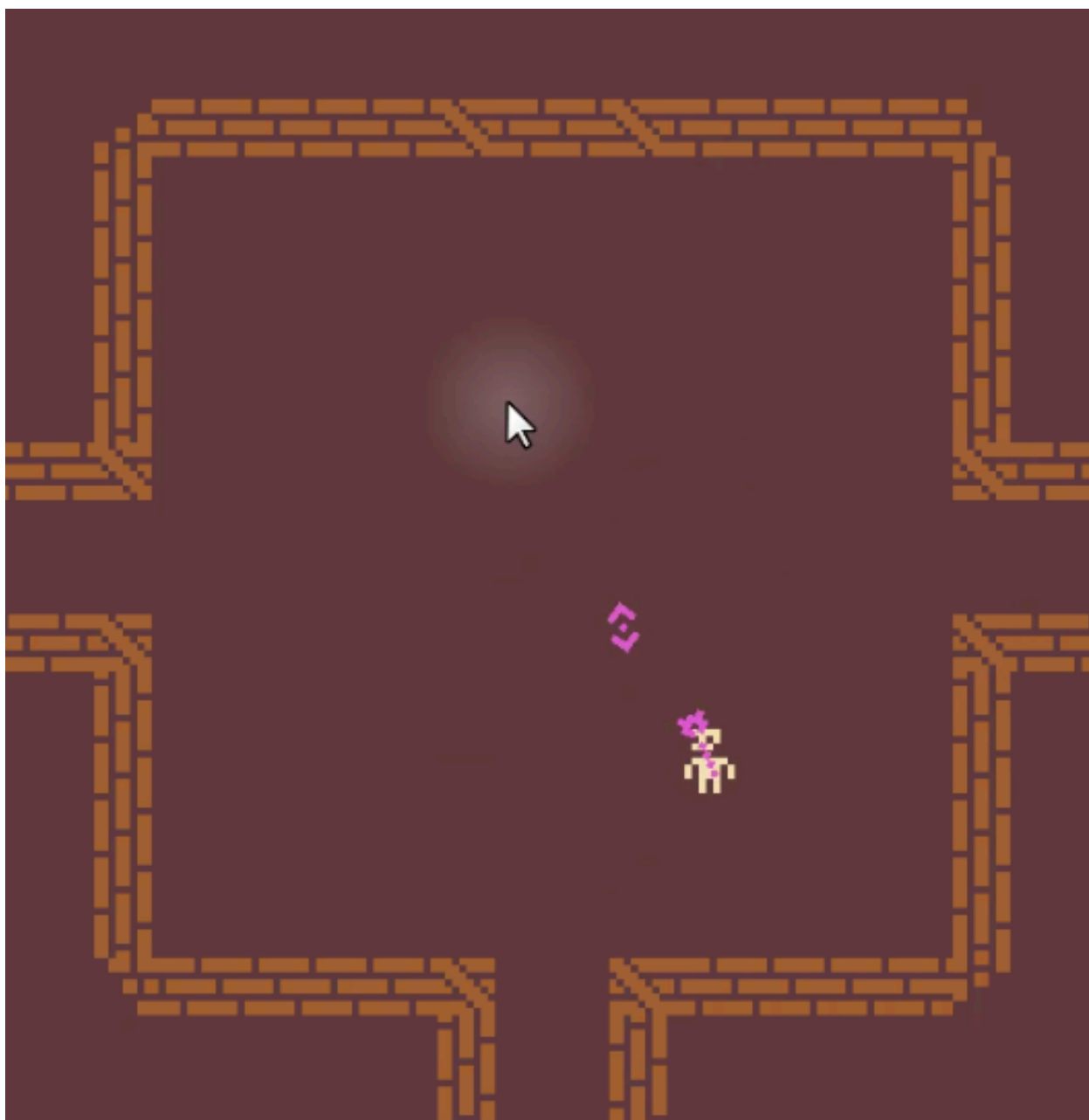
```
func _ready():  
    for child in get_children():  
        if child is Enemy:  
            enemies_in_room += 1  
            child.initialize(self)  
  
    GlobalSignals.OnDefeatEnemy.connect(_on_enemy_defeated)
```

Lastly, we must implement the _on_enemy_defeated function to decrement the count and open doors when the room is clear.

```
func _on_enemy_defeated(enemy: Enemy):  
    if enemy.get_parent() == self:  
        enemies_in_room -= 1  
  
    if enemies_in_room <= 0:  
        open_doors()
```

Now, if you place enemies in your room templates, the doors will automatically close when the player enters and only reopen once all enemies are defeated.





In the next chapter, we will expand our dungeon generation to include different types of rooms, such as item rooms and boss rooms.

Items and Dungeon Expansion

In this chapter, we will expand the variety of our dungeon by introducing different room types and collectible items. Currently, every room in our game looks identical. To make the gameplay more engaging, we will create multiple room templates: some with different enemy configurations and others containing rewards.

By the end of this chapter, you will have a system that randomly generates rooms with varying numbers of enemies, as well as special item rooms where the player can collect potions to upgrade their stats.

Different Room Types

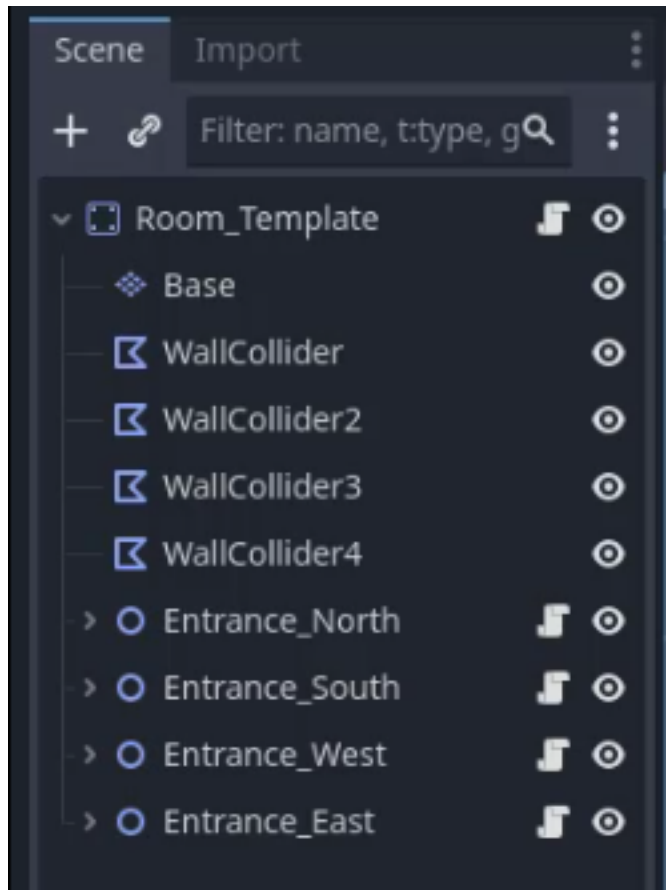
Our first goal is to set up a system that allows the generator to pick from a pool of different room layouts. This involves creating a clean base template and then deriving specific room variations from it.

Clearing the Room Template

To ensure our inheritance system works correctly, the base room template should be a blank canvas. It should contain the walls, doors, and logic, but no specific enemies or objects. Any content left in the template would appear in every inherited room, defeating the purpose of having separate variations.

As such, navigate to your `room_template.tscn` scene and delete any enemy nodes or temporary objects you may have added for testing in previous chapters.

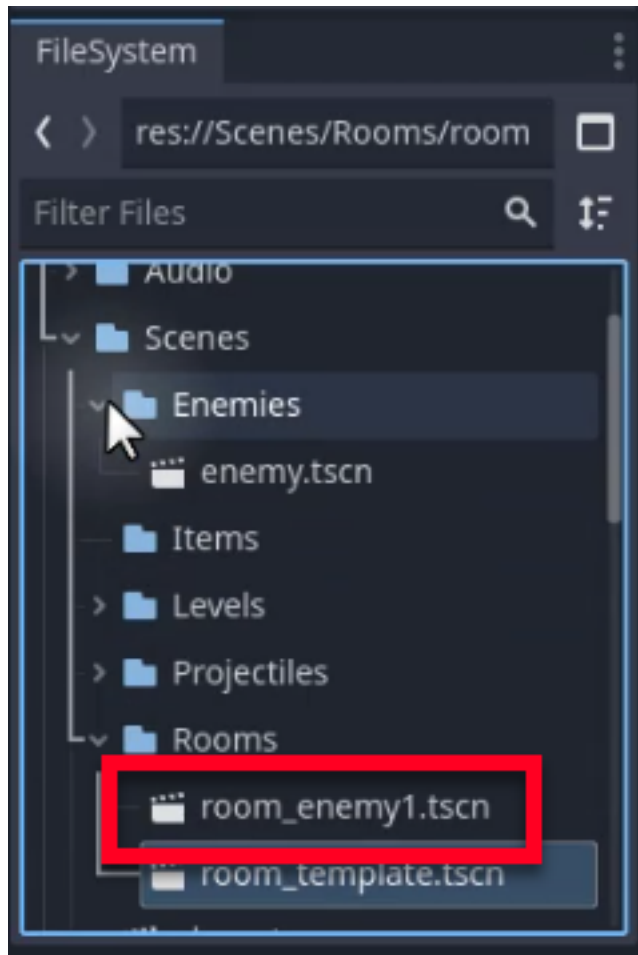
The scene tree should look clean, containing only the static geometry and entrance nodes, as shown below.



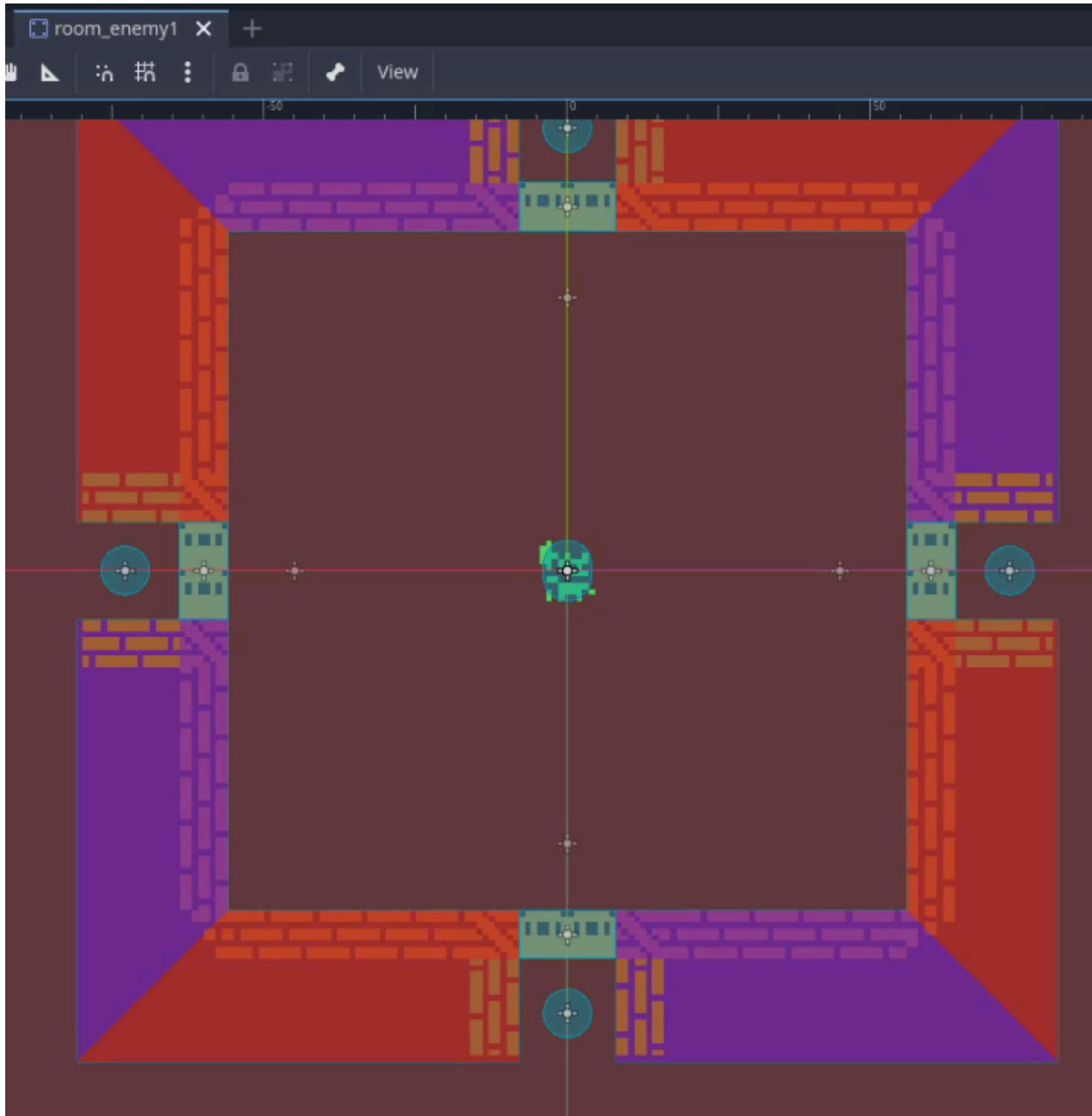
Creating Inherited Room Scenes

With a clean template, we can now create variations. We will use Godot's **Inherited Scene** feature, which allows us to create new scenes that keep all the properties of the `room_template.tscn` while allowing us to add new nodes specific to that variation.

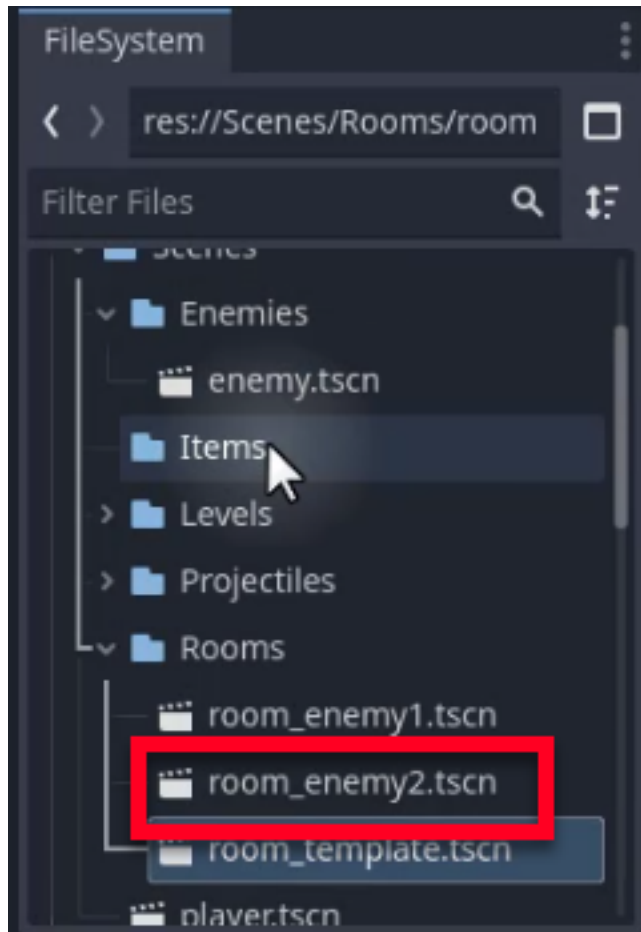
Right-click on the `room_template.tscn` file in the FileSystem and select **New Inherited Scene**. Save this new scene in the `Scenes/Rooms` folder as `room_enemy1.tscn`.



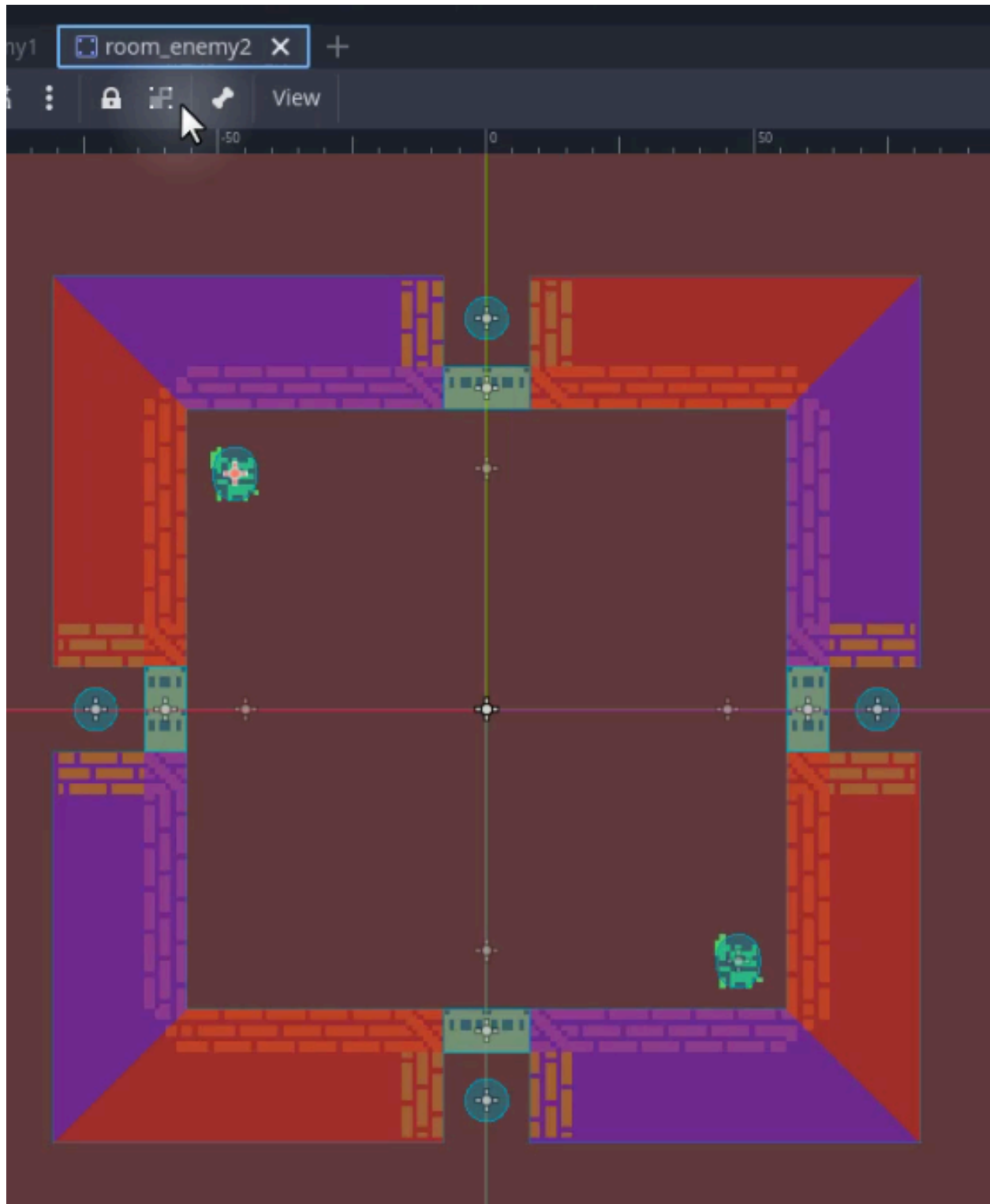
In this new scene, drag an Enemy scene from your Scenes/Enemies folder into the room. Place it in the center or any position you prefer.



Let's repeat this process to create a second variation. Right-click `room_template.tscn` again and select **New Inherited Scene**. Let's save this variation as `room_enemy2.tscn`.



To this new variation, let's add two enemies to this scene, placing them in opposite corners.



Godot Insight: Scene inheritance

Godot's inherited scenes work similarly to class inheritance in code. When you create a new inherited scene from `room_template.tscn`, the new scene starts with all the nodes and properties of the template. You can then add new nodes (like enemies) or override specific properties without duplicating the base structure.

The key advantage is maintainability: if you later adjust the wall layout or door positions in `room_template.tscn`, every inherited room scene (enemy rooms, item rooms, boss rooms) automatically picks up those changes. This “edit once, update everywhere” pattern is essential for keeping a growing project manageable.

Learn more: [PackedScene](https://docs.godotengine.org/en/4.6/classes/class_packedscene.html)

(https://docs.godotengine.org/en/4.6/classes/class_packedscene.html)

Updating the Room Generation Script

Now that we have multiple room scenes, we need to update our generation logic to use them. So, let's open the `room_generation.gd` script.

We need to replace our single `room_scene` variable with an array that can hold multiple scenes. We also want a specific variable for the starting room to ensure the player always spawns in a safe location.

Update your variables as follows:

```
extends Node
```

```
@export var first_room_scene : PackedScene  
@export var room_scenes : Array[PackedScene]
```

Next, we must update the `_instantiate_rooms` function. Instead of always instantiating the same scene, we will check if we are generating the first room. If so, we use `first_room_scene`. For all other rooms, we pick a random scene from the `room_scenes` array.

```
func _instantiate_rooms():  
    for x in range(map_size):  
        for y in range(map_size):  
            if not _get_map(x, y):
```

```

        continue

    var room : Room
    var is_first_room = first_room_x == x and first_room_y ==
y

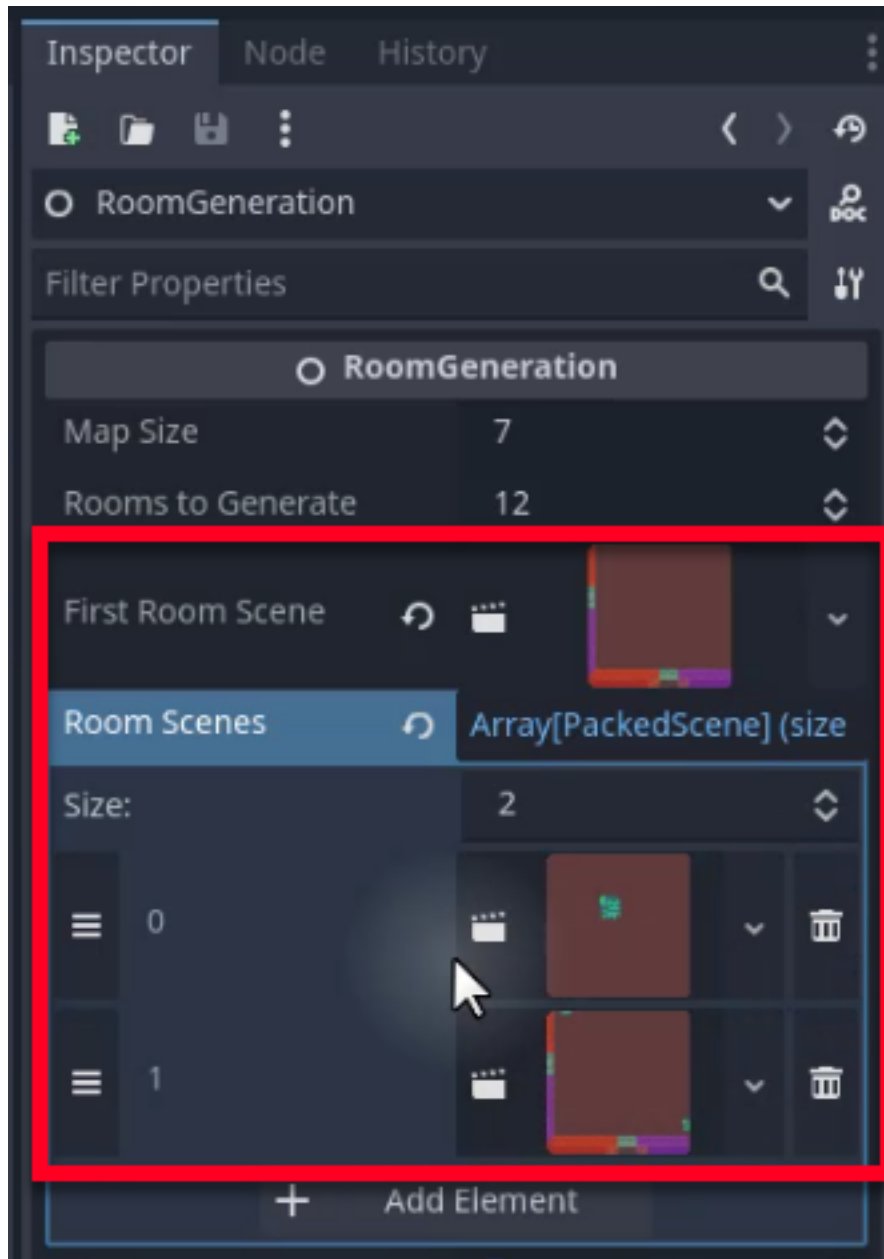
    if is_first_room:
        room = first_room_scene.instantiate()
    else:
        room = room_scenes[randi_range(0, len(room_scenes) -
1)].instantiate()

    get_tree().root.get_node("/root/Main").add_child(room)
    rooms.append(room)
    room.global_position = Vector2(x, y) * room_pos_offset

    if is_first_room:
        first_room = room

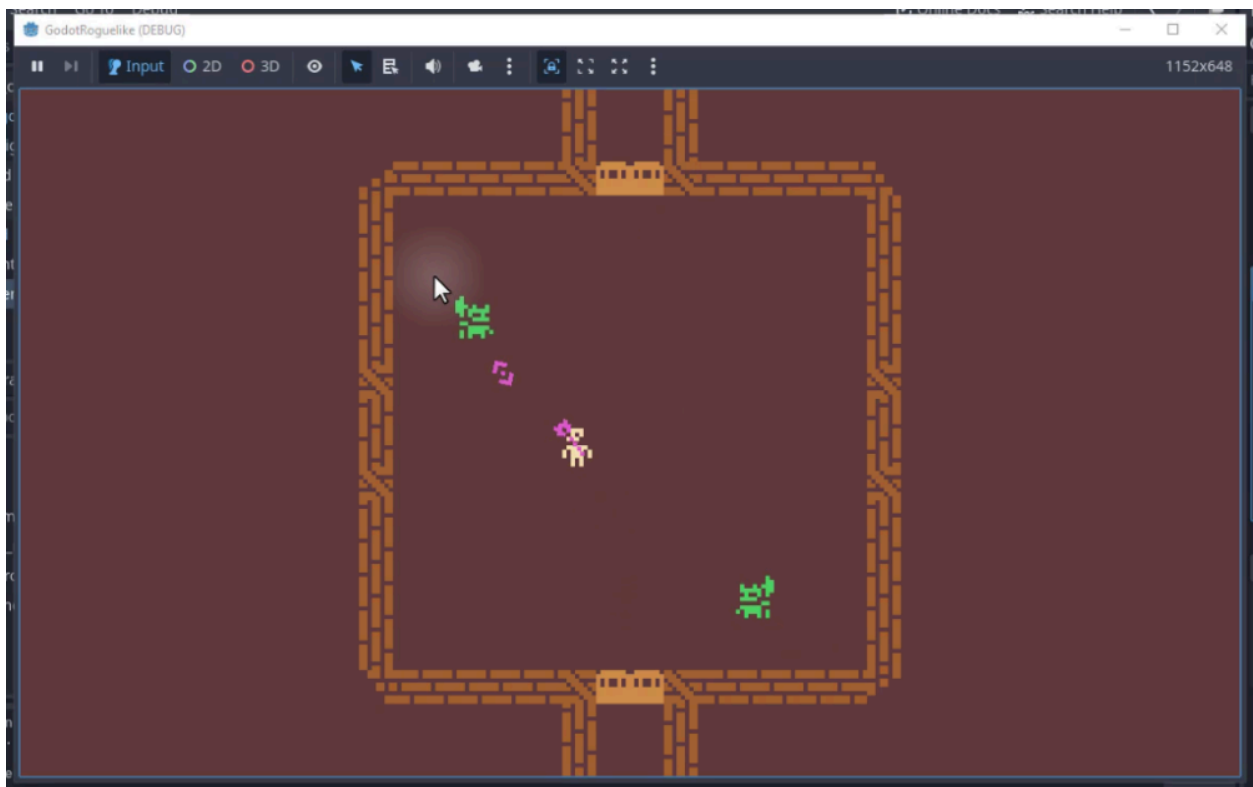
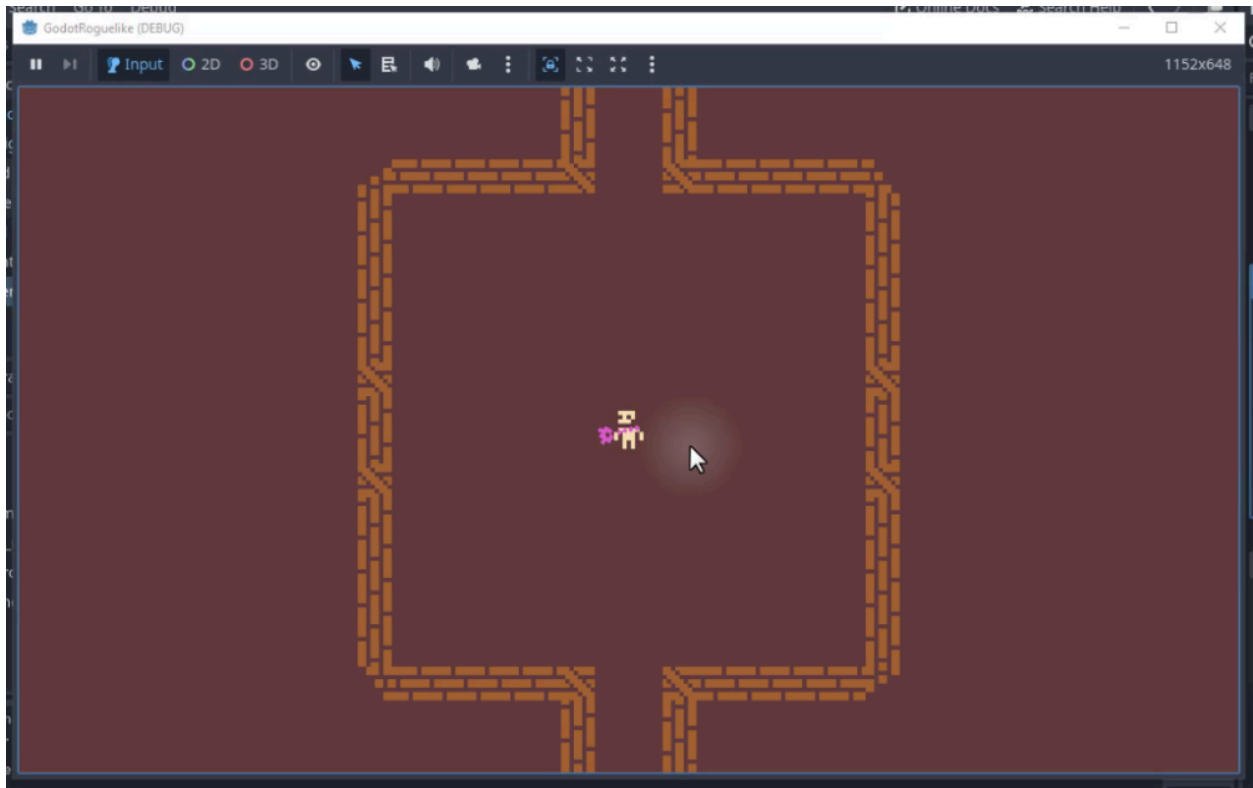
```

Back in the editor, select the RoomGeneration node in your Main scene so we can configure it. In the Inspector, assign `room_template.tscn` (the empty room) to **First Room Scene**. Then, add elements to the **Room Scenes** array and assign `room_enemy1.tscn` and `room_enemy2.tscn`.



Testing the Room Generation

We can now run the game to verify the new system. You should spawn in an empty room (the `first_room_scene`). As you explore the dungeon, you should encounter rooms with either one or two enemies, selected randomly.

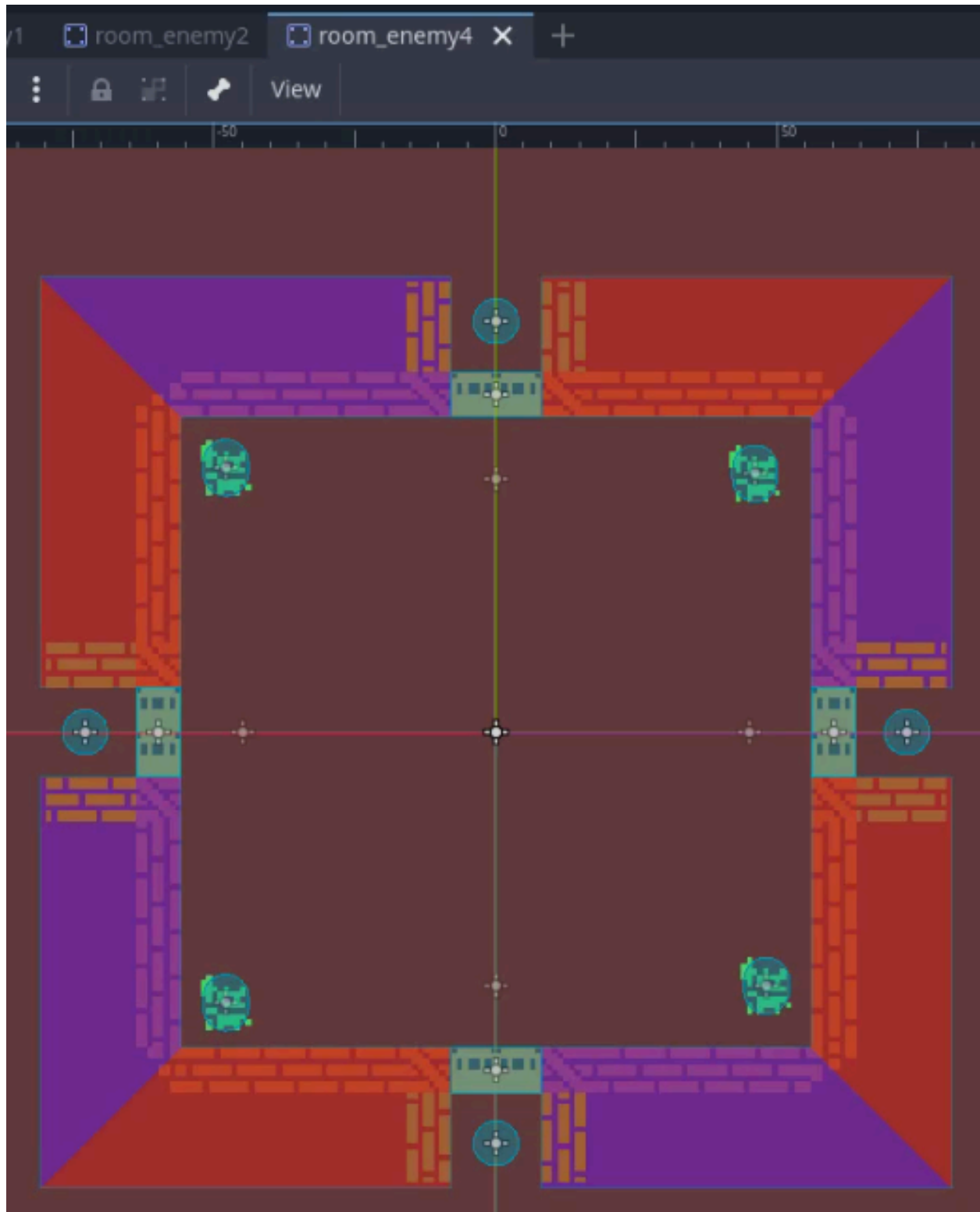


Challenge: Four Enemy Room

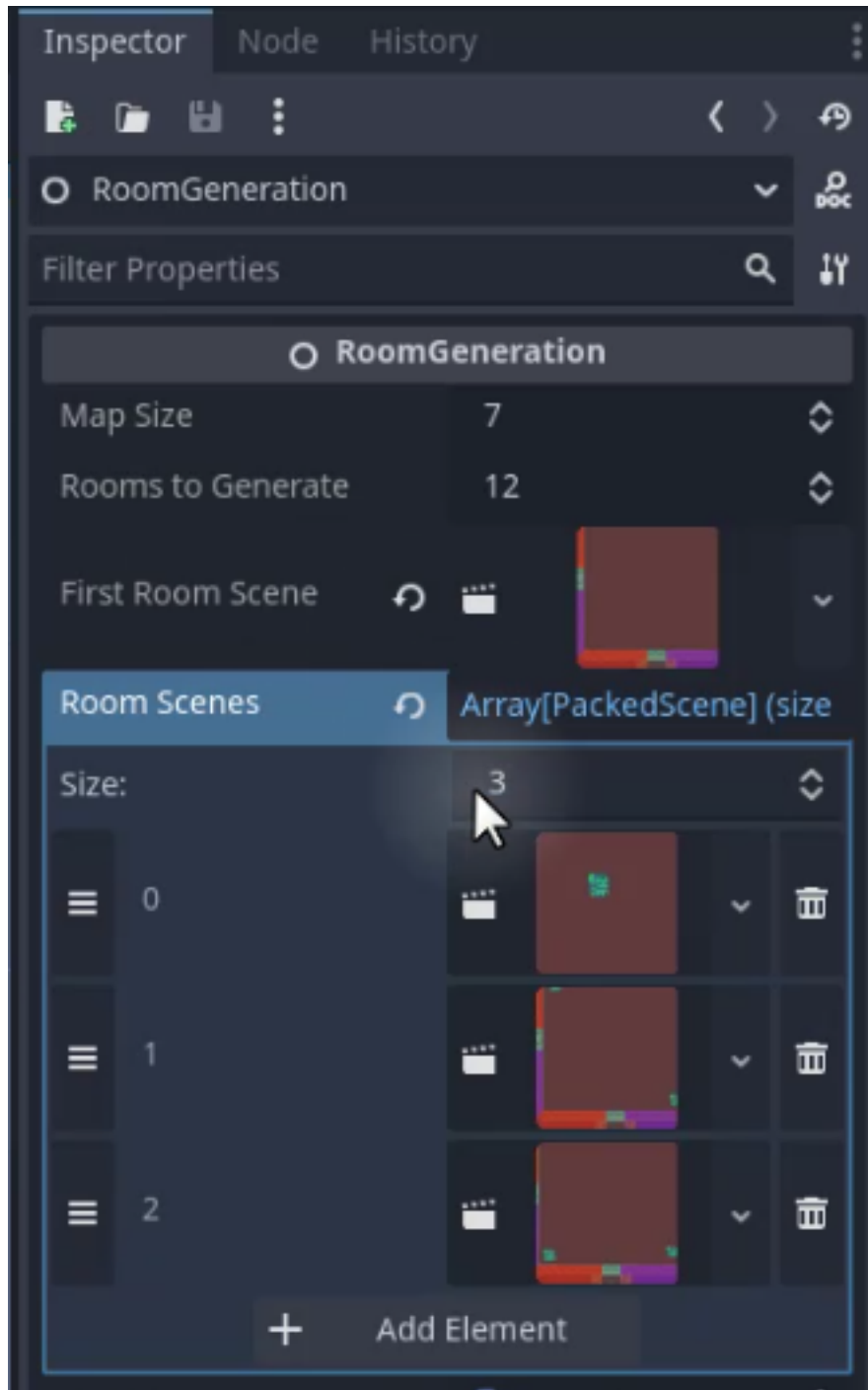
As a quick exercise, create a third room variation called `room_enemy4`. Inherit from the template and place four enemies, one in each corner. Then, add this new scene to the **Room Scenes** array in the `RoomGeneration` node.

If something doesn't feel right, we encourage you to re-review the section to make sure you understand the steps involved in creating an Inherited Scene.

Once you've tried the challenge, your new room should resemble something like the image below:



Per the challenge, ensure you update the Inspector array to include this new room so it appears in the game.



Items

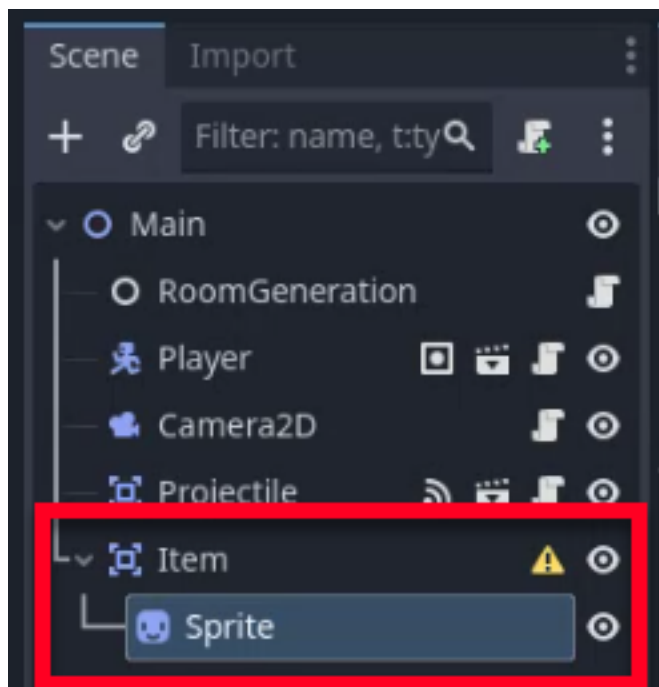
Now that we have varied combat encounters, we will add collectible items to reward the player. These items will be potions that permanently improve the player's attributes: health, movement speed, or firing rate.

Creating the Base Item Scene

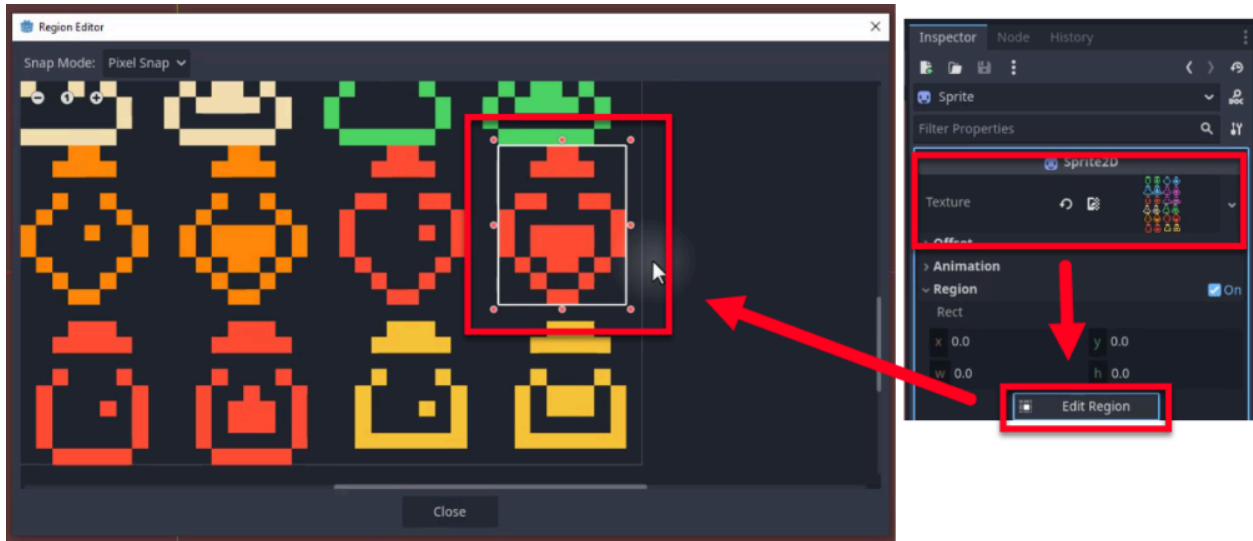
Before we can set up any rooms, we need to create the item itself. We will create a base scene that handles the logic for all items. Specific potions will then inherit from this base scene.

In the **FileSystem** dock, right-click the Scenes/Items folder (create it if it doesn't exist) and choose **New Scene**. Add an Area2D node as the root, rename it Item, and save the scene as `item.tscn`.

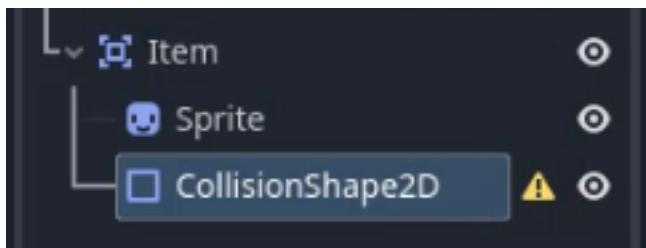
After this, add a Sprite2D child node and rename it to Sprite.



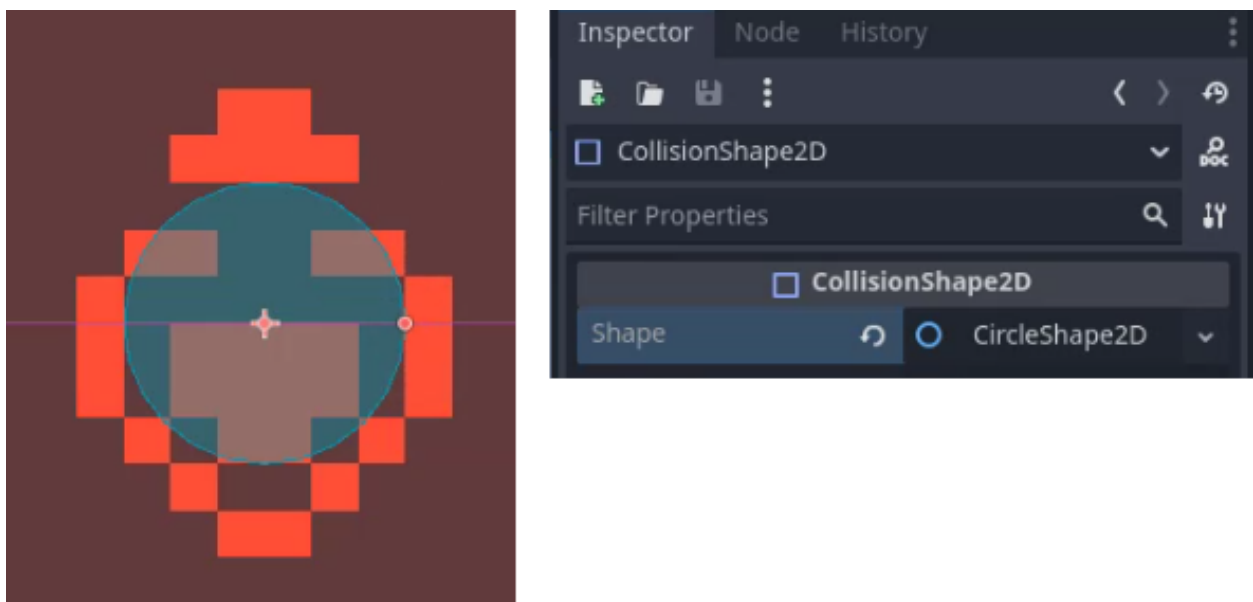
Now let's add our visual. Assign the `Potions.png` texture to the **Texture** property of the sprite. Then, use the **Region** settings to select a default potion image (e.g., the red potion). This visual is just a placeholder, as inherited scenes will override it.



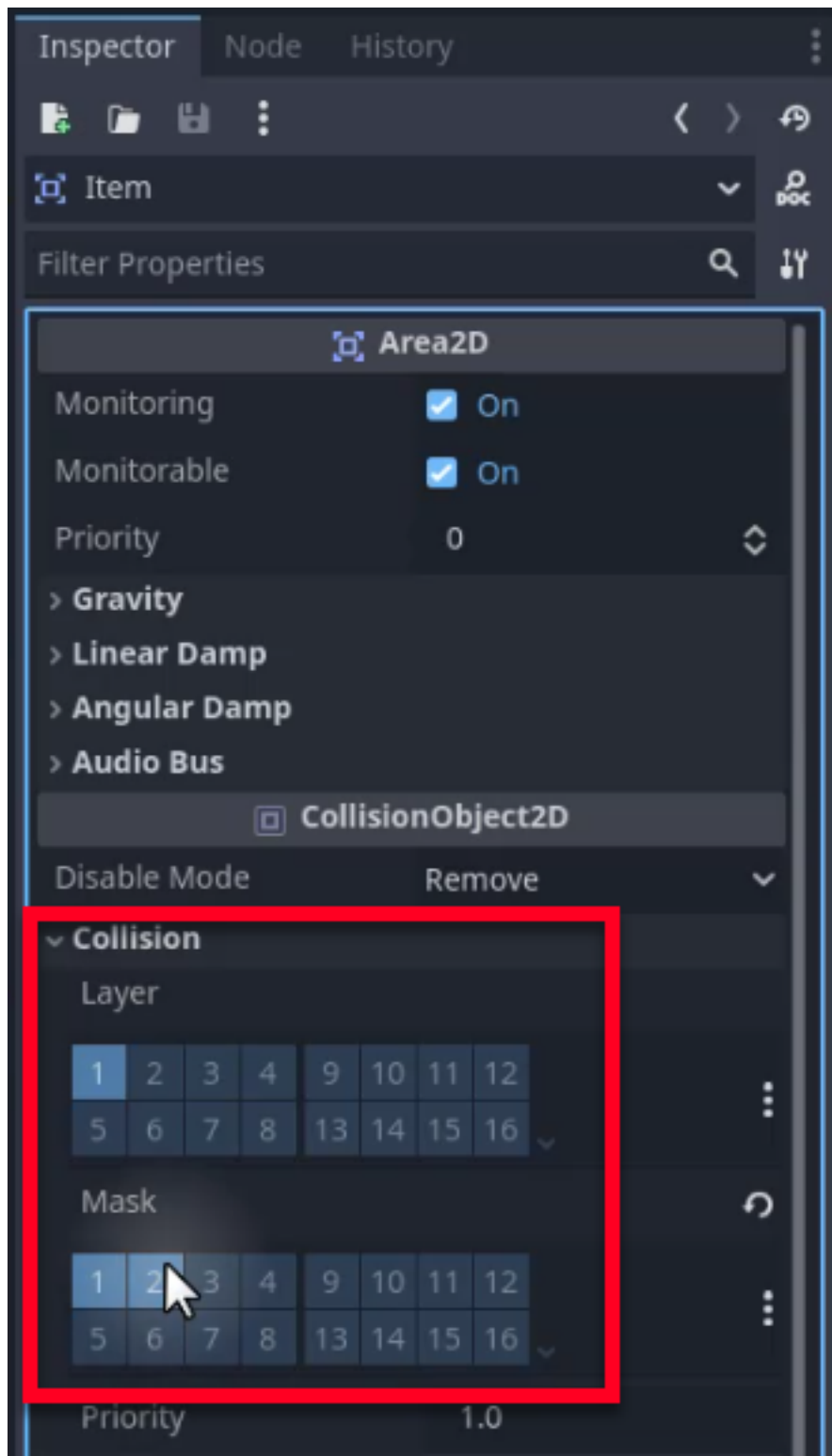
Next, in order to pick up our item, add a `CollisionShape2D` child node.



We'll set the **Shape** to `CircleShape2D` and then resize it to fit the sprite.

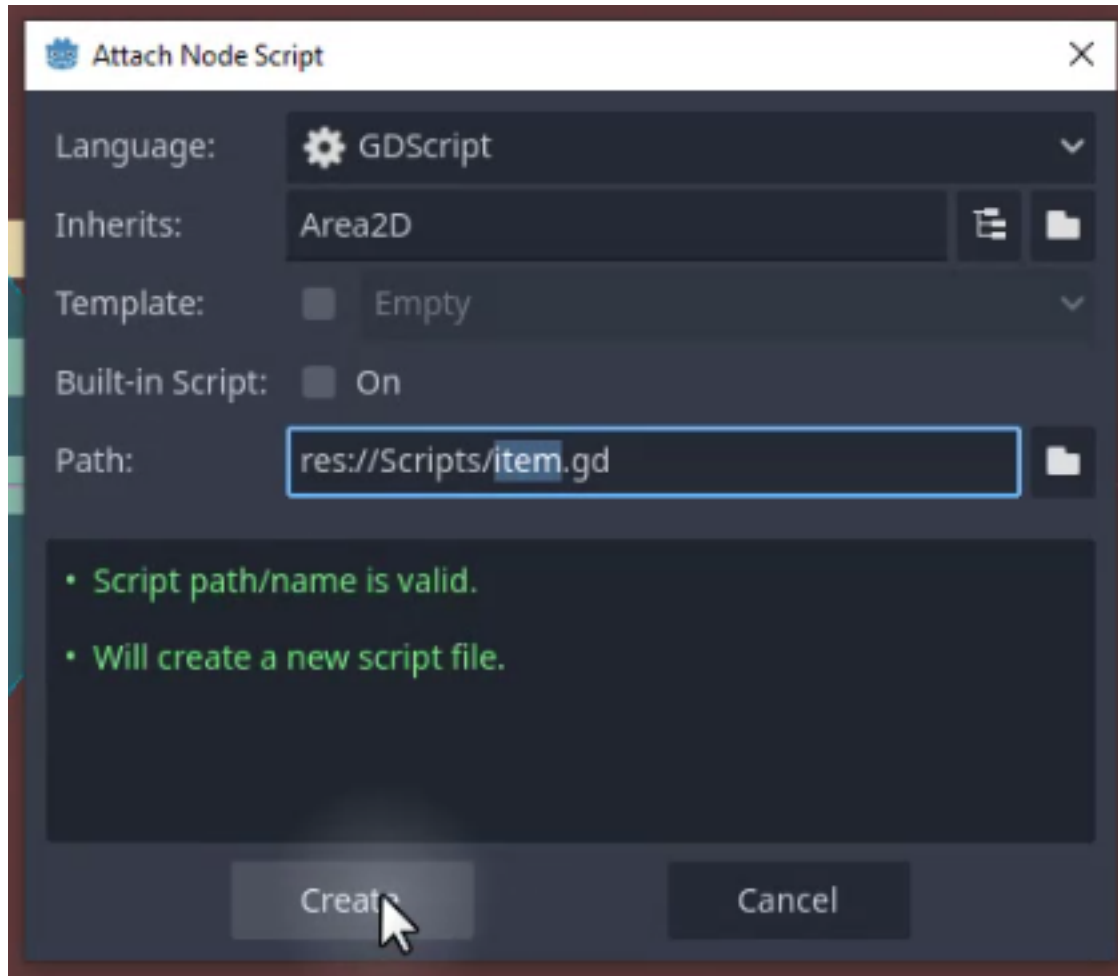


Finally, we need to configure the collision layers so the item can detect the player. Select the Item node and, in the **Collision** section of the Inspector, enable **Mask Layer 2 (Player)**.



Programming the Item Script

Our base scene is set up, so let's add our logic now. Attach a new script named `item.gd` to the Item node.



We will use an enum to define the different types of items. This allows us to use a single script for all potion types. We also need variables to store the item type and the value of the stat boost.

Add the following code to `item.gd`:

```
extends Area2D
```

```
enum ItemType
```

```
{
```

```
    HEALTH,
```

```
    SHOOT_RATE,
```

```
    MOVE_SPEED
}

@export var type : ItemType
@export var value : float
```

Godot Insight: @export and the Inspector

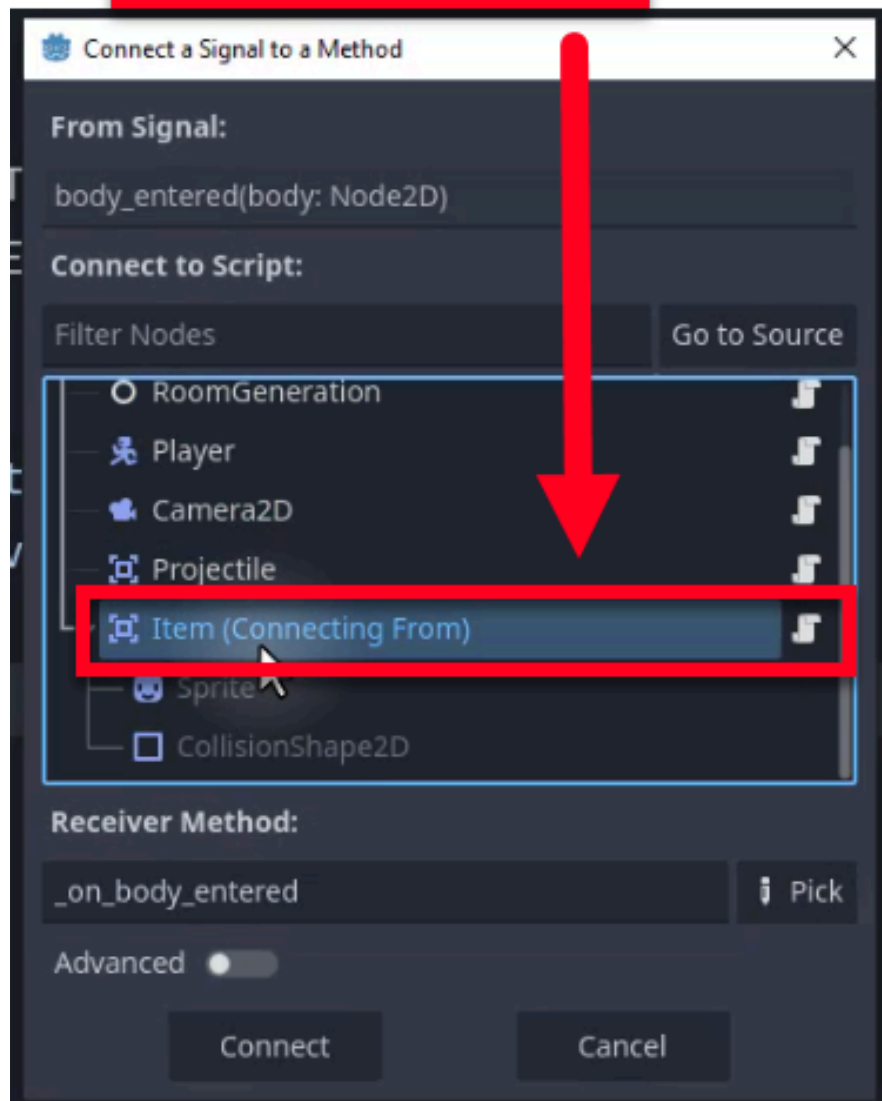
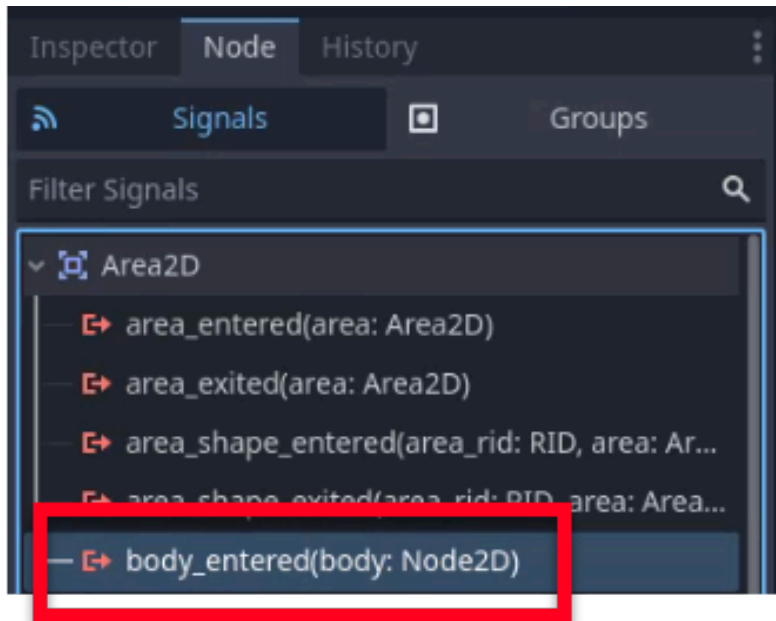
The @export annotation exposes a variable in Godot's Inspector panel, letting you adjust values per-instance without editing code. Combined with an enum, this creates a dropdown menu in the Inspector, so you can configure each item's type by selecting "Health," "Shoot Rate," or "Move Speed" from a list rather than typing numbers.

This pattern is central to Godot's design philosophy: scripts define behavior and the Inspector configures it. It means designers (or your future self) can tweak gameplay without ever opening a script file.

Learn more: [GDScript exports](https://docs.godotengine.org/en/4.6/tutorials/scripting/gdscript/gdscript_exports.html)

(https://docs.godotengine.org/en/4.6/tutorials/scripting/gdscript/gdscript_exports.html)

Now, connect the body_entered signal of the Area2D to the script. This function will trigger when the player touches the item.



In the `_on_body_entered` function, we first verify that the body is the player. Then, we can apply the effect based on the item type. We also remove the item from the scene after it has been used.

```
func _on_body_entered(body):  
    # Ignore collisions with anything that isn't the player  
    if not body.is_in_group("Player"):  
        return  
  
    # Apply a different stat effect depending on the item type  
    if type == ItemType.HEALTH:  
        body.heal(int(value))  
    elif type == ItemType.SHOOT_RATE:  
        body.shoot_rate -= value  
    elif type == ItemType.MOVE_SPEED:  
        body.move_speed += value  
  
    queue_free()
```

Extending the Player Script

The item script we just created calls functions and modifies variables on the player that don't exist yet. We need to update `player.gd` to support these stats.

Open `player.gd` and begin by adding variables for health:

```
@export var cur_hp : int = 8  
@export var max_hp : int = 8
```

Next, we'll implement the `heal` function. This function increases current health but clamps it so it doesn't exceed the maximum.

```
func heal(amount: int):  
    cur_hp += amount  
    if cur_hp > max_hp:  
        cur_hp = max_hp
```


While we're here, we also need to fully implement `take_damage` since our health stats are now available. Note that we also add a `die` function. This function we will tackle later in our project once we configure our full game loop.

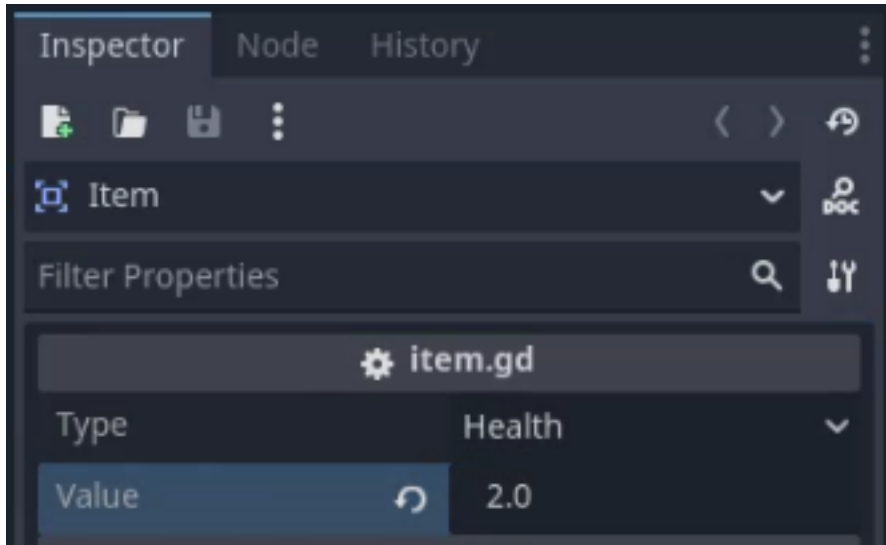
```
func take_damage (amount : int):  
    cur_hp -= amount  
  
    if cur_hp <= 0:  
        die()  
  
func die ():  
    pass
```

Creating Potion Scenes

With the base logic in place, we can now create the specific potion items.

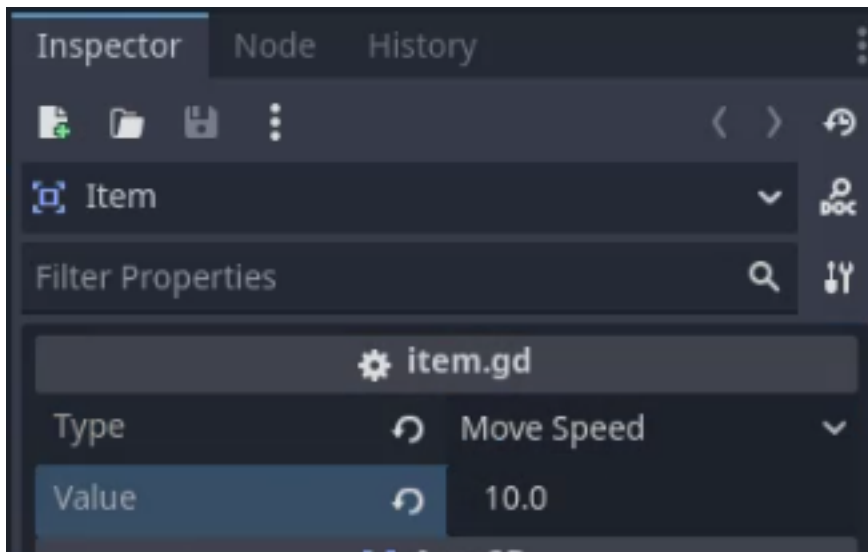
1. Health Potion

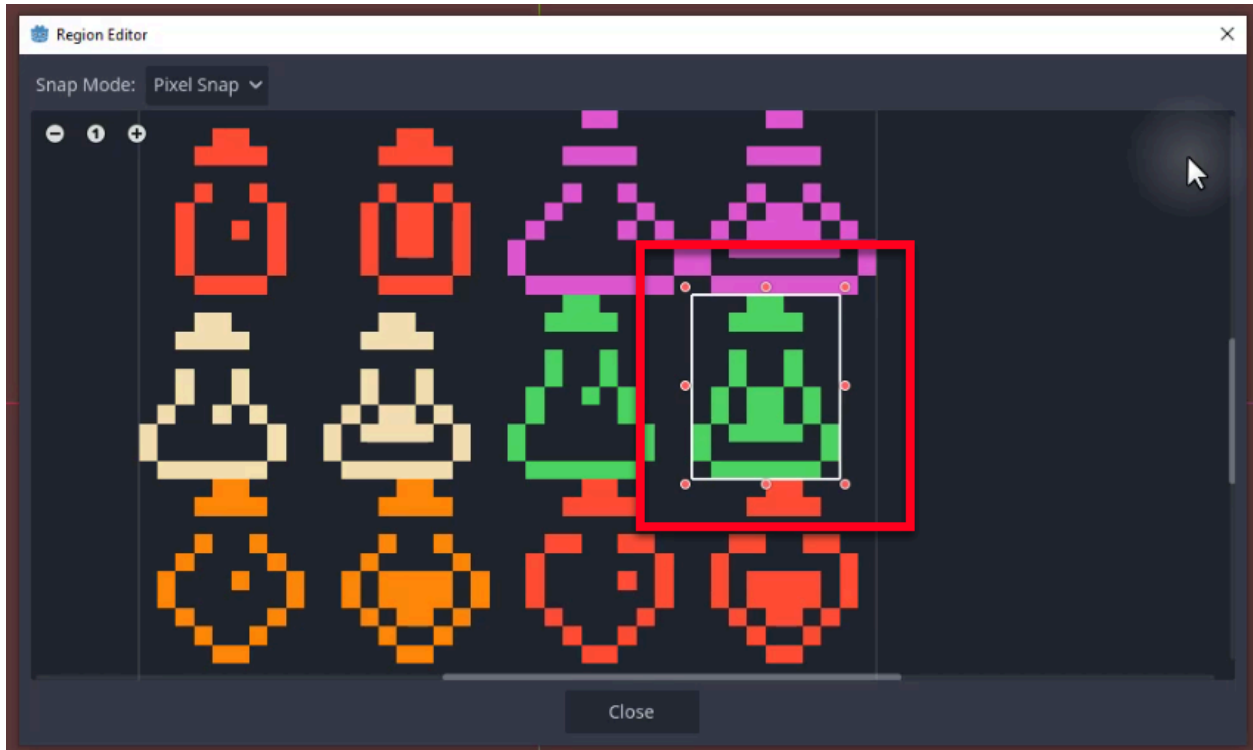
- Right-click `item.tscn` and select **New Inherited Scene**, then save it as `item_health.tscn`.
- In the Inspector, set **Type** to `HEALTH` and **Value** to 2.
- Ensure the sprite region shows a red potion. This should be the default sprite, but if you chose something different, you should change it now.



2. Speed Potion

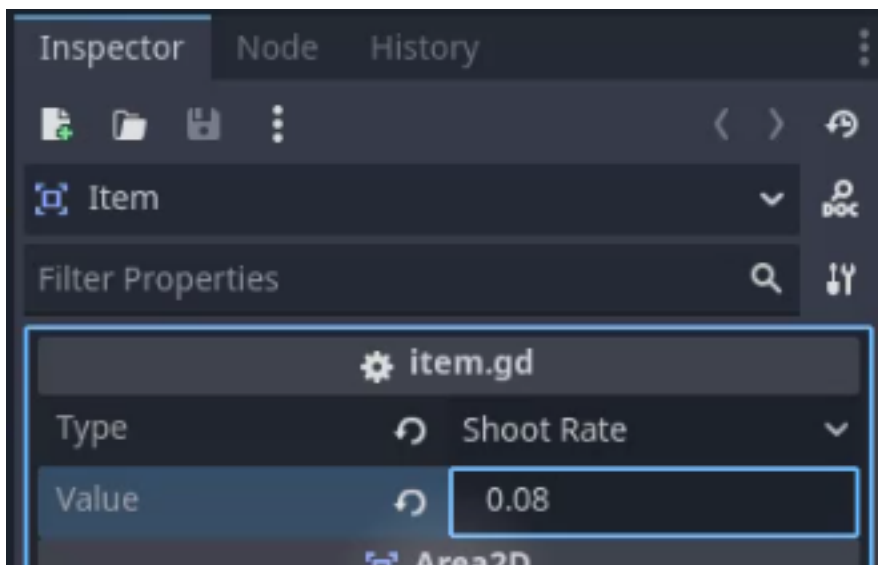
- Create another inherited scene and save it as `item_move_speed.tscn`.
- Set **Type** to `MOVE_SPEED` and **Value** to 10.
- Change the sprite region to a green potion.

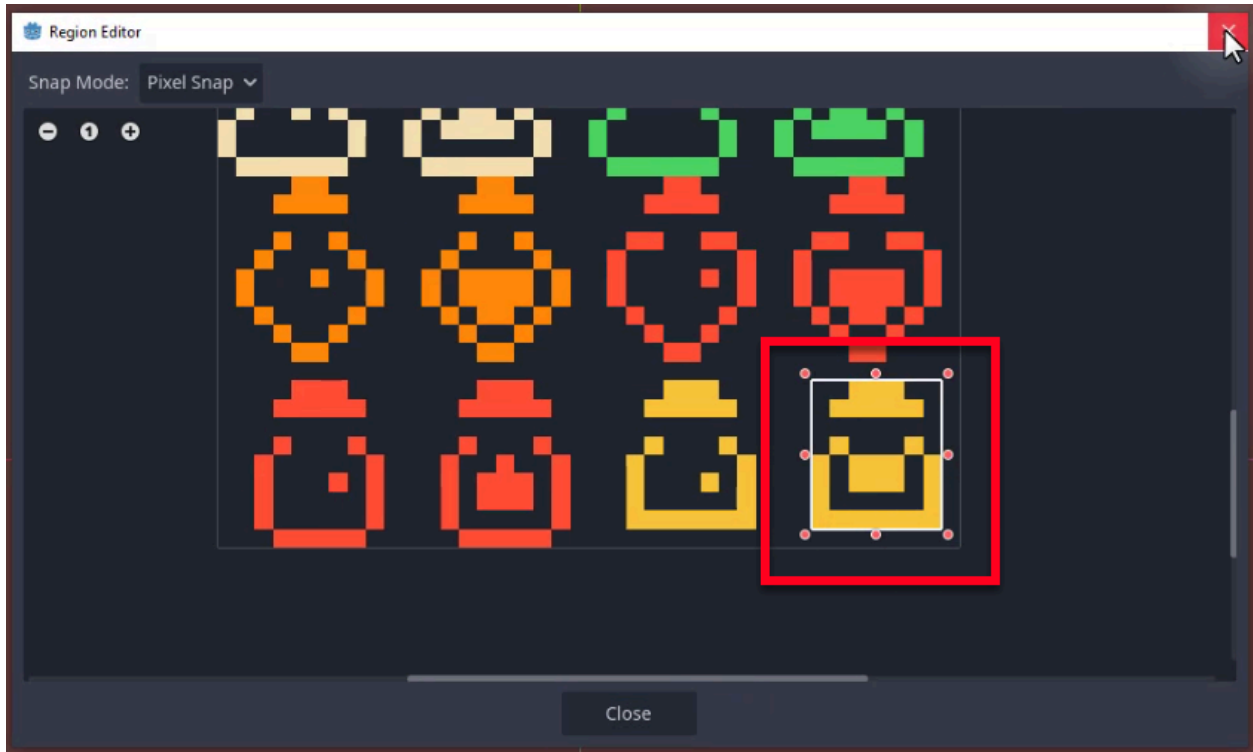




3. Fire-Rate Potion

- Create a third inherited scene and save it as `item_shoot_rate.tscn`.
- Set **Type** to `SHOOT_RATE` and **Value** to `0.08` (since `shoot_rate` is a delay, subtracting from it makes shooting faster).
- Change the sprite region to a yellow potion.





This covers all our potion types, though you can feel free to expand the project and add more using these same steps!

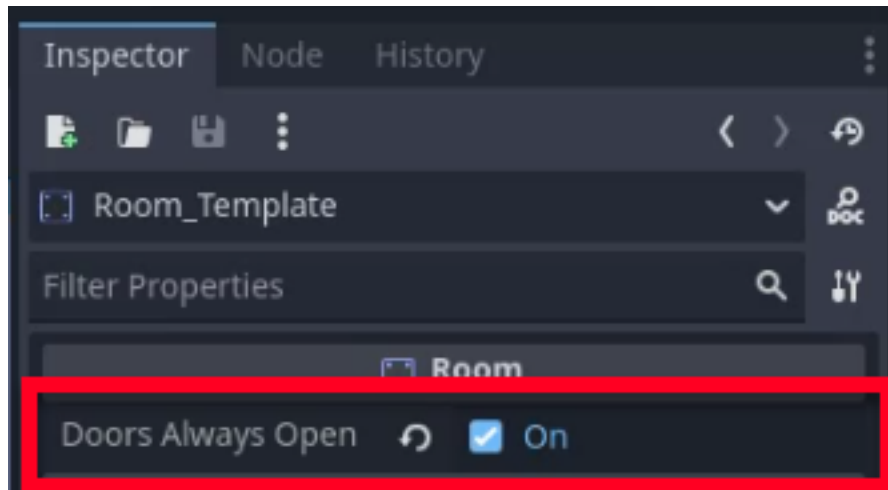
Item Room

We have our items, but we need a place for them to spawn. For this, we will create a special “Item Room” that contains a random reward.

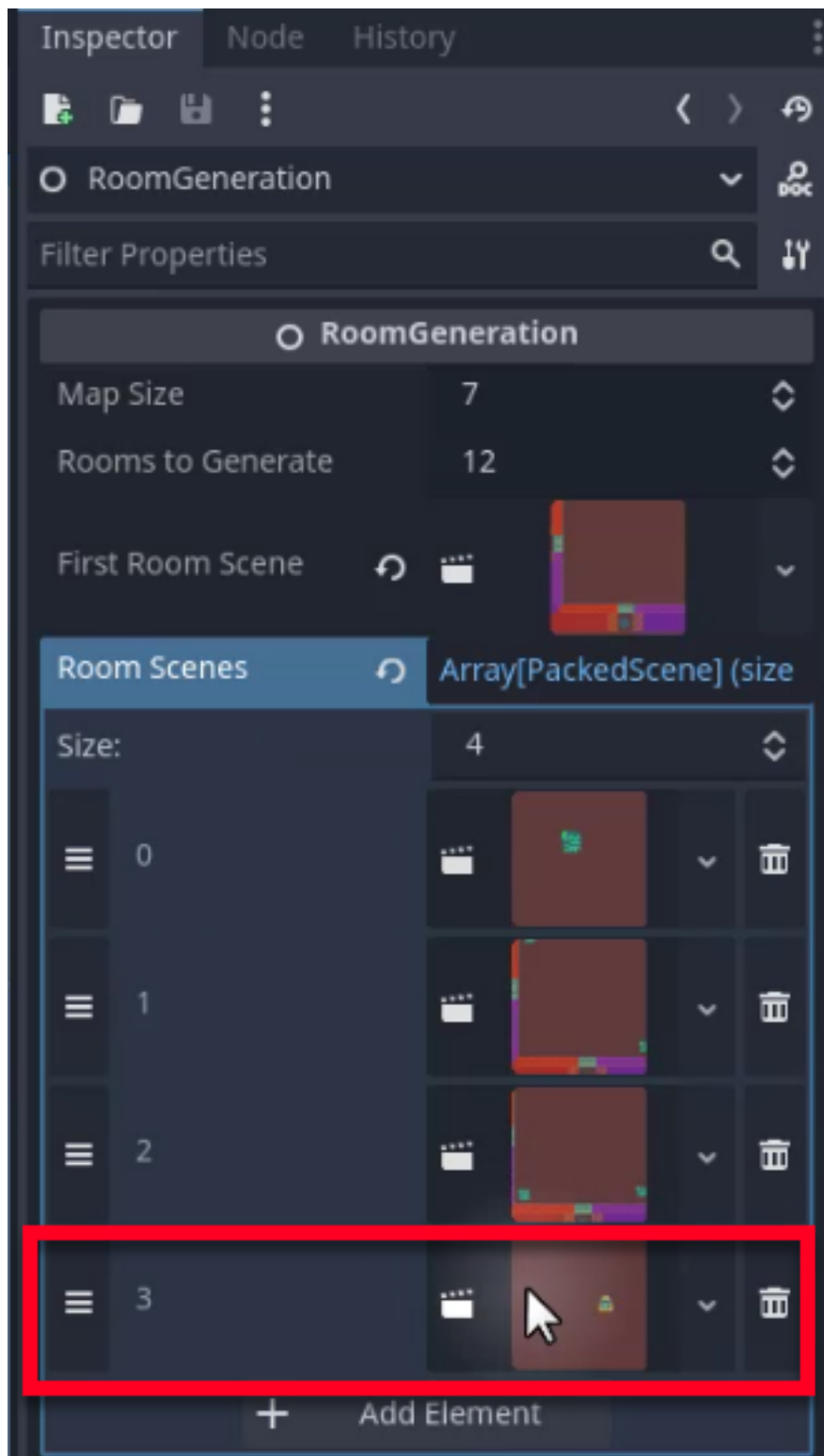
Setting Up the Item Room

To begin, let’s first create a new inherited scene from `room_template.tscn` and name it `room_item.tscn`.

Since this is a reward room, we don’t want the player to be locked in. So, select the root node and enable the **Doors Always Open** property in the Inspector. If you recall from our implementation, this ensures that our doors don’t lock at any point in time.



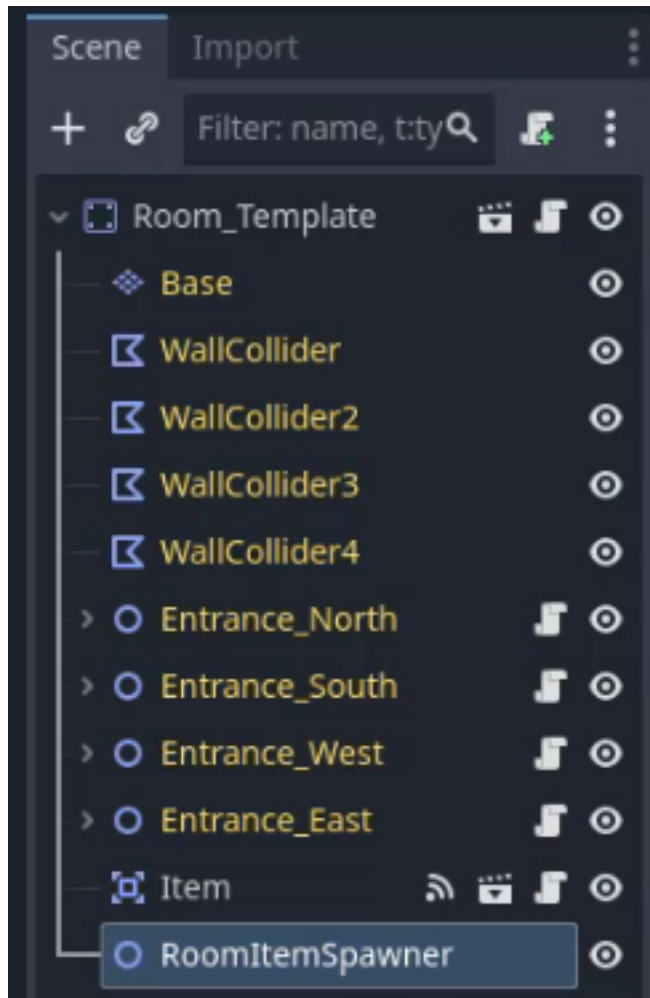
Once created, add this new room to the **Room Scenes** array in your RoomGeneration node so it has a chance to appear in the dungeon.



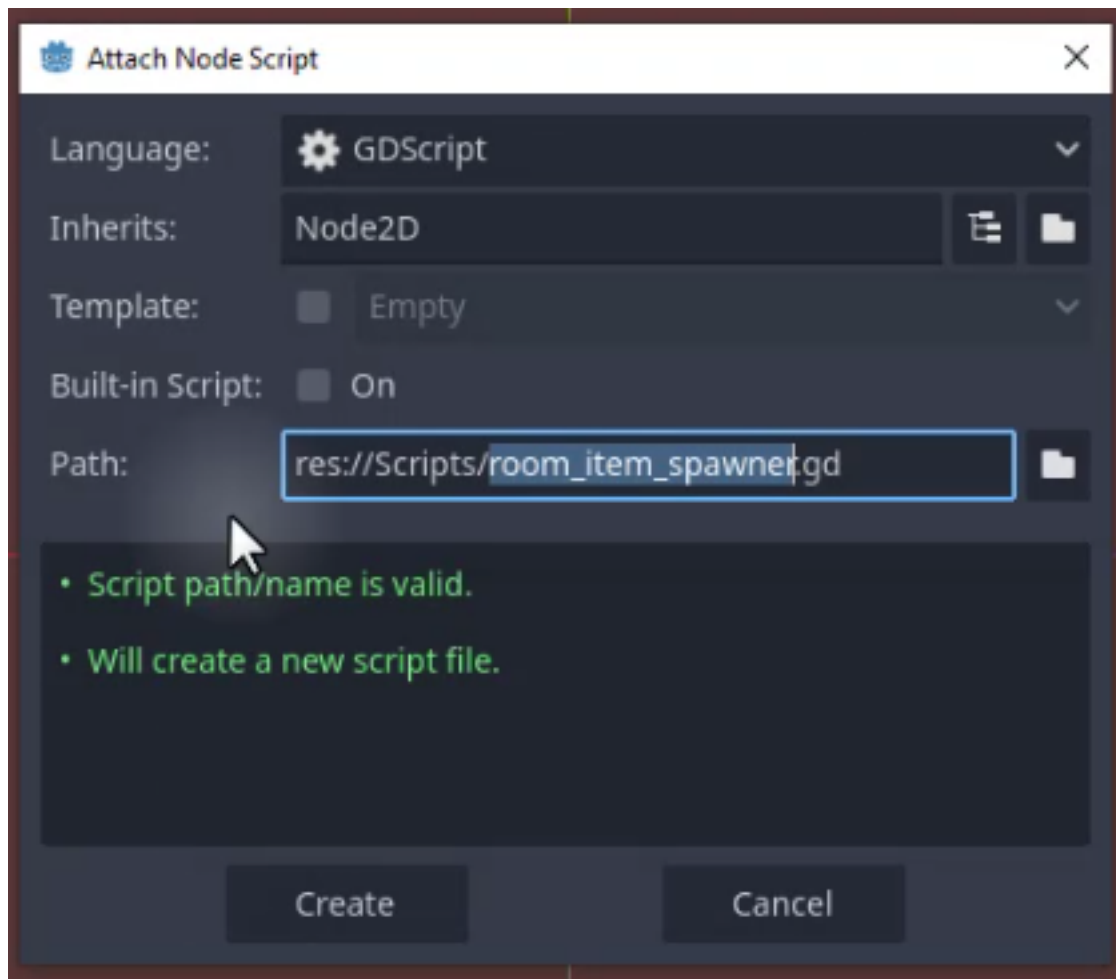
Spawning Random Items

To make the item room exciting, it should spawn a random potion each time. We will create a spawner node to handle this.

In the `room_item` scene, add a `Node2D` node and name it `RoomItemSpawner`.



We can then attach a new script to the node named `room_item_spawner.gd`.



In the script, all we need is an array to hold our potion scenes and logic to pick one at random during initialization.

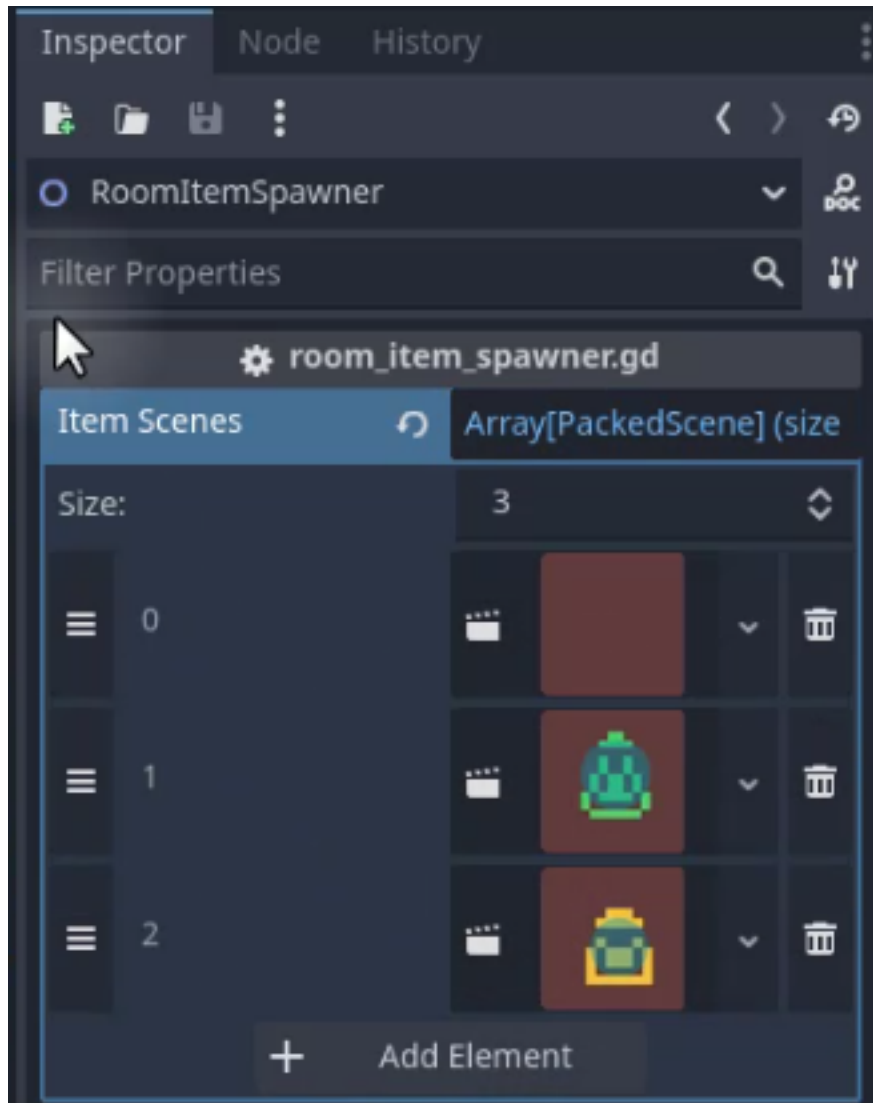
```
extends Node2D
```

```
@export var item_scenes : Array[PackedScene]
```

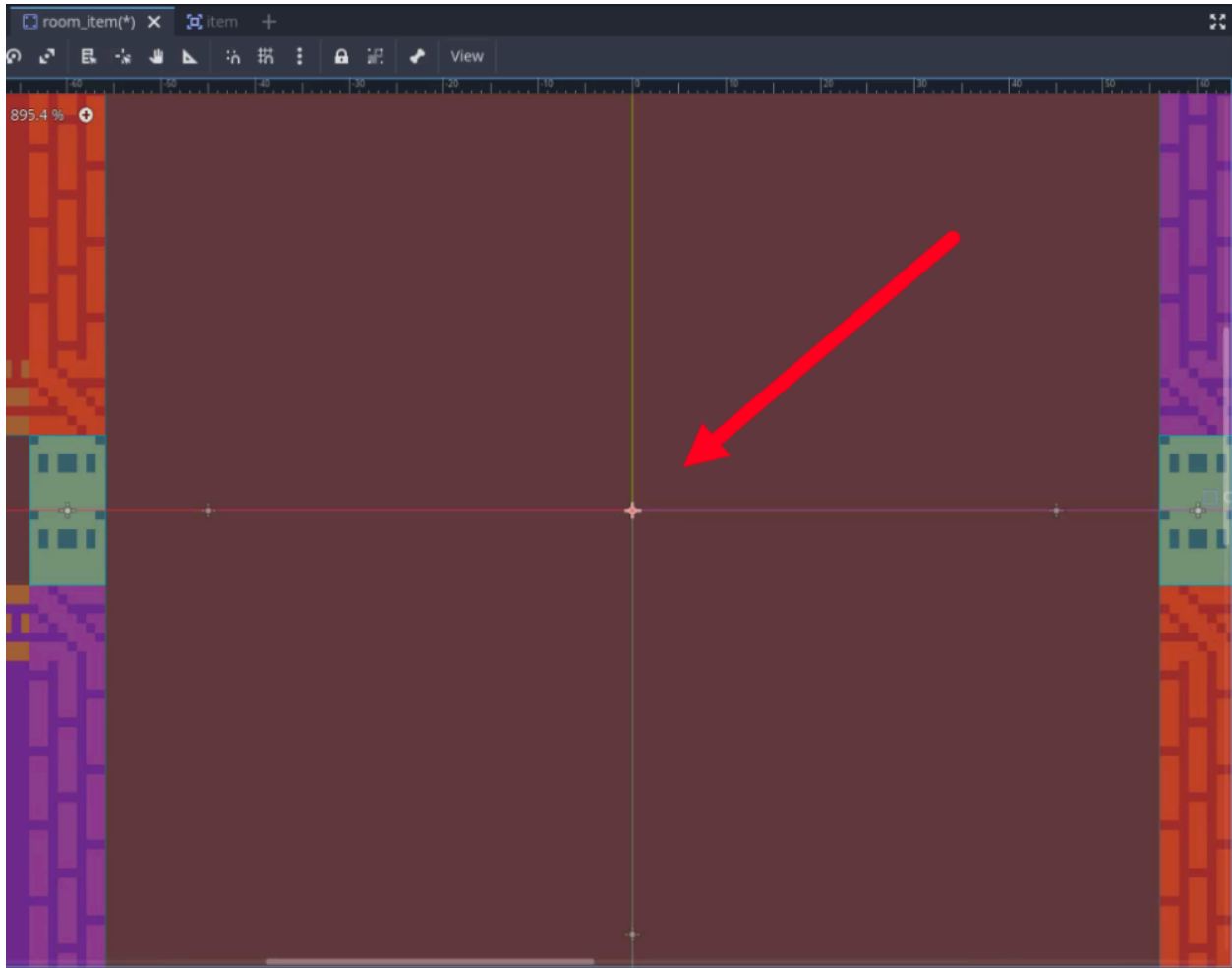
```
func _ready():
```

```
    var item = item_scenes[randi_range(0, item_scenes.size() -  
1)].instantiate()  
    add_child(item)
```

Back in the editor, select the RoomItemSpawner node. Then, in the Inspector, add your three potion scenes (item_health, item_move_speed, item_shoot_rate) to the **Item Scenes** array.

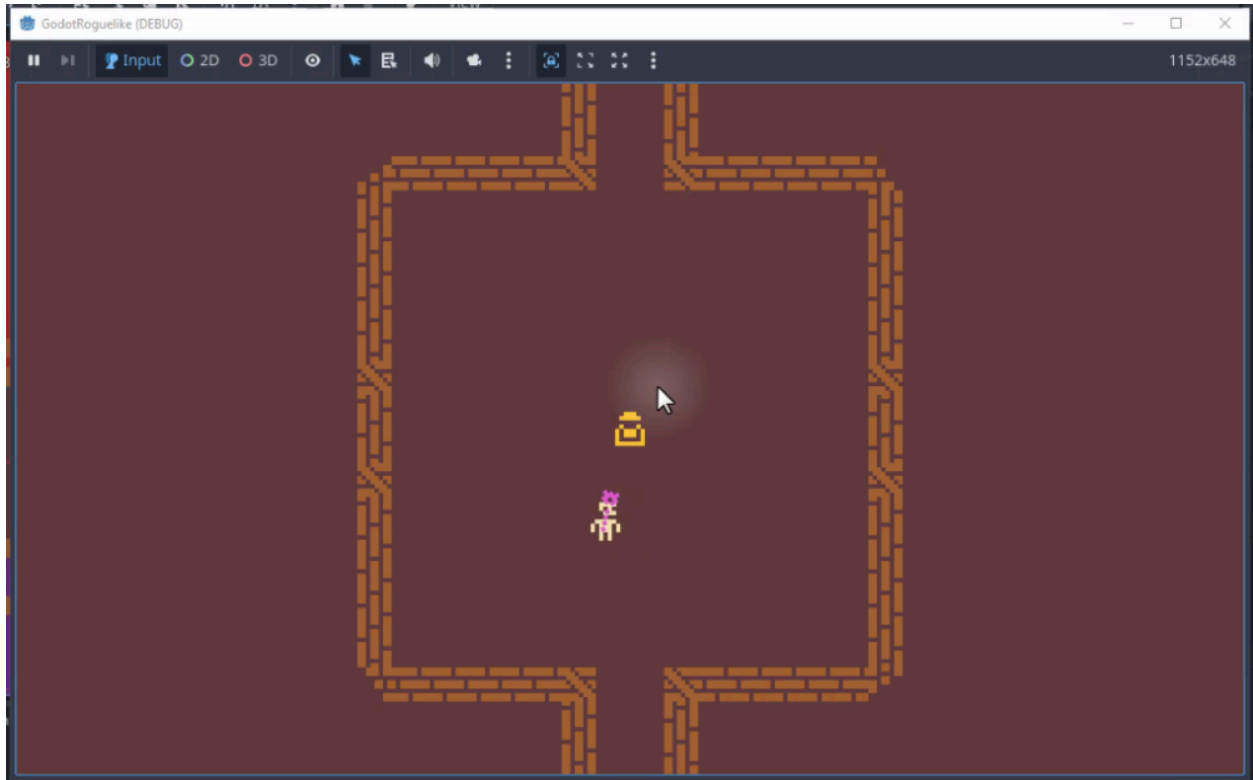


For this room, we will position the spawner in the center of the room. This way the player's attention is immediately drawn to it.



Testing the Item Room

Now, run the game and explore until you find an item room. You should see a random potion in the center. Walk over it to pick it up, and verify that it disappears. You can also use the Inspector to verify that the various stats for the Player are updated with each potion pick-up.



We now have a dungeon that generates varied combat encounters and offers rewards to the player.

In the next chapter, we will work on adding UI elements so we can communicate the player's health and map.

User Interface

In this chapter, we will focus on communicating vital information to the player through a user interface (UI). A good UI provides context, such as where the player is located and how much health they have remaining.

We will build a mini-map to help players navigate the dungeon and create a heart-based health display.

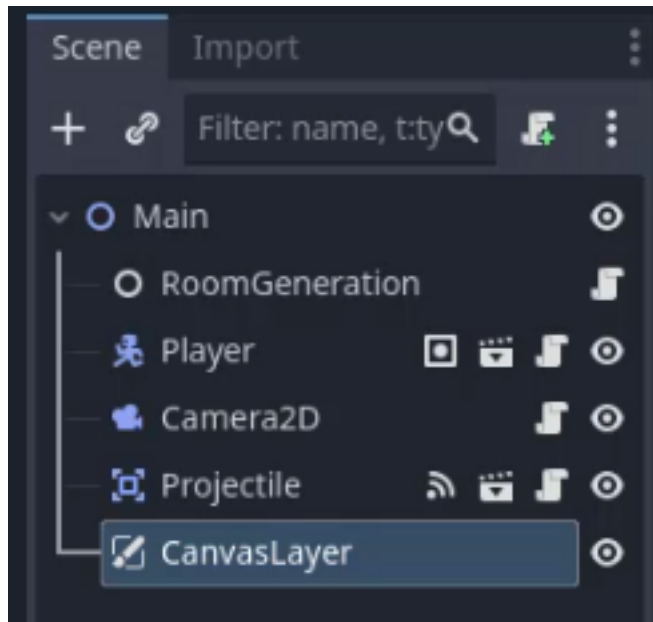
Creating the Mini-Map

The mini-map is an essential tool for dungeon crawlers, giving players a high-level view of the room layout. To create this, we first need to understand how Godot handles UI rendering.

The CanvasLayer Node

UI elements usually need to stay fixed on the screen, regardless of where the camera moves in the game world. To achieve this, we use a **CanvasLayer** node. Any node placed inside a CanvasLayer is rendered independently of the camera's transform, effectively “sticking” it to the screen.

So, in your **Main** scene, create a new node of type CanvasLayer.



Godot Insight: CanvasLayer for screen-space UI

By default, everything in a 2D scene shares the same canvas, which means it all moves together when the camera pans. A `CanvasLayer` creates a separate drawing layer with its own transform, independent of the main camera. Anything placed inside it stays fixed on screen, no matter where the camera moves in the game world.

This is exactly what we need for UI elements like health bars, mini-maps, and score displays. The `layer` property controls draw order: higher values render on top of lower ones, so a `CanvasLayer` with layer 1 (the default) always draws above the main game world at layer 0.

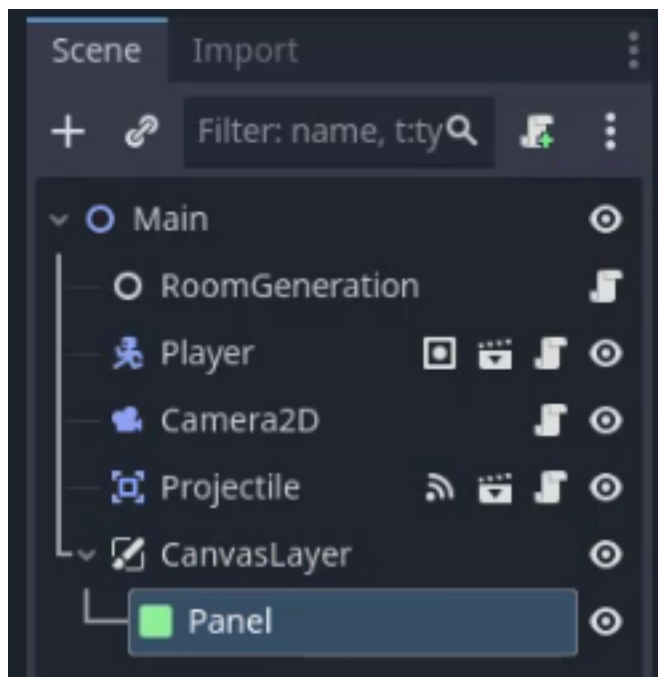
Learn more: [CanvasLayer](https://docs.godotengine.org/en/4.6/classes/class_canvaslayer.html)

(https://docs.godotengine.org/en/4.6/classes/class_canvaslayer.html)

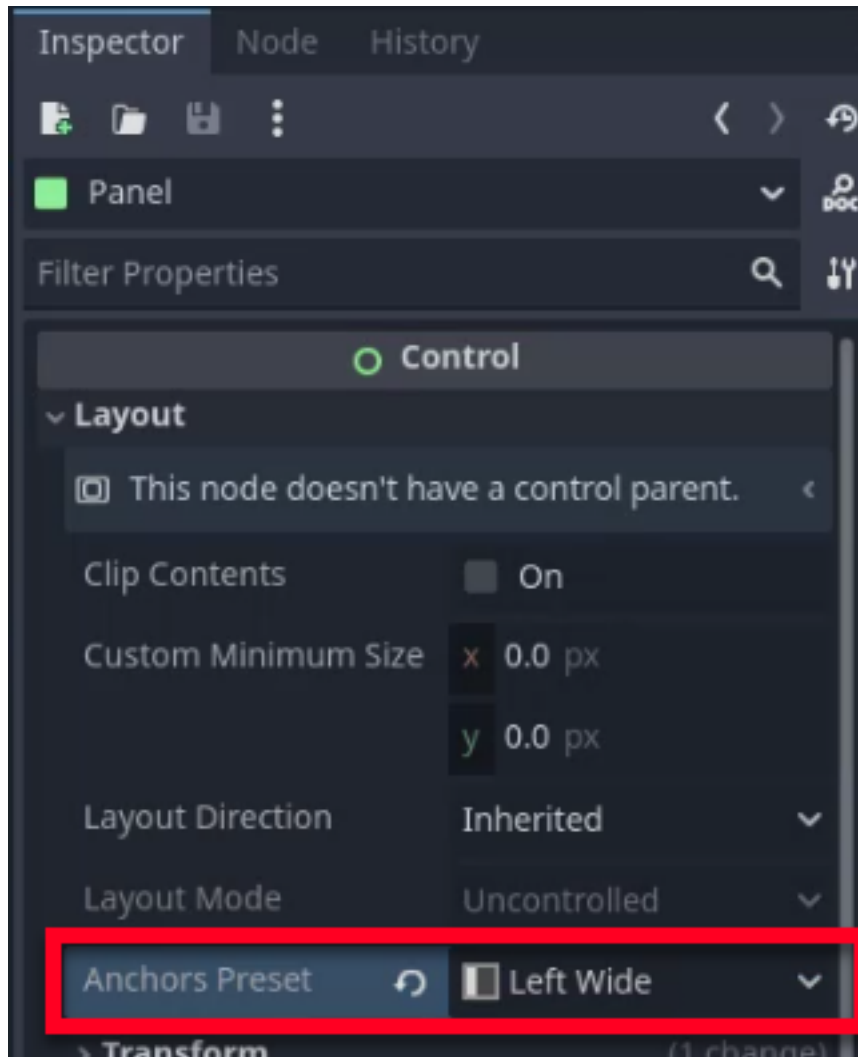
Adding UI Borders

We will frame our gameplay area with borders on the left and right sides of the screen. These borders will serve as containers for our UI elements: the health display on the left and the mini-map on the right.

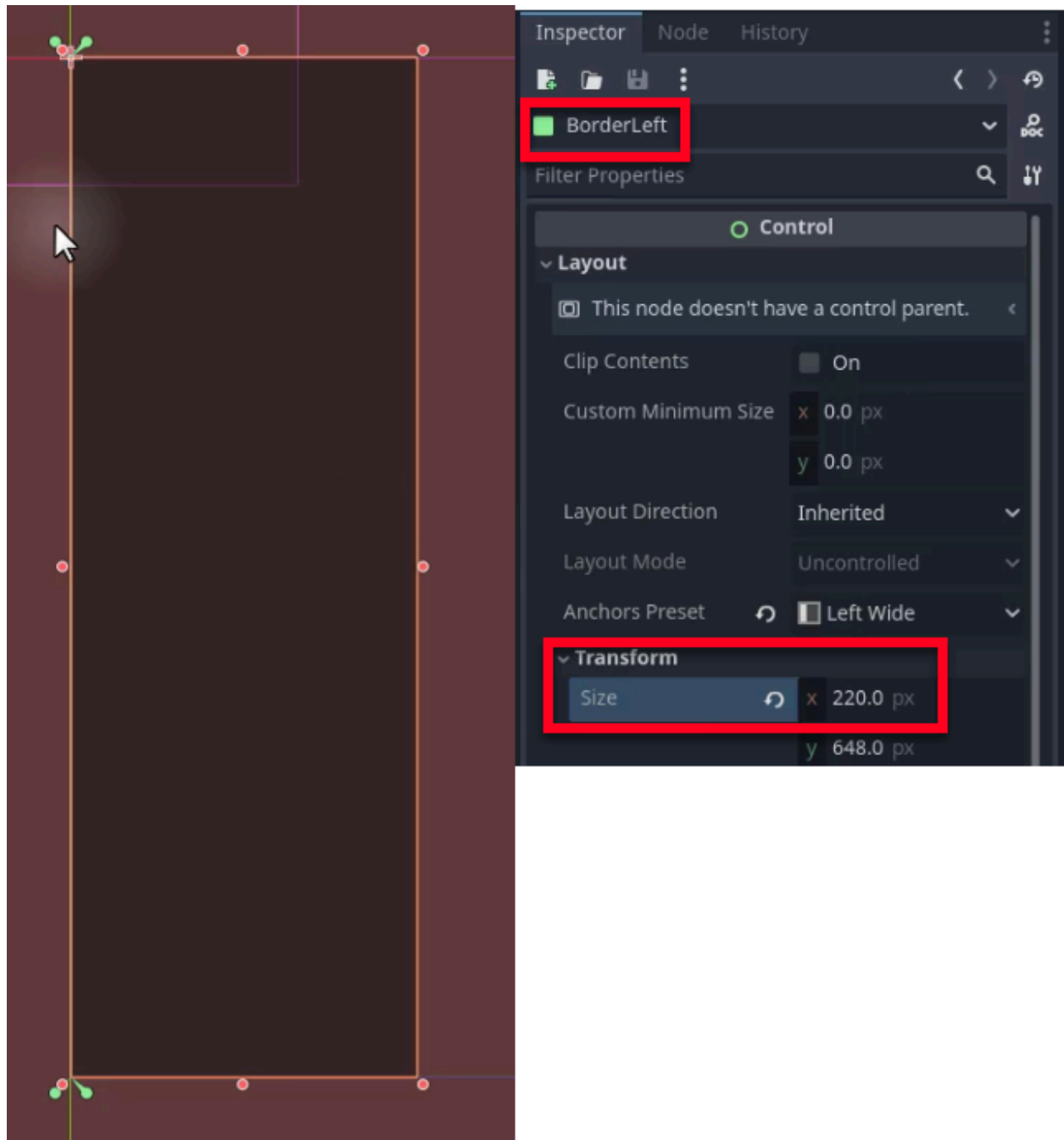
First, let's create the left border. Add a **Panel** node as a child of the `CanvasLayer`.



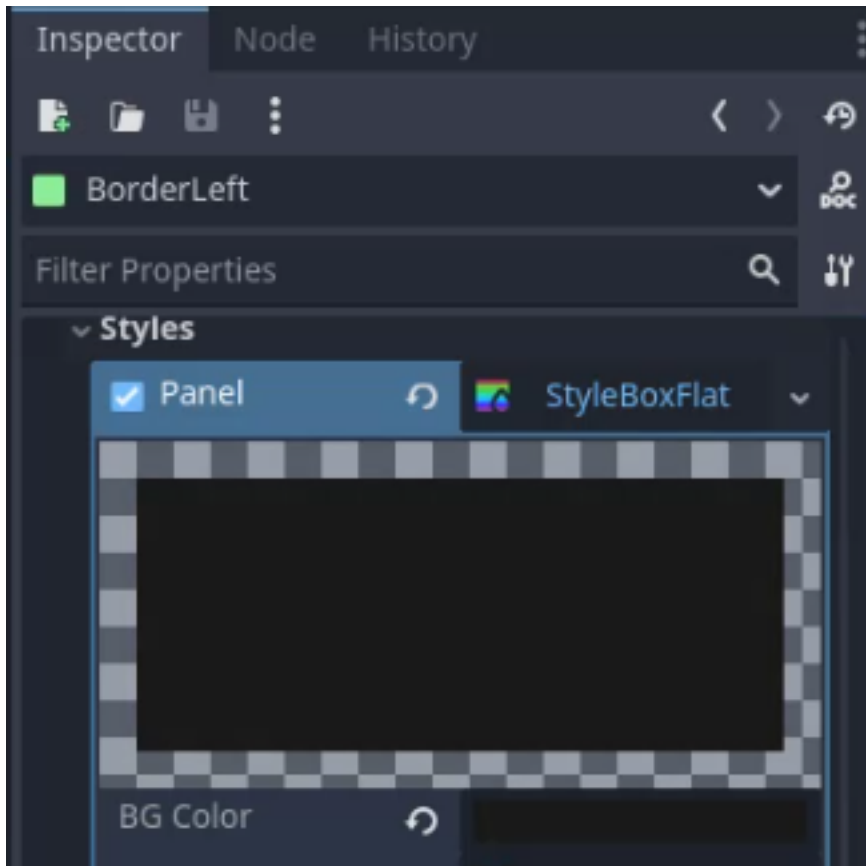
To position the panel, we use Godot's anchor system. Go to the **Layout** tab in the Inspector and set the **Anchor Preset** to **Left Wide**. This stretches the panel vertically along the left edge of the screen.



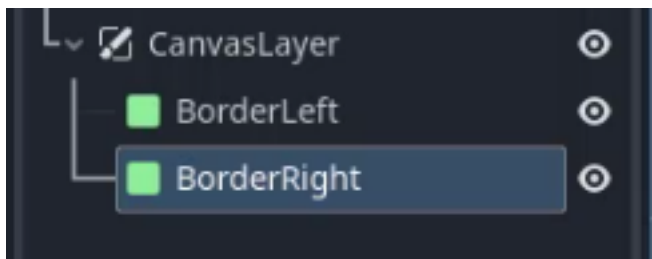
Adjust the width of the panel to approximately **220** pixels and rename it to **BorderLeft**.



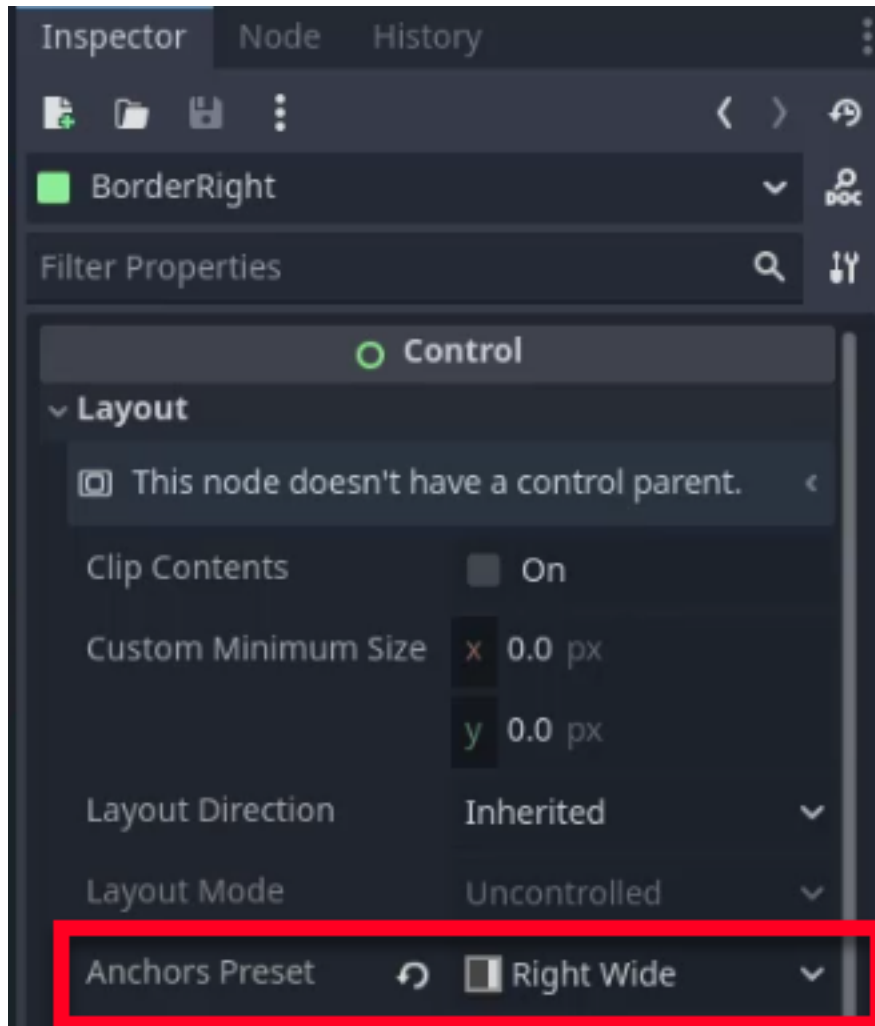
Next, let's style the panel to match our game's aesthetic. In the Inspector, go to **Theme Overrides > Styles**. We'll create a new **StyleBoxFlat** and set the **BG Color** to a dark gray (e.g., #191919).



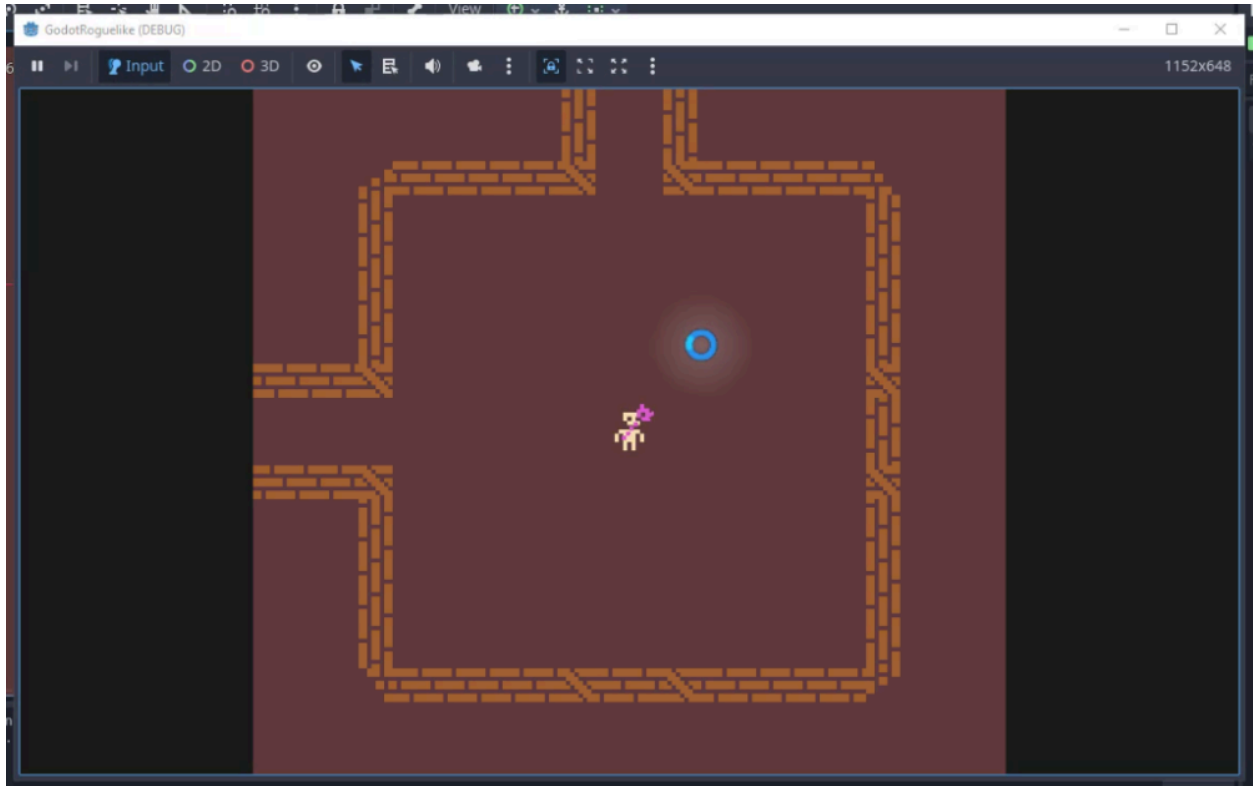
Now, let's create the right border by duplicating the BorderLeft panel. Rename the copy to BorderRight.



Then, in the Inspector for BorderRight, change the **Anchors Preset** to **Right Wide**.



If you run the game now, you will see the gameplay area framed by these two dark borders.



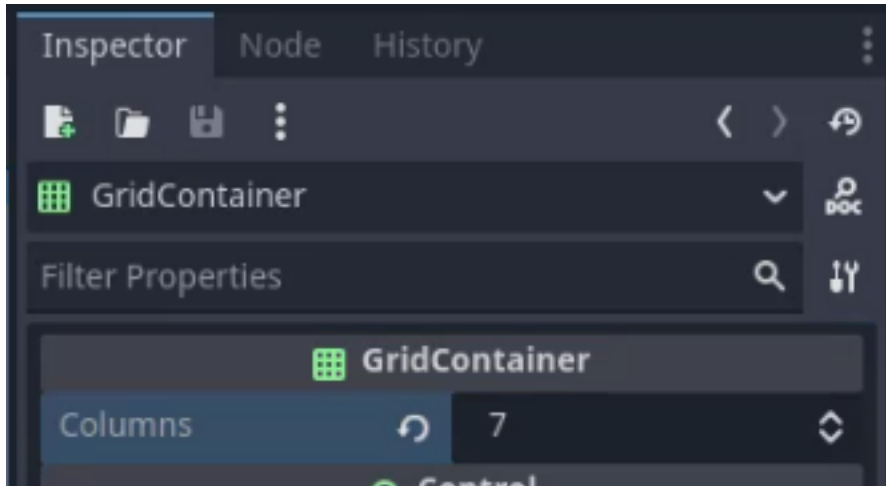
With this framing, we can now set up our mini-map.

Setting Up the Mini-Map Grid

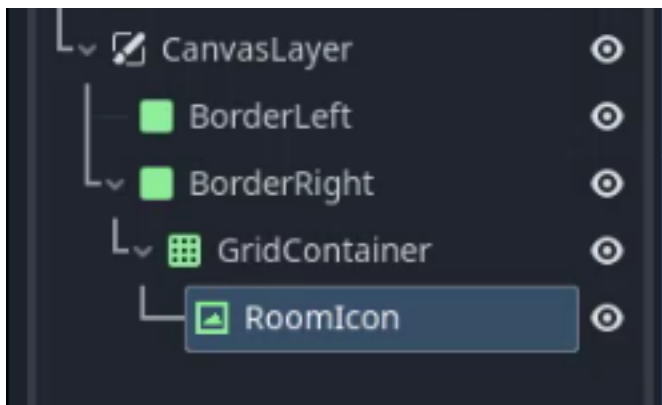
We will construct the mini-map using a grid of icons, where each icon represents a room in our 7x7 dungeon map. To begin, add a **GridContainer** node as a child of **BorderRight**.



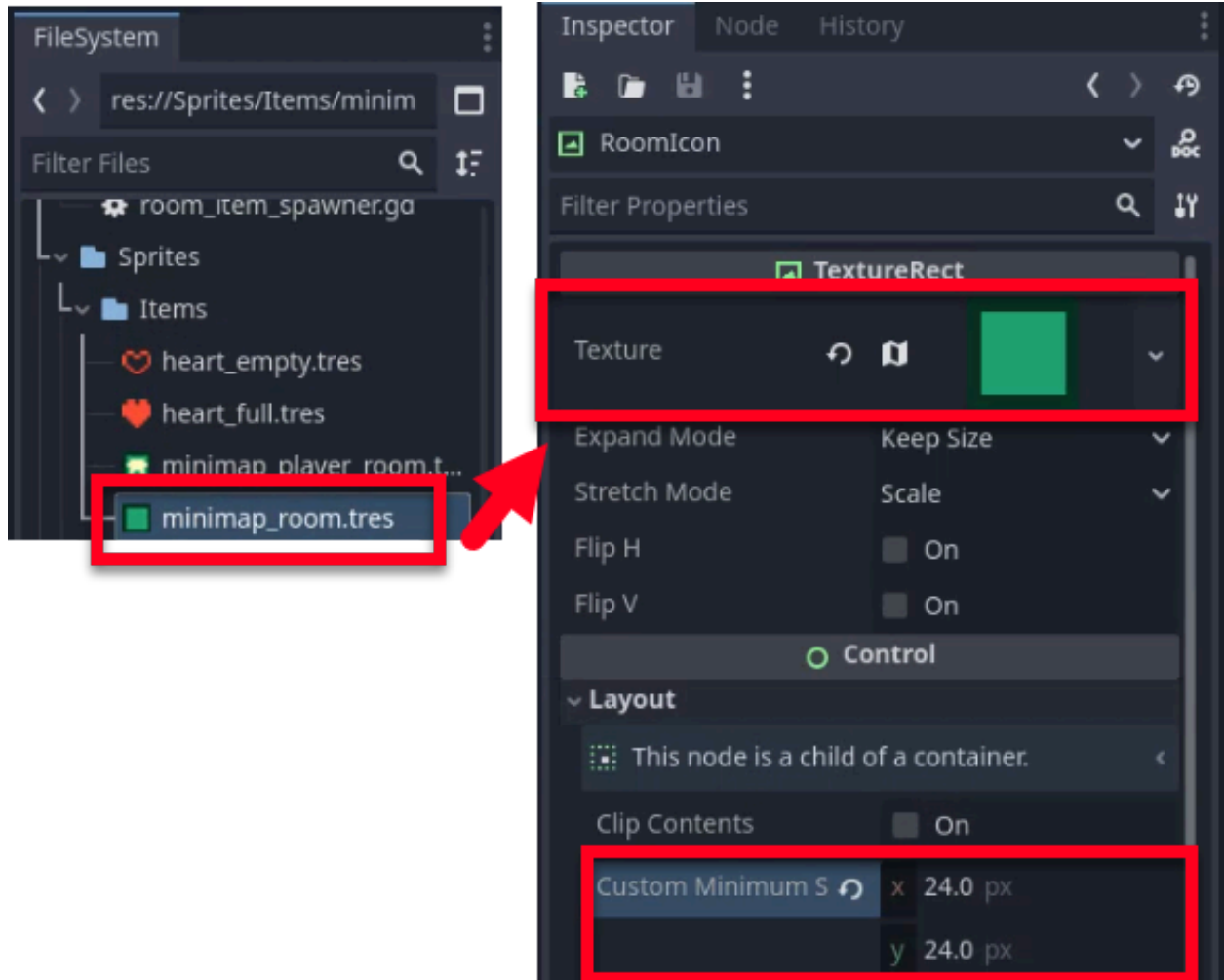
In the Inspector, set **Columns** to 7. This matches our `map_size` from the room generation script.



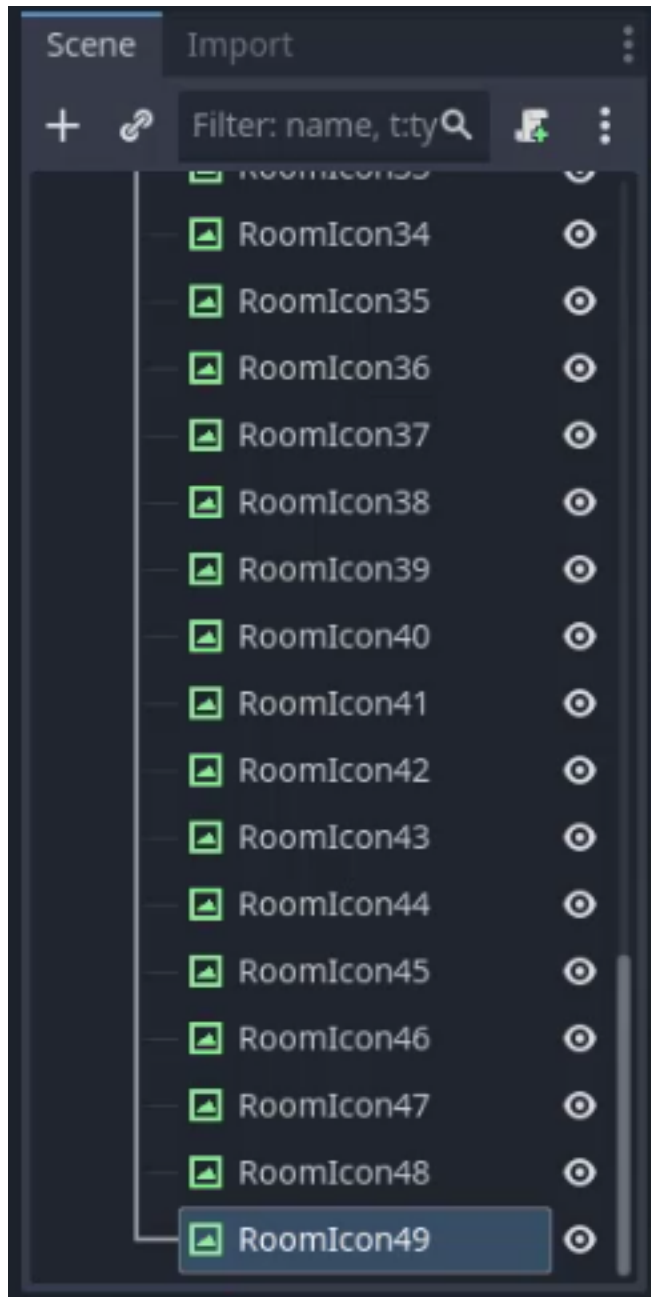
Now we need to populate this grid with icons. Let's add a **TextureRect** node as a child of the GridContainer and rename it to RoomIcon.



For RoomIcon, assign the `minimap_room.tres` texture (from Sprites/Items) to the **Texture** property. Then, set the **Custom Minimum Size** to 24x24 pixels so the icon is large enough to see clearly.

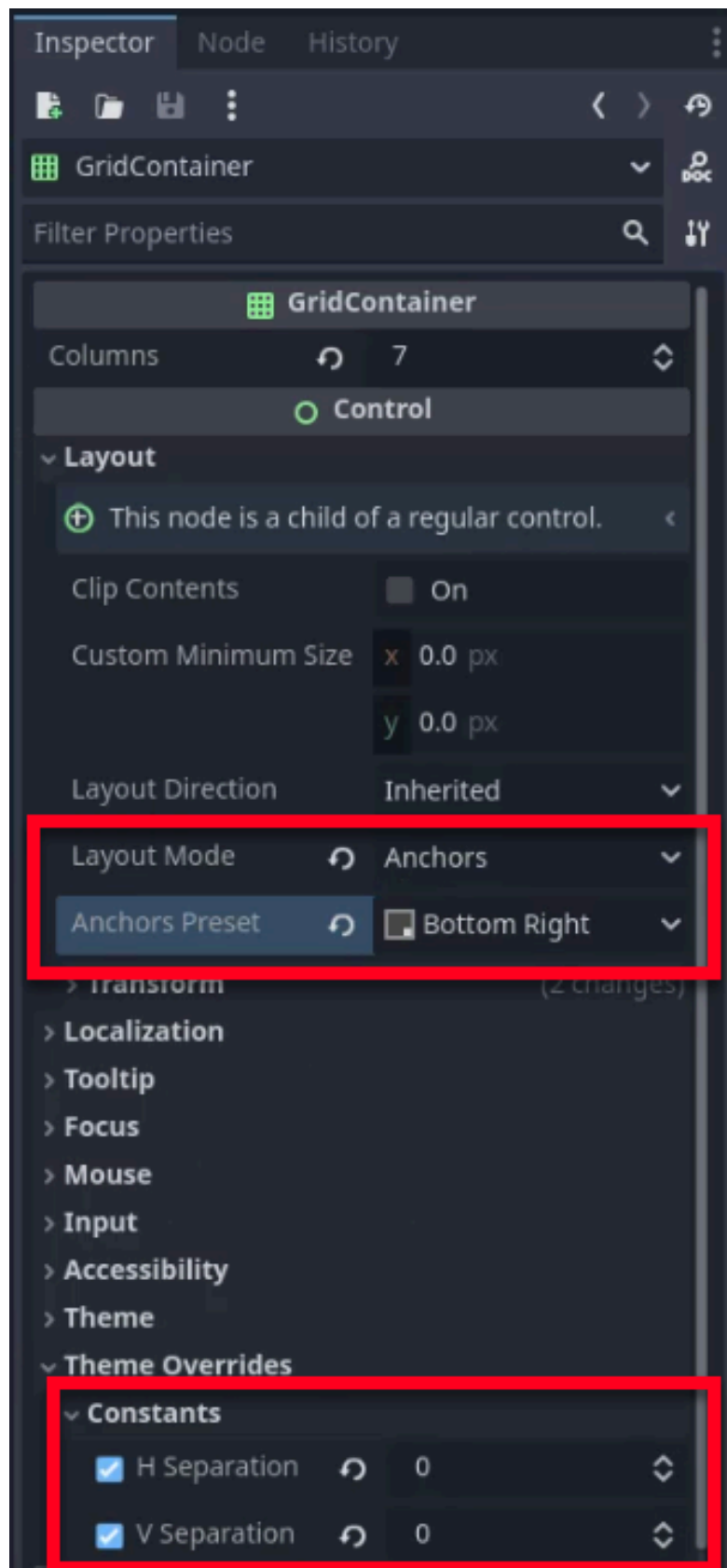


To create the full grid, we need to duplicate the RoomIcon node until we have 49 icons total (7 rows $\times$ 7 columns).

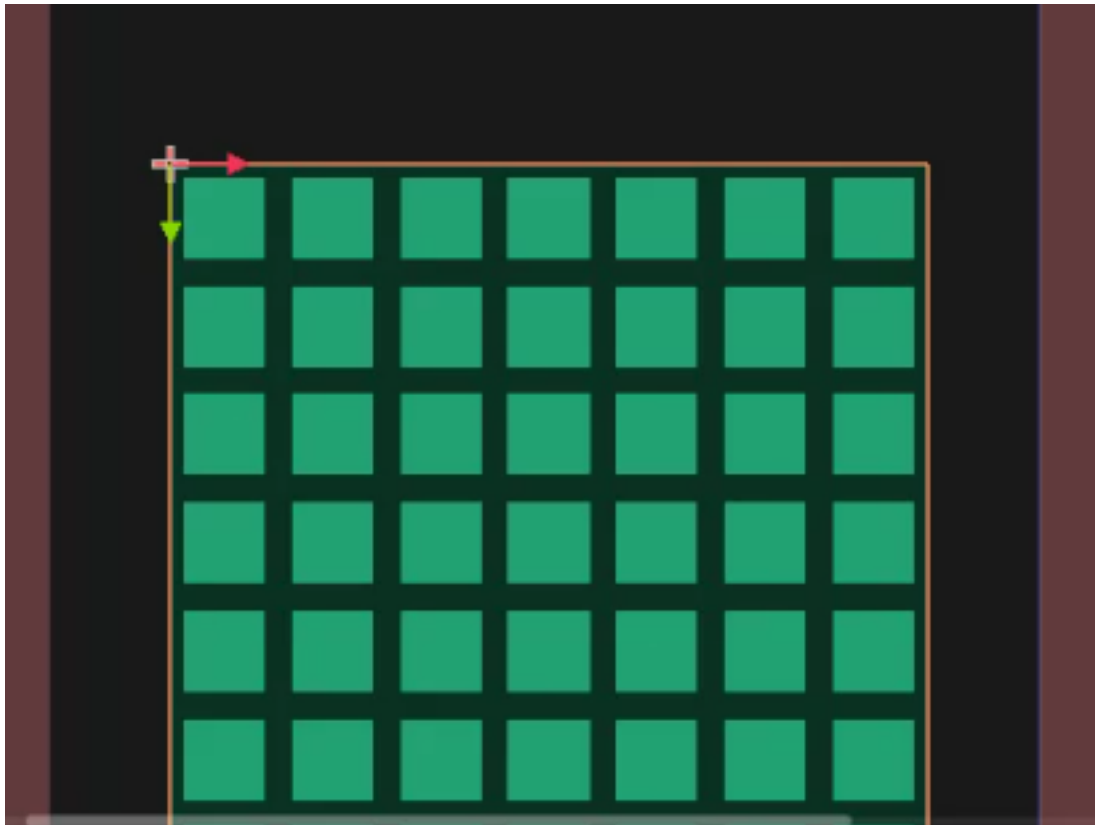


To tighten the layout, let's remove the spacing between icons. In the GridContainer properties, go to **Theme Overrides > Constants** and set **H Separation** and **V Separation** to 0.

Finally, we want to position the mini-map at the bottom of the right border by setting the **Anchors Preset** of the GridContainer to **Bottom Right**. You may need to manually adjust the position slightly to add padding.



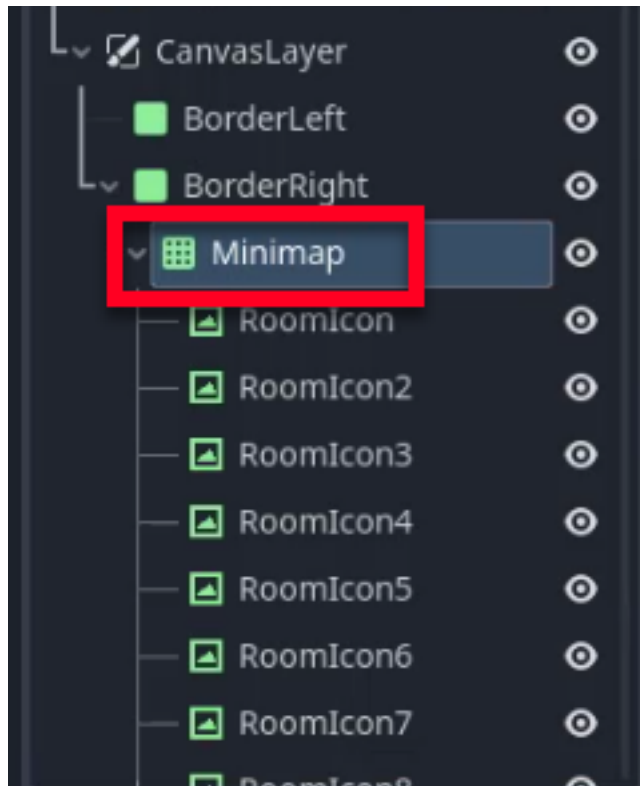
The result should look like a compact grid of green squares in the bottom-right corner.



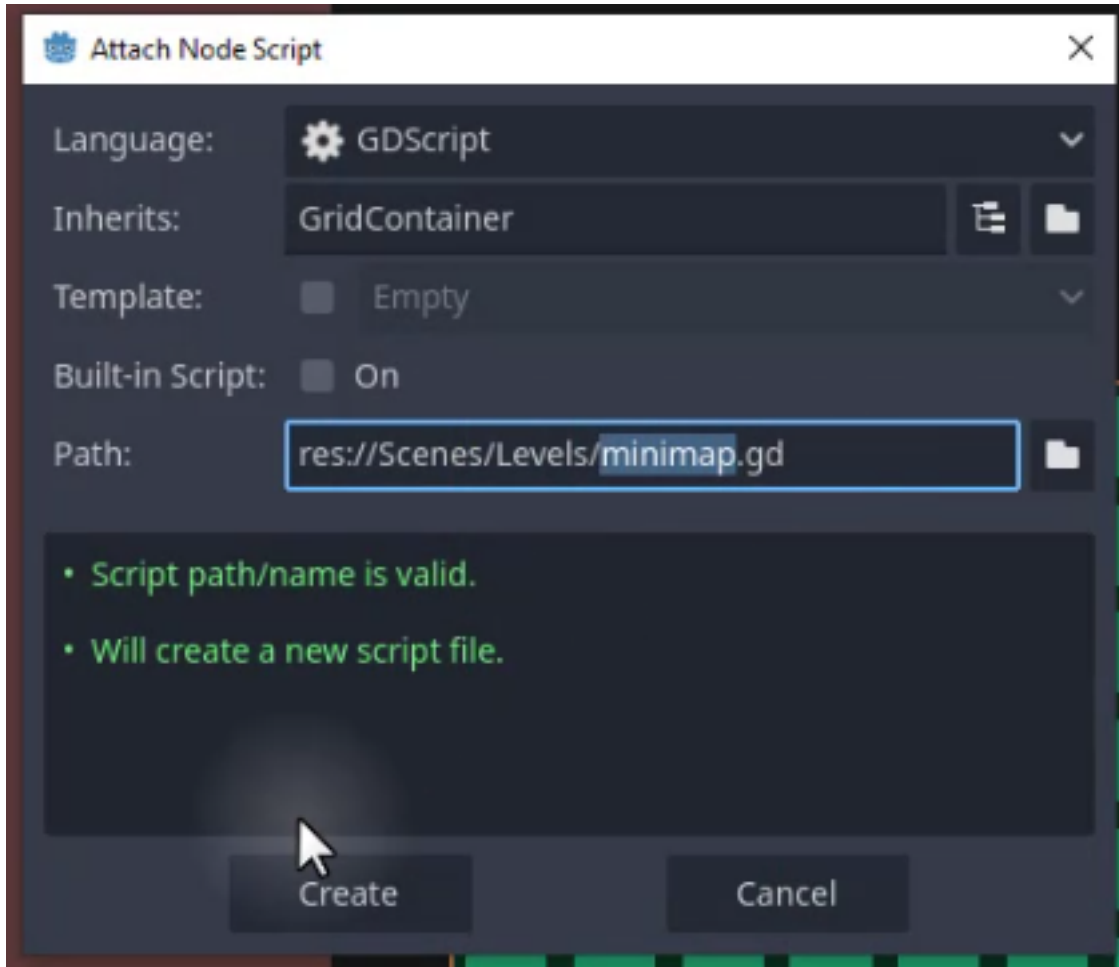
Scripting the Mini-Map Logic

The visual grid is ready, but it needs logic to update as the player moves as well as generate the correct layout of our actual map. We will write a script that listens for the player's movement and updates the icons accordingly.

First, let's rename the `GridContainer` node to `Minimap`.



Then, we will attach a new script named `minimap.gd`.



In the script, we need references to the room generation system and our icon textures. Note that for our textures, which we preload, we have one for an empty room and one for a room where the player is in.

```
extends GridContainer
```

```
var room_gen : RoomGeneration
```

```
var icons : Array[TextureRect]
```

```
var room_texture : Texture =
```

```
preload("res://Sprites/Items/minimap_room.tres")
```

```
var player_room_texture : Texture =
```

```
preload("res://Sprites/Items/minimap_player_room.tres")
```

We will next create a function called `_on_player_enter_room` that updates the grid. This function needs to:

1. Find the RoomGeneration node (if not already found).
2. Populate the icons array with the children of the grid.
3. Loop through every slot in the grid (0 to 48).
4. Check if a room exists at that coordinate.
5. Update the icon texture: empty if no room exists, player_room_texture if the player is there, or room_texture otherwise.

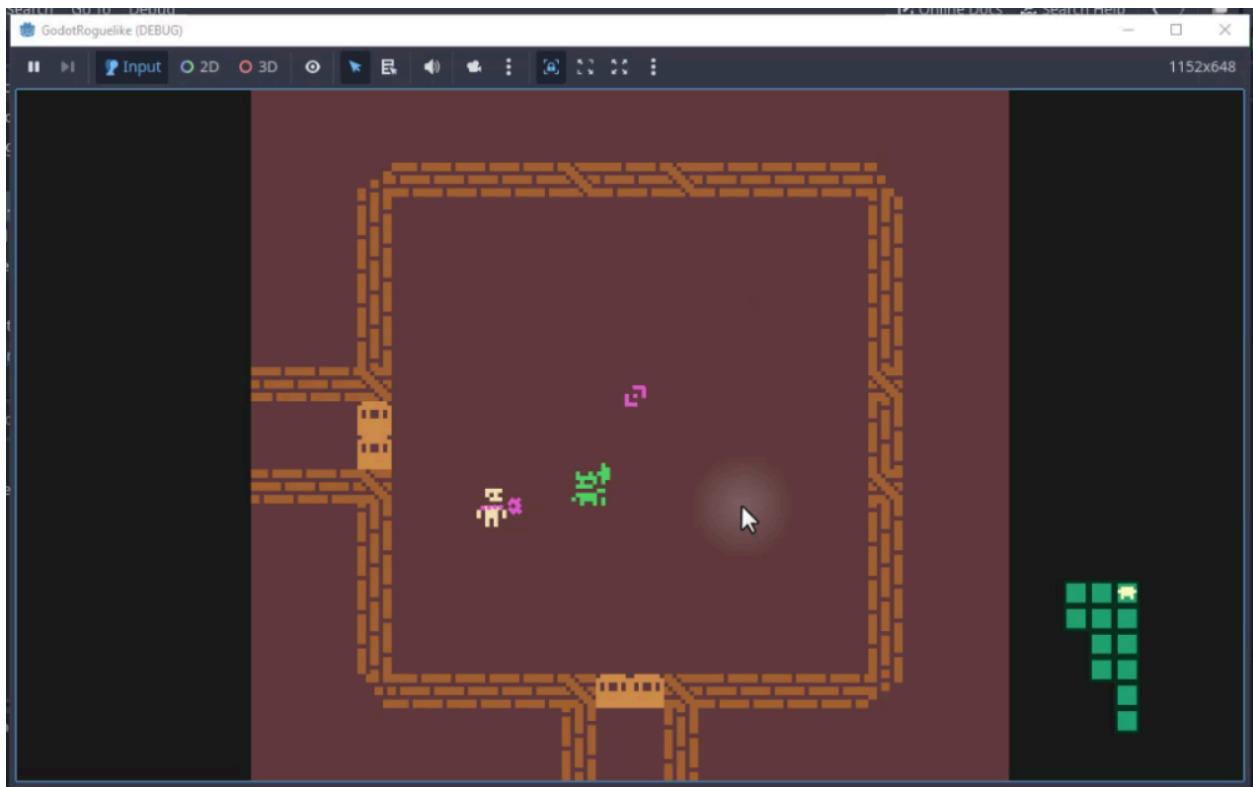
```
func _on_player_enter_room(room : Room):
    # Lazy init: fetch room_gen and icon references on first call
    if not room_gen:
        room_gen = get_node("/root/Main/RoomGeneration")
        for child in get_children():
            if child is TextureRect:
                icons.append(child)

    for x in range(room_gen.map_size):
        for y in range(room_gen.map_size):
            var r = room_gen.get_room_from_map(x, y)
            # Convert 2D grid coords to a 1D array index
            var i = x + y * room_gen.map_size
            if i >= len(icons):
                continue
            # Three visual states: empty (null), current room, or
            # visited room
            if not r:
                icons[i].texture = null
            elif r == room:
                icons[i].texture = player_room_texture
            else:
                icons[i].texture = room_texture
```

Finally, we can connect this function to our global signal in `_ready`. Since our signal is called right when the player is placed in the first room, the mini-map will be ready right as the game starts.

```
func _ready():
    GlobalSignals.OnPlayerEnterRoom.connect(_on_player_enter_room)
```

Now, as you explore the dungeon, the mini-map will highlight your current room and show the layout of the surrounding rooms.

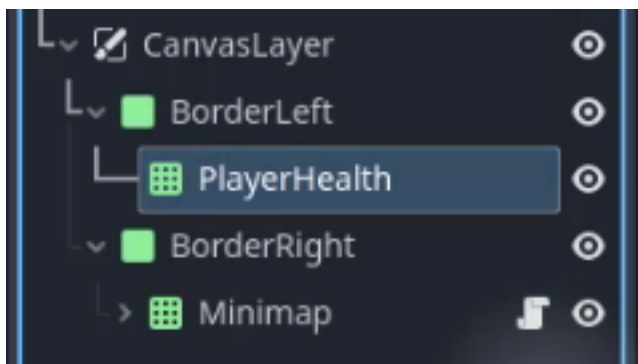


Player Health UI

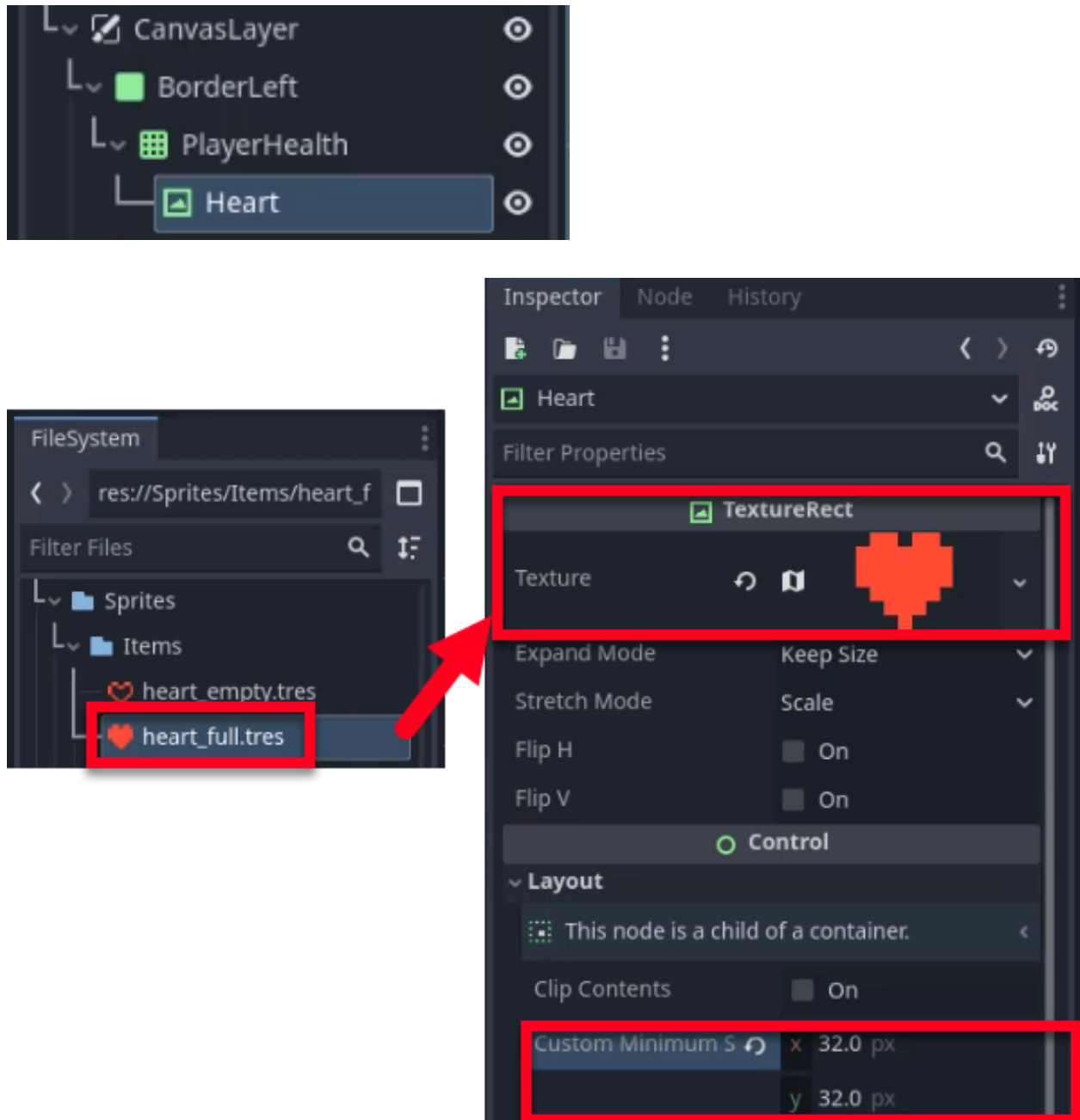
Next in our UI journey, we will create the health display on the left side of the screen. We will use a similar grid approach, but this time for heart icons.

Creating the Health Container

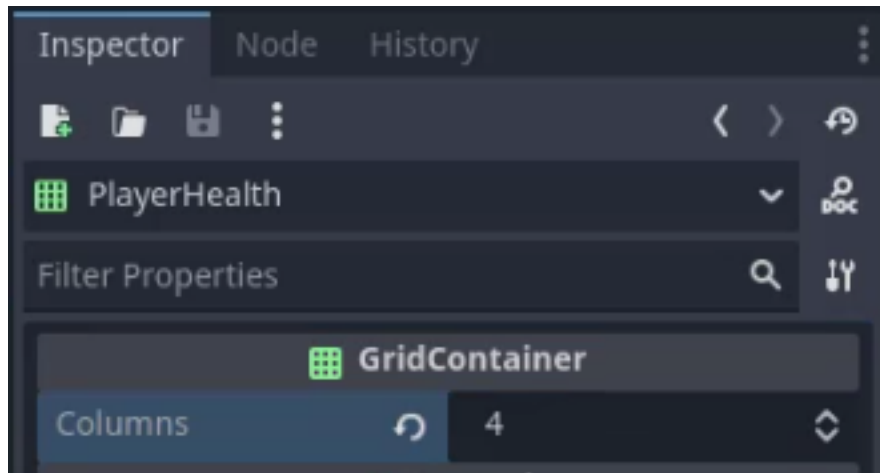
To start, add a **GridContainer** node as a child of **BorderLeft** and rename it to **PlayerHealth**.



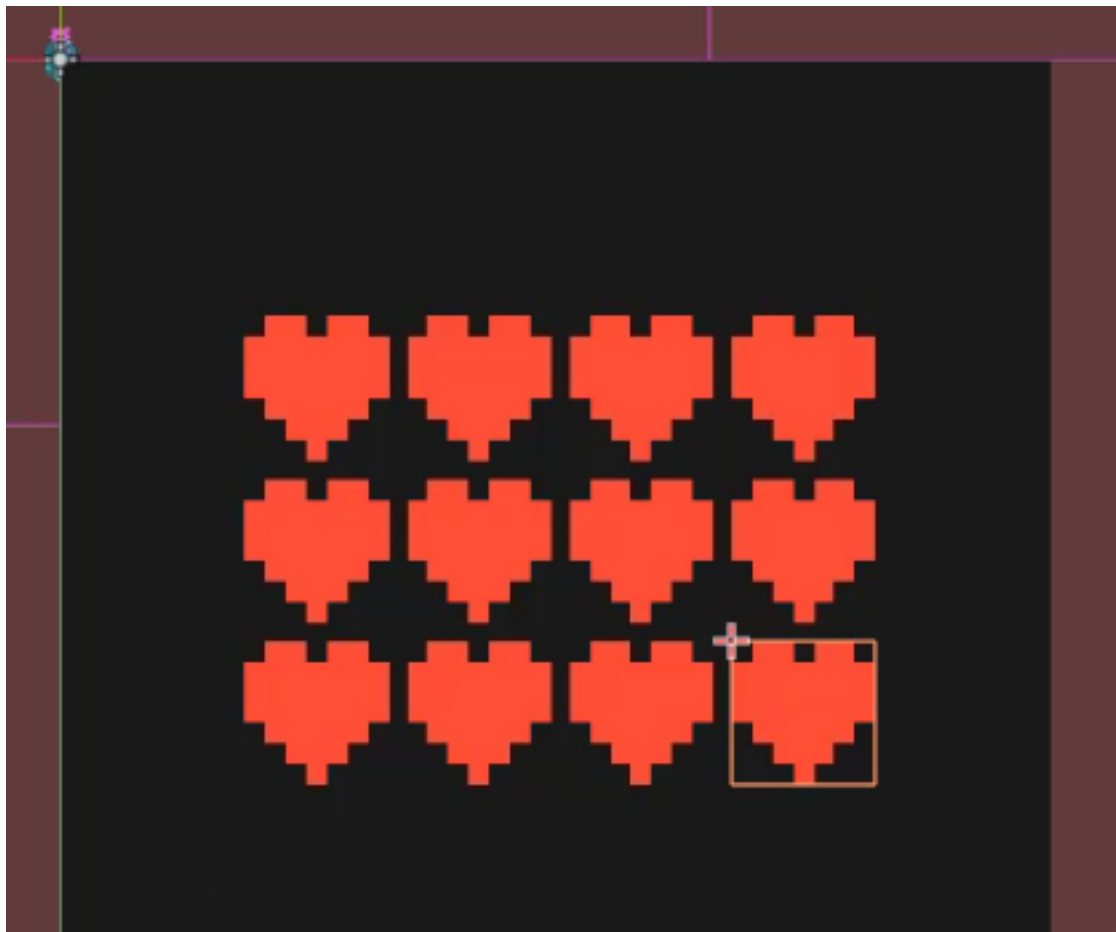
Next, we will add a **TextureRect** child to **PlayerHealth** and rename it to **Heart**. In the Inspector, assign the `heart_full.tres` texture and set the **Custom Minimum Size** to 32x32 pixels.



To fit within our border, let's set the **PlayerHealth** container's **Columns** to 4.

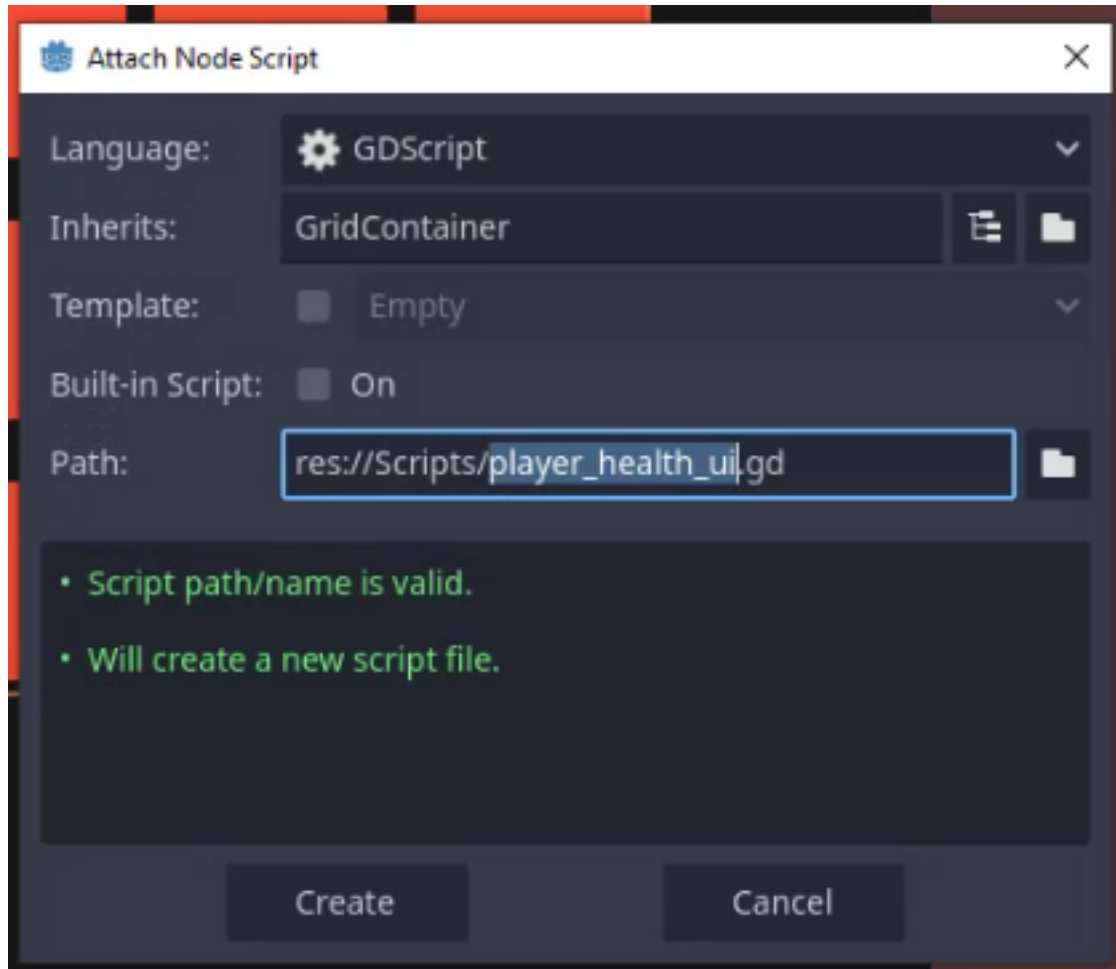


Finally, duplicate the Heart node until you have 12 hearts. This gives us a maximum displayable health of 12.



Scripting the Health UI

Our visuals are ready, so let's attach a new script named `player_health_ui.gd` to the `PlayerHealth` node so we can add our logic.



This script needs to update the hearts based on the player's current and maximum health. It will display a full heart for health we have, and an empty heart for health we don't.

The first thing we do is add variables to store our textures and an array to store our individual heart icons. We can then use the following logic to set our UI display up:

1. In `_ready`, we get all the heart icons from our scene tree and add them to our `heart_icons` array.
2. To update the UI, we loop through our array to dynamically toggle our heart icons.

- a. We first hide any icons that are beyond the value of max health.
- b. Next, we toggle whether the texture displayed is a full heart or an empty heart based on our current health value.

```
extends GridContainer

var full_heart : Texture =
preload("res://Sprites/Items/heart_full.tres")
var empty_heart : Texture =
preload("res://Sprites/Items/heart_empty.tres")

var heart_icons : Array[TextureRect]

func _ready():
    for child in get_children():
        if child is TextureRect:
            heart_icons.append(child)

# Sync heart icons with the player's current and max health
func _update_ui (cur_hp : int, max_hp : int):
    for i in len(heart_icons):
        # Hide icons beyond max health
        if i >= max_hp:
            heart_icons[i].visible = false
            continue

        heart_icons[i].visible = true

        # Full heart for remaining HP, empty heart for lost HP
        if i < cur_hp:
            heart_icons[i].texture = full_heart
        else:
            heart_icons[i].texture = empty_heart
```

Connecting the Health Signal

Although we have the functionality in place, we need a way for the player to tell the UI that their health has changed. We will add a new signal to our global event bus, so open `global_signals.gd` and add the following signal:

```
signal OnPlayerUpdateHealth(cur_hp : int, max_hp : int)
```

Now, update the `player.gd` script to emit this signal whenever health changes in both `take_damage` and `heal`.

```
func take_damage (amount : int):
    cur_hp -= amount

    GlobalSignals.OnPlayerUpdateHealth.emit(cur_hp, max_hp)

    if cur_hp <= 0:
        die()

func heal (amount: int):
    cur_hp += amount
    if cur_hp > max_hp:
        cur_hp = max_hp

    GlobalSignals.OnPlayerUpdateHealth.emit(cur_hp, max_hp)
```

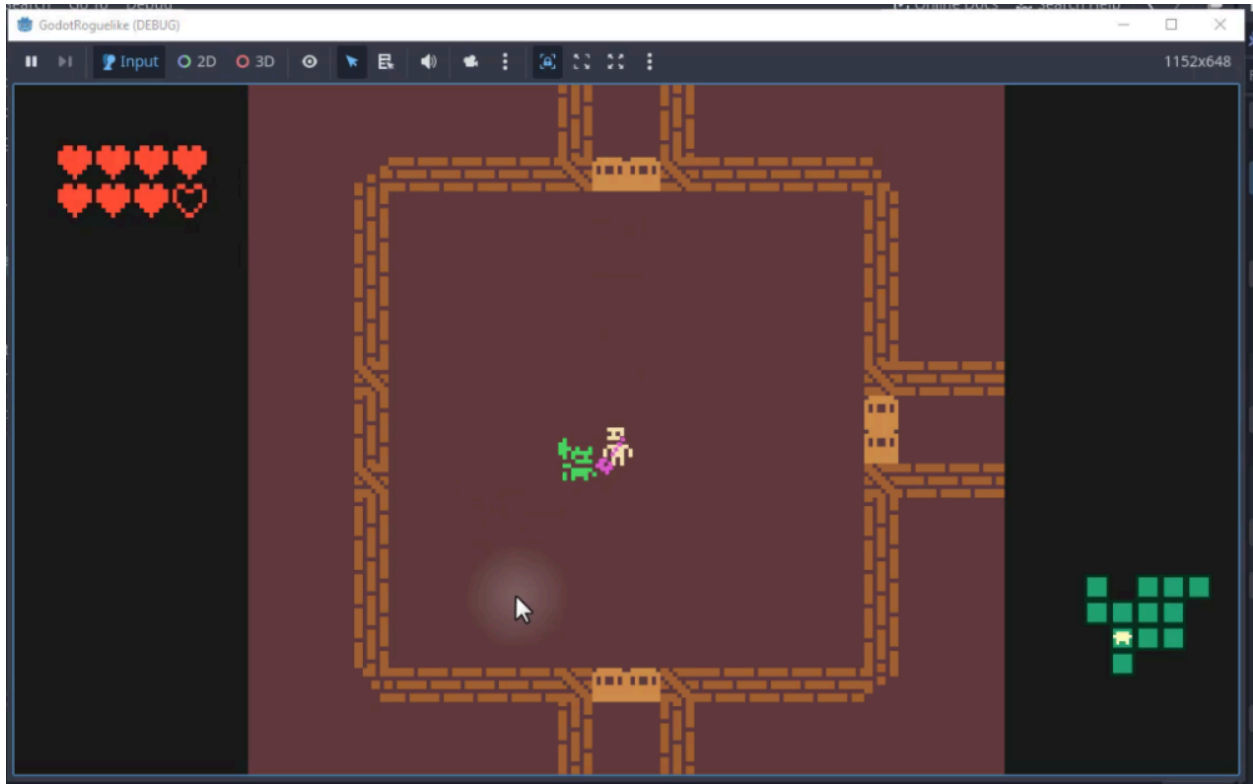
We also want to update the UI immediately when the game starts. So, emit the signal in the `_ready` function as well:

```
func _ready (amount: int):
    GlobalSignals.OnPlayerUpdateHealth.emit.call_deferred(cur_hp,
max_hp)
```

Finally, we need to connect to the signal in the `player_health_ui.gd` script. We use `_init` here to ensure the connection is established before the player emits the initial signal in `_ready`.

```
func _init():
    # Connect in _init (not _ready) so the signal is wired before the
first emit
    GlobalSignals.OnPlayerUpdateHealth.connect(_update_ui)
```

The health UI is now fully functional. Hearts will empty as you take damage and refill when you heal. Consequently, the player will have a clear understanding of where their health is at.



In this chapter, we have successfully built the core UI elements. The player can now track their health and navigate the dungeon with a mini-map. In the next chapter, we will enhance the atmosphere further by adding sound effects, music, and a useful visual effect.

Audio and Visual Feedback

In this chapter, we will focus on two major aspects of game feel: audio and visual feedback. Audio is a crucial element that enhances the player's experience by providing immediate feedback and immersion. We will add sound effects for actions such as shooting, taking damage, and collecting items, as well as background music to set the mood.

Meanwhile, visual feedback will help us vastly enhance the combat experience, making interactions feel responsive and impactful. In this case, we will implement a damage flash effect to clearly indicate when characters take hits.

Visual Feedback: Damage Flash

To make combat feel more responsive, we will add a “damage flash” effect. When a character takes damage, their sprite will briefly flash invisible, providing immediate visual confirmation of the hit.

Implementing Flash on Enemies

Let's first set this up for enemies by opening the `enemy.gd` script. Here, we will create a `_damage_flash` function that toggles visibility off and on with a short delay.

```
# Brief visibility toggle to give visual damage feedback
func _damage_flash():
    visible = false
    # await pauses this function until the timer fires, then resumes
    await get_tree().create_timer(0.07).timeout
    visible = true
```

We then simply call this function inside `take_damage`:

```
func take_damage(amount: int):
    cur_hp -= amount
    _damage_flash()
```

```
if cur_hp <= 0:
    die()
```

Godot Insight: Coroutines with await

The `await` keyword pauses a function's execution until a signal fires, then resumes exactly where it left off. In `_damage_flash()`, we create a one-shot timer with `get_tree().create_timer(0.07)` and await its `timeout` signal. This gives us a clean, readable way to write “wait 70 milliseconds, then continue” without manually managing Timer nodes or state flags.

Under the hood, `await` turns the function into a coroutine. The rest of the game keeps running normally while the coroutine is paused, so there is no performance cost. This pattern is useful for any short, timed sequence: screen shake, brief invincibility frames, fade-in effects, and more.

Learn more: [SceneTree](#)

(https://docs.godotengine.org/en/4.6/classes/class_scenetree.html)

Implementing Flash on the Player

We can now apply the exact same logic to the player. Open `player.gd` and add the same `_damage_flash` function.

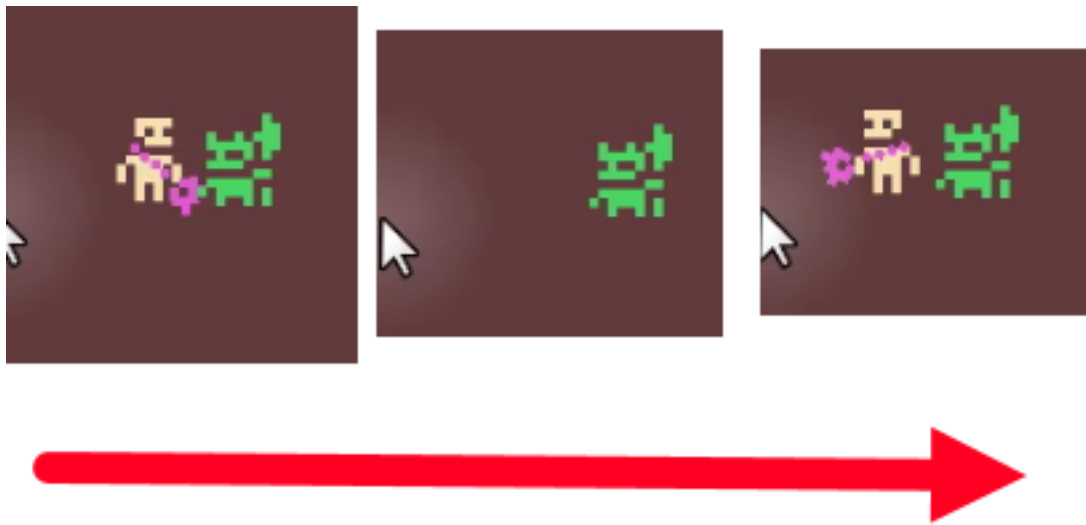
```
# Brief visibility toggle to give visual damage feedback
func _damage_flash():
    visible = false
    # await pauses this function until the timer fires, then resumes
    await get_tree().create_timer(0.07).timeout
    visible = true
```

Likewise, we will call it in the player's `take_damage` function:

```
func take_damage(amount: int):
    cur_hp -= amount
    _damage_flash()
    GlobalSignals.OnPlayerUpdateHealth.emit(cur_hp, max_hp)

    if cur_hp <= 0:
        die()
```

Now, whenever a character is hit, they will flicker briefly. This simple effect significantly improves the “feel” of combat.



Implementing Sound Effects

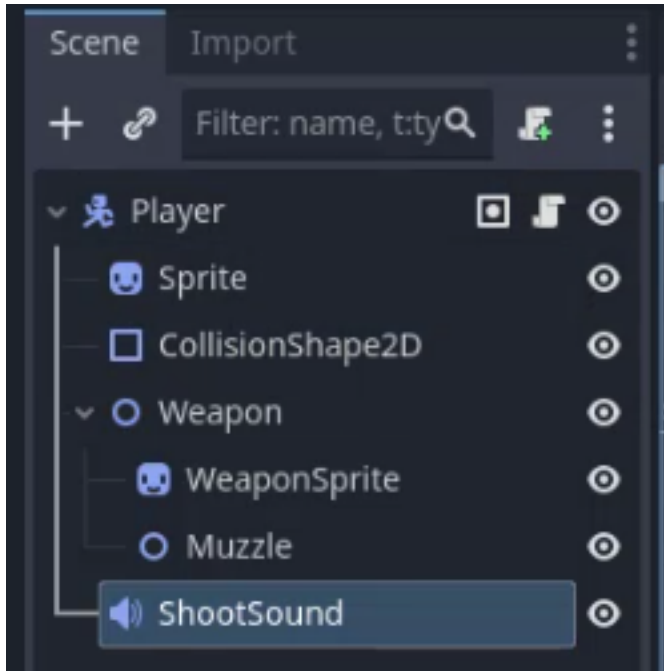
With our visuals improved, let’s now jump into adding sound effects to our player character. We will first create specific audio players for when the player shoots and when they take damage.

Player Shoot Sound

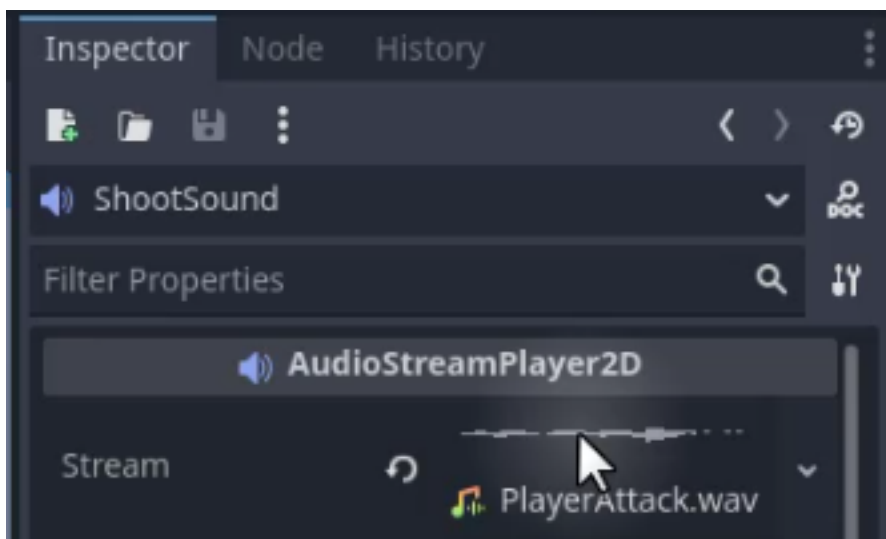
Audio feedback is one of the simplest ways to make controls feel responsive. A short sound on every shot tells the player their input registered, which is especially important during fast-paced combat.

To handle the shooting sound, we will add an audio node directly to the player scene. Open the player scene, right-click on the Player root node, and select **Add Child**. Then, choose **AudioStreamPlayer2D** and rename the node to ShootSound.

This node plays sound that is positioned in 2D space, meaning it can pan left or right depending on the player’s position relative to the camera (though for the player, it will mostly be centered).



With the ShootSound node selected, go to the Inspector. Then, drag the PlayerAttack audio file from your Audio folder into the **Stream** property.



Next, we need to trigger this sound in our code. So, open the `player.gd` script and locate the `_shoot` function. We can then add the following line to play the sound whenever a projectile is fired:

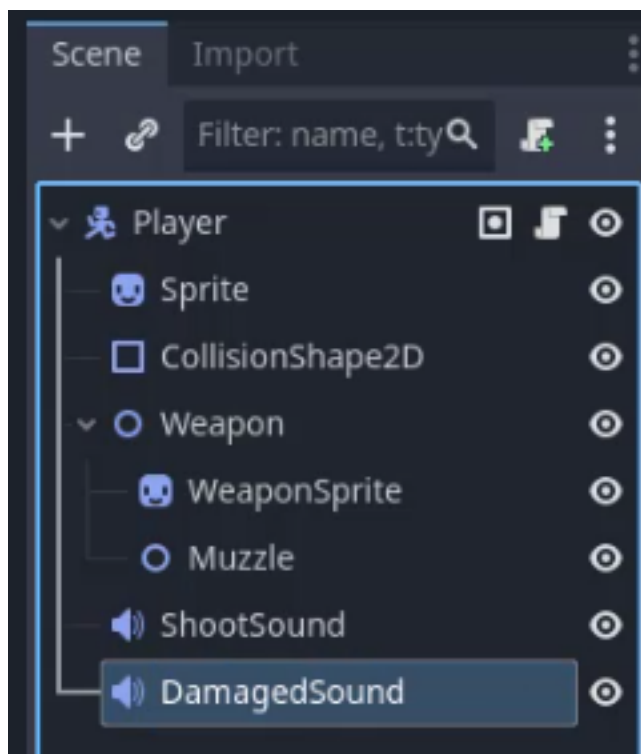
```
func _shoot ():  
    # ... Existing code ...
```

```
$ShootSound.play()
```

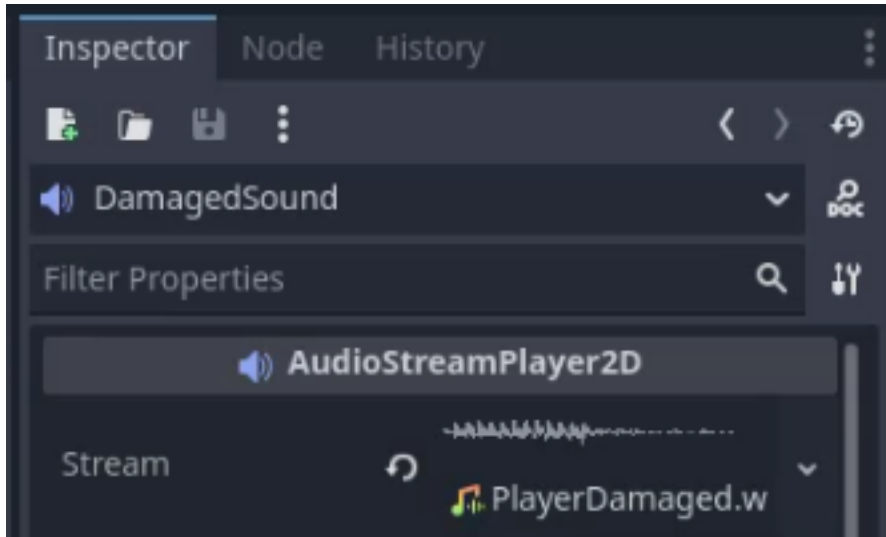
Now, whenever the player shoots, the sound effect will play, providing satisfying feedback.

Player Damage Sound

We can use a similar process to add a sound effect for when the player takes damage. In the Player scene, duplicate the ShootSound node (Ctrl+D or Cmd+D) and rename the copy to DamagedSound.



Next, in the Inspector for DamagedSound, drag the PlayerDamaged audio file into the **Stream** property.



Finally, update the `take_damage` function in the `player.gd` script to play this sound:

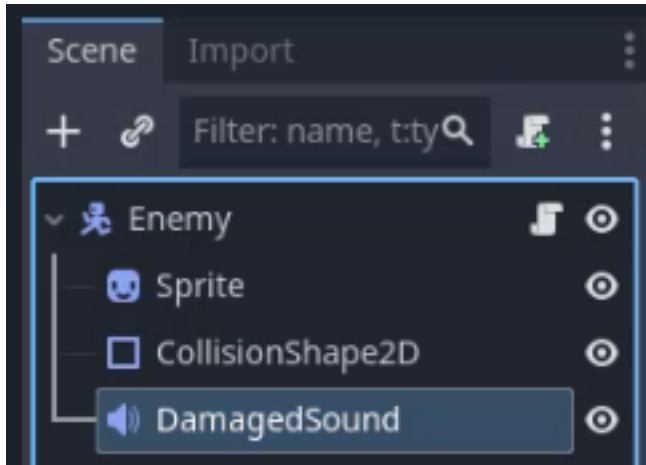
```
func take_damage (amount : int):
    # ... Existing Code ...
    # Update the health UI before checking death, so the final state is
    visible
    GlobalSignals.OnPlayerUpdateHealth.emit(cur_hp, max_hp)
    $DamagedSound.play()

if cur_hp <= 0:
    die()
```

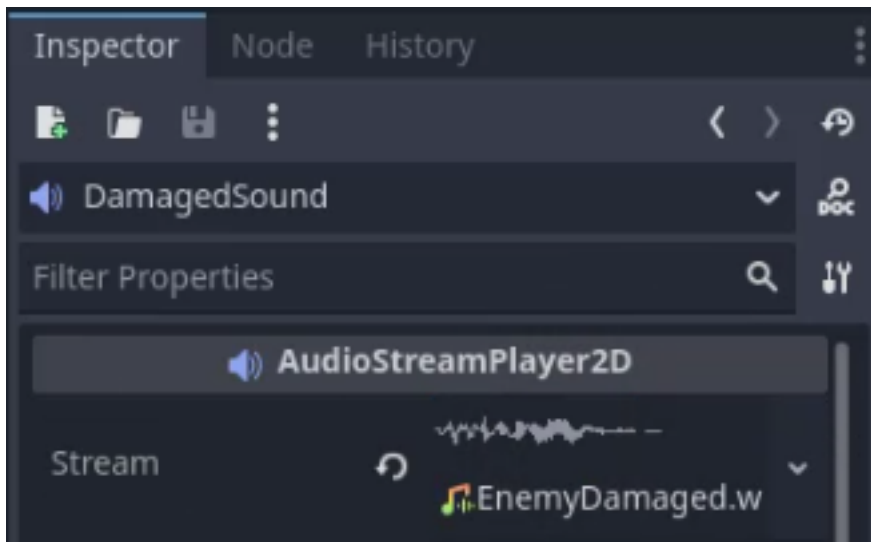
With this, we should already have a nice soundscape that makes our player much more immersive.

Enemy Damage Sound

Like the player, enemies should also provide auditory feedback when hit. As such, let's open the Enemy scene and add a new **AudioStreamPlayer2D** node as a child of the Enemy root. We will rename the node to `DamagedSound`.



In the Inspector, assign the EnemyDamaged audio file to the **Stream** property of this new node.



Then, in the enemy.gd script, locate the take_damage function and add the playback command:

```
func take_damage (amount : int):
    cur_hp -= amount

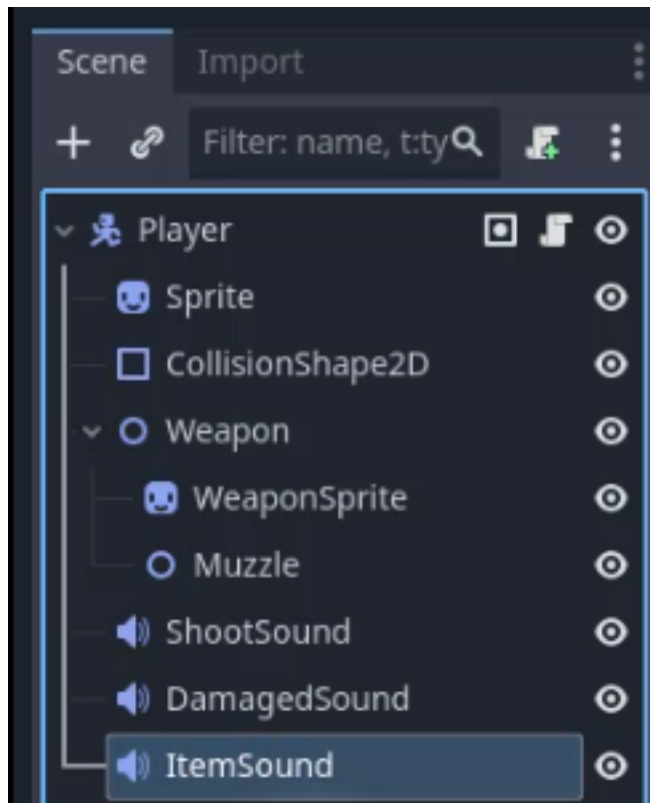
    # Brief visibility toggle to give visual damage feedback
    _damage_flash()
    $DamagedSound.play()

if cur_hp <= 0:
    die()
```

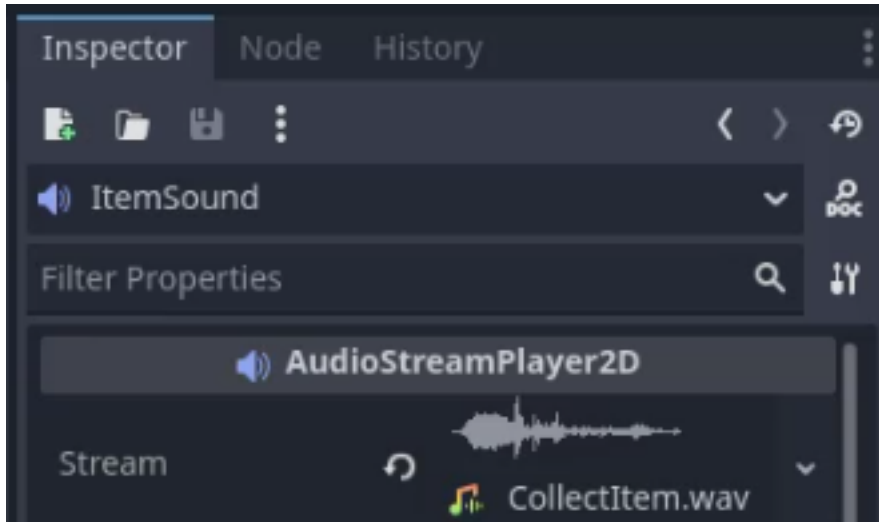

Item Collection Sound

Lastly, for sound effects, let's add a sound for collecting items. Since the item node is destroyed immediately upon pickup, we cannot play the sound from the item itself as the sound would cut off instantly. Instead, we will add the sound player to the player character, who persists in the scene.

Open the Player scene again, duplicate the DamagedSound node, and rename it to ItemSound.



For this sound, assign the CollectItem audio file to the **Stream** property.



Now, with a persistent object, we can trigger this sound from the item script. Open `item.gd` and update the `_on_body_entered` function. Before we destroy the item with `queue_free()`, we tell the player body to play its `ItemSound`.

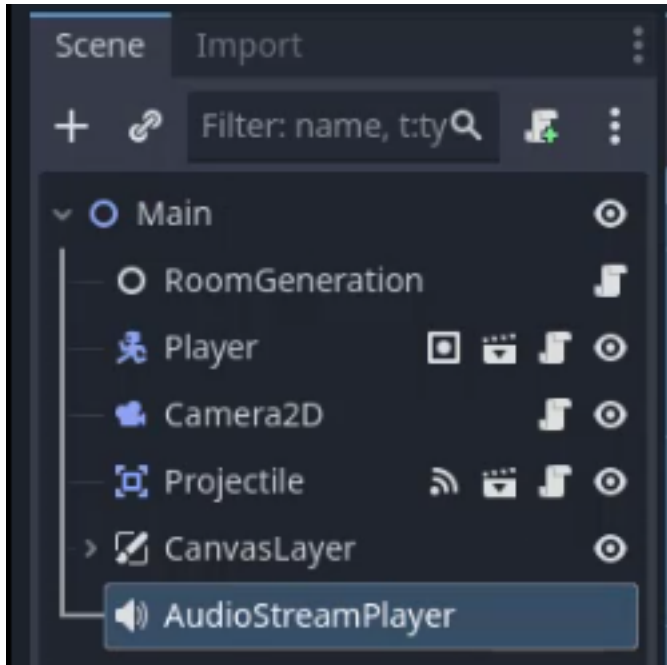
```
func _on_body_entered(body):  
    # ... Existing code ...  
  
    body.get_node("ItemSound").play()  
  
    queue_free()
```

Be sure to test out the game to make sure all the sounds play. You'll notice a huge difference the audio makes and how much more "oomph" the game has now because of them.

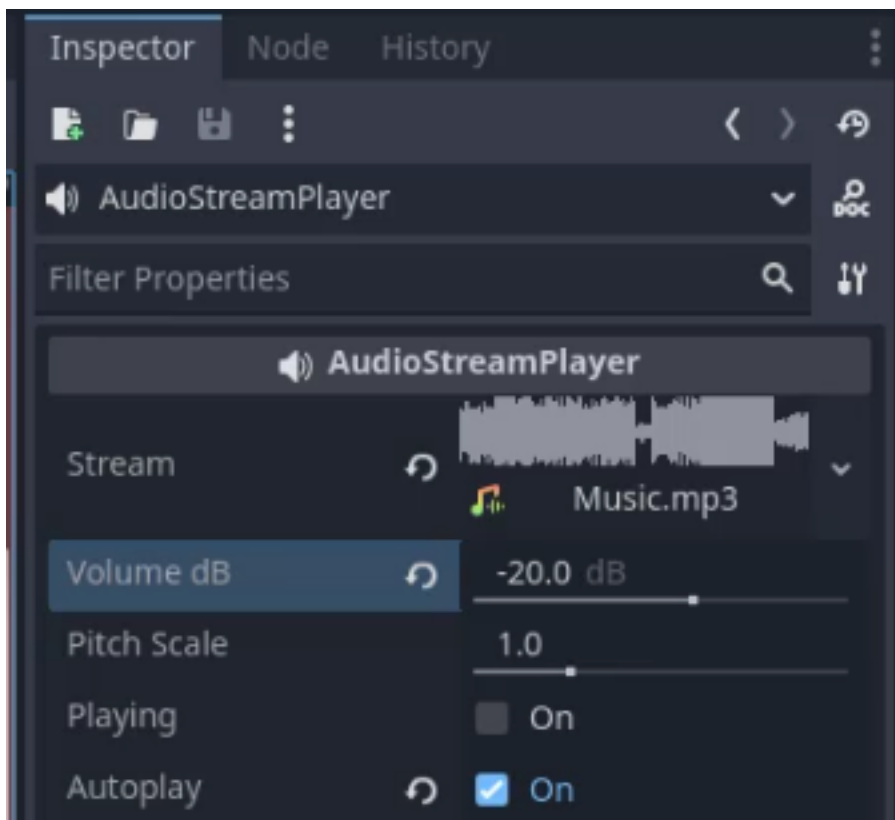
Adding Background Music

Background music sets the atmosphere for the game. Unlike sound effects, music should not be positional; it should play at a constant volume regardless of where the player is.

To begin, open your main level scene and create a new child node of type **AudioStreamPlayer** (not the 2D version).



Next, drag the Music audio file into the **Stream** property. Since music can be loud, adjust the **Volume dB** to a lower level, such as -20. Finally, enable the **Autoplay** option so the music starts automatically when the game launches.



Godot Insight: `AudioStreamPlayer` vs `AudioStreamPlayer2D`

Godot provides two audio nodes for different use cases.

`AudioStreamPlayer` plays sound at a constant volume regardless of position. It is ideal for background music, UI sounds, and any audio that should feel “global” to the player.

`AudioStreamPlayer2D` plays sound that is positioned in 2D space. The volume and stereo panning change based on the distance and direction between the audio source and the listener (usually the camera). This makes it perfect for sound effects tied to specific objects, such as a gunshot from the player or a hit sound from an enemy.

We used `AudioStreamPlayer2D` for our shooting and damage sounds because they originate from specific game objects, and `AudioStreamPlayer` for the background music because it should play consistently everywhere.

Learn more: [`AudioStreamPlayer`](https://docs.godotengine.org/en/4.6/classes/class_audiostreamplayer.html)

(https://docs.godotengine.org/en/4.6/classes/class_audiostreamplayer.html)

You can play your game at this point and hear your music in action. Feel free to play around further with the volume controls until you get a good balance.

We now have a comprehensive soundscape, as well as a better way to indicate when characters are hit in combat. In the next chapter, we’re going to learn how to add boss enemy types to our game to improve our variety even further.

Creating the Boss Battle

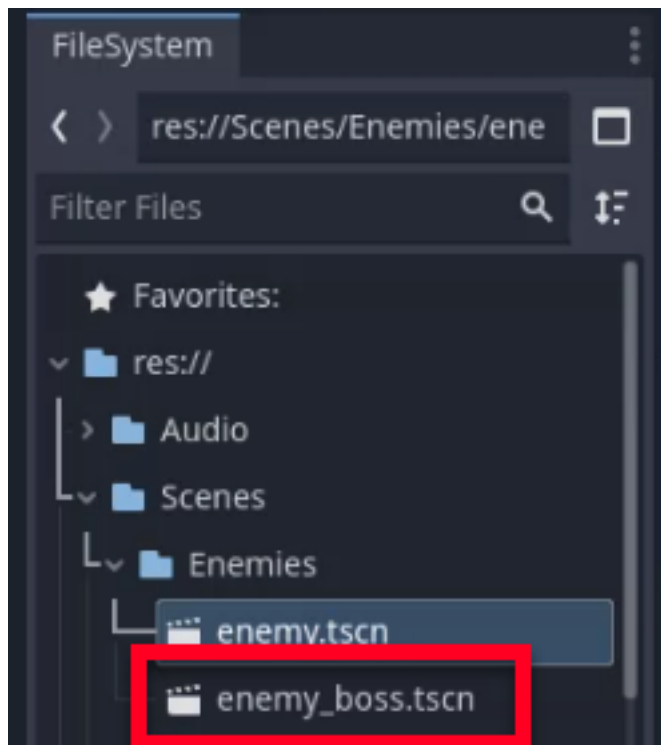
In this chapter, we will focus on improving the gameplay's progression by implementing boss battles. Boss battles are great for adding a climax to a level. Additionally, for a roguelike, the boss battle is often what controls progression to the next level, as the player must defeat the boss before receiving the means to proceed.

For our project, this system involves creating a unique boss enemy, a dedicated boss room, and a system to spawn a portal to the next level upon victory.

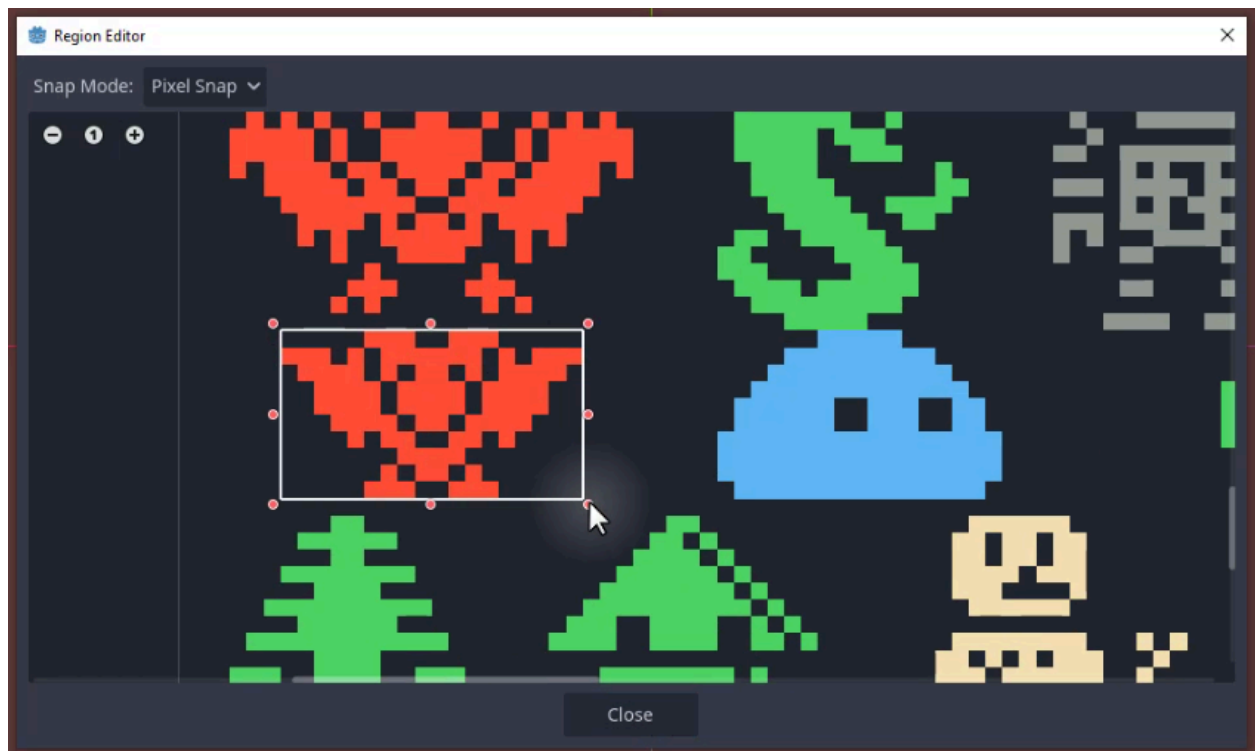
Creating the Boss Enemy

The boss will be a variant of our standard enemy, but with significantly higher stats and a distinct look.

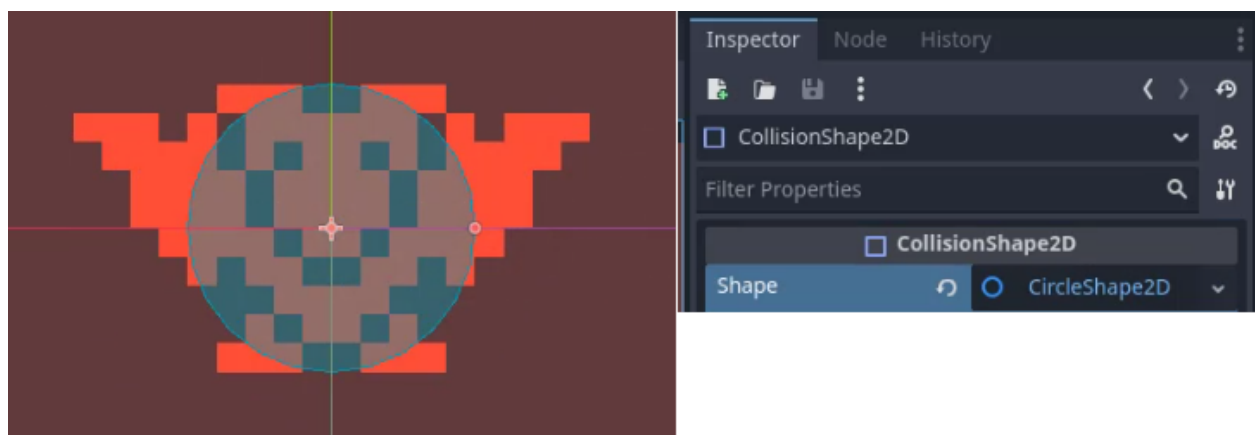
To start, create a new inherited scene from `enemy.tscn` and save it as `enemy_boss.tscn` in `Scenes/Enemies`.



The first thing we want to do in our variant is change the visuals. In the new scene, select the Sprite node and use the **Region** settings to change the sprite to a larger, more menacing creature (e.g., the red demon).



Since the sprite is likely larger or differently shaped, update the `CollisionShape2D` to match. A circle shape often works well for larger bosses, so change the shape if needed.



With the visuals handled, we just need to adjust the exported variables in the Inspector to make the boss formidable. Let's adjust them as follows:

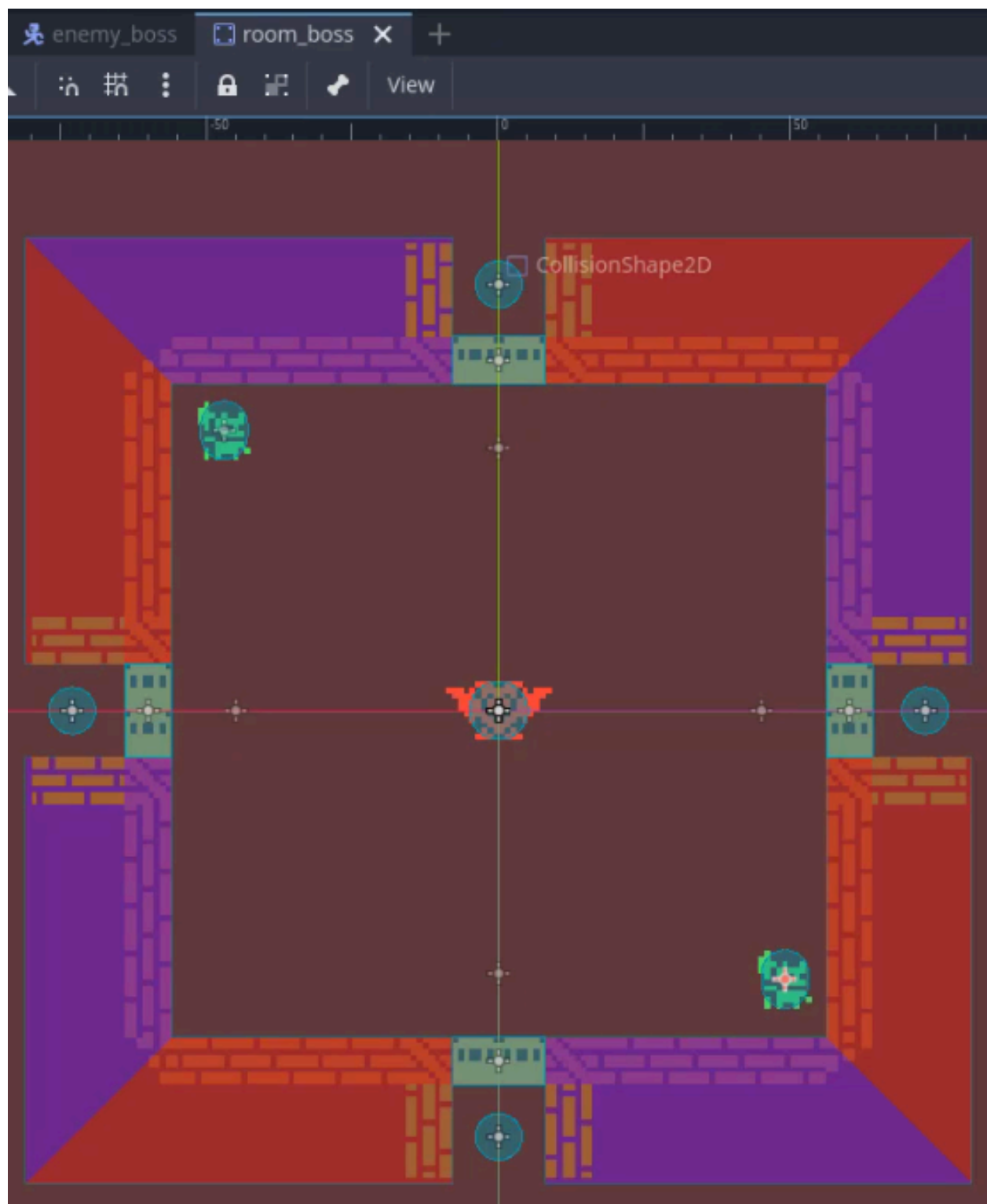
- **Max Hp and Cur Hp:** 20
- **Move Speed:** 30
- **Attack Damage:** 1
- **Attack Range:** 15
- **Attack Rate:** 0.3



Creating the Boss Room

Next on our list, we need a room to house the boss. Like we've done many times in this book, create a new inherited scene from `room_template.tscn` and save it as `room_boss.tscn`.

Then, instance the `enemy_boss` scene into the room. For good measure, let's also add a few regular enemies to increase the difficulty.

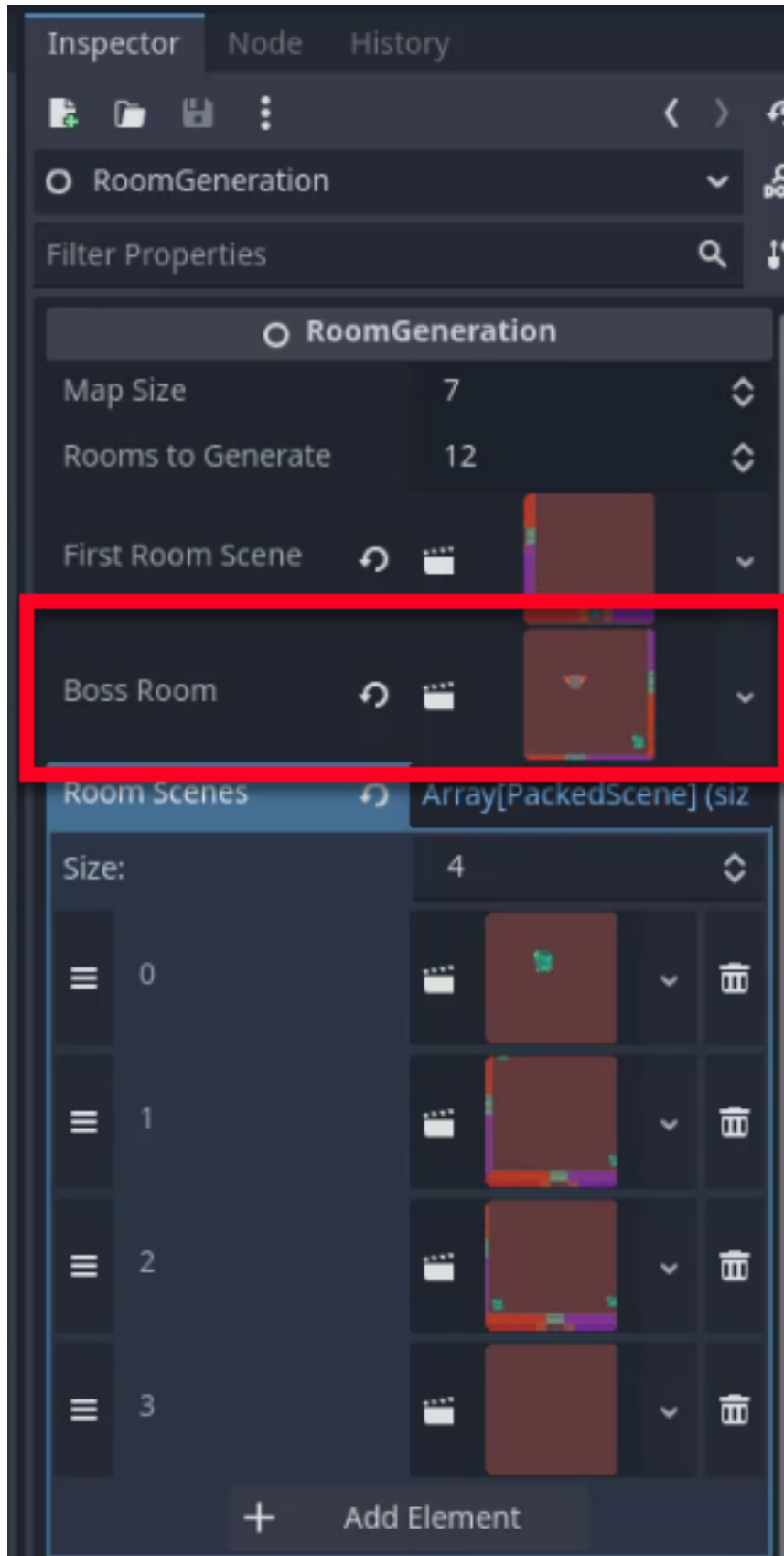


Guaranteeing the Boss Room Spawn

Unlike our previous room additions, we need to ensure that exactly one boss room spawns in every level. To implement this, first open the `room_generation.gd` script and add a new export variable:

```
@export var boss_room : PackedScene
```

Back in the editor, assign your `room_boss.tscn` to this variable in the Inspector.



Next, in our logic itself, we need a way to pick a location for the boss room. We will create a function called `_decide_boss_room` that attempts to find a valid room position that isn't the starting room from our map.

Add this function to `room_generation.gd`:

```
# Pick a random occupied cell that isn't the starting room
func _decide_boss_room() -> Vector2:
    var i = 0
    while i < 100:
        var x = randi_range(0, map_size - 1)
        var y = randi_range(0, map_size - 1)
        if first_room_x == x and first_room_y == y:
            continue
        if _get_map(x, y):
            return Vector2(x, y)
        i += 1
    return Vector2.ZERO
```

Now, we can update the `_instantiate_rooms` function to place the boss room at the decided coordinates.

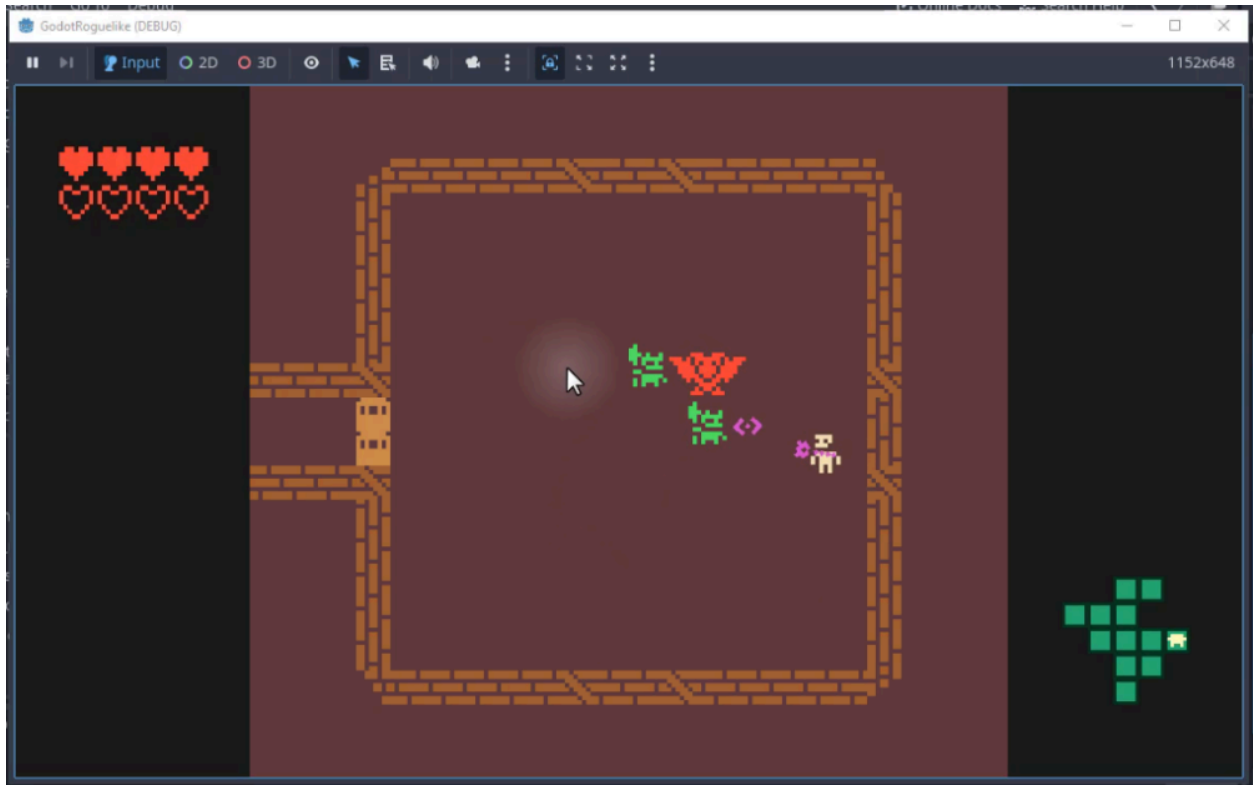
```
func _instantiate_rooms():
    var boss_room_pos : Vector2 = _decide_boss_room()

    for x in range(map_size):
        for y in range(map_size):
            if not _get_map(x, y):
                continue

            var room : Room
            var is_first_room = first_room_x == x and first_room_y ==
y
            if is_first_room:
                room = first_room_scene.instantiate()
            elif x == boss_room_pos.x and y == boss_room_pos.y:
                room = boss_room.instantiate()
            else:
                room = room_scenes[randi_range(0, len(room_scenes) -
1)].instantiate()
```

```
# ... rest of the existing code ...
```

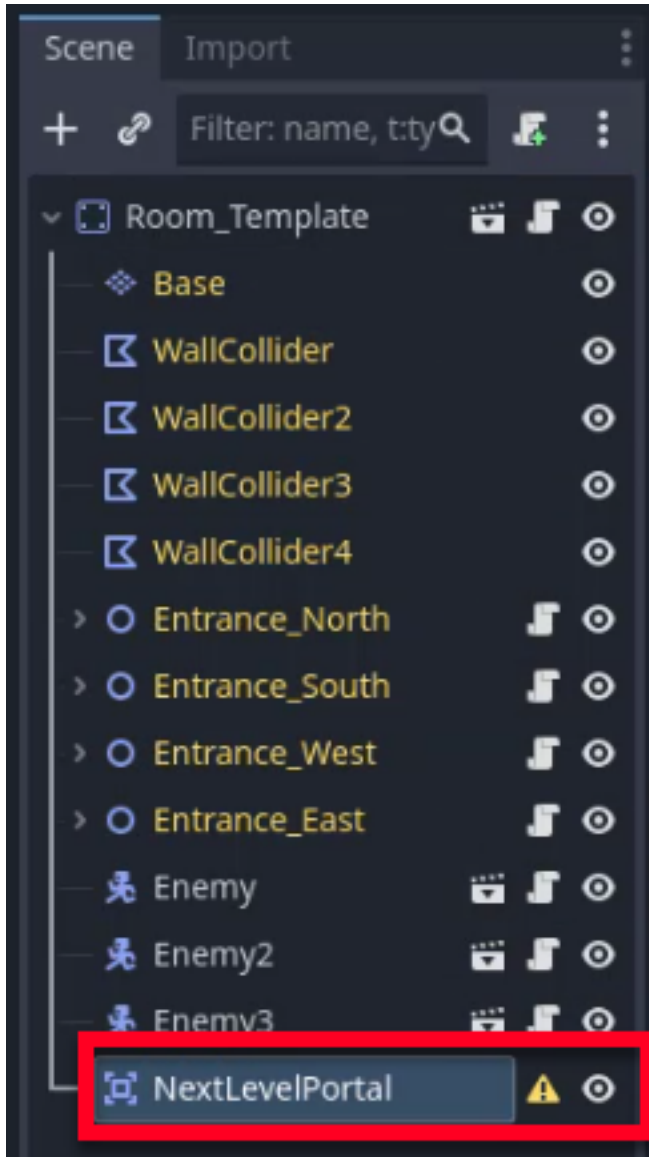
If you run the game now, one of the rooms in the dungeon will be the boss room. It may take a few moments to find it since it's chosen randomly, but our algorithm ensures there will be at least one boss room to find.



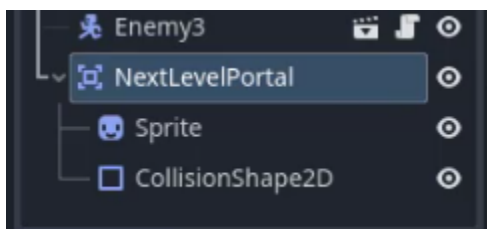
Creating the Next Level Portal

To complete the game loop, defeating the boss should reveal an exit. Without this reward, the boss fight would feel anticlimactic and our game progression would stall. We will create a portal that serves as the player's way forward.

In the `room_boss` scene, let's add an **Area2D** node and name it `NextLevelPortal`.



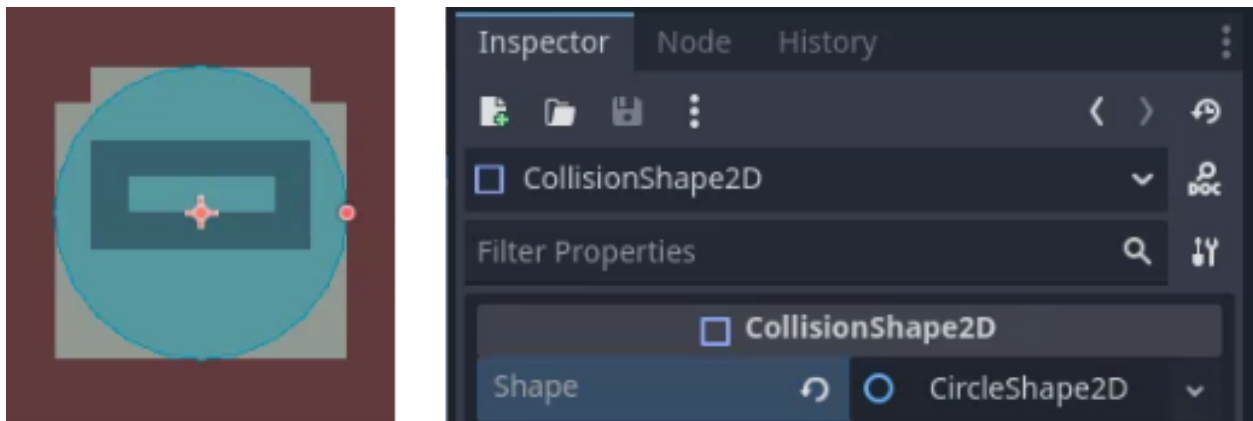
To give it a visual and a collision boundary, add a **Sprite2D** child node named **Sprite** and a **CollisionShape2D** child node.



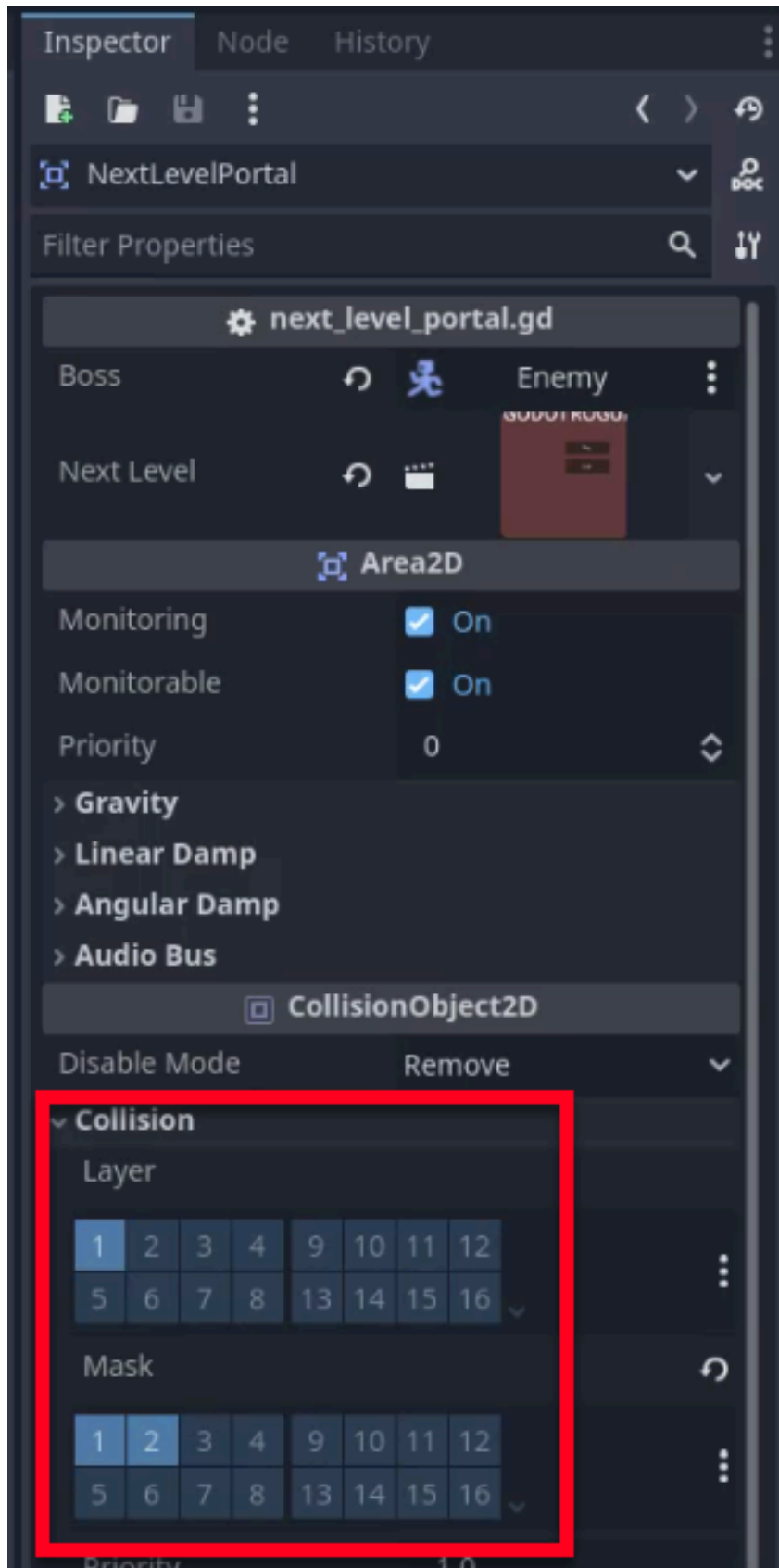
Select the **Sprite** and assign **Tiles.png** to the sprite. We can then use **Region** editing to select a door or portal graphic (e.g., the skull door).



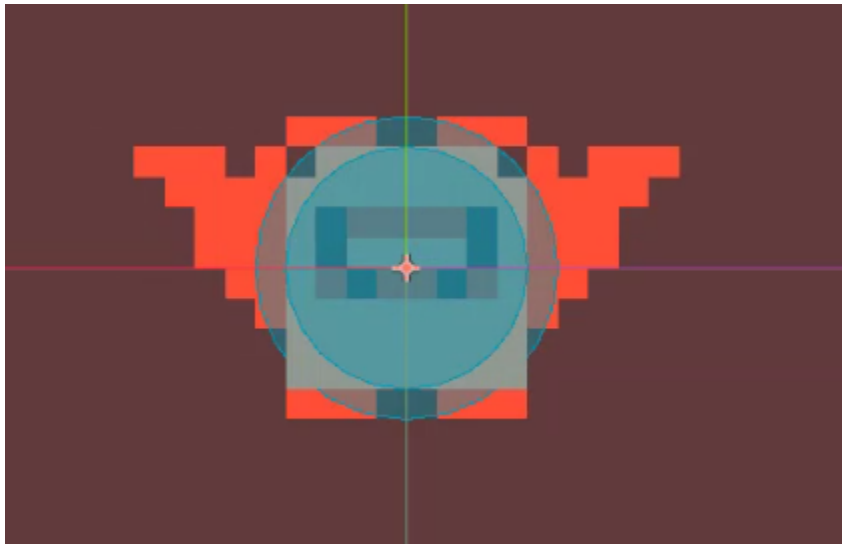
Next, for **CollisionShape2D**, set the collision shape to a circle and resize it to cover the sprite.



We also need to ensure the portal's collision mask is set correctly to detect the player (for our project, this is Layer 2).

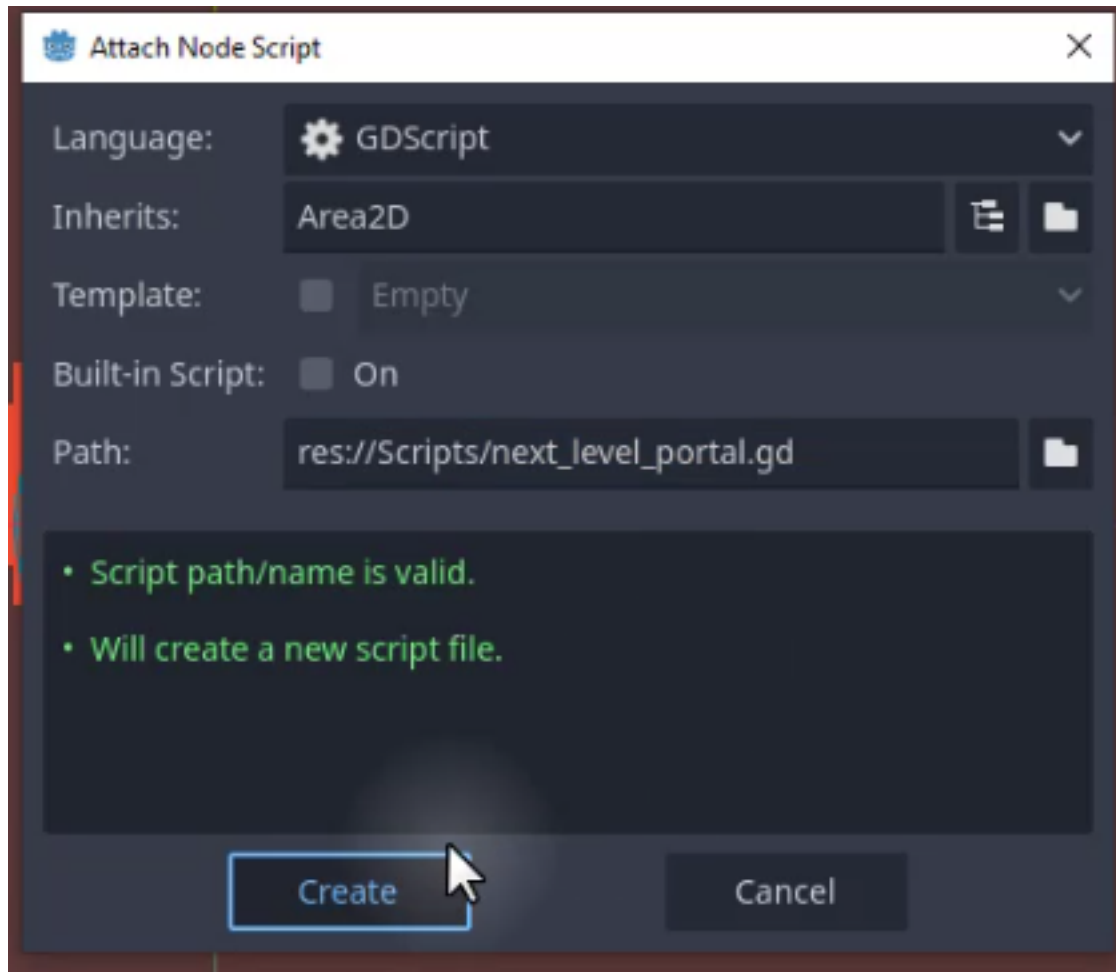


Once configured, position the portal in the center of the room which is where we'll want it to appear.

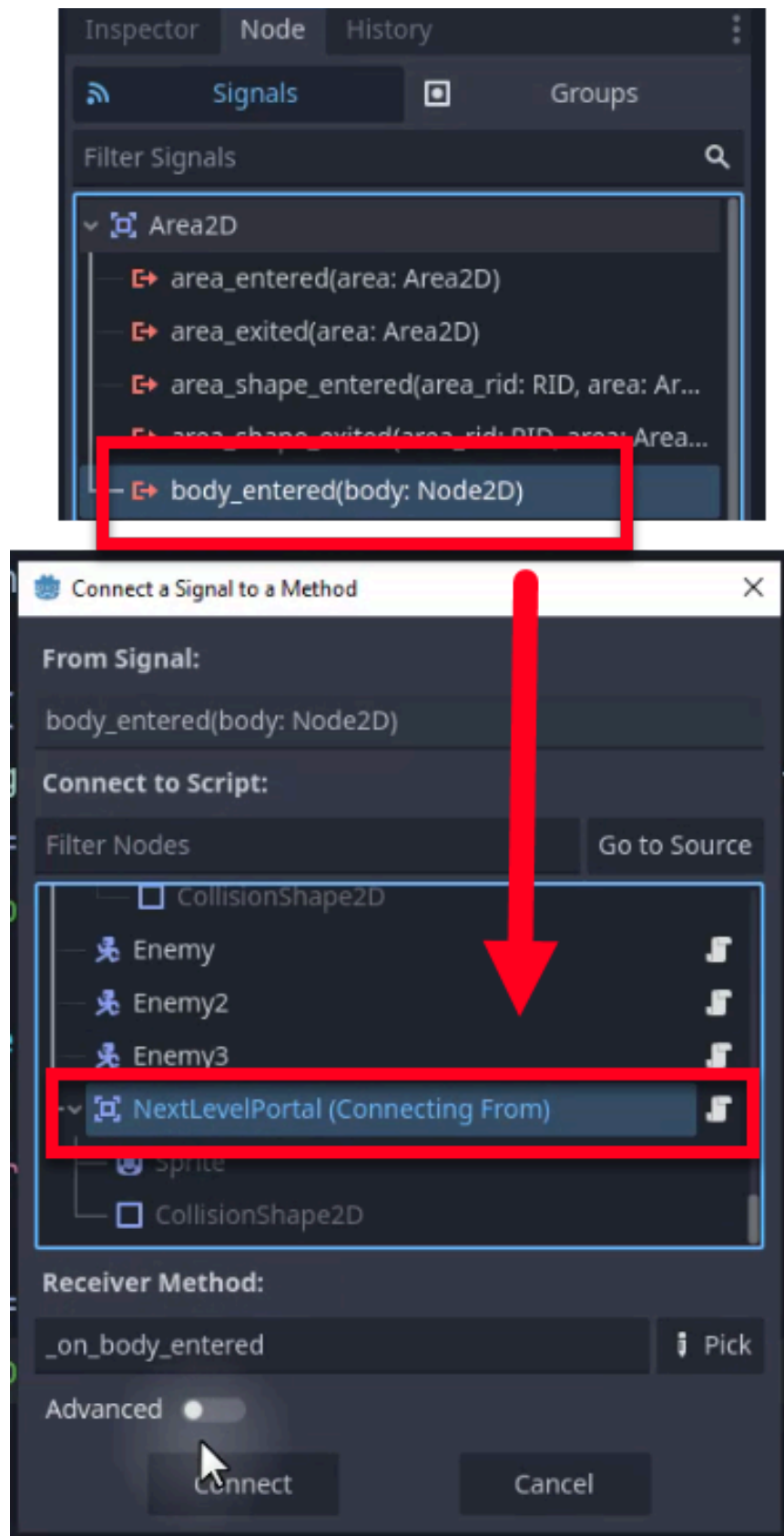


Scripting the Portal

Our portal is set up, so now we have to work on the logic that makes it appear. To begin, attach a new script named `next_level_portal.gd` to the `NextLevelPortal` node.



Before we work on the logic, we need to know when the player interacts with the portal. As such, let's connect the `body_entered` signal of the `NextLevelPortal` (which is an `Area2d` node) to an `_on_body_entered` function in the script.



Now we can open `next_level_portal.gd` and work on it. At the top, this script needs to know which enemy is the boss and what level to load next.

```
extends Area2D
```

```
@export var boss : Enemy
```

```
@export var next_level : PackedScene
```

Next, we'll add our logic:

1. In `_ready`, we want to connect to our global enemy defeated signal. We also want to make sure our portal is non-interactable, since we should only see it when the boss is defeated.
2. In the `_on_defeat_enemy` function, we will check if the defeated enemy is our boss. If it is, we'll toggle our portal on.
3. Finally, in our `_on_body_entered` function, we check if the object triggering the interaction is the Player and load the `next_level` scene.

```
func _ready():
    GlobalSignals.OnDefeatEnemy.connect(_on_defeat_enemy)
    # Start hidden and non-interactive until the boss is defeated
    visible = false
    $CollisionShape2D.disabled = true

# Only activate the portal when the assigned boss is killed
func _on_defeat_enemy(enemy : Enemy):
    if enemy != boss:
        return

    visible = true
    $CollisionShape2D.set_deferred("disabled", false)

func _on_body_entered(body):
    if not body.is_in_group("Player"):
        return
    get_tree().change_scene_to_packed.call_deferred(next_level)
```

Godot Insight: Safe physics property changes with `set_deferred`

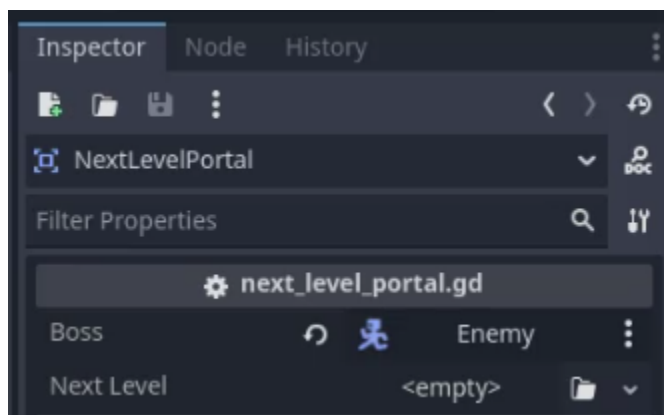
Godot does not allow you to modify certain physics properties (like disabling a `CollisionShape2D`) while the physics engine is actively processing collisions. Attempting to do so triggers a warning and can cause unpredictable behavior.

`set_deferred("property", value)` solves this by scheduling the property change for the next idle frame, after physics processing is complete. Similarly, `call_deferred("method")` postpones a method call. These patterns appear throughout this project whenever we toggle collision shapes inside signal callbacks like `body_entered`.

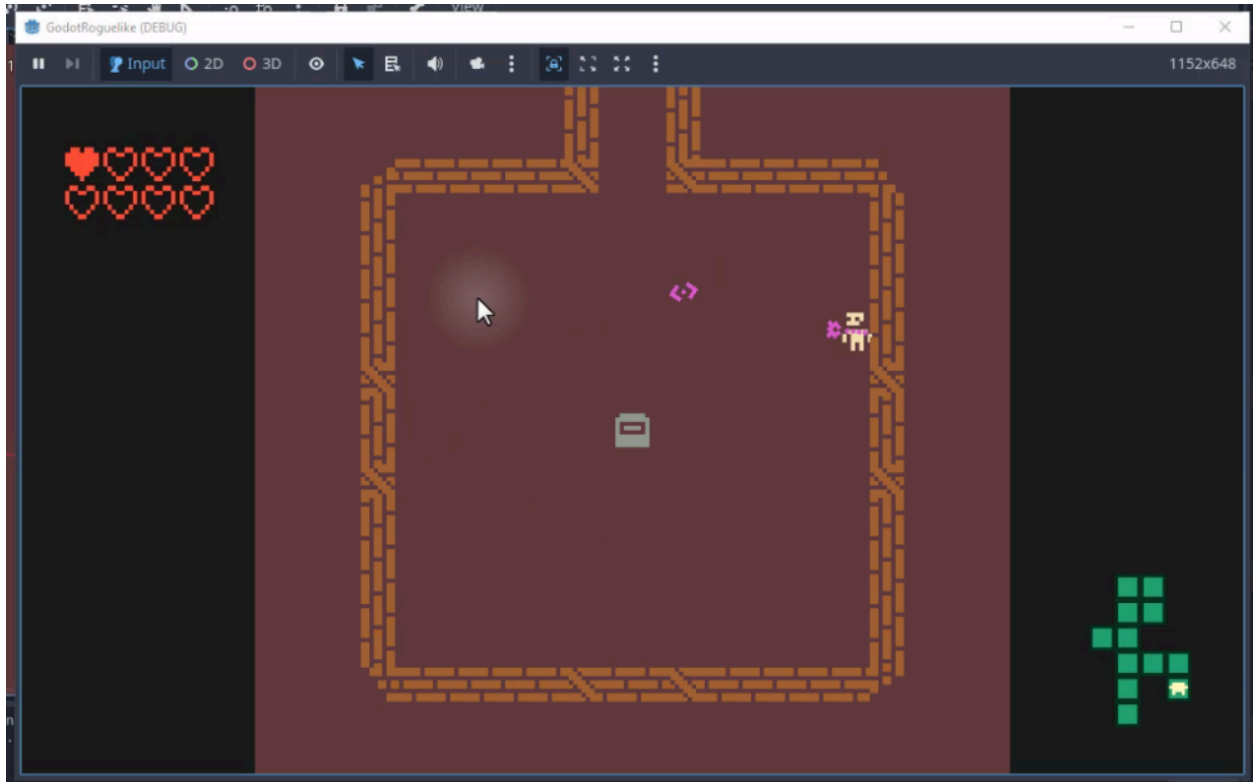
Learn more: [Object](#)

(https://docs.godotengine.org/en/4.6/classes/class_object.html)

Finally, in the Inspector for the `NextLevelPortal`, assign the `enemy_boss` node in the room to the **Boss** variable. We won't assign the next level variable just yet, as we haven't yet created anything to complete the loop (though we will next chapter).



With this setup, the portal will remain hidden until the specific boss enemy assigned to it is defeated. Once visible, walking into it will trigger the scene change (once we configure this later).



In the next chapter, we will wrap up our project by creating a main menu and adding final polish to the game.

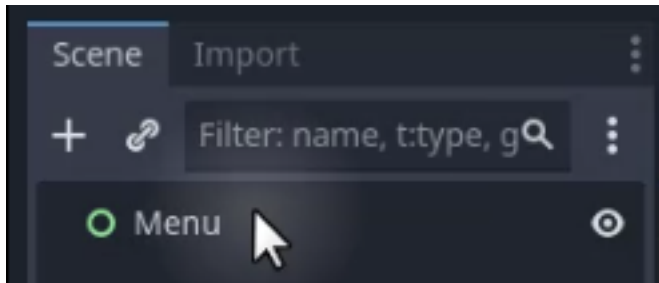
Main Menu and Game Loop

In this final chapter, we will complete our game loop by creating a main menu. This menu will serve as the entry point for players, allowing them to start a new run or quit the application. We will also connect this menu to the game's win and loss conditions, ensuring a smooth flow between the gameplay and the interface.

Finally, we will address some common bugs related to scene transitions and physics collisions to ensure a polished final product.

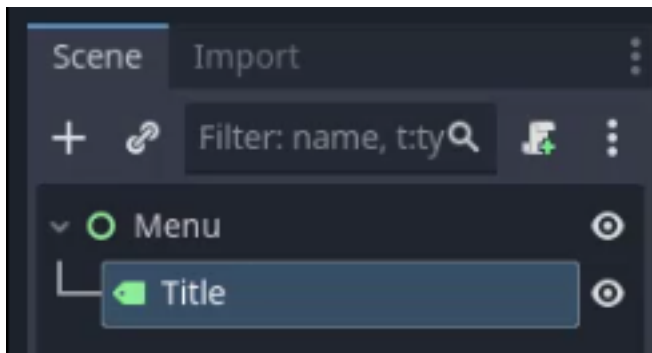
Creating the Menu Scene

To begin, let's create a new scene that will hold our user interface. Navigate to the Scenes folder in the FileSystem dock and create a new scene of type **User Interface** (Control node). Let's name the root node **Menu**.



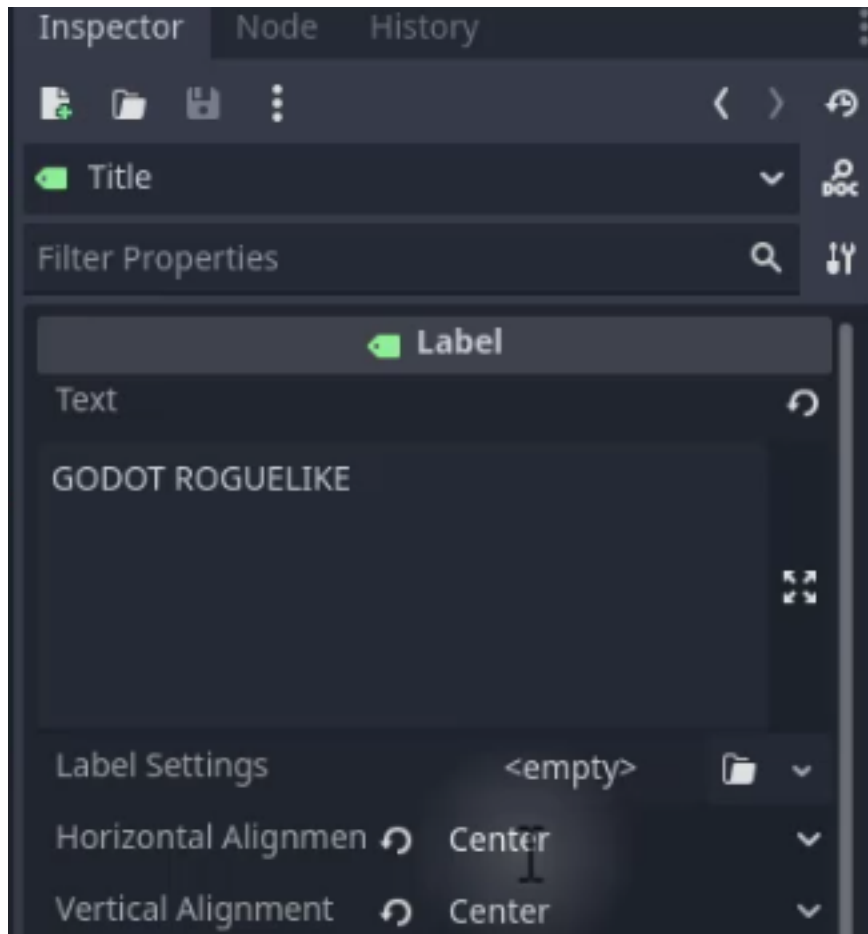
Setting Up the Title

We will start by adding a title to the menu. For this, add a **Label** node as a child of the **Menu** node and rename it to **Title**.

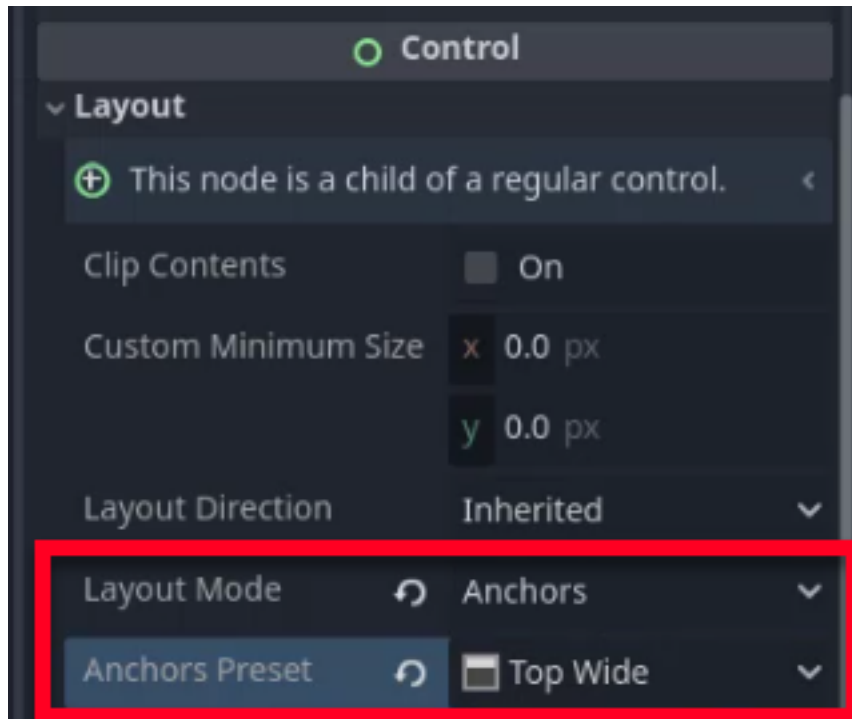


With the `Title` node selected, configure its properties in the Inspector as follows:

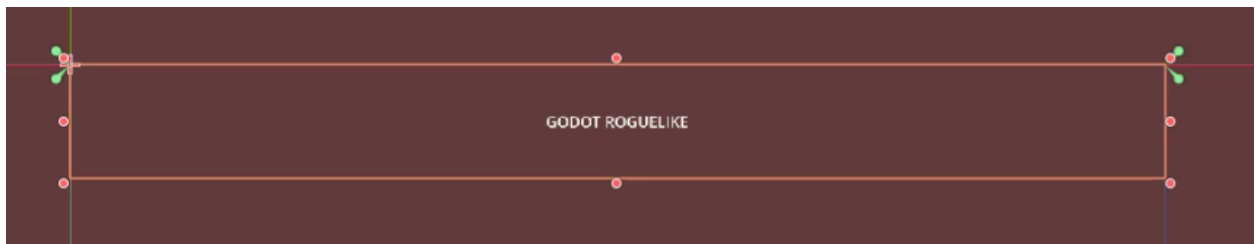
- Set the **Text** to `GODOT ROGUELIKE`.
- Set **Horizontal Alignment** and **Vertical Alignment** to `Center`.



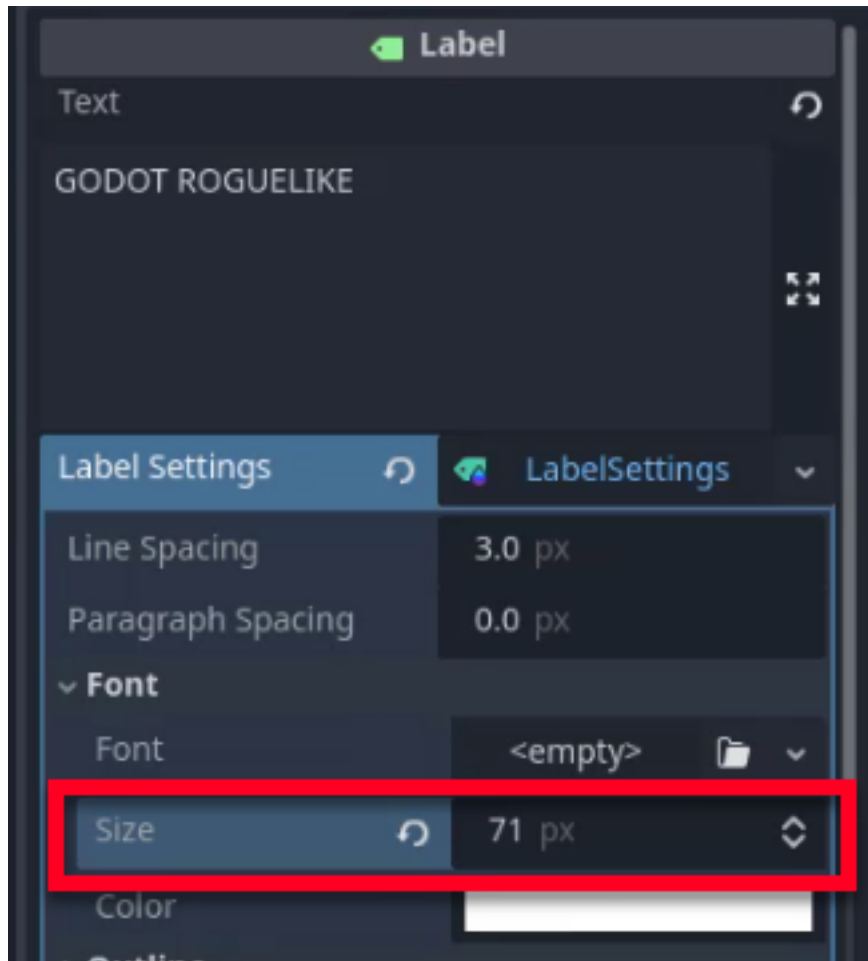
To ensure the title stays centered regardless of screen size, we will use anchors. In the Inspector, change the **Layout Mode** to `Anchors` and set the **Anchors Preset** to `Top Wide`. This will stretch the label across the top of the screen.



You can now resize the label vertically to give it some breathing room at the top of the screen.



To make the title stand out, we need to increase the font size. In the Inspector, find the **Label Settings** property and create a new **LabelSettings** resource. Expand the resource and set the **Font Size** to a larger value, such as 71 px.

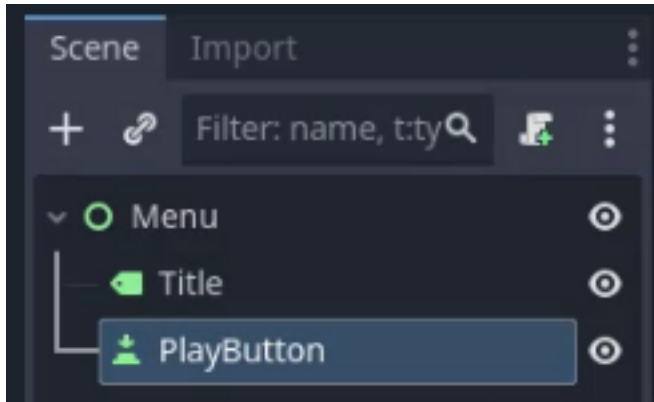


After adjusting the font size, make any final positional adjustments in the viewport to ensure the title looks balanced.

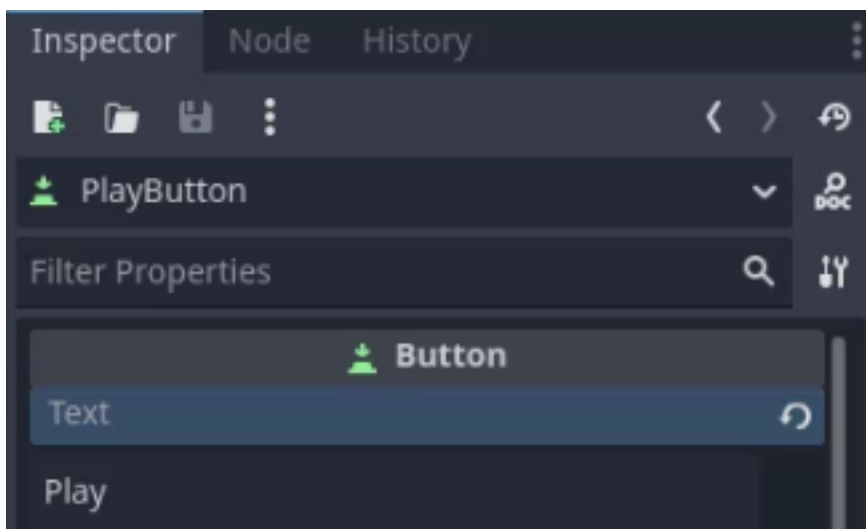


Adding Buttons

Next, we will add the interactive elements for the player. To begin, create a new Button node as a child of Menu and name it PlayButton.



As the name suggests, set the **Text** of the button to Play.

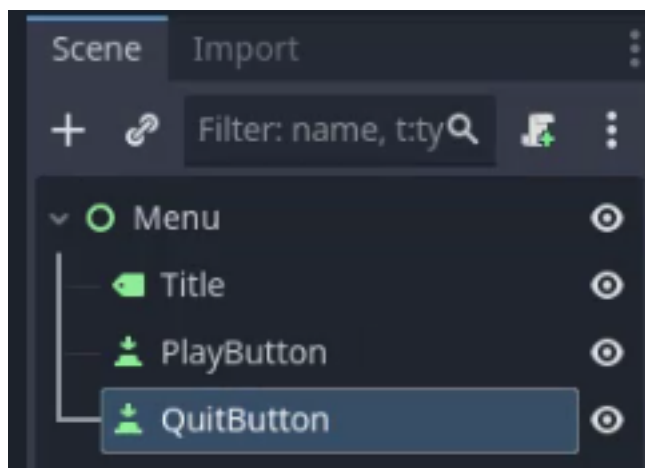


Then, resize the button to be easily clickable and position it in the center of the screen, below the title.

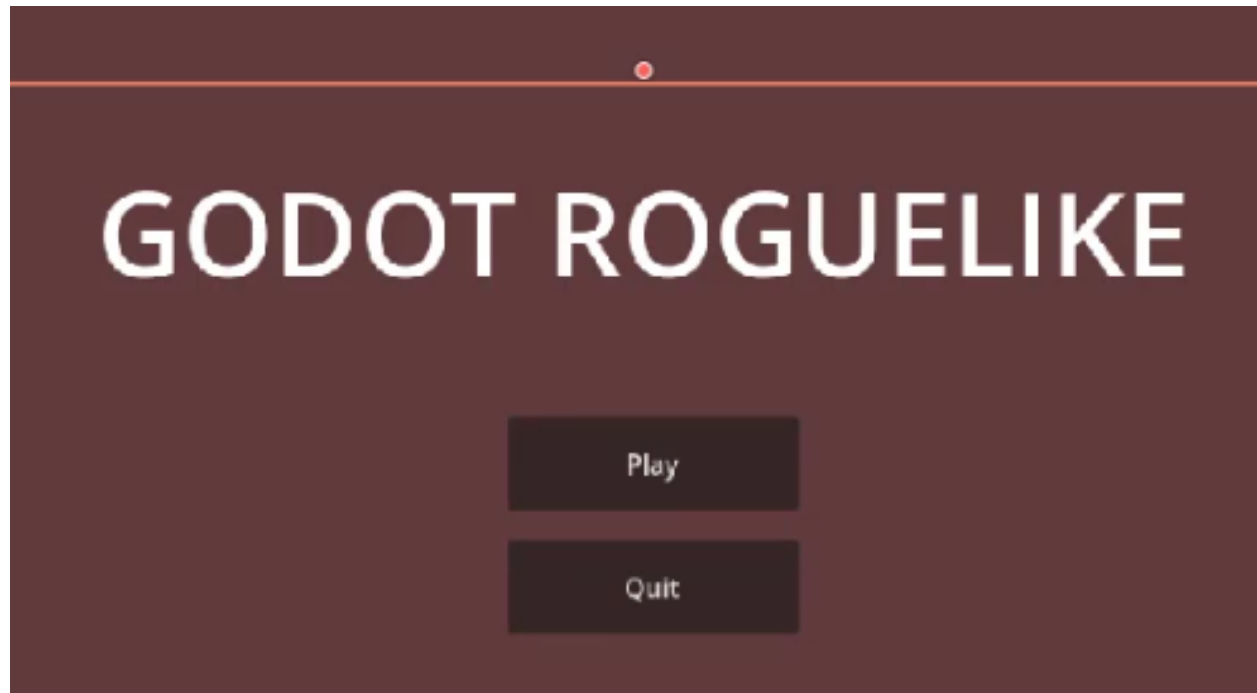
GODOT ROGUELIKE



To create the Quit button, we can simply duplicate the PlayButton (Ctrl+D or Cmd+D) and rename it to QuitButton.

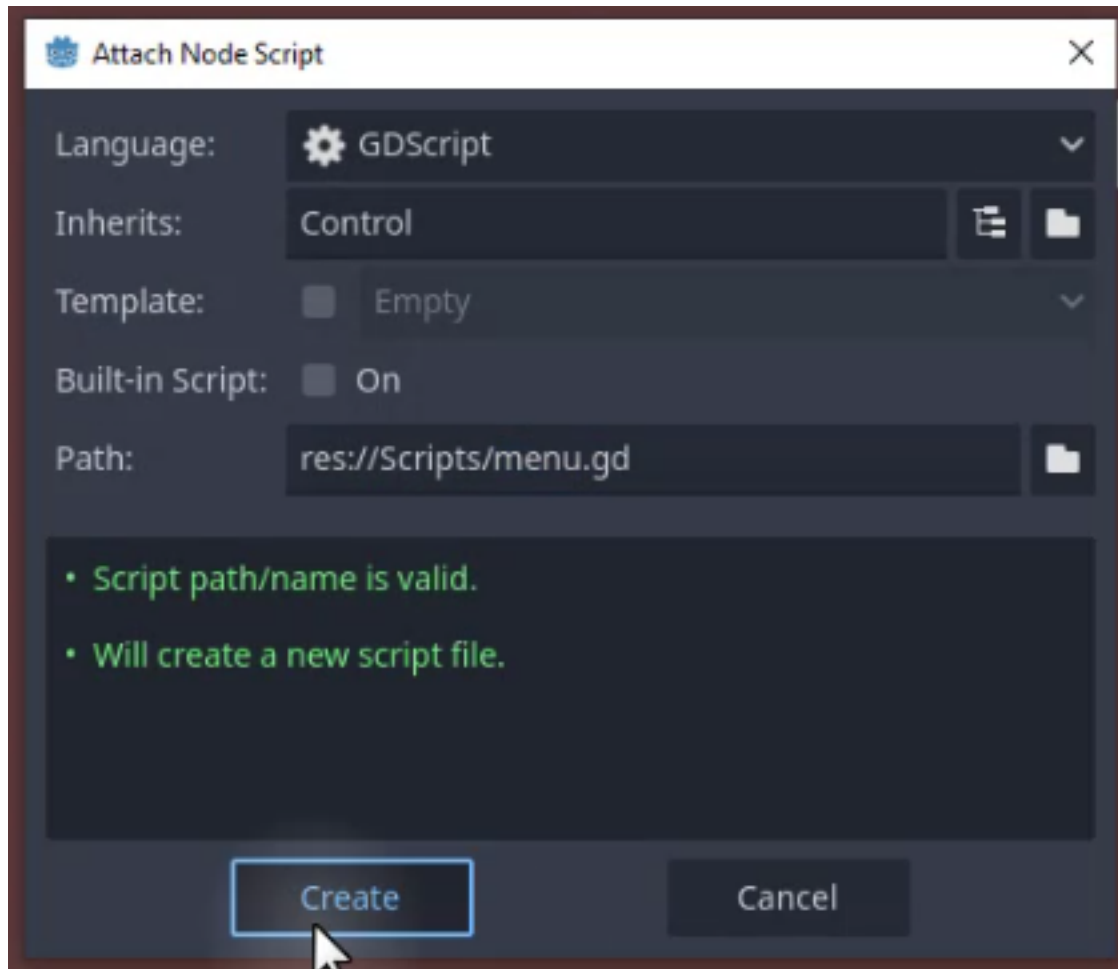


Then, we just change the text of the new button to Quit and position it below the Play button.

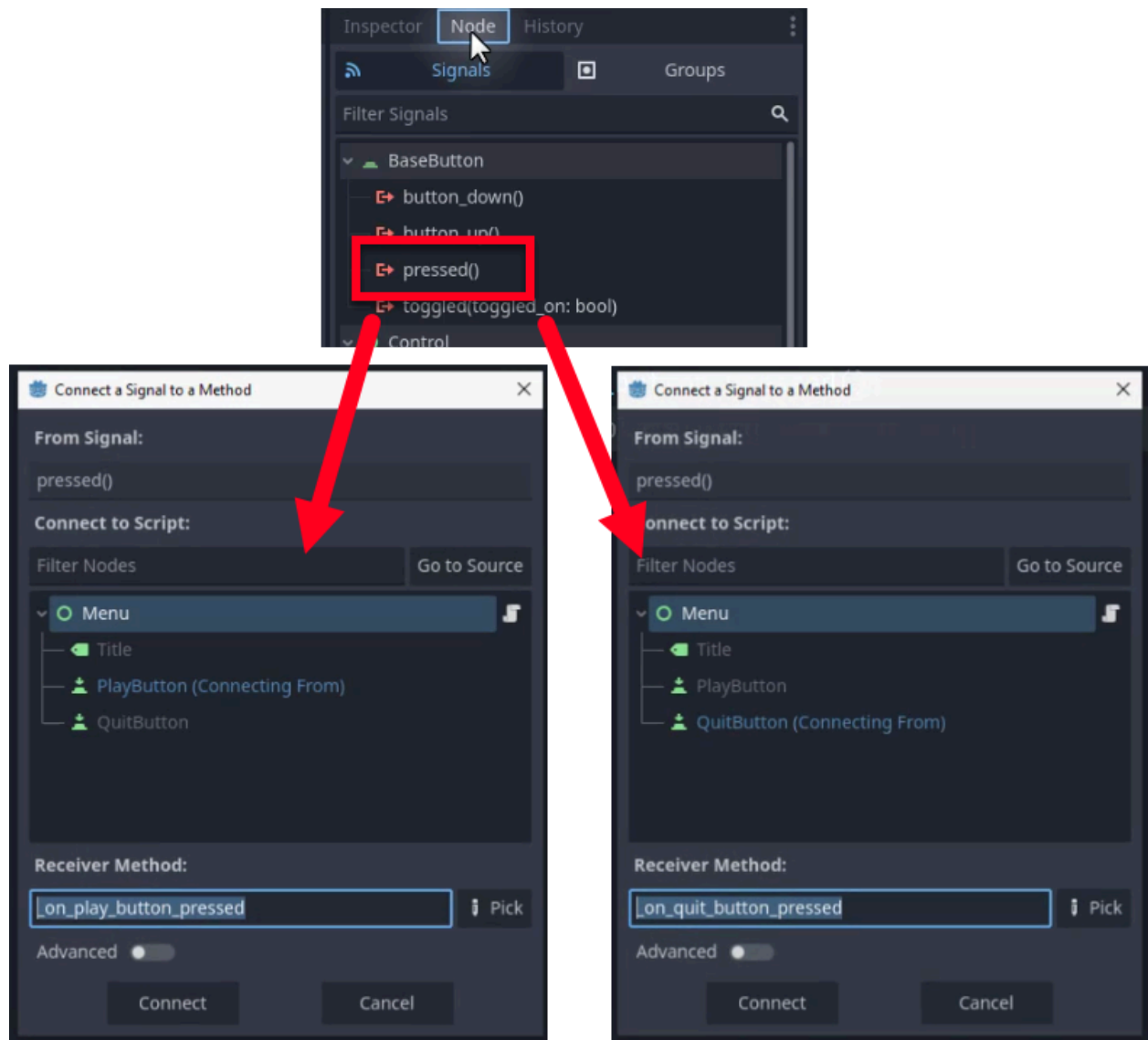


Creating the Menu Script

Now that our visual elements are in place, we need to add functionality. Let's select the Menu root node and attach a new script named `menu.gd`.



In order for our menu to work, we need to connect the pressed signals from both buttons to this script. Select PlayButton, go to the **Node** tab, and connect the `pressed()` signal to the `menu.gd` script. Then, repeat this process for the QuitButton.



We can now add the following code to `menu.gd` to handle the scene transition and application exit:

```
extends Control
```

```
func _on_play_button_pressed():
```

```
    # Transition from the menu to the first gameplay level
```

```
get_tree().change_scene_to_file("res://Scenes/Levels/level_1.tscn")
```

```
func _on_quit_button_pressed():
```

```
    get_tree().quit()
```

Godot Insight: Scene transitions

Godot offers two main ways to switch scenes at runtime.

change_scene_to_file(path) takes a file path string (like "res://Scenes/Levels/level_1.tscn") and loads the scene from disk. This is convenient when you know the path at the call site, but means the scene is loaded synchronously, which can cause a brief freeze on large scenes.

change_scene_to_packed(packed_scene) takes a PackedScene resource that has already been loaded (via preload or load). This is useful when you want to preload scenes in advance or pass them as exported variables, as we do with the boss portal's next_level property.

Both methods free the current scene tree and replace it with the new one. Any nodes added to the root outside the main scene (as we initially did with rooms) will persist across transitions unless explicitly cleaned up.

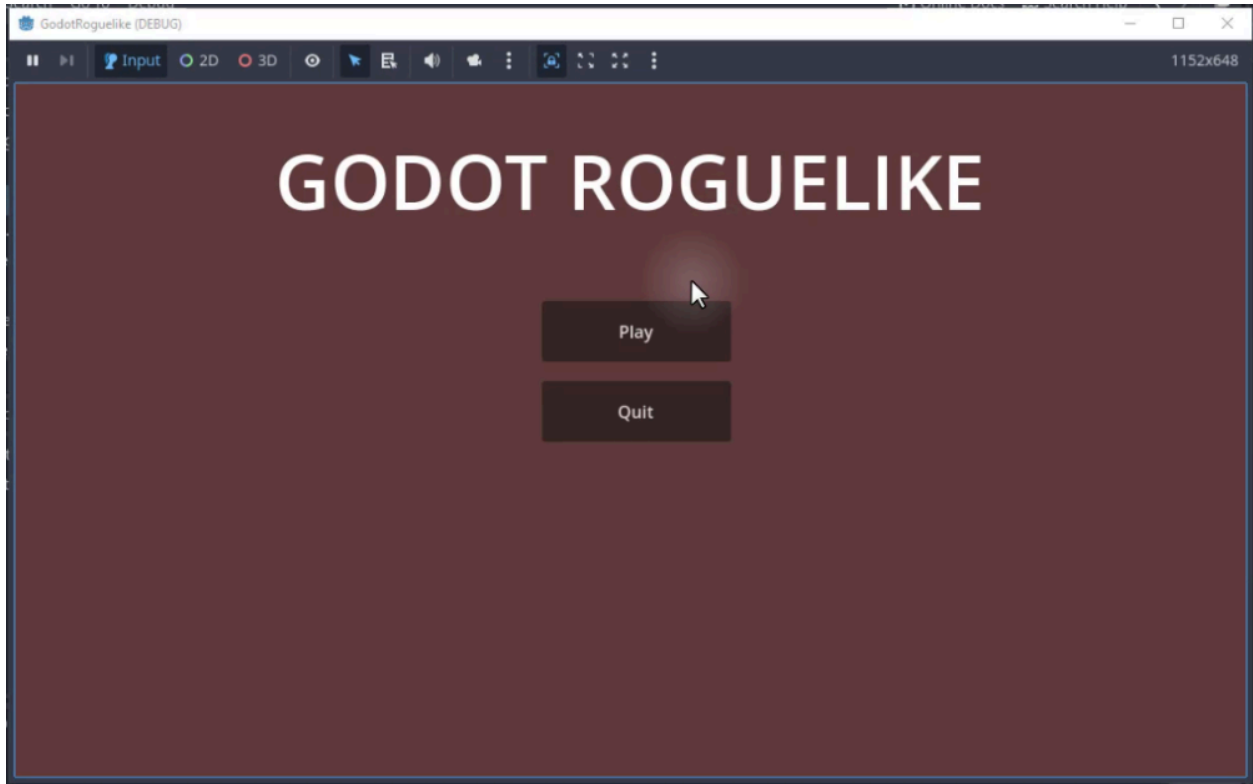
Learn more: [SceneTree](#)

(https://docs.godotengine.org/en/4.6/classes/class_scenetree.html)

Testing the Menu

Let's verify that the menu works as expected. First, run the Menu scene and click the **Play** button. It should load level_1.

Then, stop the game and run the menu again. This time click the **Quit** button and confirm that the game window closes.



You can feel free to style the menu further. For our purposes, however, it has all the functionality we need.

Connecting the Menu to the Game

Currently, the menu we just created is isolated. We need to integrate it into the game loop so that the player returns to the menu upon death or completing a level.

Handling Player Death

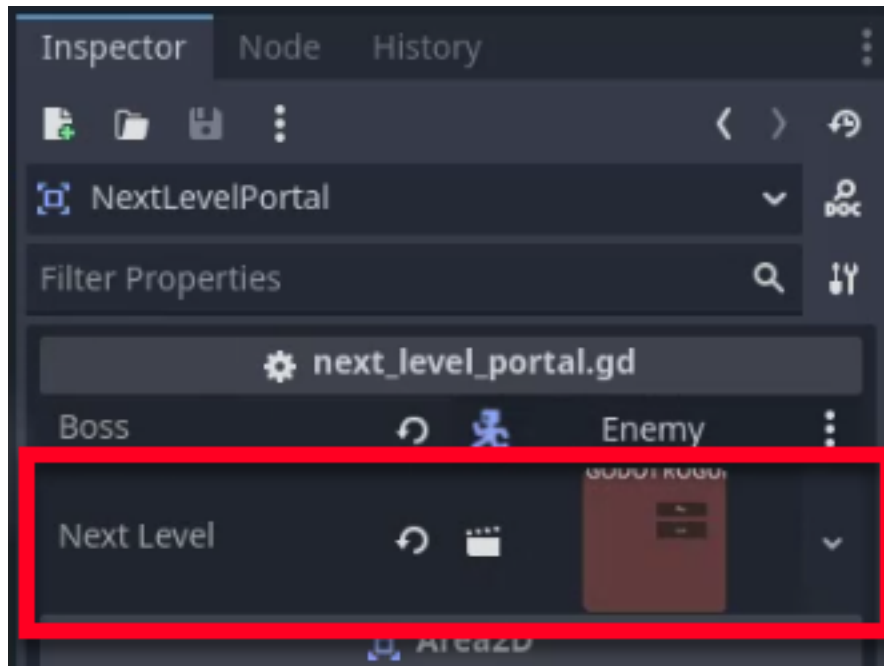
Let's first open the `player.gd` script. In the `die` function, load the menu scene when the player is defeated.

```
func die():  
    get_tree().change_scene_to_file("res://Scenes/menu.tscn")
```


Handling Level Completion

We also want to return to the menu when the player beats the boss (you could load a “Level 2” if you have one, but since we don’t, the menu will suffice).

Open the `room_boss` scene and select the `NextLevelPortal` node. In the Inspector, we can assign the `menu.tscn` scene to the **Next Level** property.



With this, our game loop is successfully set up.

Conclusion

Congratulations! You have successfully built a complete roguelike game in Godot. You now have a project with procedural generation, combat, items, a boss battle, audio, and a full game loop. In the closing chapter, we will reflect on everything you accomplished and explore ideas for where to go next.

Closing Words

Congratulations on reaching the end of this journey. Over these chapters, you stuck with it through debugging, problem-solving, and creative experimentation.

You have gone from an empty project folder to a fully playable, procedurally generated dungeon crawler. This is a significant milestone. Many people start game development projects, but far fewer finish them. The game you have built is not just a collection of scripts and sprites; it is a working system that handles input, physics, artificial intelligence, and user interfaces, all running in real-time.

Take a moment to appreciate the complexity of what you have built. You did not simply type out code; you solved problems.

What You Accomplished

Throughout these chapters, you have developed a versatile set of skills that apply to almost any game you will build in Godot.

You mastered the Node system. You learned how to compose complex objects like the player and enemies out of smaller, single-purpose nodes. You understand how inheritance works in Godot scenes, allowing you to create templates that share logic while maintaining unique properties.

You conquered the physics engine. You worked with `CharacterBody2D` to handle movement and collisions. You used `Area2D` to detect interactions without blocking movement, creating hitboxes and detection zones that make combat feel responsive.

You built systems that talk to each other. By using Signals, you decoupled your game logic. Your UI updates when health changes, but the player script does not need to know the UI exists. This architectural pattern is crucial for keeping larger projects manageable and bug-free.

You generated infinite content. You tackled the logic of procedural generation. You learned how to manipulate `TileMaps` through code, placing rooms and corridors dynamically to ensure no two playthroughs are exactly the same.

You polished the experience. A game is more than just mechanics. You added “juice” to your project through audio cues, visual feedback, floating text, and menus. You learned that the small details are often what make a game feel complete.

Where to Go From Here

The best way to solidify what you have learned is to keep using it. Here are several paths you can take to continue your development journey:

Expand this project

The roguelike game you built is a solid foundation, but it can be much more. Try adding these features:

- **A Save System:** Add scoring and use the `FileAccess` class to save the player’s high score between runs.
- **Inventory and Loot:** Create a system where enemies drop weapons with different stats.
- **Multiple Levels:** When the player defeats the boss, generate a new, harder dungeon and increase the difficulty.

Explore the Documentation

We covered a lot of ground, but Godot has hundreds of nodes we did not touch. Make a habit of reading the official documentation. It is the most up-to-date resource available and often contains code snippets that solve the exact problem you are facing.

Visit the documentation here: docs.godotengine.org
(<https://docs.godotengine.org>)

Join the Community

Game development can be solitary, but you do not have to do it alone. The Godot community (<https://godotengine.org/community/>) is known for being welcoming and helpful. Whether you hang out on the official Discord, the Godot Forum, or Reddit, sharing your work and asking questions is a great way to learn.

Start a New Project

If you feel ready for a change of pace, try building a completely different genre. A platformer will teach you about gravity and jump physics. A top-down shooter will test your vector math. A puzzle game will challenge your logic skills.

Continue Your Learning

Discover Zenva Academy (<https://academy.zenva.com>) for practical, project-led instruction in Godot and programming. With a catalog of over 300 courses and 40+ Learning Pathways, we provide structured paths that move you from basic concepts to advanced, real-world skills.

Final Thoughts

Game development is a practice. There will be days when your code breaks, your logic fails, and the engine does not do what you expect. This is normal. Every error message is just a puzzle waiting to be solved. The fact that you completed this book proves you have the persistence to work through those challenges.

You now have the tools to turn your ideas into playable experiences. Trust in the skills you have learned, keep experimenting, and most importantly, keep creating.

Good luck with your next game.

Appendix: Full Source Code

This appendix contains the complete source code for the roguelike project developed throughout this book. You can use this reference to debug your own code, verify your implementation, or simply review the logic of specific systems.

All scripts are located in the `Scripts` folder of the project files.

camera_controller.gd

Location: `/Scripts/camera_controller.gd`

This script manages the camera's movement. It extends `Camera2D` and connects to the global `OnPlayerEnterRoom` signal. When the player enters a new room, this script updates the camera's global position to center on that room.

```
extends Camera2D

func _ready ():
    # Listen for room changes via the global event bus
    GlobalSignals.OnPlayerEnterRoom.connect(_on_player_enter_room)

# Snap the camera to the center of the newly entered room
func _on_player_enter_room (room : Room):
    global_position = room.global_position
```

enemy.gd

Location: `/Scripts/enemy.gd`

This script defines the behavior for enemy characters. It handles state management (active/inactive), pathfinding towards the player, attacking when in range, and taking damage. The enemy only becomes active when the player enters its specific room.

```
class_name Enemy
extends CharacterBody2D
```

```

@export var cur_hp : int = 4
@export var max_hp : int = 4
@export var move_speed : float = 20

@export var attack_damage : int = 1
@export var attack_range : float = 10
@export var attack_rate : float = 0.5
var last_attack_time : float

@onready var sprite : Sprite2D = $Sprite

var room : Room
var is_active : bool = false
var player : CharacterBody2D

var player_direction : Vector2
var player_distance : float

func _ready():
    # Find the player node via the "Player" group instead of a hardcoded path
    player = get_tree().get_first_node_in_group("Player")
    GlobalSignals.OnPlayerEnterRoom.connect(_on_player_enter_room)

func initialize (in_room : Room):
    is_active = false
    room = in_room

# Activate only when the player is in this enemy's room
func _on_player_enter_room (player_room : Room):
    is_active = player_room == room

func _physics_process(delta):
    # Skip processing when inactive or if the player reference is lost
    if not is_active or not player:
        return

    player_direction =
    global_position.direction_to(player.global_position)
    player_distance =
    global_position.distance_to(player.global_position)

    sprite.flip_h = player_direction.x < 0

    # Stop moving and attack when close enough to the player

```

```

    if player_distance < attack_range:
        _try_attack()
        return

    velocity = player_direction * move_speed
    move_and_slide()

# Attempts an attack if the cooldown period has elapsed
func _try_attack ():
    # Enforce cooldown using real time to prevent attack spamming
    if Time.get_unix_time_from_system() - last_attack_time <
attack_rate:
        return

    last_attack_time = Time.get_unix_time_from_system()

    player.take_damage(attack_damage)

func take_damage (amount : int):
    cur_hp -= amount

    _damage_flash()
    $DamagedSound.play()

    if cur_hp <= 0:
        die()

# Brief visibility toggle to give visual damage feedback
func _damage_flash ():
    visible = false
    # await pauses this function until the timer fires, then resumes
    await get_tree().create_timer(0.07).timeout
    visible = true

func die ():
    # Emit signal before queue_free so listeners can react while the
node still exists
    GlobalSignals.OnDefeatEnemy.emit(self)
    queue_free()

```

global_signals.gd

Location: /Scripts/global_signals.gd

This script acts as a central hub for communication between different systems. It defines signals for room transitions, enemy defeat, and player health updates, allowing nodes to communicate without direct dependencies. This script should be set as an AutoLoad in the Project Settings.

```
extends Node
```

```
# Global event bus (AutoLoad): lets any node broadcast or listen for game events
```

```
# without needing direct references to other nodes
```

```
signal OnPlayerEnterRoom (room : Room)
```

```
signal OnDefeatEnemy (enemy : Enemy)
```

```
signal OnPlayerUpdateHealth (cur_hp : int, max_hp : int)
```

item.gd

Location: /Scripts/item.gd

This script controls collectible items. It defines an enumeration for item types (Health, Shoot Rate, Move Speed) and handles the logic for applying stat boosts to the player upon collision.

```
extends Area2D
```

```
enum ItemType
```

```
{  
    HEALTH,  
    SHOOT_RATE,  
    MOVE_SPEED  
}
```

```
@export var type : ItemType
```

```
@export var value : float
```

```
func _on_body_entered(body):
```

```
    # Ignore collisions with anything that isn't the player
```

```
    if not body.is_in_group("Player"):
```

```
        return
```

```
    # Apply a different stat effect depending on the item type
```



```

if type == ItemType.HEALTH:
    body.heal(int(value))
elif type == ItemType.SHOOT_RATE:
    body.shoot_rate -= value
elif type == ItemType.MOVE_SPEED:
    body.move_speed += value

body.get_node("ItemSound").play()

# Remove the item from the scene after pickup
queue_free()

```

menu.gd

Location: /Scripts/menu.gd

This script manages the main menu interface. It handles button inputs to start the game (loading the first level) or quit the application.

```

extends Control

func _on_play_button_pressed():
    # Transition from the menu to the first gameplay level

get_tree().change_scene_to_file("res://Scenes/Levels/level_1.tscn")

func _on_quit_button_pressed():
    get_tree().quit()

```

minimap.gd

Location: /Scripts/minimap.gd

This script controls the mini-map UI. It listens for player movement and updates a grid of icons to reflect the current room and explored areas. It differentiates between the player's current room, visited rooms, and empty space.

```

extends GridContainer

var room_gen : RoomGeneration

```

```

var icons : Array[TextureRect]

var room_texture : Texture =
preload("res://Sprites/Items/minimap_room.tres")
var player_room_texture : Texture =
preload("res://Sprites/Items/minimap_player_room.tres")

func _ready ():
    GlobalSignals.OnPlayerEnterRoom.connect(_on_player_enter_room)

# Updates the minimap grid each time the player moves to a new room
func _on_player_enter_room (room : Room):
    # Lazy init: fetch room_gen and icon references on first call,
    # since the RoomGeneration node may not be ready during _ready
    if not room_gen:
        room_gen = get_node("/root/Main/RoomGeneration")

        for child in get_children():
            if child is TextureRect:
                icons.append(child)

    for x in range(room_gen.map_size):
        for y in range(room_gen.map_size):
            var r = room_gen.get_room_from_map(x, y)
            # Convert 2D grid coords to a 1D array index
            var i = x + y * room_gen.map_size

            if i >= len(icons):
                continue

            # Three visual states: empty (null), current room, or
visited room
            if not r:
                icons[i].texture = null
            elif r == room:
                icons[i].texture = player_room_texture
            else:
                icons[i].texture = room_texture

```

next_level_portal.gd

Location: /Scripts/next_level_portal.gd

This script manages the portal to the next level. It remains inactive and invisible until a specific boss enemy is defeated. Once active, it detects player collision to transition to the next scene.

```
extends Area2D
```

```
@export var boss : Enemy
```

```
@export var next_level : PackedScene
```

```
func _ready ():
```

```
    GlobalSignals.OnDefeatEnemy.connect(_on_defeat_enemy)
```

```
    # Start hidden and non-interactive until the boss is defeated
```

```
    visible = false
```

```
    $CollisionShape2D.disabled = true
```

```
func _on_defeat_enemy (enemy : Enemy):
```

```
    # Only activate the portal when the assigned boss is killed
```

```
    if enemy != boss:
```

```
        return
```

```
    visible = true
```

```
    # set_deferred avoids toggling physics shapes mid-physics step
```

```
    $CollisionShape2D.set_deferred("disabled", false)
```

```
func _on_body_entered(body):
```

```
    if not body.is_in_group("Player"):
```

```
        return
```

```
    # call_deferred prevents changing scenes while the engine is mid-frame
```

```
    get_tree().change_scene_to_packed.call_deferred(next_level)
```

player.gd

Location: /Scripts/player.gd

This script defines the player character. It handles movement input, weapon aiming (rotating towards the mouse), shooting projectiles, taking damage, and healing. It also emits signals to update the health UI.

```
extends CharacterBody2D
```

```

@export var cur_hp : int = 8
@export var max_hp : int = 8

@export var move_speed : float = 50

@export var shoot_rate : float = 0.4
var last_shoot_time : float

var projectile_scene : PackedScene =
preload("res://Scenes/Projectiles/projectile.tscn")

@onready var sprite : Sprite2D = $Sprite
@onready var weapon_origin : Node2D = $Weapon
@onready var muzzle : Node2D = $Weapon/Muzzle

func _ready ():
    # Defer the initial health emit; the UI node may not be ready yet
    GlobalSignals.OnPlayerUpdateHealth.emit.call_deferred(cur_hp,
max_hp)

    # Handle WASD movement each physics frame
    func _physics_process(delta):
        var move_input : Vector2 = Input.get_vector("move_left",
"move_right", "move_up", "move_down")
        velocity = move_input * move_speed
        move_and_slide()

    # Aim weapon toward the mouse and handle shooting input
    func _process (delta):
        var mouse_pos : Vector2 = get_global_mouse_position()
        var mouse_dir : Vector2 = (mouse_pos -
global_position).normalized()
        # +90 because the sprite points up, but Godot's angle() 0 points
right
        weapon_origin.rotation_degrees = rad_to_deg(mouse_dir.angle()) +
90

        sprite.flip_h = mouse_dir.x < 0

        if Input.is_action_pressed("attack"):
            # Enforce cooldown between shots
            if Time.get_unix_time_from_system() - last_shoot_time >
shoot_rate:
                _shoot()

```

```

# Spawn a projectile at the muzzle and orient it to match the weapon
func _shoot ():
    last_shoot_time = Time.get_unix_time_from_system()

    var proj = projectile_scene.instantiate()
    get_tree().root.add_child(proj)
    proj.global_position = muzzle.global_position
    proj.rotation = weapon_origin.rotation
    # Tag projectile so it won't damage the player who fired it
    proj.owner_character = self

    $ShootSound.play()

func take_damage (amount : int):
    cur_hp -= amount

    _damage_flash()
    # Update the health UI before checking death, so the final state is
visible
    GlobalSignals.OnPlayerUpdateHealth.emit(cur_hp, max_hp)
    $DamagedSound.play()

    if cur_hp <= 0:
        die()

# Brief flicker to give visual hit feedback
func _damage_flash ():
    visible = false
    await get_tree().create_timer(0.07).timeout
    visible = true

# Return to the main menu on death
func die ():
    get_tree().change_scene_to_file("res://Scenes/menu.tscn")

func heal (amount : int):
    cur_hp += amount

    # Clamp health to the maximum
    if cur_hp > max_hp:
        cur_hp = max_hp

    GlobalSignals.OnPlayerUpdateHealth.emit(cur_hp, max_hp)

```

player_health_ui.gd

Location: /Scripts/player_health_ui.gd

This script manages the visual representation of the player's health. It updates a grid of heart icons based on the player's current and maximum health values received from the global signal.

```
extends GridContainer

var full_heart : Texture =
preload("res://Sprites/Items/heart_full.tres")
var empty_heart : Texture =
preload("res://Sprites/Items/heart_empty.tres")

var heart_icons : Array[TextureRect]

# Connect in _init (not _ready) so the signal is wired before the first emit
func _init ():
    GlobalSignals.OnPlayerUpdateHealth.connect(_update_ui)

func _ready ():
    for child in get_children():
        if child is TextureRect:
            heart_icons.append(child)

# Sync heart icons with the player's current and max health
func _update_ui (cur_hp : int, max_hp : int):
    for i in len(heart_icons):
        # Hide icons beyond max health
        if i >= max_hp:
            heart_icons[i].visible = false
            continue

        heart_icons[i].visible = true

        # Full heart for remaining HP, empty heart for lost HP
        if i < cur_hp:
            heart_icons[i].texture = full_heart
        else:
            heart_icons[i].texture = empty_heart
```

projectile.gd

Location: /Scripts/projectile.gd

This script defines the behavior of projectiles fired by the player. It handles movement in a straight line, collision detection with enemies, and self-destruction upon impact or timeout.

extends Area2D

@export **var** base_speed : float = 100

var owner_character : CharacterBody2D

func _process (delta):

Move along local up (-Y); rotation determines travel direction

translate(-transform.y * base_speed * delta)

func _on_body_entered(body):

Ignore the character who fired this projectile

if body == owner_character:

return

Duck-typed damage: any body with take_damage() can be hurt

if body.has_method("take_damage"):

body.take_damage(1)

queue_free()

func _on_timer_timeout():

queue_free()

room.gd

Location: /Scripts/room.gd

This script manages individual room behavior. It tracks enemies within the room, controls door states (open/closed), and handles player entry events. It also coordinates the four directional entrances.

class_name Room

extends StaticBody2D

```

enum Direction
{
    NORTH,
    SOUTH,
    EAST,
    WEST
}

@export var doors_always_open : bool = false

@onready var entrance_north : RoomEntrance = $Entrance_North
@onready var entrance_south : RoomEntrance = $Entrance_South
@onready var entrance_east : RoomEntrance = $Entrance_East
@onready var entrance_west : RoomEntrance = $Entrance_West

var enemies_in_room : int

func _ready ():
    GlobalSignals.OnDefeatEnemy.connect(_on_enemy_defeated)

    # Count child enemies and bind them to this room
    for child in get_children():
        if child is Enemy:
            enemies_in_room += 1
            child.initialize(self)

func set_neighbor (neighbor_direction : Direction, neighbor_room :
Room):
    if neighbor_direction == Direction.NORTH:
        entrance_north.set_neighbor(neighbor_room)
    elif neighbor_direction == Direction.SOUTH:
        entrance_south.set_neighbor(neighbor_room)
    elif neighbor_direction == Direction.EAST:
        entrance_east.set_neighbor(neighbor_room)
    elif neighbor_direction == Direction.WEST:
        entrance_west.set_neighbor(neighbor_room)

# called when the player enters the room
func player_enter (entry_direction : Direction, player :
CharacterBody2D, first_room : bool = false):
    # set their position based on where they entered from
    if entry_direction == Direction.NORTH:
        player.global_position =
entrance_north.player_spawn.global_position
    elif entry_direction == Direction.SOUTH:

```



```

        player.global_position =
entrance_south.player_spawn.global_position
    elif entry_direction == Direction.EAST:
        player.global_position =
entrance_east.player_spawn.global_position
    elif entry_direction == Direction.WEST:
        player.global_position =
entrance_west.player_spawn.global_position

    # if this is the first room, spawn the player in the middle
    if first_room:
        player.global_position = global_position

    # Notify all listeners (camera, minimap, enemies) of room change
    GlobalSignals.OnPlayerEnterRoom.emit(self)

    # close the doors if there are enemies to defeat
    if enemies_in_room > 0 and not doors_always_open:
        close_doors()
    else:
        open_doors()

func _on_defeat_enemy (enemy):
    pass

# call_deferred avoids toggling physics bodies mid-callback
func open_doors ():
    entrance_north.open_door.call_deferred()
    entrance_south.open_door.call_deferred()
    entrance_east.open_door.call_deferred()
    entrance_west.open_door.call_deferred()

# call_deferred avoids toggling physics bodies mid-callback
func close_doors ():
    entrance_north.close_door.call_deferred()
    entrance_south.close_door.call_deferred()
    entrance_east.close_door.call_deferred()
    entrance_west.close_door.call_deferred()

func _on_enemy_defeated (enemy : Enemy):
    # Only count enemies that belong to this room
    if enemy.get_parent() == self:
        enemies_in_room -= 1

```

```

    if enemies_in_room <= 0:
        open_doors()

```

room_entrance.gd

Location: /Scripts/room_entrance.gd

This script controls specific room entrances. It manages the visual state of doors and barriers, and detects when the player passes through to a neighboring room.

```

class_name RoomEntrance
extends Node2D

@export var direction : Room.Direction = Room.Direction.NORTH

@onready var barrier : StaticBody2D = $Barrier
@onready var barrier_collider : CollisionShape2D =
    $Barrier/CollisionShape2D

@onready var door : Node2D = $Door
@onready var door_shut : StaticBody2D = $Door/Door_Shut
@onready var door_shut_collider : CollisionShape2D =
    $Door/Door_Shut/CollisionShape2D

@onready var player_spawn : Node2D = $PlayerSpawn
@onready var exit_trigger : Area2D = $ExitTrigger

var neighbor : Room

func _ready ():
    exit_trigger.body_entered.connect(_on_body_entered_exit_trigger)
    # Start as a solid wall; set_neighbor will swap to a door if linked
    toggle_barrier(true)

# Reveal a passable door when this entrance connects to another room
func set_neighbor (neighbor_room : Room):
    neighbor = neighbor_room
    toggle_barrier(false)

# Swap between impassable wall (barrier) and openable door
func toggle_barrier (toggle : bool):
    barrier.visible = toggle

```

```

barrier_collider.disabled = !toggle

door.visible = !toggle

func open_door ():
    # Do nothing if this entrance is a barrier (no neighbor)
    if barrier.visible:
        return

    door_shut.visible = false
    door_shut_collider.disabled = true

func close_door ():
    # Do nothing if this entrance is a barrier (no neighbor)
    if barrier.visible:
        return

    door_shut.visible = true
    door_shut_collider.disabled = false

# Transition the player into the neighboring room
func _on_body_entered_exit_trigger (body):
    if body.is_in_group("Player"):
        neighbor.player_enter(_get_neighbor_entry_direction(), body)

# Map to the opposite direction so the player enters from the correct side
func _get_neighbor_entry_direction () -> Room.Direction:
    if direction == Room.Direction.NORTH:
        return Room.Direction.SOUTH
    elif direction == Room.Direction.SOUTH:
        return Room.Direction.NORTH
    elif direction == Room.Direction.EAST:
        return Room.Direction.WEST
    else:
        return Room.Direction.EAST

```

room_generation.gd

Location: /Scripts/room_generation.gd

This script handles the procedural generation of the dungeon. It creates a logical grid of rooms, instantiates room scenes, connects neighbors, and places the boss room.

```

class_name RoomGeneration
extends Node

@export var map_size : int = 7
@export var rooms_to_generate : int = 12
var room_count : int = 0
var map : Array[bool]
var rooms : Array[Room]
var room_pos_offset : float = 160

var first_room_x : int = 3
var first_room_y : int = 3
var first_room : Room

var room_scene : PackedScene =
    preload("res://Scenes/Rooms/room_template.tscn")

@export var first_room_scene : PackedScene
@export var boss_room : PackedScene
@export var room_scenes : Array[PackedScene]

@export var player : CharacterBody2D

func _ready ():
    _generate()

    # room layout preview
    #for x in range(map_size):
        #var line : String
        #for y in range(map_size):
            #line += "# " if _get_map(x, y) else " "
        #print(line)

func _generate ():
    room_count = 0
    map.resize(map_size * map_size)

    # start the check room chain at the center of the grid
    _check_room(first_room_x, first_room_y, Vector2.ZERO, true)

    # once we've defined our map, spawn in the room scenes
    _instantiate_rooms()

func _check_room (x : int, y : int, desired_direction : Vector2,
is_first_room : bool = false):

```

```

# return if we've created enough rooms
if room_count >= rooms_to_generate:
    return

# return if out of bounds
if x < 0 or x > map_size - 1 or y < 0 or y > map_size - 1:
    return

# return if room already exists
if _get_map(x, y) == true:
    return

room_count += 1
_set_map(x, y, true)

# define the directions we want to expand in
var go_north : bool = randf() > (0.2 if desired_direction ==
Vector2.UP else 0.8)
var go_south : bool = randf() > (0.2 if desired_direction ==
Vector2.DOWN else 0.8)
var go_east : bool = randf() > (0.2 if desired_direction ==
Vector2.RIGHT else 0.8)
var go_west : bool = randf() > (0.2 if desired_direction ==
Vector2.LEFT else 0.8)

# continue the check room chain in those directions
if go_north or is_first_room:
    _check_room(x, y - 1, Vector2.UP if is_first_room else
desired_direction)
if go_south or is_first_room:
    _check_room(x, y + 1, Vector2.DOWN if is_first_room else
desired_direction)
if go_east or is_first_room:
    _check_room(x + 1, y, Vector2.RIGHT if is_first_room else
desired_direction)
if go_west or is_first_room:
    _check_room(x - 1, y, Vector2.LEFT if is_first_room else
desired_direction)

func _instantiate_rooms ():
    var boss_room_pos : Vector2 = _decide_boss_room()

# instantiate each room scene at their designated position
for x in range(map_size):

```

```

    for y in range(map_size):
        if _get_map(x, y) == false:
            continue

        var room : Room
        var is_first_room : bool = first_room_x == x and
first_room_y == y

        if is_first_room:
            room = first_room_scene.instantiate()
        elif x == boss_room_pos.x and y == boss_room_pos.y:
            room = boss_room.instantiate()
        else:
            room = room_scenes[randi_range(0, len(room_scenes) -
1)].instantiate()

get_tree().root.get_node("/root/Main").add_child.call_deferred(room)
rooms.append(room)

room.global_position = Vector2(x, y) * room_pos_offset

if is_first_room:
    first_room = room

# set room neighbors
for room in rooms:
    var map_pos : Vector2 = _get_map_index(room)
    var x = map_pos.x
    var y = map_pos.y

    if y > 0 and _get_map(x, y - 1):
        room.set_neighbor.call_deferred(Room.Direction.NORTH,
get_room_from_map(x, y - 1))
    if y < map_size - 1 and _get_map(x, y + 1):
        room.set_neighbor.call_deferred(Room.Direction.SOUTH,
get_room_from_map(x, y + 1))
    if x < map_size - 1 and _get_map(x + 1, y):
        room.set_neighbor.call_deferred(Room.Direction.EAST,
get_room_from_map(x + 1, y))
    if x > 0 and _get_map(x - 1, y):
        room.set_neighbor.call_deferred(Room.Direction.WEST,
get_room_from_map(x - 1, y))

```

```

    # put the player in the first room
    first_room.player_enter.call_deferred(Room.Direction.NORTH,
    player, true)

# Pick a random occupied cell that isn't the starting room
func _decide_boss_room () -> Vector2:
    var i = 0

    while i < 100:
        var x = randi_range(0, map_size - 1)
        var y = randi_range(0, map_size - 1)

        if first_room_x == x and first_room_y == y:
            continue

        if _get_map(x, y):
            return Vector2(x, y)

    return Vector2.ZERO

# Find the Room node at the given grid coordinates
func get_room_from_map (x : int, y : int) -> Room:
    for room in rooms:
        var pos = _get_map_index(room)

        if pos.x != x or pos.y != y:
            continue

    return room

return null

# Convert a room's world position back to grid coordinates
func _get_map_index (room : Room) -> Vector2i:
    return Vector2i(room.global_position / room_pos_offset)

# 2D-to-1D index: row-major layout in a flat array
func _get_map (x : int, y : int) -> bool:
    return map[x + y * map_size]

func _set_map (x : int, y : int, value : bool):
    map[x + y * map_size] = value

```

room_item_spawner.gd

Location: /Scripts/room_item_spawner.gd

This script spawns a random item in the item room. It selects a scene from a predefined list and instantiates it when the room loads.

extends Node2D

@export **var** item_scenes : Array[PackedScene]

func _ready ():

Pick a random item from the exported list and spawn it here

var item = item_scenes[randi_range(0, len(item_scenes) -

1)].instantiate()

add_child(item)